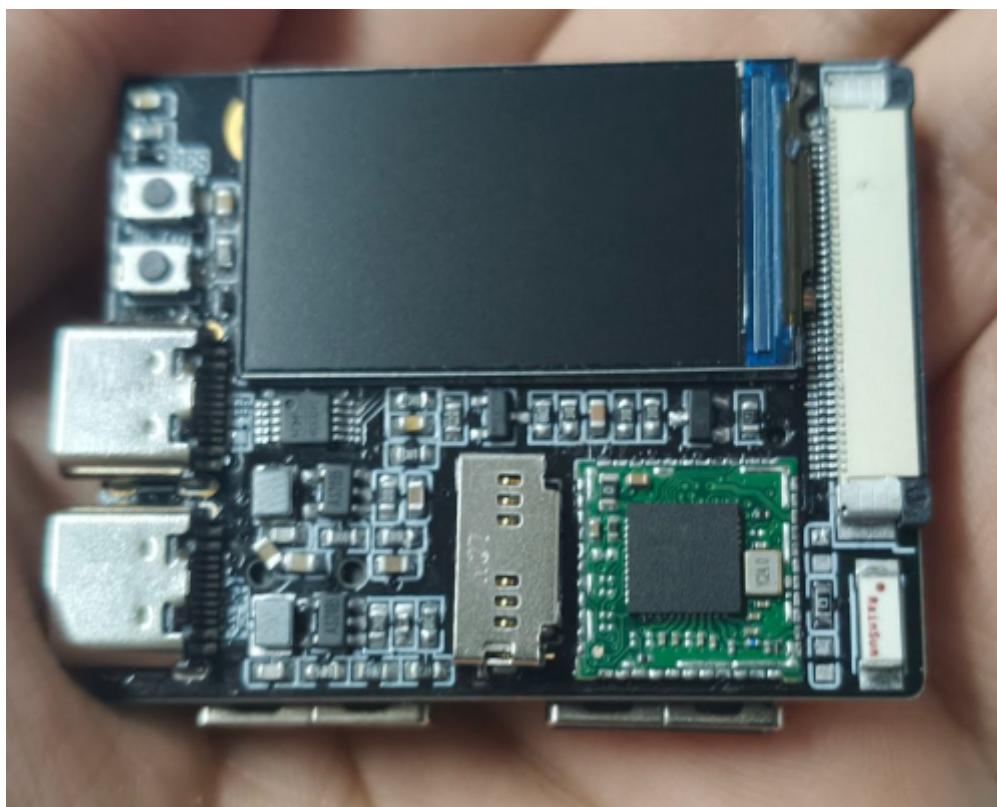


*Rvicard-Linux*驱动移植与应用 开发指南

Drivers and Application Development Guide for
Rvicard-Linux

陈过一上 (B站) 著



前言

第一章 Rvicard平台概述

1.1 Rvicard 简介

Rvicard 是一款基于瑞芯微 RV1106G3 处理器设计的高集成度开发板，采用 **核心板 + 底板分离** 的模块化结构，便于功能扩展和快速产品化。核心板尺寸仅 **2.2cm × 3.0cm**，底板尺寸仅 **3.5cm × 4.5cm**，非常紧凑。

RV1106G3 处理器内置 **256MB DDR3 内存**，同时集成 **1TOPS 级别的 AI 算力 (NPU)**，可满足图像识别、视频处理、智能控制等多种应用场景的需求。该平台具有以下特点：

- **高性能与低功耗兼顾**：Cortex-A7 架构处理器主频高，配合硬件 NPU 加速 AI 算法，适合边缘智能应用。
- **丰富的接口资源**：提供 USB、SPI、I²C、UART、CAN、RGB/LCD 显示接口等，可灵活连接传感器、外设模块和显示屏。
- **模块化设计**：核心板封装主要芯片与高速信号，底板可根据不同产品需求定制，缩短开发周期。
- **良好的软件支持**：官方提供基于 Linux 的 SDK，支持设备树配置、驱动开发与应用移植，开发环境成熟。
- **易于调试**：支持串口调试、USB ADB、网络调试，便于开发。

Rvicard 是我出于个人兴趣设计的一款嵌入式开发板，目标是做一套 **体积小、性能强、开发体验友好** 的软硬件平台，方便我进行 **Linux 驱动研究、应用移植和边缘 AI 实验**

1.2 硬件资源概览

核心板与底板均采用 **4 层 PCB 设计**。

1.2.1 核心板集成资源

核心板仅需 **5V 电源输入** 即可工作，板上集成多颗 **MT3520B DC/DC 转换芯片**，依次输出 3.3V、1.8V 和 0.9V，为 RV1106G3 及其外设供电。

核心板主要资源如下：

- **存储**：板载 256MB SPI Flash
- **状态指示**：板载可编程 LED（可作为心跳灯）
- **外设接口**：
 - 1 路 I²C 接口
 - 1 路 SPI 接口
 - RGB666 显示接口
 - 调试串口 UART0
 - 蓝牙串口（可连接 RTL8723DS 等模块）
 - SDIO 接口（支持 Wi-Fi 模块或 TF 卡）
 - MIPI 摄像头接口
 - 1 路 USB 接口（连接外部设备）

1.2.2 底板集成资源

底板用于验证核心板各项功能资源，设计紧凑，并提供丰富的扩展能力。主要资源如下：

- **USB 资源**
 - 2 路 Type-C 接口：
 - 一路为调试串口
 - 一路为烧录串口（可用于 ADB 调试或固件烧录）
 - 板载 USB 选择芯片 + USB HUB 扩展，提供双 USB Host 口
- **显示与音频**
 - 板载 1.14 英寸 SPI 屏幕
 - RGB666 屏幕接口（集成升压电路）
 - 板载全指向麦克风 + 扬声器接口
- **存储与网络**
 - 板载 TF 卡槽
 - 板载 RTL8723DS 无线网卡（Wi-Fi + BT）
- **调试与控制**
 - BOOT/RESET 按键（便于固件烧录与复位）
 - I²C 总线挂载 MPU6050 六轴传感器

底板可基于核心板引出的接口进行二次定制开发，也可作为完整的参考设计使用。

1.3 软件支持与开发环境

Rvicard 平台的软件开发主要基于瑞芯微官方提供的 **SDK**，开发者可以通过官方文档快速搭建开发环境。官方教程可参考：

[上手教程](#) | [LUCKFOX WIKI](#)

本书教程主要围绕 **SPI NAND Flash** 的应用开发与演示进行说明，帮助读者理解驱动开发和应用移植的全过程。

Rvicard 平台的软件环境主要特点如下：

- **操作系统**：基于 Linux 内核，支持常用 Linux 工具和命令行操作。
- **开发工具链**：官方 SDK 包含交叉编译器、调试工具、示例程序，支持快速编译和烧录。
- **驱动支持**：提供针对核心板和底板各类外设的驱动接口，包括 UART、I²C、SPI、SDIO、USB、RGB666 显示接口等。
- **调试与测试**：
 - **串口调试**：支持 UART/USB 连接，可通过终端查看内核输出和应用日志。
 - **网络调试**：Wi-Fi 模块可用于远程 SSH 访问和文件传输。
 - **烧录与固件更新**：通过 BOOT/RESET 按键及 USB 烧录接口进行固件写入。
- **示例与扩展**：SDK 提供丰富的示例程序，便于学习和扩展开发，包括 GPIO 控制、摄像头采集、显示输出、网络通信等。

通过上述软件环境，读者可以在 Rvicard 平台上完成从驱动开发、应用移植到实验验证的全流程操作。本书后续章节将结合具体示例对 **驱动开发、应用开发及适配** 进行详细讲解

1.4 其他补充

第二章 Linux驱动开发与移植

2.1 Linux驱动开发简单介绍

在正式学习驱动前，需要了解 Linux 系统中 **驱动、设备树、内核与根文件系统** 的基本概念和关系。只有理解了这些基础框架，才能更高效地进行后续的移植与调试。

2.1.1 设备树和linux内核介绍

在传统的单片机开发中，如果要使用某个引脚或外设（UART、SPI 等），我们通常要在代码里声明引脚、初始化硬件资源，然后再写程序去控制。一旦硬件改动或移植，就得修改大量代码。由于代码和硬件强耦合，擅自改动（尤其是工程体量很大时）很容易引入意想不到的 bug。

Linux 的做法不一样：**它把硬件信息和驱动逻辑分开**。硬件资源由设备树（Device Tree）描述，而控制硬件的程序就是驱动代码。这样，驱动可以做到“一处编写，多处使用”，只需要在设备树里调整硬件描述，而无需大动驱动源码，大大降低了软硬件耦合带来的风险。

设备树本质上是个树形结构，用节点和属性来描述硬件资源。内核启动时会读取设备树，知道“有哪些硬件、该分配哪些资源”，再把这些信息交给对应的驱动去使用。对初学者来说，设备树语法和解析方式可能有些陌生，所以本教程会以实践为主，遇到概念再补理论。

Linux 内核本身是整个系统的核心，相当于硬件和应用之间的大管家。它负责管理 CPU、内存、外设资源，调度进程和线程运行，并通过驱动框架为上层应用提供统一接口。内核是一个庞大的 C 工程（带少量汇编），包括内存管理、进程调度、文件系统、网络协议栈以及各种外设驱动支持。运行内核还需要一个 **根文件系统（RootFS）** 提供命令、库和启动脚本，这样系统才能真正“活起来”。

我们常说的**交叉编译**，就是在 PC（通常是 x86 架构）上，用 ARM 的 gcc 或 g++ 工具链编译出能在开发板（ARM 架构）运行的程序。我们不仅要交叉编译应用程序，还要交叉编译内核和驱动模块。比如：

```
arm-linux-gnueabi-gcc -o hello hello.c
```

这条命令会生成一个 `hello` 可执行文件，拷到 ARM 板子上执行：

```
./hello
```

就会打印 `hello`。注意：必须先在 Ubuntu 虚拟环境或 WSL 中下载好交叉编译工具链，否则会提示“找不到命令”。

当我们编译 Linux 内核时，最终会得到：

- **zimage / uimage**（内核镜像）；
- **.dtb 文件**（设备树二进制文件）；
- **.ko 模块**（驱动模块，可按需加载）。

在后续教程中，我们会反复修改设备树和内核，并重新编译它们，然后将结果移到开发板上运行

2.1.2 根文件系统

根文件系统（Root File System，简称 RootFS）是 Linux 启动后第一个挂载的文件系统，它为内核和用户提供最基本的运行环境。简单来说，内核负责让硬件运转，但想让系统“活”起来、能运行各种程序，还需要一个包含命令、库文件、配置文件和启动脚本的完整目录结构（如 `/bin`、`/lib`、`/etc`、`/dev` 等）。根文件系统可以非常小（仅包含启动必需的工具），也可以非常完整（类似桌面 Linux），根据开发需求选择。嵌入式系统常见的根文件系统有 BusyBox 制作的精简版，也有 Buildroot、Yocto 生成的完整版本。简单来说，**内核是引擎，根文件系统是车身和驾驶室，二者结合才能真正跑起来。**

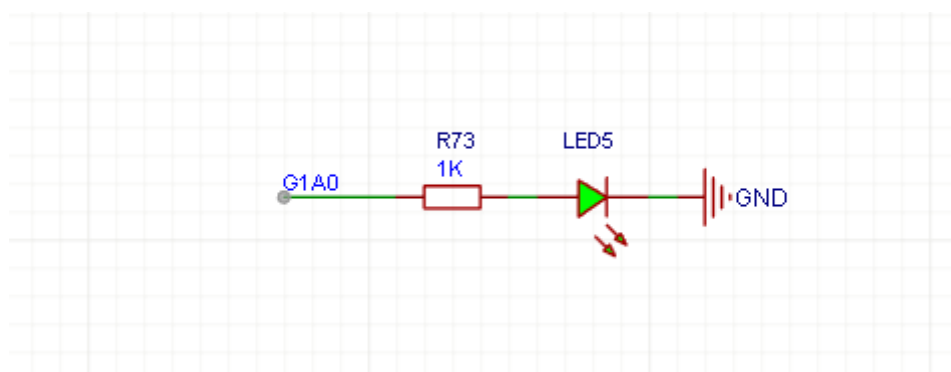
后续我们将使用 buildroot 文件系统进行应用与驱动的开发，如果我们需要移植 QT 应用，需要把 QT 运行库放到根文件系统中，或者搞一个自动启动的脚本。

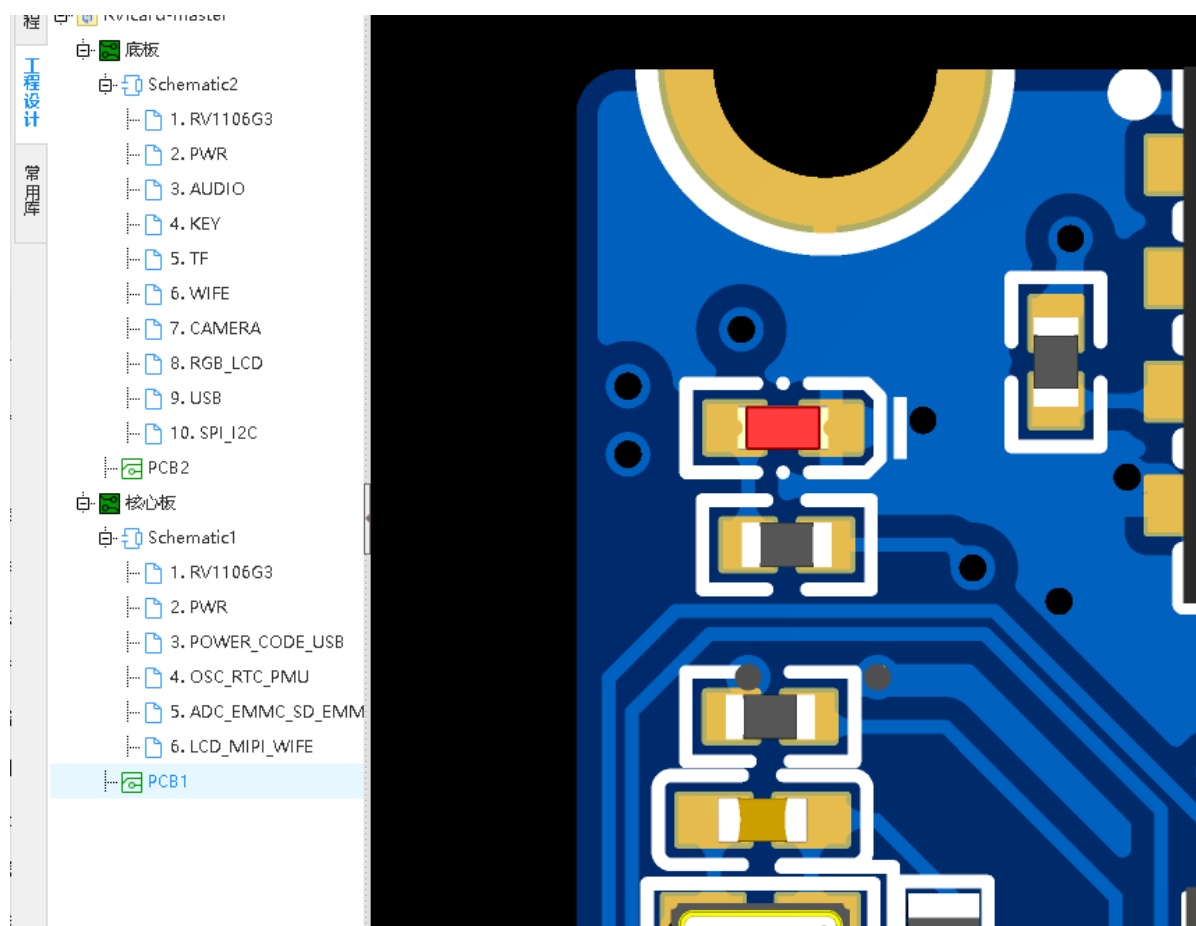
2.2 点亮一颗“心跳”灯

因为是第一次操作设备树和 linux 内核，所以会讲的详细一些

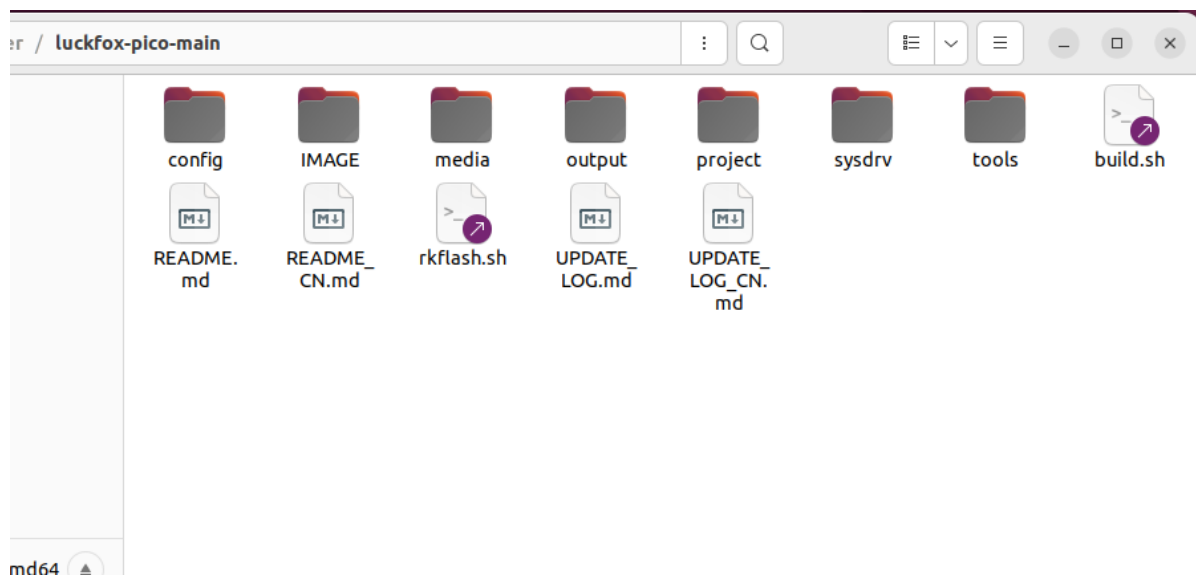
2.2.1 设备树及内核修改

在我们实际操作之前，我们先来看看硬件资源，我们驱动这颗灯在核心板这里，以及它对应的原理图，如图





由此我们可以看出，这颗灯需要GPIO1_A0输出高电平进行点亮，好接下来就正式开始，我们先来找设备树，来描述这个灯。相信你已经下载好了SDK，他的结构如图，而他的设备树文件就在当前文件夹下的 luckfox-pico-main/sysdrv/source/kernel/arch/arm/boot/dts 这个路径，名叫 rv1106g-luckfox-pico-pro-max.dts。



我们先来添加这个设备树节点，我们在它的根节点 (/) 下添加 leds 这个节点，如图所示（当然如果你添加失败了，可以在我提供的资料Materials/2.2文件夹中直接替换设备树）

```

5
6 /dts-v1/;
7
8 #include "rv1106.dtsi"
9 #include "rv1106-evb.dtsi"
10 #include "rv1106-luckfox-pico-pro-max-ipc.dtsi"
11 #include <dt-bindings/input/input.h>
12 #include <dt-bindings/display/media-bus-format.h>
13
14
15 / {
16     model = "Luckfox Pico Max";
17     compatible = "rockchip,rv1103g-38x38-ipc-v10", "rockchip,rv1106";
18
19
20
21     leds {
22         compatible = "gpio-leds";
23
24         heartbeat_led {
25             label = "heartbeat";
26             gpios = <&gpio1 RK_PA0 GPIO_ACTIVE_LOW>;
27             default-state = "on";
28             //max-brightness = <255>; // 手动设置最大亮度
29             linux,default-trigger = "heartbeat";
30             status = "okay"; // 这应该在每个 LED 节点下
31         };
32     };
33
34     backlight: backlight {
35         compatible = "gpio-backlight";
36         gpios = <&gpio2 RK_PA7 GPIO_ACTIVE_HIGH>;
37         default-on; // 默认开启背光
38         status = "okay";
39 };

```

```

leds {
    compatible = "gpio-leds";

    heartbeat_led {
        label = "heartbeat";
        gpios = <&gpio1 RK_PA0 GPIO_ACTIVE_LOW>;
        default-state = "on";
        //max-brightness = <255>; // 手动设置最大亮度
        linux,default-trigger = "heartbeat";
        status = "okay"; // 这应该在每个 LED 节点下
    };
};

```

`leds`：这是一个节点，表示节点的名称，是和led有关的。

`compatible = "gpio-leds"`：告诉内核这是一个 GPIO 控制的 LED 设备，内核会匹配 `gpio-leds` 驱动。

`heartbeat_led`：LED 的节点名字，这个需要和内核软件所寻找的一致。

`label = "heartbeat"`：内核给这个 LED 的名字，在 `/sys/class/leds/` 下会生成一个同名目录。
(`/sys/class/leds/heartbeat`)。

`gpios` 定义它的gpio引脚。

`&gpio1`：引用的 GPIO 控制器节点（通常在设备树中定义，可参考设备树中其他设备是如何声明gpio的）。

`RK_PA0`：GPIO 号，表示具体哪一个引脚。

`GPIO_ACTIVE_LOW`：LED 点亮时引脚为低电平。

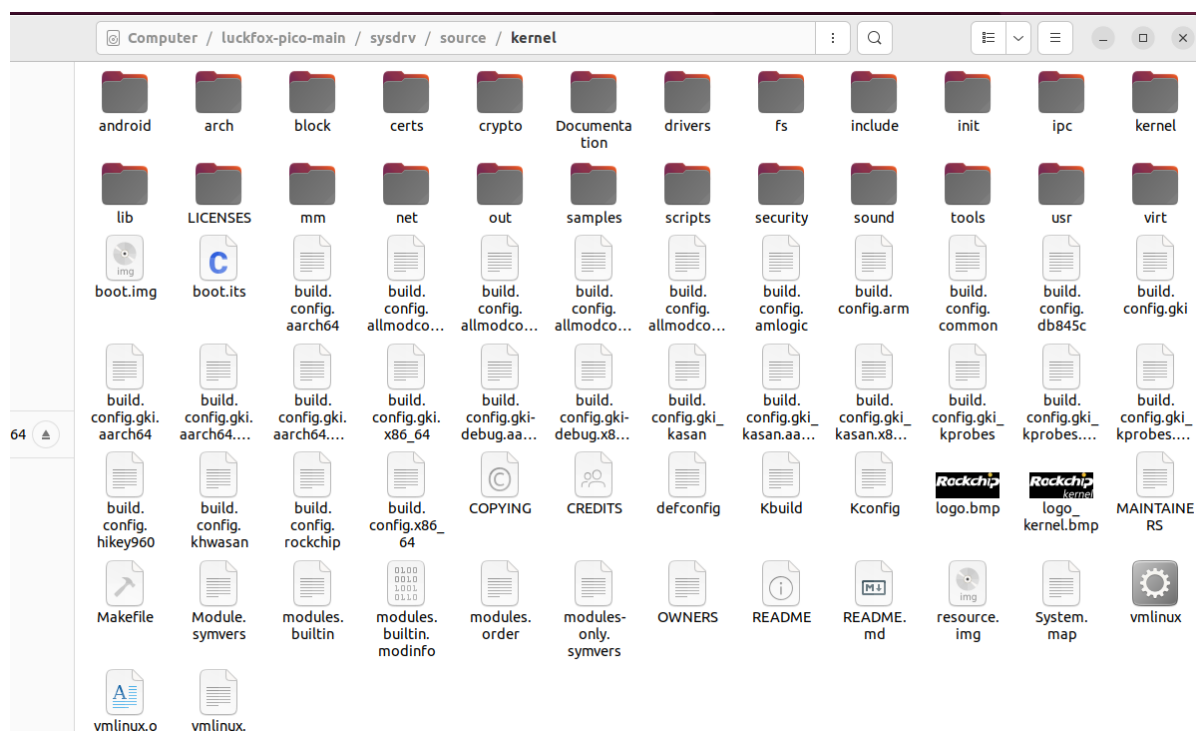
`default-state = "on";`: 系统启动后 LED 默认点亮 (on/off/off-on/blink 可选)。

`max-brightness = <255>;` 可选属性, 用于 PWM LED 的最大亮度 (up把它注释了, 表示使用默认值)。

`linux,default-trigger = "heartbeat";`: 内核触发器, 把 LED 设置为心跳指示 (会根据系统负载闪烁)。

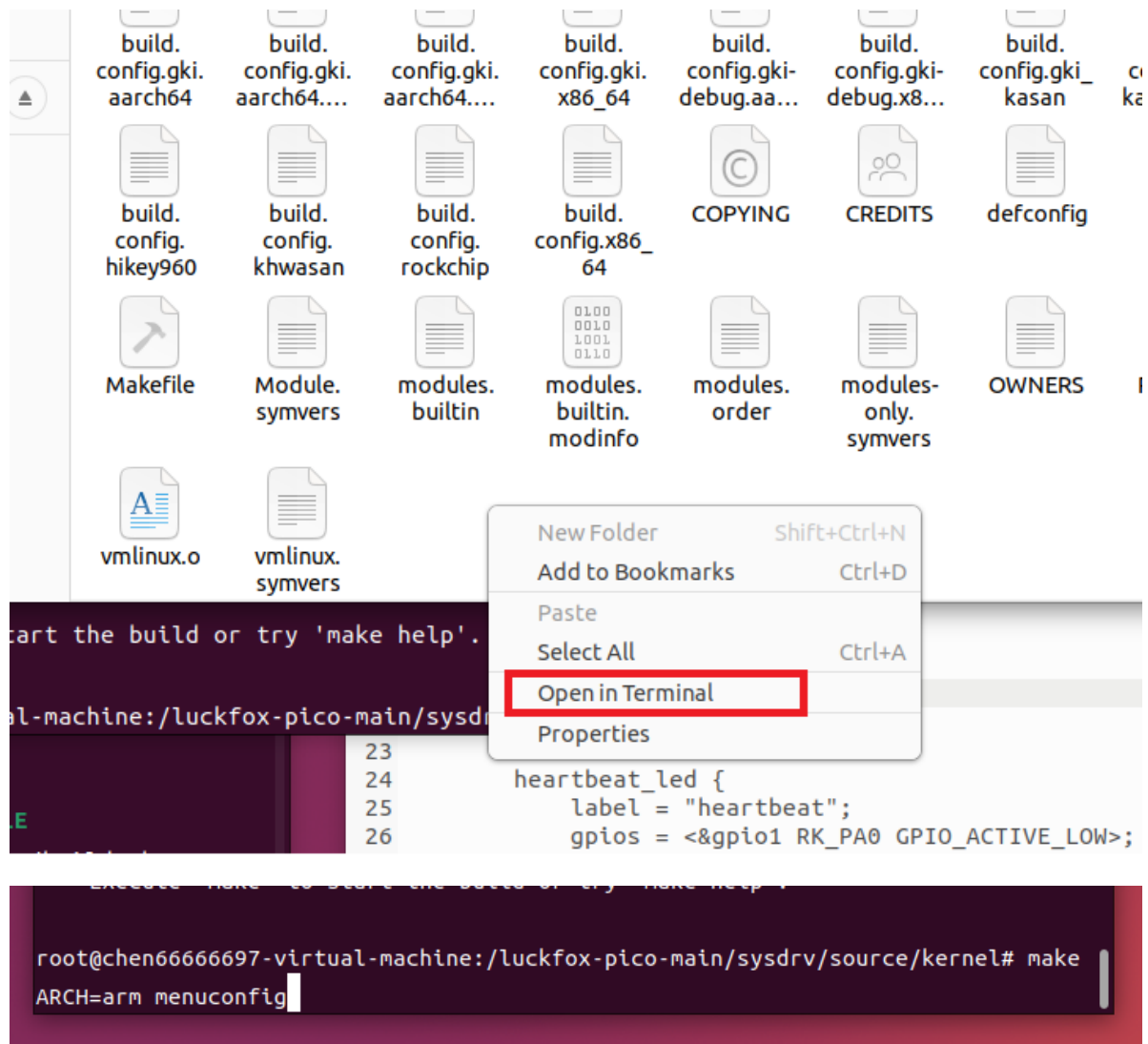
`status = "okay";`: 标记这个节点有效, 内核会启用该设备。如果设置 `disabled`, 内核会忽略, 如果我们要使用这个设备, 就必须设置为okay。

现在我们设置好了设备树, 我们该开始写驱动代码了吧? 哎, 等一下, 前面说过, Linux将软件硬件分离, 方便开发者更快捷的开发。所以不妨找找linux内核中是否已经已经有大佬已经实现了相关软件 (事实是linux内核中已经有了相关软件, 那我们直接激活这个软件代码吧)。我们直接进入linux内核界面, linux内核路径在 luckfox-pico-main/sysdrv/source/kernel, 如图所示

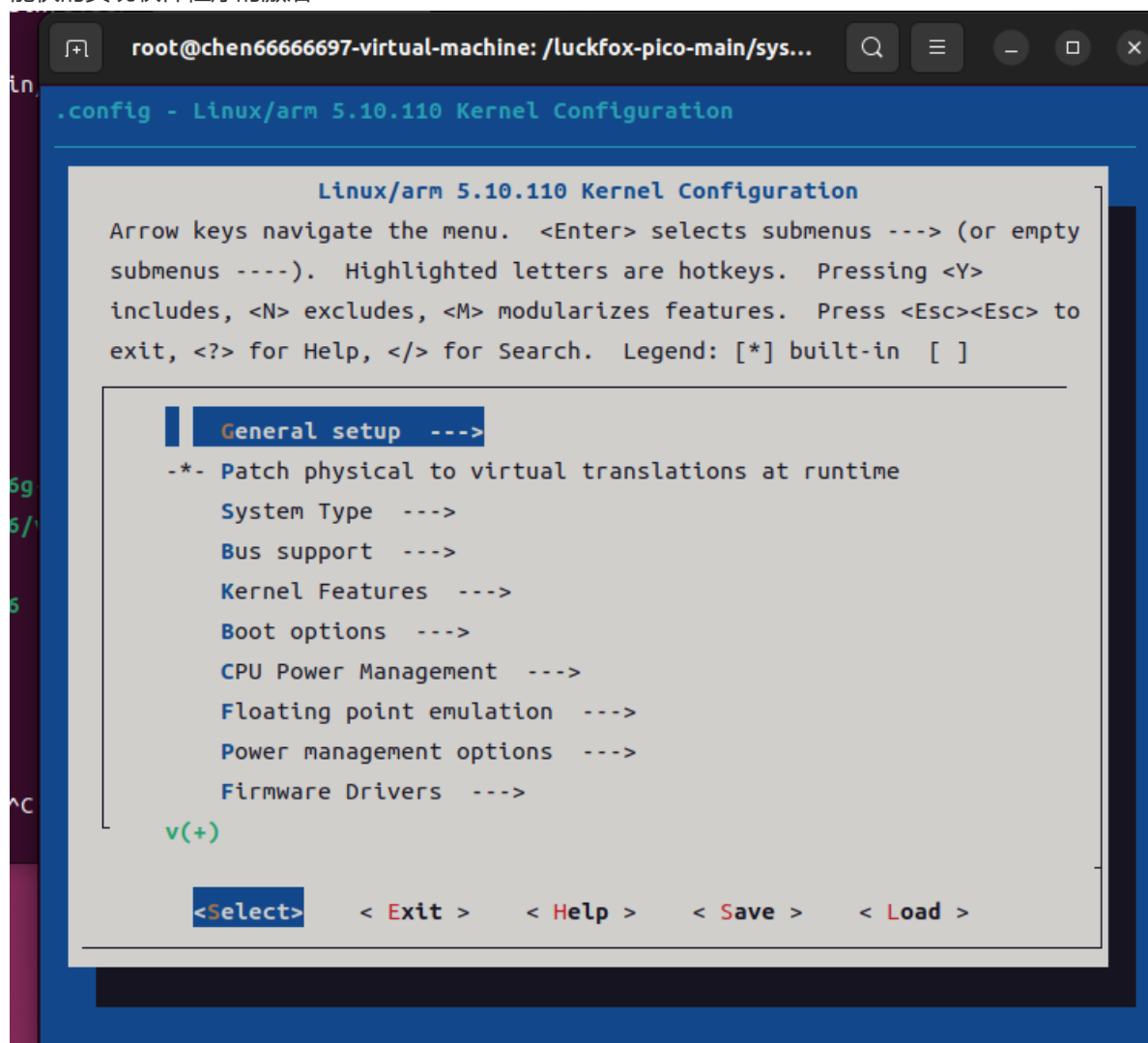


再当前路径右键点击执行open in Terminal, 执行

```
make ARCH=arm menuconfig
```



执行完后，你会看到这样的界面，这个就是内核的可视化配置界面，内核是十分庞大的，通过它，我们能快的实现软件程序的激活



使用上键和下键可以选择我们需要配置的软件模块或功能选项，使用确定键进入这个选择的模块内部，他的内部可能还有其他模块，或者就是我们需要激活的目标功能，`<*>` 表示编译进内核（内建），`<M>` 表示编译为模块（module），`<>` 表示未启用，`[ ]` 表示简单的开关选项，使用空格键（Space）在不同状态之间切换（如从未启用 `<>` 变为模块 `<M>`，或内建 `<*>`），使用 ESC 键或左右方向键返回上一级菜单，修改完成后，选择 **Save** 保存配置，或退出时系统会提示是否保存到 `.config` 文件。好了，现在我们需要找到先辈写的功能选项了，通过我说的操作方法，我们要走到如下路径，我们要把 LED Class Support 变成*号，LED Support for GPIO connected LEDs 也变成*号。

```
config - Linux/arm 5.10.110 Kernel Configuration
Device Drivers > LED Support

LED Support
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----). Highlighted letters are hotkeys. Press
<Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] built-in [ ] excluded <M> module < > module capable

--- LED Support
[*] LED Class Support
< > LED Flash Class Support
< > LED Multicolor Class Support
[ ] LED Class brightness_hw_changed attribute support
*** LED drivers ***
< > LED support for Panasonic AN30259A
< > LED support for Awinic AW2013
< > LED Support for Broadcom BCM6328
< > LED Support for Broadcom BCM6358
< > LED Support for Crane CR0014114
< > LED Support for Crane EL15203000
< > LCD Backlight driver for LM3530
< > LCD Backlight driver for LM3532
< > LED support for LM3642 Chip
< > LED support for LM3692x Chips
< > LED driver for PCA9532 dimmer
[*] LED Support for GPIO connected LEDs
< > LED Support for N.S. LP3944 (Fun Light) I2C chip
< > LED Support for TI LP3952 2 channel LED driver
v(+)
```

我们在当前目录下找到 LED Trigger support ---> ，并使用空格键将他变成* 号，并进入，并勾选如图所示的功能。

```
< > LED driver for LT3593 controllers
< > LED Support for TCA6507 I2C chip
< > LED driver for TLC59108 and TLC59116 controllers
< > LED support for LM3554 and LM3556 chips
< > LED Support for ISSI IS31FL319x I2C LED controller family
< > LED support for ISSI IS31FL32XX I2C LED controller family
*** LED driver for blink(1) USB RGB LED is under Special HID driver ***
< > LED support for the BlinkM I2C RGB LED
[ ] LED support for LEDs on system controllers
< > LED support for the Mellanox switches management control
< > Userspace LED support
< > LED support for SPI LED controller with a single byte
< > LED driver for TI LMU
*** LED Triggers ***
[*] LED Trigger support --->
```

> selects submenus ---> (or empty submenus ----). Highlighted letters are hotkeys. Pressing for Search. Legend: [*] built-in [] excluded <M> module < > module capable

--- LED Trigger support

<*> LED Timer Trigger

< > LED One-shot Trigger

[] LED MTD (NAND/NOR) Trigger

<*> LED Heartbeat Trigger

<*> LED backlight Trigger

[] LED CPU Trigger

< > LED activity Trigger

< > LED GPIO Trigger

< > LED Default ON Trigger

*** iptables trigger is under Netfilter config (LED target) ***

< > LED Transient Trigger

< > LED Camera Flash/Torch Trigger

[] LED Panic Trigger

< > LED Netdev Trigger

< > LED Pattern Trigger

< > Audio Mute LED Trigger

Timer Trigger: 基于定时器控制 LED 闪烁。

One-shot Trigger: 单次闪烁触发。

MTD Trigger: 用于 NAND/NOR 存储设备操作时触发 LED。

Heartbeat Trigger: 让 LED 根据系统负载像心跳一样闪烁。

Backlight Trigger: 控制 LED 亮度与屏幕背光相关。

CPU Trigger: LED 根据 CPU 使用率闪烁。

Activity Trigger: LED 根据磁盘或系统活动闪烁。

LED GPIO Trigger: 让一个 LED 受另一个 GPIO 信号控制。

LED Transient Trigger: 用于短暂脉冲闪灯, 类似 One-shot。

LED Default ON Trigger: 上电后默认点亮 LED。

LED GPIO Trigger: 让一个 LED 受另一个 GPIO 信号控制。

我们用左键右键选择save 确定 再确定保存内核配置文件, 然后按多次esc 回到命令行界面, 执行如图所示命令。

```
make ARCH=arm savedefconfig
```

```
cp defconfig arch/arm/configs/luckfox_rv1106_linux_defconfig
```

```
root@chen66666697-virtual-machine:/luckfox-pico-main/sysdrv/source/kernel# make ARCH=arm savedefconfig
root@chen66666697-virtual-machine:/luckfox-pico-main/sysdrv/source/kernel# cp defconfig arch/arm/configs/luckfox_rv1106_linux_defconfig
root@chen66666697-virtual-machine:/luckfox-pico-main/sysdrv/source/kernel#
```

回到SDK根目录执行./build kernel ,注意如果你是第一次执行, 你需要执行./build lunch 按如下执行

```
root@chen66666697-virtual-machine:/luckfox-pico-main# ./build.sh lunch
You're building on Linux
Lunch menu...pick the Luckfox Pico hardware version:
  选择 Luckfox Pico 硬件版本:
      [0] RV1103_Luckfox_Pico
      [1] RV1103_Luckfox_Pico_Mini_A
      [2] RV1103_Luckfox_Pico_Mini_B
      [3] RV1103_Luckfox_Pico_Plus
      [4] RV1106_Luckfox_Pico_Pro_Max
      [5] RV1106_Luckfox_Pico_Ultra
      [6] RV1106_Luckfox_Pico_Ultra_W
      [7] custom
Which would you like? [0~7][default:0]: 4
Lunch menu...pick the boot medium:
  选择启动媒介:
      [0] SD_CARD
      [1] SPI_NAND
Which would you like? [0~1][default:0]: 1
Lunch menu...pick the system version:
  选择系统版本:
      [0] Buildroot(Support Rockchip official features)
Which would you like? [0~1][default:0]: 0
[build.sh:info] Lunching for Default BoardConfig_IPC/BoardConfig-SPI_NAND-Buildroot-RV1106_Luckfox_Pico_Pro_Max-IPC.mk boards...
[build.sh:info] Running build_select_board succeeded.
root@chen66666697-virtual-machine:/luckfox-pico-main#
```

好了, 可以执行,第一次执行可能比较慢, 后面执行再次编译就快了。

```
./build kernel
```

执行完成后, 你会得到如下提示

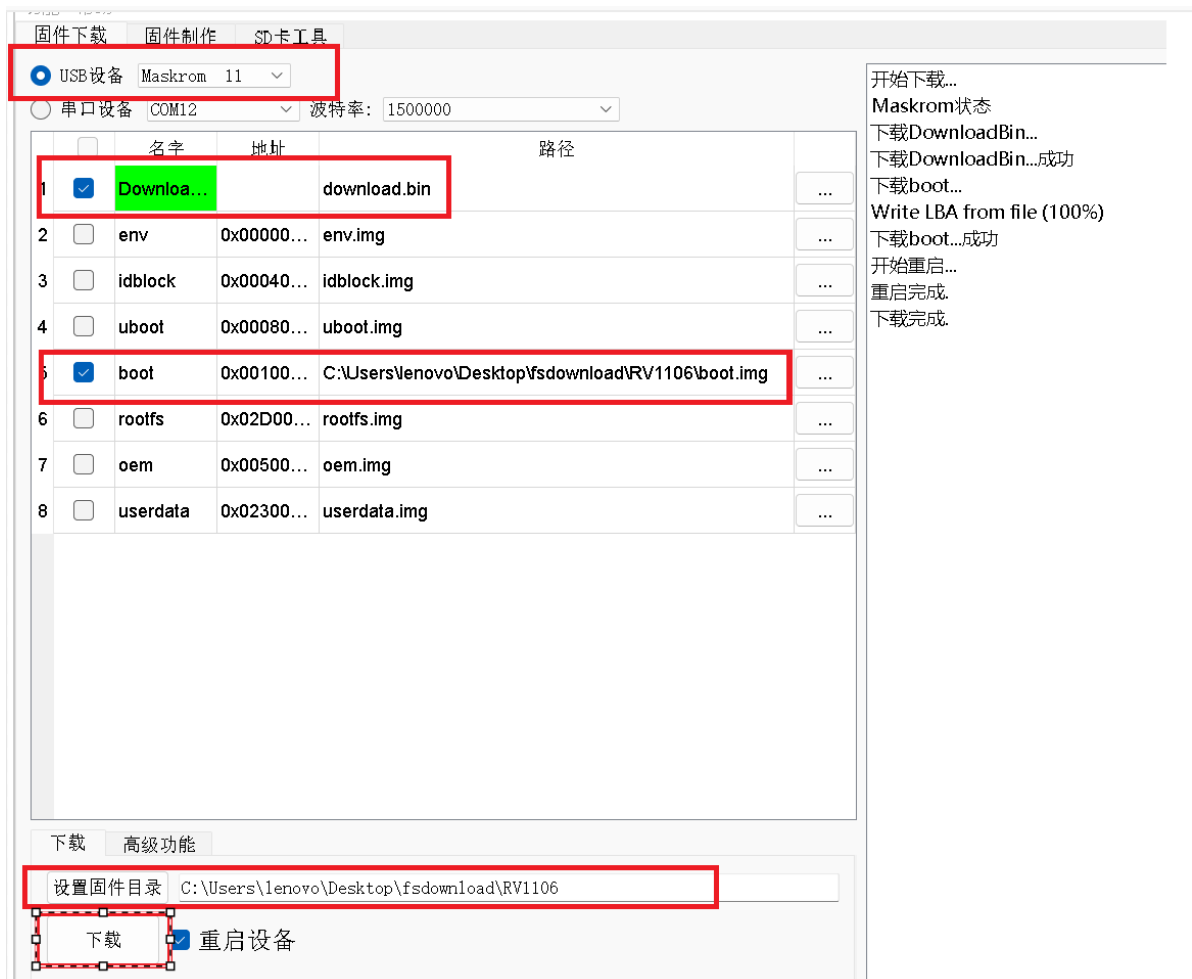
```
GZIP arch/arm/boot/Image.gz
Kernel: arch/arm/boot/zImage is ready
Kernel: arch/arm/boot/Image.gz is ready
Image: resource.img (with rv1106g-luckfox-pico-pro-max.dtb logo.bmp logo_kernel.bmp) is ready
Image: boot.img (FIT image with Linux kernel, FDT blob and resource) is ready
make[1]: Leaving directory '/luckfox-pico-main/sysdrv/source/kernel'
cp -fv /luckfox-pico-main/sysdrv/source/kernel/arch/arm/boot/dts/rv1106g-luckfox-pico-pro-max.dtb /luckfox-pico-main/sysdrv/out/bin/board_uclibc_rv1106
'/luckfox-pico-main/sysdrv/source/kernel/arch/arm/boot/dts/rv1106g-luckfox-pico-pro-max.dtb' -> '/luckfox-pico-main/sysdrv/out/bin/board_uclibc_rv1106/rv1106g-luckfox-pico-pro-max.dtb'
# Start install kernel drivers
cp -fv /luckfox-pico-main/sysdrv/source/kernel/boot.img /luckfox-pico-main/sysdrv/out/image_uclibc_rv1106/boot.img
'/luckfox-pico-main/sysdrv/source/kernel/boot.img' -> '/luckfox-pico-main/sysdrv/out/image_uclibc_rv1106/boot.img'

[INSTALL] /luckfox-pico-main/sysdrv/out/image_uclibc_rv1106/boot.img
TO /luckfox-pico-main/output/image

ntu 22.04.4 LTS amd64 /luckfox-pico-main/sysdrv/out/bin/board_uclibc_rv1106/rv1106g-luckfox-pico-pro-max.dtb /luckfox-pico-main/sysdrv/out/bin/board_uclibc_rv1106/vmlinux
TO /luckfox-pico-main/output/out/sysdrv_out/board_uclibc_rv1106

make: Leaving directory '/luckfox-pico-main/sysdrv'
[build.sh:info] Running build_kernel succeeded.
root@chen66666697-virtual-machine:/luckfox-pico-main#
```

将output/image下的文件全部 复制到windows 界面的文件夹下，设置固件地址，并勾选boot.img (up 还勾选了download.bin), 然后使我们的开发板进入烧录模式（按住boot，再按rest，就进入烧录模式），点击下载，



2.2.2 心跳灯的相关操作

现在我们已经添加相关驱动了，那么可以查看核心板的灯是否开始闪烁。

现在up来介绍一下心跳灯设备的一些操作，`/sys/class/leds/heartbeat` 目录下的文件与 LED 控制相关。

1.brightness

1. 这个文件控制 LED 的亮度，通常的值是 0（关）到 `max_brightness` 之间的一个值。你可以通过向这个文件写入一个值来调整 LED 的亮度：

```
echo 255 > /sys/class/leds/heartbeat/brightness # 最大亮度
```

2.invert

1. 这个文件控制 LED 的极性（反转）。如果设备支持反转，写入 1 将使 LED 的极性反转（高电平灭，低电平亮）。写入 0 则将 LED 恢复为默认极性（高电平亮，低电平灭）：

2.

```
echo 1 > /sys/class/leds/heartbeat/invert # 反转极性
```



```
echo 0 > /sys/class/leds/heartbeat/invert # 恢复默认极性
```

3.power

1. 这个文件通常用于控制 LED 的电源状态。写入 on 可以开启 LED，off 则关闭 LED：

2.

```
echo on > /sys/class/leds/heartbeat/powe
```

4.trigger

1. 这个文件控制 LED 的触发机制。常见的触发器有：

none：没有触发器，LED 由 brightness 控制。

heartbeat：LED 会根据系统的心跳闪烁。

activity：基于某些系统活动（如磁盘读写、网络活动等）控制 LED。

```
echo heartbeat > /sys/class/leds/heartbeat/trigger
```

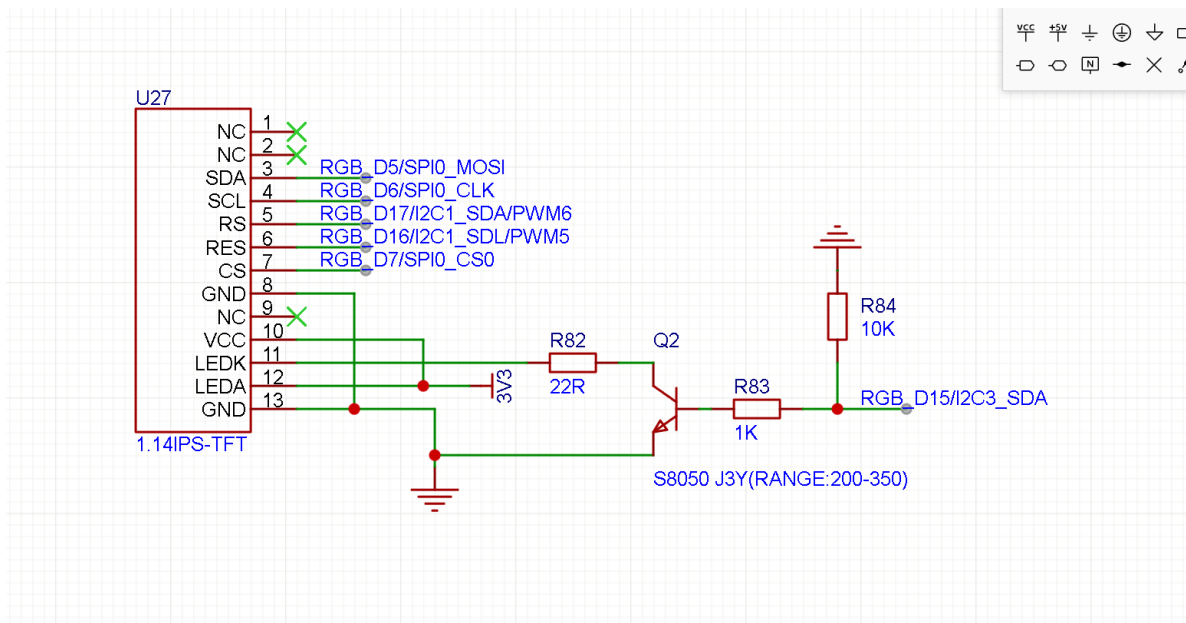
以上就是心跳灯的一些基本操作了，快去试试led灯的反应吧！

2.3 驱动1.14寸 spi小屏幕

现在我们要开始上难度了。

2.3.1 设备树及内核修改

在开始操作之前，我们先来看看我们的硬件资源,可以看出，RGB666与我们的spi小屏幕起了冲突，所以后续要使用时，只能二选一。除了spi 通信所必要的引脚，驱动屏幕我们还需要其他引脚，这告诉我们需要在资源中声明。



还是熟悉的修改设备树，但是我们这次还要修改 rv1106.dtsi。 .dtsi 文件是设备树的可复用包含文件，用于描述 SoC 或通用硬件的配置，而 .dts 文件是板卡专用的设备树源文件，它可以引用一个或多个 .dtsi 文件来继承通用硬件描述，同时添加或覆盖板卡特有的外设配置，最终通过编译生成 .dtb 供内核使用，实现硬件信息的分层管理和复用。

我们打开目标文标文件 sysdrv/source/kernel/arch/arm/boot/dts/rv1106.dtsi 修改spi0这个节点。

```
};
};
spi0: spi@ff500000 {
    compatible = "rockchip,rv1106-spi", "rockchip,rk3066-spi";
    reg = <0xff500000 0x1000>;
    interrupts = <GIC_SPI 23 IRQ_TYPE_LEVEL_HIGH>;
    status = "disabled";
    #address-cells = <1>;
    #size-cells = <0>;
    clocks = <&cru CLK_SPI0>, <&cru PCLK_SPI0>, <&cru SCLK_IN_SPI0>;
    clock-names = "spiclk", "apb_pclk", "sclk_in";
    dmas = <&dmac 1>, <&dmac 0>;
    dma-names = "tx", "rx";
    pinctrl-names = "default";
    pinctrl-0 = <&spi0m0_pins &spi0m0_cs0>;
    //&spi0m0_cs1
};
};
};
```

再修改下

```

33
34     backlight: backlight {
35         compatible = "gpio-backlight";
36         gpios = <&gpio2 RK_PA7 GPIO_ACTIVE_HIGH>;
37         default-on; // 默认开启背光
38         status = "okay";
39     };
40
41     /*LCD_BL*/
42     gpio2pa7:gpio2pa7 {
43         compatible = "regulator-fixed";
44         pinctrl-names = "default";
45         pinctrl-0 = <&gpio2_pa7>;
46         regulator-name = "gpio2_pa7";
47         regulator-always-on;
48     };
49
50     /*LCD_DC*/
51     gpio2pb1:gpio2pb1 {
52         compatible = "regulator-fixed";
53         pinctrl-names = "default";
54         pinctrl-0 = <&gpio2_pb1>;
55         regulator-name = "gpio2_pb1";
56         regulator-always-on;
57     };
58
59     /*LCD_RES*/
60     gpio2pb0:gpio2pb0 {
61         compatible = "regulator-fixed";
62         pinctrl-names = "default";
63         pinctrl-0 = <&gpio2_pb0>;
64         regulator-name = "gpio2_pb0";
65         regulator-always-on;
66     };
67

```

以及开启spi0，和他的资源描述

```

86
87 &spi0 {
88
89     status = "okay";
90
91     pinctrl-names = "default";
92
93     pinctrl-0 = <&spi0m0_pins>;           // 包含clk,mi,mo
94     cs-gpios=<&spi0m0_cs0>;
95     #address-cells = <1>;
96     #size-cells = <0>;
97
98
99
100     st7789v@0{
101
102         status = "okay";
103
104         compatible = "sitronix,st7789v";
105
106         reg = <0>;
107         rotate = <270>;
108         spi-max-frequency = <32000000>;
109         rgb;
110         spi-cpol;
111         spi-cpha;
112         width = <135>; // 修改为实际的宽度
113         height = <240>; // 修改为实际的高度
114
115         fps = <30>;
116
117         buswidth = <8>;
118
119         // 包含clk,mi,mo
120         // 修改为实际的宽度
121         // 修改为实际的高度
122         cs-gpios = <&gpio1 RK_PC0 GPIO_ACTIVE_LOW>;
123         dc-gpios = <&gpio2 RK_PB1 GPIO_ACTIVE_HIGH>;
124         reset-gpios = <&gpio2 RK_PB0 GPIO_ACTIVE_LOW>;
125         backlight = <&backlight>; // 引用背光节点
126     };
127 };
128
34
35 &pinctrl {
36
37
38     /*dc*/
39     gpio2-pb1 {
40         gpio2_pb1:gpio2-pb1 {
41             rockchip,pins = <2 RK_PB1 RK_FUNC_GPIO &pcfg_pull_none>;
42         };
43     };
44
45     /*LCD_RES*/
46     gpio2-pb0 {
47         gpio2_pb0:gpio2-pb0 {
48             rockchip,pins = <2 RK_PB0 RK_FUNC_GPIO &pcfg_pull_none>;
49         };
50     };
51
52     /*LCD_BACKLIGHT*/
53     gpio2-pa7 {
54         gpio2_pa7:gpio2-pa7 {
55             rockchip,pins = <2 RK_PA7 RK_FUNC_GPIO &pcfg_pull_none>;
56         };
57     };
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999

```

```
backlight: backlight {
compatible = "gpio-backlight";
gpios = <&gpio2 RK_PA7 GPIO_ACTIVE_HIGH>;
default-on; // 默认开启背光
status = "okay";
};
```

```
/*dc*/
gpio2-pb1 {
gpio2_pb1:gpio2-pb1 {
rockchip,pins = <2 RK_PB1 RK_FUNC_GPIO &pcfg_pull_none>;
};
};

/*LCD_RES*/
gpio2-pb0 {
gpio2_pb0:gpio2-pb0 {
rockchip,pins = <2 RK_PB0 RK_FUNC_GPIO &pcfg_pull_none>;
};
};

/*LCD_BACKLIGHT*/
gpio2-pa7 {
gpio2_pa7:gpio2-pa7 {
rockchip,pins = <2 RK_PA7 RK_FUNC_GPIO &pcfg_pull_none>;
};
};
```

```
/*LCD_BL*/
gpio2pa7:gpio2pa7 {
compatible = "regulator-fixed";
pinctrl-names = "default";
pinctrl-0 = <&gpio2_pa7>;
regulator-name = "gpio2_pa7";
regulator-always-on;
};

/*LCD_DC*/
gpio2pb1:gpio2pb1 {
compatible = "regulator-fixed";
pinctrl-names = "default";
pinctrl-0 = <&gpio2_pb1>;
regulator-name = "gpio2_pb1";
regulator-always-on;
};

/*LCD_RES*/
gpio2pb0:gpio2pb0 {
compatible = "regulator-fixed";
pinctrl-names = "default";
pinctrl-0 = <&gpio2_pb0>;
regulator-name = "gpio2_pb0";
regulator-always-on;
};
```

```

&spi0 {

    status = "okay";

    pinctrl-names = "default";

    pinctrl-0 = <&spi0m0_pins>;    // 包含clk,mi,mo
    cs-gpios=<&spi0m0_cs0>;
    #address-cells = <1>;
    #size-cells = <0>;

    st7789v@0{

        status = "okay";

        compatible = "sitronix,st7789v";

        reg = <0>;
        rotate = <270>;
        spi-max-frequency = <32000000>;
        rgb;
        spi-cpol;
        spi-cpha;
        width = <135>; // 修改为实际的宽度
        height = <240>; // 修改为实际的高度

        fps = <30>;

        buswidth = <8>;

        debug = <0x0>;

        cs-gpios = <&gpio1 RK_PC0 GPIO_ACTIVE_LOW>;
        dc-gpios = <&gpio2 RK_PB1 GPIO_ACTIVE_HIGH>;
        reset-gpios = <&gpio2 RK_PB0 GPIO_ACTIVE_LOW>;
        backlight = <&backlight>; // 引用背光节点
    };
};

```

`pinctrl-names / pinctrl-0`：配置 SPI 引脚复用和电气属性 (clk、MISO、MOSI)。

`cs-gpios`：如果使用 GPIO 作为片选信号。

`#address-cells = <1> / #size-cells = <0>`：SPI 总线下子设备地址单元配置，1 表示子设备用一个单元表示地址，0 表示没有大小单元 (typical for SPI)。

```
compatible = "sitronix,st7789v"
```

指定驱动匹配的硬件型号，内核根据这个加载对应驱动。

```
reg = <0>
```

SPI 总线上的设备地址（0 对应第一个片选）。

```
rotate = <270>
```

屏幕旋转角度（驱动处理显示方向）

```
spi-max-frequency = <32000000>
```

SPI 最大通信频率 32MHz。

```
rgb
```

表示使用 RGB 模式（具体驱动解析）。

```
spi-cpol / spi-cpha
```

SPI 时钟极性和相位（模式设置）。

```
width / height
```

屏幕物理分辨率（单位像素）。

```
fps
```

帧率（可选，用于驱动刷新控制）。

```
buswidth
```

数据总线宽度，通常 SPI 用 8bit。

```
debug
```

驱动调试开关。

```
cs-gpios
```

片选引脚（GPIO 控制）。

```
dc-gpios
```

数据/命令控制引脚。

```
reset-gpios
```

屏幕复位引脚。

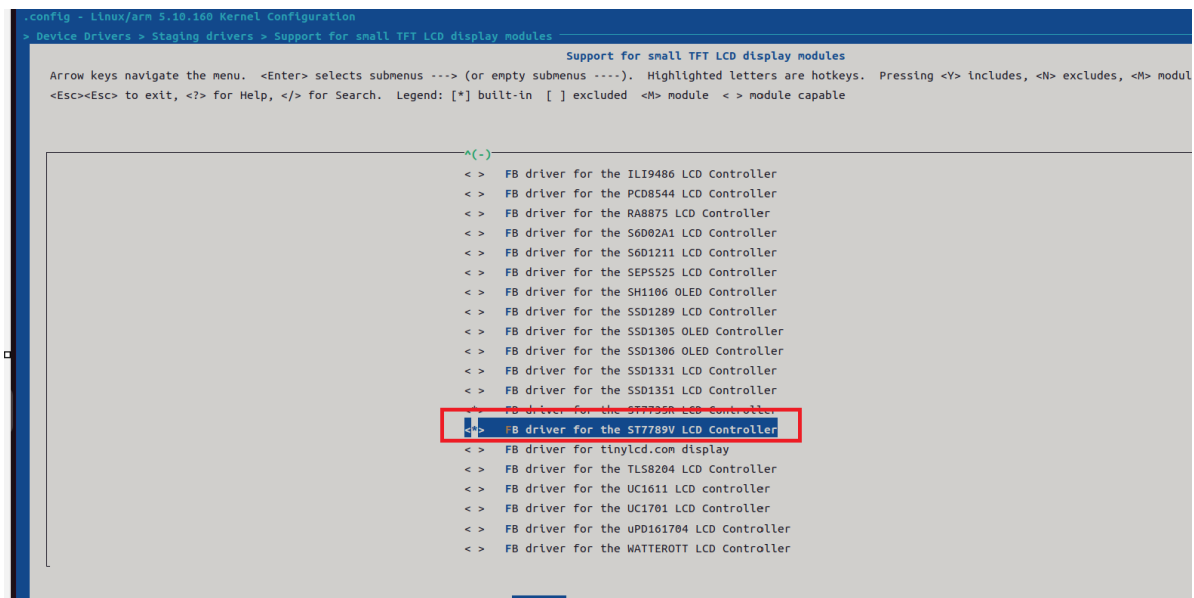
backlight

引用背光控制节点（通常是 PWM 或 GPIO 控制背光）。

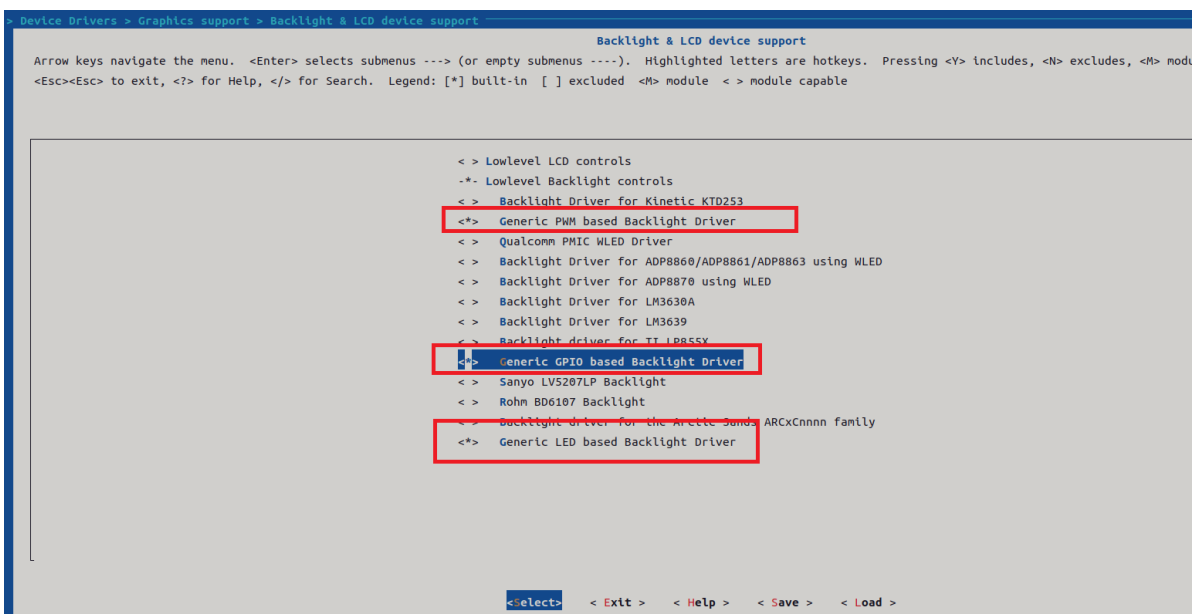
好了，接下来我们开始编写st7789v的驱动代码，事实上，linux内核中已经有了相关代码，但是，代码是有问题的，up这里提供正确代码我们需要修改 /kernel/drivers/staging/fbtf/ 下的 fb_st7789V.c fbtf-core.c这两个文件，参考2.3.3，当然，我也提供源代码，可以直接替换。

修改内核配置，执行，然后将对应功能激活

```
make ARCH=arm menuconfig
```



背光驱动也要打开

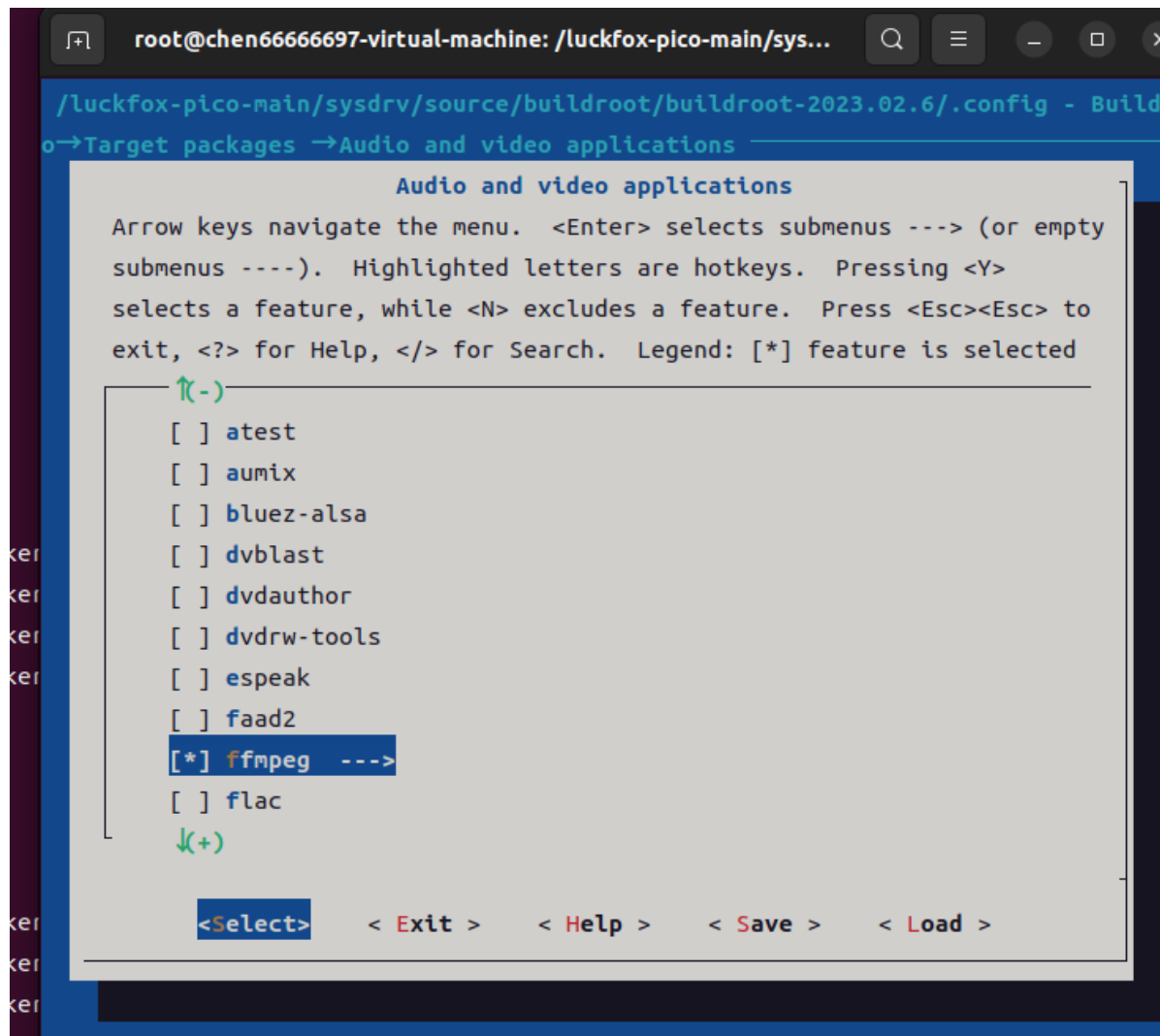


保存退出，记得，把配置cp一下。

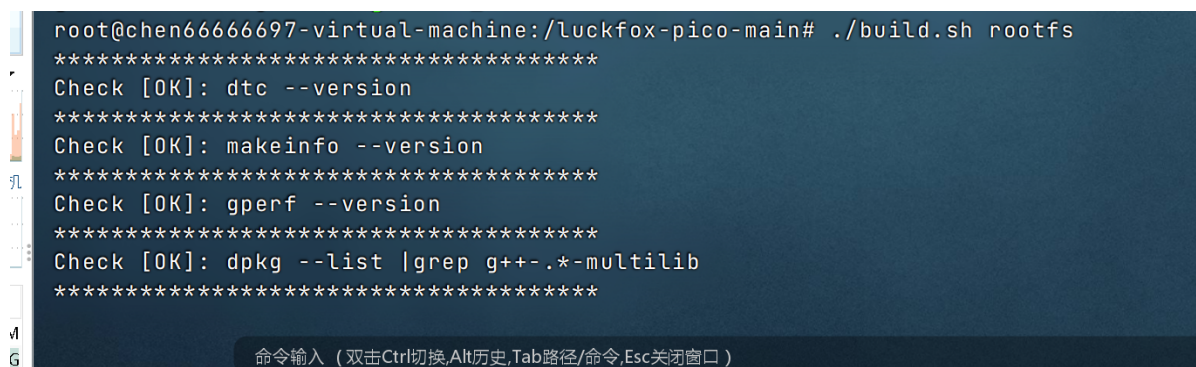
```
make ARCH=arm savedefconfig
cp defconfig arch/arm/configs/luckfox_rv1106_linux_defconfig
cd ~/SDK目录
./build kernel
```

接下来我们需要操作一下文件系统，添加一个显示图片或视频的命令，进入路径 luckfox-pico-main/sysdrv/source/buildroot/buildroot-2023.02.6。

在SDK文件夹下执行./build.sh buildrootconfig，找到功能项，将ffmpeg勾选，然后保存退出。



切换到SDK根目录，执行./build.sh rootfs 再打包 ./build.sh firmware 注意烧录的时候要勾选文件系统镜像。



2.3.2 小屏幕的相关操作

清屏操作: `dd if=/dev/zero of=/dev/fb0 bs=64800 count=1`

随机数操作: `dd if=/dev/urandom of=/dev/fb0 bs=64800 count=1`

修改屏幕亮度:

`echo 0 >/sys/class/backlight/backlight/brightness`

修改旋转角度: 设备树角度只能设置为90 270

`echo 360 >> /sys/class/graphics/fb0/rotate`

查看显示设备: `fbset`

```
mode "240x135-0"
    # D: 0.000 MHz, H: 0.000 kHz, V: 0.000 Hz
    geometry 240 135 240 135 16
    timings 0 0 0 0 0 0 0
    accel false
    rgba 5/11,6/5,5/0,0/0
endmode
```

好了我们显示张图片或者播放下视频吧, (演示的图片和视频, 可查看2.3 的资料) 那么我们的视频和图片如何上传到开发板呢?

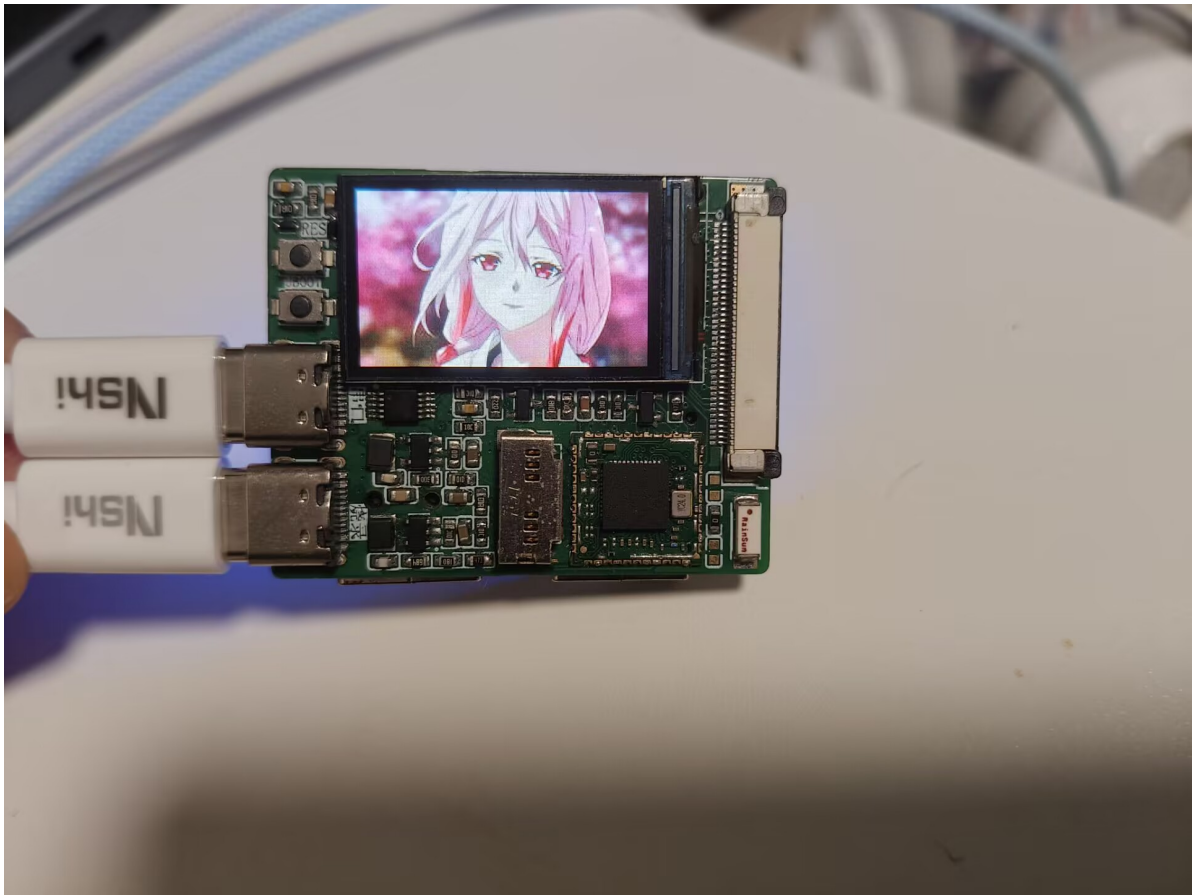
我们可以使用`adb push`命令将文件传到开发板上, 如图

```
PS E:\RV1106\驱动及应用开发资料\Materials\2.3> adb push .\OIP.jpg ./root
* daemon not running; starting now at tcp:5037
* daemon started successfully
.\OIP.jpg: 1 file pushed. 1.2 MB/s (11292 bytes in 0.009s)
PS E:\RV1106\驱动及应用开发资料\Materials\2.3> adb push .\output_bad_apple.mp4 ./root
.\output_bad_apple.mp4: 1 file pushed. 4.7 MB/s (7027994 bytes in 1.418s)
PS E:\RV1106\驱动及应用开发资料\Materials\2.3>
```

显示一张图片, 执行以下命令

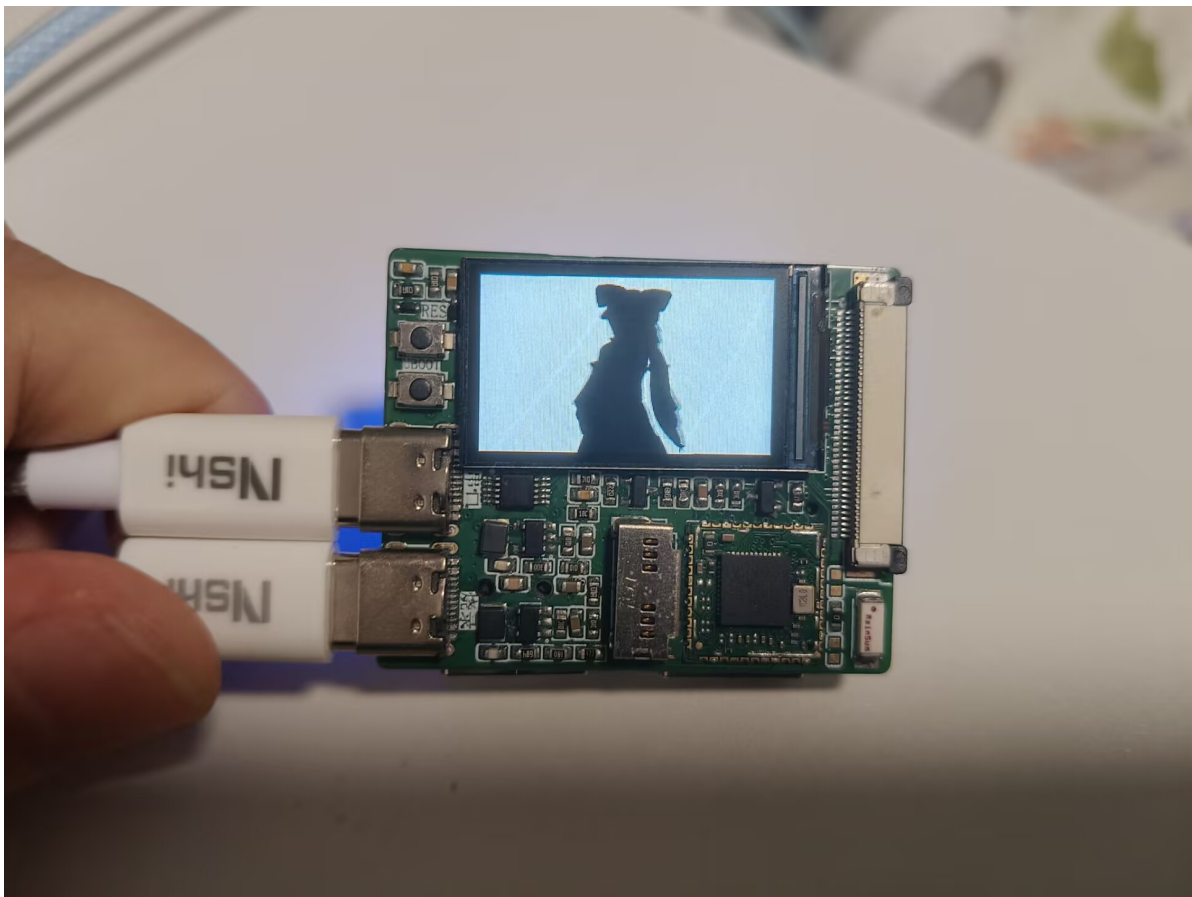
```
ffmpeg -i OIP.jpg -vf "scale=240:135" -pix_fmt rgb565 output.rgb

dd if=output.rgb of=/dev/fb0 bs=64800
```



接下来播放视频

```
ffmpeg -re -i output_240_135.mp4 -vf "scale=240:135" -pix_fmt rgb565 -r 30 -f  
fbdev /dev/fb0  
#或者带音频的输出  
ffmpeg -re -i output_bad_apple.mp4 -vf "scale=240:135" -pix_fmt rgb565 -r 30 -f  
fbdev /dev/fb0 -f wav - | aplay -D hw:0,0
```



2.3.3 驱动相关代码

fb_st7789v.c

```
// SPDX-License-Identifier: GPL-2.0+
/*
 * FB driver for the ST7789V LCD Controller
 *
 * Copyright (C) 2015 Dennis Menschel
 */

#include <linux/bitops.h>
#include <linux/delay.h>
#include <linux/init.h>
#include <linux/kernel.h>
#include <linux/module.h>
#include <video/mipi_display.h>

#include "fbtft.h"

#define DRVNAME "fb_st7789v"

#define DEFAULT_GAMMA \
    "70 2C 2E 15 10 09 48 33 53 0B 19 18 20 25\n" \
    "70 2C 2E 15 10 09 48 33 53 0B 19 18 20 25"

#define HSD20_IPS_GAMMA \
    "D0 05 0A 09 08 05 2E 44 45 0F 17 16 2B 33\n" \
    "D0 05 0A 09 08 05 2E 43 45 0F 16 16 2B 33"
```

```

#define HSD20_IPS 1

/**
 * enum st7789v_command - ST7789V display controller commands
 *
 * @PORCTRL: porch setting
 * @GCTRL: gate control
 * @VCOMS: VCOM setting
 * @VDVVRHEN: VDV and VRH command enable
 * @VRHS: VRH set
 * @VDVS: VDV set
 * @VCMOFSET: VCOM offset set
 * @PWCTRL1: power control 1
 * @PVGAMCTRL: positive voltage gamma control
 * @NVGAMCTRL: negative voltage gamma control
 *
 * The command names are the same as those found in the datasheet to ease
 * looking up their semantics and usage.
 *
 * Note that the ST7789V display controller offers quite a few more commands
 * which have been omitted from this list as they are not used at the moment.
 * Furthermore, commands that are compliant with the MIPI DCS have been left
 * out as well to avoid duplicate entries.
 */
enum st7789v_command {
    PORCTRL = 0xB2,
    GCTRL = 0xB7,
    VCOMS = 0xBB,
    VDVVRHEN = 0xC2,
    VRHS = 0xC3,
    VDVS = 0xC4,
    VCMOFSET = 0xC5,
    PWCTRL1 = 0xD0,
    PVGAMCTRL = 0xE0,
    NVGAMCTRL = 0xE1,
};

#define MADCTL_BGR BIT(3) /* bitmask for RGB/BGR order */
#define MADCTL_MV BIT(5) /* bitmask for page/column order */
#define MADCTL_MX BIT(6) /* bitmask for column address order */
#define MADCTL_MY BIT(7) /* bitmask for page address order */

/**
 * init_display() - initialize the display controller
 *
 * @par: FBTFT parameter object
 *
 * Most of the commands in this init function set their parameters to the
 * same default values which are already in place after the display has been
 * powered up. (The main exception to this rule is the pixel format which
 * would default to 18 instead of 16 bit per pixel.)
 * Nonetheless, this sequence can be used as a template for concrete
 * displays which usually need some adjustments.
 */

```

```

* Return: 0 on success, < 0 if error occurred.
*/
static int init_display(struct fbtft_par *par)
{
    par->fbtftops.reset(par);
    mdelay(50);
    write_reg(par,0x11); //Sleep exit
    mdelay(12);
    write_reg(par,0x11);
    mdelay(10);
    write_reg(par,0x3A,0x05); //65k mode
    write_reg(par,0xc5,0x1a);
    write_reg(par,0x36,0x70); // 屏幕显示方向设置
//-----ST7789V Frame rate setting-----//
    write_reg(par,0xb2,0x05,0x05,0x00,0x33,0x33);
    write_reg(par,0xb7,0x35);
//-----ST7789V Power setting-----//
    write_reg(par,0xbb,0x3f);
    write_reg(par,0xc0,0x2c);
    write_reg(par,0xc2,0x01);
    write_reg(par,0xc3,0x0f);
    write_reg(par,0xc4,0x20);
    write_reg(par,0xc6,0x11);
    write_reg(par,0xd0,0xa4,0xa1);
    write_reg(par,0xe8,0x03);
    write_reg(par,0xe9,0x09,0x09,0x08);

    write_reg(par,0xe0,0xd0,0x05,0x09,0x09,0x08,0x14,0x28,0x33,0x3f,0x07,0x13,0x14,0
x28,0x30);

    write_reg(par,0xe1,0xd0,0x05,0x09,0x09,0x08,0x03,0x24,0x32,0x32,0x3b,0x14,0x13,0
x28,0x2f);
    write_reg(par,0x21);
    write_reg(par,0x11);
    mdelay(120); //Delay 120ms
    write_reg(par,0x29);
    mdelay(200);
    return 0;
}

/**
 * set_var() - apply LCD properties like rotation and BGR mode
 *
 * @par: FBTF parameter object
 *
 * Return: 0 on success, < 0 if error occurred.
 */
static int set_var(struct fbtft_par *par)
{
    u8 madctl_par = 0;

    if (par->bgr)
        madctl_par |= MADCTL_BGR;
    switch (par->info->var.rotate) {
    case 0:

```

```

        break;
    case 90:
        madctl_par |= (MADCTL_MV | MADCTL_MY);
        break;
    case 180:
        madctl_par |= (MADCTL_MX | MADCTL_MY);
        break;
    case 270:
        madctl_par |= (MADCTL_MV | MADCTL_MX);
        break;
    default:
        return -EINVAL;
    }
    write_reg(par, MIPI_DCS_SET_ADDRESS_MODE, madctl_par);
    return 0;
}

/**
 * set_gamma() - set gamma curves
 *
 * @par: FBTF parameter object
 * @curves: gamma curves
 *
 * Before the gamma curves are applied, they are preprocessed with a bitmask
 * to ensure syntactically correct input for the display controller.
 * This implies that the curves input parameter might be changed by this
 * function and that illegal gamma values are auto-corrected and not
 * reported as errors.
 *
 * Return: 0 on success, < 0 if error occurred.
 */
static int set_gamma(struct fbtft_par *par, u32 *curves)
{
    int i;
    int j;
    int c; /* curve index offset */

    /**
     * Bitmasks for gamma curve command parameters.
     * The masks are the same for both positive and negative voltage
     * gamma curves.
     */
    static const u8 gamma_par_mask[] = {
        0xFF, /* v63[3:0], v0[3:0] */
        0x3F, /* v1[5:0] */
        0x3F, /* v2[5:0] */
        0x1F, /* v4[4:0] */
        0x1F, /* v6[4:0] */
        0x3F, /* j0[1:0], v13[3:0] */
        0x7F, /* v20[6:0] */
        0x77, /* v36[2:0], v27[2:0] */
        0x7F, /* v43[6:0] */
        0x3F, /* j1[1:0], v50[3:0] */
        0x1F, /* v57[4:0] */
        0x1F, /* v59[4:0] */
    };

```

```

        0x3F, /* V61[5:0] */
        0x3F, /* V62[5:0] */
    };

    for (i = 0; i < par->gamma.num_curves; i++) {
        c = i * par->gamma.num_values;
        for (j = 0; j < par->gamma.num_values; j++)
            curves[c + j] &= gamma_par_mask[j];
        write_reg(par, PVGAMCTRL + i,
            curves[c + 0], curves[c + 1], curves[c + 2],
            curves[c + 3], curves[c + 4], curves[c + 5],
            curves[c + 6], curves[c + 7], curves[c + 8],
            curves[c + 9], curves[c + 10], curves[c + 11],
            curves[c + 12], curves[c + 13]);
    }
    return 0;
}

/**
 * blank() - blank the display
 *
 * @par: FBTFT parameter object
 * @on: whether to enable or disable blanking the display
 *
 * Return: 0 on success, < 0 if error occurred.
 */
static int blank(struct fbtft_par *par, bool on)
{
    if (on)
        write_reg(par, MIPI_DCS_SET_DISPLAY_OFF);
    else
        write_reg(par, MIPI_DCS_SET_DISPLAY_ON);
    return 0;
}

static struct fbtft_display display = {
    .regwidth = 8,
    .width = 135,
    .height = 240,
    .gamma_num = 2,
    .gamma_len = 14,
    .gamma = HSD20_IPS_GAMMA,
    .fbtftops = {
        .init_display = init_display,
        .set_var = set_var,
        .set_gamma = set_gamma,
        .blank = blank,
    },
};

FBTFT_REGISTER_DRIVER(DRVNAME, "sitronix,st7789v", &display);

MODULE_ALIAS("spi:" DRVNAME);
MODULE_ALIAS("platform:" DRVNAME);
MODULE_ALIAS("spi:st7789v");

```

```

MODULE_ALIAS("platform:st7789v");

MODULE_DESCRIPTION("FB driver for the ST7789V LCD Controller");
MODULE_AUTHOR("Dennis Menschel");
MODULE_LICENSE("GPL");

```

fbtft-core.c

```

// SPDX-License-Identifier: GPL-2.0+
/*
 * Copyright (C) 2013 Noralf Trønnes
 *
 * This driver is inspired by:
 *   st7735fb.c, Copyright (C) 2011, Matt Porter
 *   broadsheetfb.c, Copyright (C) 2008, Jaya Kumar
 */

#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/errno.h>
#include <linux/string.h>
#include <linux/mm.h>
#include <linux/vmalloc.h>
#include <linux/slab.h>
#include <linux/init.h>
#include <linux/fb.h>
#include <linux/gpio/consumer.h>
#include <linux/spi/spi.h>
#include <linux/delay.h>
#include <linux/uaccess.h>
#include <linux/backlight.h>
#include <linux/platform_device.h>
#include <linux/property.h>
#include <linux/spinlock.h>

#include <video/mipi_display.h>

#include "fbtft.h"
#include "internal.h"

#include <linux/gpio.h>
#include <linux/of_gpio.h>

static unsigned long debug;
module_param(debug, ulong, 0000);
MODULE_PARM_DESC(debug, "override device debug level");

int fbtft_write_buf_dc(struct fbtft_par *par, void *buf, size_t len, int dc)
{
    int ret;

    gpiod_set_value(par->gpio.dc, dc);

    ret = par->fbtftops.write(par, buf, len);

```

```

        if (ret < 0)
            dev_err(par->info->device,
                    "write() failed and returned %d\n", ret);
        return ret;
    }
EXPORT_SYMBOL(fbtft_write_buf_dc);

void fbtft_dbg_hex(const struct device *dev, int groupsize,
                  void *buf, size_t len, const char *fmt, ...)
{
    va_list args;
    static char textbuf[512];
    char *text = textbuf;
    size_t text_len;

    va_start(args, fmt);
    text_len = vsnprintf(text, sizeof(textbuf), fmt, args);
    va_end(args);

    hex_dump_to_buffer(buf, len, 32, groupsize, text + text_len,
                      512 - text_len, false);

    if (len > 32)
        dev_info(dev, "%s ...\n", text);
    else
        dev_info(dev, "%s\n", text);
}
EXPORT_SYMBOL(fbtft_dbg_hex);

static int fbtft_request_one_gpio(struct fbtft_par *par,
                                  const char *name, int index,
                                  struct gpio_desc **gpiop)
{
    struct device *dev = par->info->device;
    struct device_node *node = dev->of_node;
    int gpio, flags, ret = 0;
    enum of_gpio_flags of_flags;
    if (of_find_property(node, name, NULL)) {
        gpio = of_get_named_gpio_flags(node, name, index, &of_flags);
        if (gpio == -ENOENT)
            return 0;
        if (gpio == -EPROBE_DEFER)
            return gpio;
        if (gpio < 0) {
            dev_err(dev,
                    "failed to get '%s' from DT\n", name);
            return gpio;
        }
        //active low translates to initially low
        flags = (of_flags & OF_GPIO_ACTIVE_LOW) ? GPIOF_OUT_INIT_LOW :
        GPIOF_OUT_INIT_HIGH;
        ret = devm_gpio_request_one(dev, gpio, flags,
                                    dev->driver->name);
        if (ret) {
            dev_err(dev,

```

```

        "gpio_request_one('%s'=%d) failed with %d\n",
        name, gpio, ret);
    return ret;
}

*giop = gpio_to_desc(gpio);
fbtft_par_dbg(DEBUG_REQUEST_GPIOS, par, "%s: '%s' = GPIO%d\n",
    __func__, name, gpio);
}

return ret;
}

static int fbtft_request_gpios(struct fbtft_par *par)
{
    int i;
    int ret;

    ret = fbtft_request_one_gpio(par, "reset-gpios", 0, &par->gpio.reset);
    if (ret)
        return ret;
    ret = fbtft_request_one_gpio(par, "dc-gpios", 0, &par->gpio.dc);
    if (ret)
        return ret;
    ret = fbtft_request_one_gpio(par, "rd-gpios", 0, &par->gpio.rd);
    if (ret)
        return ret;
    ret = fbtft_request_one_gpio(par, "wr-gpios", 0, &par->gpio.wr);
    if (ret)
        return ret;
    ret = fbtft_request_one_gpio(par, "cs-gpios", 0, &par->gpio.cs);
    if (ret)
        return ret;
    ret = fbtft_request_one_gpio(par, "latch-gpios", 0, &par->gpio.latch);
    if (ret)
        return ret;
    for (i = 0; i < 16; i++) {
        ret = fbtft_request_one_gpio(par, "db-gpios", i,
            &par->gpio.db[i]);
        if (ret)
            return ret;
        ret = fbtft_request_one_gpio(par, "led-gpios", i,
            &par->gpio.led[i]);
        if (ret)
            return ret;
        ret = fbtft_request_one_gpio(par, "aux-gpios", i,
            &par->gpio.aux[i]);
        if (ret)
            return ret;
    }

    return 0;
}

```

```

static int fbtft_backlight_update_status(struct backlight_device *bd)
{
    struct fbtft_par *par = bl_get_data(bd);
    bool polarity = par->polarity;

    fbtft_par_dbg(DEBUG_BACKLIGHT, par,
        "%s: polarity=%d, power=%d, fb_blank=%d\n",
        __func__, polarity, bd->props.power, bd->props.fb_blank);

    if ((bd->props.power == FB_BLANK_UNBLANK) &&
        (bd->props.fb_blank == FB_BLANK_UNBLANK))
        gpiod_set_value(par->gpio.led[0], polarity);
    else
        gpiod_set_value(par->gpio.led[0], !polarity);

    return 0;
}

static int fbtft_backlight_get_brightness(struct backlight_device *bd)
{
    return bd->props.brightness;
}

void fbtft_unregister_backlight(struct fbtft_par *par)
{
    if (par->info->bl_dev) {
        par->info->bl_dev->props.power = FB_BLANK_POWERDOWN;
        backlight_update_status(par->info->bl_dev);
        backlight_device_unregister(par->info->bl_dev);
        par->info->bl_dev = NULL;
    }
}

EXPORT_SYMBOL(fbtft_unregister_backlight);

static const struct backlight_ops fbtft_bl_ops = {
    .get_brightness = fbtft_backlight_get_brightness,
    .update_status = fbtft_backlight_update_status,
};

void fbtft_register_backlight(struct fbtft_par *par)
{
    struct backlight_device *bd;
    struct backlight_properties bl_props = { 0, };

    if (!par->gpio.led[0]) {
        fbtft_par_dbg(DEBUG_BACKLIGHT, par,
            "%s(): led pin not set, exiting.\n", __func__);
        return;
    }

    bl_props.type = BACKLIGHT_RAW;
    /* Assume backlight is off, get polarity from current state of pin */
    bl_props.power = FB_BLANK_POWERDOWN;
    if (!gpiod_get_value(par->gpio.led[0]))
        par->polarity = true;
}

```

```

    bd = backlight_device_register(dev_driver_string(par->info->device),
        par->info->device, par,
        &fbtft_bl_ops, &bl_props);
    if (IS_ERR(bd)) {
        dev_err(par->info->device,
            "cannot register backlight device (%ld)\n",
            PTR_ERR(bd));
        return;
    }
    par->info->bl_dev = bd;

    if (!par->fbtftops.unregister_backlight)
        par->fbtftops.unregister_backlight = fbtft_unregister_backlight;
}
EXPORT_SYMBOL(fbtft_register_backlight);

static void fbtft_set_addr_win(struct fbtft_par *par, int xs, int ys, int xe,
    int ye)
{
    write_reg(par, MIPI_DCS_SET_COLUMN_ADDRESS, (xs+40) >> 8, xs+40, ((xe+40) >>
8) & 0xFF, (xe+40) & 0xFF);

    write_reg(par, MIPI_DCS_SET_PAGE_ADDRESS, ((ys+53) >> 8) & 0xFF, (ys+53) &
0xFF, ((ye+53) >> 8) & 0xFF, (ye+53) & 0xFF);

    write_reg(par, MIPI_DCS_WRITE_MEMORY_START);
}

static void fbtft_reset(struct fbtft_par *par)
{
    if (!par->gpio.reset)
        return;
    fbtft_par_dbg(DEBUG_RESET, par, "%s()\n", __func__);
    gpiod_set_value_cansleep(par->gpio.reset, 1);
    msleep(10);
    gpiod_set_value_cansleep(par->gpio.reset, 0);
    msleep(200);
    gpiod_set_value_cansleep(par->gpio.reset, 1);
    msleep(10);
}

static void fbtft_update_display(struct fbtft_par *par, unsigned int start_line,
    unsigned int end_line)
{
    size_t offset, len;
    ktime_t ts_start, ts_end;
    long fps, throughput;
    bool timeit = false;
    int ret = 0;

    if (unlikely(par->debug & (DEBUG_TIME_FIRST_UPDATE |
        DEBUG_TIME_EACH_UPDATE))) {
        if ((par->debug & DEBUG_TIME_EACH_UPDATE) ||
            ((par->debug & DEBUG_TIME_FIRST_UPDATE) &&

```

```

        !par->first_update_done)) {
            ts_start = ktime_get();
            timeit = true;
        }
    }

    /* Sanity checks */
    if (start_line > end_line) {
        dev_warn(par->info->device,
            "%s: start_line=%u is larger than end_line=%u. Shouldn't happen, will
do full display update\n",
            __func__, start_line, end_line);
        start_line = 0;
        end_line = par->info->var.yres - 1;
    }
    if (start_line > par->info->var.yres - 1 ||
        end_line > par->info->var.yres - 1) {
        dev_warn(par->info->device,
            "%s: start_line=%u or end_line=%u is larger than max=%d. Shouldn't
happen, will do full display update\n",
            __func__, start_line,
            end_line, par->info->var.yres - 1);
        start_line = 0;
        end_line = par->info->var.yres - 1;
    }

    fbtft_par_dbg(DEBUG_UPDATE_DISPLAY, par, "%s(start_line=%u, end_line=%u)\n",
        __func__, start_line, end_line);

    if (par->fbtftops.set_addr_win)
        par->fbtftops.set_addr_win(par, 0, start_line,
            par->info->var.xres - 1, end_line);

    offset = start_line * par->info->fix.line_length;
    len = (end_line - start_line + 1) * par->info->fix.line_length;
    ret = par->fbtftops.write_vmem(par, offset, len);
    if (ret < 0)
        dev_err(par->info->device,
            "%s: write_vmem failed to update display buffer\n",
            __func__);

    if (unlikely(timeit)) {
        ts_end = ktime_get();
        if (!ktime_to_ns(par->update_time))
            par->update_time = ts_start;

        fps = ktime_us_delta(ts_start, par->update_time);
        par->update_time = ts_start;
        fps = fps ? 1000000 / fps : 0;

        throughput = ktime_us_delta(ts_end, ts_start);
        throughput = throughput ? (len * 1000) / throughput : 0;
        throughput = throughput * 1000 / 1024;

        dev_info(par->info->device,

```

```

        "Display update: %ld kB/s, fps=%ld\n",
        throughput, fps);
    par->first_update_done = true;
}
}

static void fbtft_mkdirty(struct fb_info *info, int y, int height)
{
    struct fbtft_par *par = info->par;
    struct fb_deferred_io *fbdefio = info->fbdefio;

    /* special case, needed ? */
    if (y == -1) {
        y = 0;
        height = info->var.yres;
    }

    /* Mark display lines/area as dirty */
    spin_lock(&par->dirty_lock);
    if (y < par->dirty_lines_start)
        par->dirty_lines_start = y;
    if (y + height - 1 > par->dirty_lines_end)
        par->dirty_lines_end = y + height - 1;
    spin_unlock(&par->dirty_lock);

    /* Schedule deferred_io to update display (no-op if already on queue)*/
    schedule_delayed_work(&info->deferred_work, fbdefio->delay);
}

static void fbtft_deferred_io(struct fb_info *info, struct list_head *pagelist)
{
    struct fbtft_par *par = info->par;
    unsigned int dirty_lines_start, dirty_lines_end;
    struct page *page;
    unsigned long index;
    unsigned int y_low = 0, y_high = 0;
    int count = 0;

    spin_lock(&par->dirty_lock);
    dirty_lines_start = par->dirty_lines_start;
    dirty_lines_end = par->dirty_lines_end;
    /* set display line markers as clean */
    par->dirty_lines_start = par->info->var.yres - 1;
    par->dirty_lines_end = 0;
    spin_unlock(&par->dirty_lock);

    /* Mark display lines as dirty */
    list_for_each_entry(page, pagelist, lru) {
        count++;
        index = page->index << PAGE_SHIFT;
        y_low = index / info->fix.line_length;
        y_high = (index + PAGE_SIZE - 1) / info->fix.line_length;
        dev_dbg(info->device,
            "page->index=%lu y_low=%d y_high=%d\n",
            page->index, y_low, y_high);
    }
}

```

```

        if (y_high > info->var.yres - 1)
            y_high = info->var.yres - 1;
        if (y_low < dirty_lines_start)
            dirty_lines_start = y_low;
        if (y_high > dirty_lines_end)
            dirty_lines_end = y_high;
    }

    par->fbtftops.update_display(info->par,
                                dirty_lines_start, dirty_lines_end);
}

static void fbtft_fb_fillrect(struct fb_info *info,
                              const struct fb_fillrect *rect)
{
    struct fbtft_par *par = info->par;

    dev_dbg(info->dev,
            "%s: dx=%d, dy=%d, width=%d, height=%d\n",
            __func__, rect->dx, rect->dy, rect->width, rect->height);
    sys_fillrect(info, rect);

    par->fbtftops.mkdirty(info, rect->dy, rect->height);
}

static void fbtft_fb_copyarea(struct fb_info *info,
                              const struct fb_copyarea *area)
{
    struct fbtft_par *par = info->par;

    dev_dbg(info->dev,
            "%s: dx=%d, dy=%d, width=%d, height=%d\n",
            __func__, area->dx, area->dy, area->width, area->height);
    sys_copyarea(info, area);

    par->fbtftops.mkdirty(info, area->dy, area->height);
}

static void fbtft_fb_imageblit(struct fb_info *info,
                               const struct fb_image *image)
{
    struct fbtft_par *par = info->par;

    dev_dbg(info->dev,
            "%s: dx=%d, dy=%d, width=%d, height=%d\n",
            __func__, image->dx, image->dy, image->width, image->height);
    sys_imageblit(info, image);

    par->fbtftops.mkdirty(info, image->dy, image->height);
}

static ssize_t fbtft_fb_write(struct fb_info *info, const char __user *buf,
                              size_t count, loff_t *ppos)
{
    struct fbtft_par *par = info->par;

```

```

ssize_t res;

dev_dbg(info->dev,
        "%s: count=%zd, ppos=%llu\n", __func__, count, *ppos);
res = fb_sys_write(info, buf, count, ppos);

/* TODO: only mark changed area update all for now */
par->fbtftops.mkdirty(info, -1, 0);

return res;
}

/* from pxaafb.c */
static unsigned int chan_to_field(unsigned int chan, struct fb_bitfield *bf)
{
    chan &= 0xffff;
    chan >>= 16 - bf->length;
    return chan << bf->offset;
}

static int fbtft_fb_setcolreg(unsigned int regno, unsigned int red,
    unsigned int green, unsigned int blue,
    unsigned int transp, struct fb_info *info)
{
    unsigned int val;
    int ret = 1;

    dev_dbg(info->dev,
        "%s(regno=%u, red=0x%X, green=0x%X, blue=0x%X, trans=0x%X)\n",
        __func__, regno, red, green, blue, transp);

    switch (info->fix.visual) {
    case FB_VISUAL_TRUECOLOR:
        if (regno < 16) {
            u32 *pal = info->pseudo_palette;

            val = chan_to_field(red, &info->var.red);
            val |= chan_to_field(green, &info->var.green);
            val |= chan_to_field(blue, &info->var.blue);

            pal[regno] = val;
            ret = 0;
        }
        break;
    }
    return ret;
}

static int fbtft_fb_blank(int blank, struct fb_info *info)
{
    struct fbtft_par *par = info->par;
    int ret = -EINVAL;

    dev_dbg(info->dev, "%s(blank=%d)\n",
        __func__, blank);

```

```

    if (!par->fbtftops.blank)
        return ret;

    switch (blank) {
    case FB_BLANK_POWERDOWN:
    case FB_BLANK_VSYNC_SUSPEND:
    case FB_BLANK_HSYNC_SUSPEND:
    case FB_BLANK_NORMAL:
        ret = par->fbtftops.blank(par, true);
        break;
    case FB_BLANK_UNBLANK:
        ret = par->fbtftops.blank(par, false);
        break;
    }
    return ret;
}

static void fbtf_merge_fbtfops(struct fbtf_ops *dst, struct fbtf_ops *src)
{
    if (src->write)
        dst->write = src->write;
    if (src->read)
        dst->read = src->read;
    if (src->write_vmem)
        dst->write_vmem = src->write_vmem;
    if (src->write_register)
        dst->write_register = src->write_register;
    if (src->set_addr_win)
        dst->set_addr_win = src->set_addr_win;
    if (src->reset)
        dst->reset = src->reset;
    if (src->mkdirty)
        dst->mkdirty = src->mkdirty;
    if (src->update_display)
        dst->update_display = src->update_display;
    if (src->init_display)
        dst->init_display = src->init_display;
    if (src->blank)
        dst->blank = src->blank;
    if (src->request_gpios_match)
        dst->request_gpios_match = src->request_gpios_match;
    if (src->request_gpios)
        dst->request_gpios = src->request_gpios;
    if (src->verify_gpios)
        dst->verify_gpios = src->verify_gpios;
    if (src->register_backlight)
        dst->register_backlight = src->register_backlight;
    if (src->unregister_backlight)
        dst->unregister_backlight = src->unregister_backlight;
    if (src->set_var)
        dst->set_var = src->set_var;
    if (src->set_gamma)
        dst->set_gamma = src->set_gamma;
}

```

```

/**
 * fbtft_framebuffer_alloc - creates a new frame buffer info structure
 *
 * @display: pointer to structure describing the display
 * @dev: pointer to the device for this fb, this can be NULL
 * @pdata: platform data for the display in use
 *
 * Creates a new frame buffer info structure.
 *
 * Also creates and populates the following structures:
 *   info->fbops
 *   info->fbdefio
 *   info->pseudo_palette
 *   par->fbtftops
 *   par->txbuf
 *
 * Returns the new structure, or NULL if an error occurred.
 */
struct fb_info *fbtft_framebuffer_alloc(struct fbtft_display *display,
    struct device *dev,
    struct fbtft_platform_data *pdata)
{
    struct fb_info *info;
    struct fbtft_par *par;
    struct fb_ops *fbops = NULL;
    struct fb_deferred_io *fbdefio = NULL;
    u8 *vmem = NULL;
    void *txbuf = NULL;
    void *buf = NULL;
    unsigned int width;
    unsigned int height;
    int txbuflen = display->txbuflen;
    unsigned int bpp = display->bpp;
    unsigned int fps = display->fps;
    int vmem_size;
    const s16 *init_sequence = display->init_sequence;
    char *gamma = display->gamma;
    u32 *gamma_curves = NULL;

    /* sanity check */
    if (display->gamma_num * display->gamma_len >
        FBTFT_GAMMA_MAX_VALUES_TOTAL) {
        dev_err(dev, "FBTFT_GAMMA_MAX_VALUES_TOTAL=%d is exceeded\n",
            FBTFT_GAMMA_MAX_VALUES_TOTAL);
        return NULL;
    }

    /* defaults */
    if (!fps)
        fps = 20;
    if (!bpp)
        bpp = 16;

```

```

if (!pdata) {
    dev_err(dev, "platform data is missing\n");
    return NULL;
}

/* override driver values? */
if (pdata->fps)
    fps = pdata->fps;
if (pdata->txbuflen)
    txbuflen = pdata->txbuflen;
if (pdata->display.init_sequence)
    init_sequence = pdata->display.init_sequence;
if (pdata->gamma)
    gamma = pdata->gamma;
if (pdata->display.debug)
    display->debug = pdata->display.debug;
if (pdata->display.backlight)
    display->backlight = pdata->display.backlight;
if (pdata->display.width)
    display->width = pdata->display.width;
if (pdata->display.height)
    display->height = pdata->display.height;
if (pdata->display.buswidth)
    display->buswidth = pdata->display.buswidth;
if (pdata->display.regwidth)
    display->regwidth = pdata->display.regwidth;

display->debug |= debug;
fbtft_expand_debug_value(&display->debug);

switch (pdata->rotate) {
case 90:
case 270:
    width = display->height;
    height = display->width;
    break;
default:
    width = display->width;
    height = display->height;
}

vmem_size = display->width * display->height * bpp / 8;
vmem = vzalloc(vmem_size);
if (!vmem)
    goto alloc_fail;

fbops = devm_kzalloc(dev, sizeof(struct fb_ops), GFP_KERNEL);
if (!fbops)
    goto alloc_fail;

fbdefio = devm_kzalloc(dev, sizeof(struct fb_deferred_io), GFP_KERNEL);
if (!fbdefio)
    goto alloc_fail;

buf = devm_kzalloc(dev, 128, GFP_KERNEL);

```

```

if (!buf)
    goto alloc_fail;

if (display->gamma_num && display->gamma_len) {
    gamma_curves = devm_kcalloc(dev,
        display->gamma_num *
        display->gamma_len,
        sizeof(gamma_curves[0]),
        GFP_KERNEL);
    if (!gamma_curves)
        goto alloc_fail;
}

info = framebuffer_alloc(sizeof(struct fbtft_par), dev);
if (!info)
    goto alloc_fail;

info->screen_buffer = vmem;
info->fbops = fbops;
info->fbdefio = fbdefio;

fbops->owner      = dev->driver->owner;
fbops->fb_read    = fb_sys_read;
fbops->fb_write   = fbtft_fb_write;
fbops->fb_fillrect = fbtft_fb_fillrect;
fbops->fb_copyarea = fbtft_fb_copyarea;
fbops->fb_imageblit = fbtft_fb_imageblit;
fbops->fb_setcolreg = fbtft_fb_setcolreg;
fbops->fb_blank   = fbtft_fb_blank;

fbdefio->delay = HZ / fps;
fbdefio->deferred_io = fbtft_deferred_io;
fb_deferred_io_init(info);

snprintf(info->fix.id, sizeof(info->fix.id), "%S", dev->driver->name);
info->fix.type = FB_TYPE_PACKED_PIXELS;
info->fix.visual = FB_VISUAL_TRUECOLOR;
info->fix.xpanstep = 0;
info->fix.ypanstep = 0;
info->fix.ywrapstep = 0;
info->fix.line_length = width * bpp / 8;
info->fix.accel = FB_ACCEL_NONE;
info->fix.smem_len = vmem_size;

info->var.rotate = pdata->rotate;
info->var.xres = width;
info->var.yres = height;
info->var.xres_virtual = info->var.xres;
info->var.yres_virtual = info->var.yres;
info->var.bits_per_pixel = bpp;
info->var.nonstd = 1;

/* RGB565 */
info->var.red.offset = 11;
info->var.red.length = 5;

```

```

info->var.green.offset = 5;
info->var.green.length = 6;
info->var.blue.offset = 0;
info->var.blue.length = 5;
info->var.transp.offset = 0;
info->var.transp.length = 0;

info->flags = FBINFO_FLAG_DEFAULT | FBINFO_VIRTFB;

par = info->par;
par->info = info;
par->pdata = pdata;
par->debug = display->debug;
par->buf = buf;
spin_lock_init(&par->dirty_lock);
par->bgr = pdata->bgr;
par->startbyte = pdata->startbyte;
par->init_sequence = init_sequence;
par->gamma.curves = gamma_curves;
par->gamma.num_curves = display->gamma_num;
par->gamma.num_values = display->gamma_len;
mutex_init(&par->gamma.lock);
info->pseudo_palette = par->pseudo_palette;

if (par->gamma.curves && gamma) {
    if (fbtft_gamma_parse_str(par, par->gamma.curves, gamma,
        strlen(gamma)))
        goto release_framebuf;
}

/* Transmit buffer */
if (txbuflen == -1)
    txbuflen = vmem_size + 2; /* add in case startbyte is used */
if (txbuflen >= vmem_size + 2)
    txbuflen = 0;

#ifdef __LITTLE_ENDIAN
    if ((!txbuflen) && (bpp > 8))
        txbuflen = PAGE_SIZE; /* need buffer for byteswapping */
#endif

if (txbuflen > 0) {
    txbuf = devm_kzalloc(par->info->device, txbuflen, GFP_KERNEL);
    if (!txbuf)
        goto release_framebuf;
    par->txbuf.buf = txbuf;
    par->txbuf.len = txbuflen;
}

/* default fbtft operations */
par->fbtftops.write = fbtft_write_spi;
par->fbtftops.read = fbtft_read_spi;
par->fbtftops.write_vmem = fbtft_write_vmem16_bus8;
par->fbtftops.write_register = fbtft_write_reg8_bus8;
par->fbtftops.set_addr_win = fbtft_set_addr_win;

```

```

    par->fbtftops.reset = fbtft_reset;
    par->fbtftops.mkdirty = fbtft_mkdirty;
    par->fbtftops.update_display = fbtft_update_display;
    if (display->backlight)
        par->fbtftops.register_backlight = fbtft_register_backlight;

    /* use driver provided functions */
    fbtft_merge_fbtftops(&par->fbtftops, &display->fbtftops);

    return info;

release_framebuf:
framebuffer_release(info);

alloc_fail:
vfree(vmem);

    return NULL;
}
EXPORT_SYMBOL(fbtft_framebuffer_alloc);

/**
 * fbtft_framebuffer_release - frees up all memory used by the framebuffer
 *
 * @info: frame buffer info structure
 */
void fbtft_framebuffer_release(struct fb_info *info)
{
    fb_deferred_io_cleanup(info);
    vfree(info->screen_buffer);
    framebuffer_release(info);
}
EXPORT_SYMBOL(fbtft_framebuffer_release);

/**
 * fbtft_register_framebuffer - registers a tft frame buffer device
 * @fb_info: frame buffer info structure
 *
 * Sets SPI driverdata if needed
 * Requests needed gpios.
 * Initializes display
 * Updates display.
 * Registers a frame buffer device @fb_info.
 *
 * Returns negative errno on error, or zero for success.
 */
int fbtft_register_framebuffer(struct fb_info *fb_info)
{
    int ret;
    char text1[50] = "";
    char text2[50] = "";
    struct fbtft_par *par = fb_info->par;
    struct spi_device *spi = par->spi;

```

```

/* sanity checks */
if (!par->fbtftops.init_display) {
    dev_err(fb_info->device, "missing fbtftops.init_display()\n");
    return -EINVAL;
}

if (spi)
    spi_set_drvdata(spi, fb_info);
if (par->pdev)
    platform_set_drvdata(par->pdev, fb_info);

ret = par->fbtftops.request_gpios(par);
if (ret < 0)
    goto reg_fail;

if (par->fbtftops.verify_gpios) {
    ret = par->fbtftops.verify_gpios(par);
    if (ret < 0)
        goto reg_fail;
}

ret = par->fbtftops.init_display(par);
if (ret < 0)
    goto reg_fail;
if (par->fbtftops.set_var) {
    ret = par->fbtftops.set_var(par);
    if (ret < 0)
        goto reg_fail;
}

/* update the entire display */
par->fbtftops.update_display(par, 0, par->info->var.yres - 1);

if (par->fbtftops.set_gamma && par->gamma.curves) {
    ret = par->fbtftops.set_gamma(par, par->gamma.curves);
    if (ret)
        goto reg_fail;
}

if (par->fbtftops.register_backlight)
    par->fbtftops.register_backlight(par);

ret = register_framebuffer(fb_info);
if (ret < 0)
    goto reg_fail;

fbtft_sysfs_init(par);

if (par->txbuf.buf && par->txbuf.len >= 1024)
    sprintf(text1, "%zu KiB buffer memory", par->txbuf.len >> 10);
if (spi)
    sprintf(text2, "spi%d.%d at %d MHz", spi->master->bus_num,
            spi->chip_select, spi->max_speed_hz / 1000000);
dev_info(fb_info->dev,

```

```

        "%s frame buffer, %dx%d, %d KiB video memory%s, fps=%lu%s\n",
        fb_info->fix.id, fb_info->var.xres, fb_info->var.yres,
        fb_info->fix.smem_len >> 10, text1,
        HZ / fb_info->fbdefio->delay, text2);

    /* Turn on backlight if available */
    if (fb_info->bl_dev) {
        fb_info->bl_dev->props.power = FB_BLANK_UNBLANK;
        fb_info->bl_dev->ops->update_status(fb_info->bl_dev);
    }

    return 0;

reg_fail:
    if (par->fbtftops.unregister_backlight)
        par->fbtftops.unregister_backlight(par);

    return ret;
}
EXPORT_SYMBOL(fbtft_register_framebuffer);

/**
 * fbtft_unregister_framebuffer - releases a tft frame buffer device
 * @fb_info: frame buffer info structure
 *
 * Frees SPI driverdata if needed
 * Frees gpios.
 * Unregisters frame buffer device.
 *
 */
int fbtft_unregister_framebuffer(struct fb_info *fb_info)
{
    struct fbtft_par *par = fb_info->par;

    if (par->fbtftops.unregister_backlight)
        par->fbtftops.unregister_backlight(par);
    fbtft_sysfs_exit(par);
    unregister_framebuffer(fb_info);

    return 0;
}
EXPORT_SYMBOL(fbtft_unregister_framebuffer);

/**
 * fbtft_init_display_from_property() - Device Tree init_display() function
 * @par: Driver data
 *
 * Return: 0 if successful, negative if error
 */
static int fbtft_init_display_from_property(struct fbtft_par *par)
{
    struct device *dev = par->info->device;
    int buf[64], count, index, i, j, ret;
    u32 *values;
    u32 val;

```

```

count = device_property_count_u32(dev, "init");
if (count < 0)
    return count;
if (count == 0)
    return -EINVAL;

values = kmalloc_array(count + 1, sizeof(*values), GFP_KERNEL);
if (!values)
    return -ENOMEM;

ret = device_property_read_u32_array(dev, "init", values, count);
if (ret)
    goto out_free;

par->fbtftops.reset(par);

index = -1;
val = values[++index];

while (index < count) {
    if (val & FBTFT_OF_INIT_CMD) {
        val &= 0xFFFF;
        i = 0;
        while ((index < count) && !(val & 0xFFFF0000)) {
            if (i > 63) {
                dev_err(dev,
                    "%s: Maximum register values exceeded\n",
                    __func__);
                ret = -EINVAL;
                goto out_free;
            }
            buf[i++] = val;
            val = values[++index];
        }
        /* make debug message */
        fbtft_par_dbg(DEBUG_INIT_DISPLAY, par,
            "init: write_register:\n");
        for (j = 0; j < i; j++)
            fbtft_par_dbg(DEBUG_INIT_DISPLAY, par,
                "buf[%d] = %02X\n", j, buf[j]);

        par->fbtftops.write_register(par, i,
            buf[0], buf[1], buf[2], buf[3],
            buf[4], buf[5], buf[6], buf[7],
            buf[8], buf[9], buf[10], buf[11],
            buf[12], buf[13], buf[14], buf[15],
            buf[16], buf[17], buf[18], buf[19],
            buf[20], buf[21], buf[22], buf[23],
            buf[24], buf[25], buf[26], buf[27],
            buf[28], buf[29], buf[30], buf[31],
            buf[32], buf[33], buf[34], buf[35],
            buf[36], buf[37], buf[38], buf[39],
            buf[40], buf[41], buf[42], buf[43],
            buf[44], buf[45], buf[46], buf[47],

```

```

        buf[48], buf[49], buf[50], buf[51],
        buf[52], buf[53], buf[54], buf[55],
        buf[56], buf[57], buf[58], buf[59],
        buf[60], buf[61], buf[62], buf[63]);
    } else if (val & FBTFT_OF_INIT_DELAY) {
        fbtft_par_dbg(DEBUG_INIT_DISPLAY, par,
            "init: msleep(%u)\n", val & 0xFFFF);
        msleep(val & 0xFFFF);
        val = values[++index];
    } else {
        dev_err(dev, "illegal init value 0x%X\n", val);
        ret = -EINVAL;
        goto out_free;
    }
}

out_free:
kfree(values);
return ret;
}

/**
 * fbtft_init_display() - Generic init_display() function
 * @par: Driver data
 *
 * Uses par->init_sequence to do the initialization
 *
 * Return: 0 if successful, negative if error
 */
int fbtft_init_display(struct fbtft_par *par)
{
    int buf[64];
    char msg[128];
    char str[16];
    int i = 0;
    int j;

    /* sanity check */
    if (!par->init_sequence) {
        dev_err(par->info->device,
            "error: init_sequence is not set\n");
        return -EINVAL;
    }

    /* make sure stop marker exists */
    for (i = 0; i < FBTFT_MAX_INIT_SEQUENCE; i++)
        if (par->init_sequence[i] == -3)
            break;
    if (i == FBTFT_MAX_INIT_SEQUENCE) {
        dev_err(par->info->device,
            "missing stop marker at end of init sequence\n");
        return -EINVAL;
    }

    par->fbtftops.reset(par);

```

```

i = 0;
while (i < FBTFT_MAX_INIT_SEQUENCE) {
    if (par->init_sequence[i] == -3) {
        /* done */
        return 0;
    }
    if (par->init_sequence[i] >= 0) {
        dev_err(par->info->device,
            "missing delimiter at position %d\n", i);
        return -EINVAL;
    }
    if (par->init_sequence[i + 1] < 0) {
        dev_err(par->info->device,
            "missing value after delimiter %d at position %d\n",
            par->init_sequence[i], i);
        return -EINVAL;
    }
    switch (par->init_sequence[i]) {
    case -1:
        i++;
        /* make debug message */
        strcpy(msg, "");
        j = i + 1;
        while (par->init_sequence[j] >= 0) {
            sprintf(str, "0x%02X ", par->init_sequence[j]);
            strcat(msg, str);
            j++;
        }
        fbtft_par_dbg(DEBUG_INIT_DISPLAY, par,
            "init: write(0x%02X) %s\n",
            par->init_sequence[i], msg);

        /* write */
        j = 0;
        while (par->init_sequence[i] >= 0) {
            if (j > 63) {
                dev_err(par->info->device,
                    "%s: Maximum register values exceeded\n",
                    __func__);
                return -EINVAL;
            }
            buf[j++] = par->init_sequence[i++];
        }
        par->fbtftops.write_register(par, j,
            buf[0], buf[1], buf[2], buf[3],
            buf[4], buf[5], buf[6], buf[7],
            buf[8], buf[9], buf[10], buf[11],
            buf[12], buf[13], buf[14], buf[15],
            buf[16], buf[17], buf[18], buf[19],
            buf[20], buf[21], buf[22], buf[23],
            buf[24], buf[25], buf[26], buf[27],
            buf[28], buf[29], buf[30], buf[31],
            buf[32], buf[33], buf[34], buf[35],
            buf[36], buf[37], buf[38], buf[39],

```

```

        buf[40], buf[41], buf[42], buf[43],
        buf[44], buf[45], buf[46], buf[47],
        buf[48], buf[49], buf[50], buf[51],
        buf[52], buf[53], buf[54], buf[55],
        buf[56], buf[57], buf[58], buf[59],
        buf[60], buf[61], buf[62], buf[63]);
    break;
case -2:
    i++;
    fbtft_par_dbg(DEBUG_INIT_DISPLAY, par,
        "init: mdelay(%d)\n",
        par->init_sequence[i]);
    mdelay(par->init_sequence[i++]);
    break;
default:
    dev_err(par->info->device,
        "unknown delimiter %d at position %d\n",
        par->init_sequence[i], i);
    return -EINVAL;
}
}

dev_err(par->info->device,
    "%s: something is wrong. Shouldn't get here.\n", __func__);
return -EINVAL;
}

EXPORT_SYMBOL(fbtft_init_display);

/**
 * fbtft_verify_gpios() - Generic verify_gpios() function
 * @par: Driver data
 *
 * Uses @spi, @pdev and @buswidth to determine which GPIOs is needed
 *
 * Return: 0 if successful, negative if error
 */
static int fbtft_verify_gpios(struct fbtft_par *par)
{
    struct fbtft_platform_data *pdata = par->pdata;
    int i;

    fbtft_par_dbg(DEBUG_VERIFY_GPIO, par, "%s()\n", __func__);

    if (pdata->display.buswidth != 9 && par->startbyte == 0 &&
        !par->gpio.dc) {
        dev_err(par->info->device,
            "Missing info about 'dc' gpio. Aborting.\n");
        return -EINVAL;
    }

    if (!par->pdev)
        return 0;

    if (!par->gpio.wr) {
        dev_err(par->info->device, "Missing 'wr' gpio. Aborting.\n");

```

```

        return -EINVAL;
    }
    for (i = 0; i < pdata->display.buswidth; i++) {
        if (!par->gpio.db[i]) {
            dev_err(par->info->device,
                    "Missing 'db%02d' gpio. Aborting.\n", i);
            return -EINVAL;
        }
    }

    return 0;
}

/* returns 0 if the property is not present */
static u32 fbtft_property_value(struct device *dev, const char *propname)
{
    int ret;
    u32 val = 0;

    ret = device_property_read_u32(dev, propname, &val);
    if (ret == 0)
        dev_info(dev, "%s: %s = %u\n", __func__, propname, val);

    return val;
}

static struct fbtft_platform_data *fbtft_properties_read(struct device *dev)
{
    struct fbtft_platform_data *pdata;

    if (!dev_fwnode(dev)) {
        dev_err(dev, "Missing platform data or properties\n");
        return ERR_PTR(-EINVAL);
    }

    pdata = devm_kzalloc(dev, sizeof(*pdata), GFP_KERNEL);
    if (!pdata)
        return ERR_PTR(-ENOMEM);

    pdata->display.width = fbtft_property_value(dev, "width");
    pdata->display.height = fbtft_property_value(dev, "height");
    pdata->display.regwidth = fbtft_property_value(dev, "regwidth");
    pdata->display.buswidth = fbtft_property_value(dev, "buswidth");
    pdata->display.backlight = fbtft_property_value(dev, "backlight");
    pdata->display.bpp = fbtft_property_value(dev, "bpp");
    pdata->display.debug = fbtft_property_value(dev, "debug");
    pdata->rotate = fbtft_property_value(dev, "rotate");
    pdata->bgr = device_property_read_bool(dev, "bgr");
    pdata->fps = fbtft_property_value(dev, "fps");
    pdata->txbuflen = fbtft_property_value(dev, "txbuflen");
    pdata->startbyte = fbtft_property_value(dev, "startbyte");
    device_property_read_string(dev, "gamma", (const char **)&pdata->gamma);

    if (device_property_present(dev, "led-gpios"))
        pdata->display.backlight = 1;
}

```

```

        if (device_property_present(dev, "init"))
            pdata->display.fbtftops.init_display =
                fbtft_init_display_from_property;

        pdata->display.fbtftops.request_gpios = fbtft_request_gpios;

        return pdata;
    }

/**
 * fbtft_probe_common() - Generic device probe() helper function
 * @display: Display properties
 * @sdev: SPI device
 * @pdev: Platform device
 *
 * Allocates, initializes and registers a framebuffer
 *
 * Either @sdev or @pdev should be NULL
 *
 * Return: 0 if successful, negative if error
 */
int fbtft_probe_common(struct fbtft_display *display,
                      struct spi_device *sdev,
                      struct platform_device *pdev)
{
    struct device *dev;
    struct fb_info *info;
    struct fbtft_par *par;
    struct fbtft_platform_data *pdata;
    int ret;

    if (sdev)
        dev = &sdev->dev;
    else
        dev = &pdev->dev;

    if (unlikely(display->debug & DEBUG_DRIVER_INIT_FUNCTIONS))
        dev_info(dev, "%s()\n", __func__);

    pdata = dev->platform_data;
    if (!pdata) {
        pdata = fbtft_properties_read(dev);
        if (IS_ERR(pdata))
            return PTR_ERR(pdata);
    }

    info = fbtft_framebuffer_alloc(display, dev, pdata);
    if (!info)
        return -ENOMEM;

    par = info->par;
    par->spi = sdev;
    par->pdev = pdev;

    if (display->buswidth == 0) {

```

```

    dev_err(dev, "buswidth is not set\n");
    return -EINVAL;
}

/* write register functions */
if (display->regwidth == 8 && display->buswidth == 8)
    par->fbtftops.write_register = fbtft_write_reg8_bus8;
else if (display->regwidth == 8 && display->buswidth == 9 && par->spi)
    par->fbtftops.write_register = fbtft_write_reg8_bus9;
else if (display->regwidth == 16 && display->buswidth == 8)
    par->fbtftops.write_register = fbtft_write_reg16_bus8;
else if (display->regwidth == 16 && display->buswidth == 16)
    par->fbtftops.write_register = fbtft_write_reg16_bus16;
else
    dev_warn(dev,
        "no default functions for regwidth=%d and buswidth=%d\n",
        display->regwidth, display->buswidth);

/* write_vmem() functions */
if (display->buswidth == 8)
    par->fbtftops.write_vmem = fbtft_write_vmem16_bus8;
else if (display->buswidth == 9)
    par->fbtftops.write_vmem = fbtft_write_vmem16_bus9;
else if (display->buswidth == 16)
    par->fbtftops.write_vmem = fbtft_write_vmem16_bus16;

/* GPIO write() functions */
if (par->pdev) {
    if (display->buswidth == 8)
        par->fbtftops.write = fbtft_write_gpio8_wr;
    else if (display->buswidth == 16)
        par->fbtftops.write = fbtft_write_gpio16_wr;
}

/* 9-bit SPI setup */
if (par->spi && display->buswidth == 9) {
    if (par->spi->master->bits_per_word_mask & SPI_BPW_MASK(9)) {
        par->spi->bits_per_word = 9;
    } else {
        dev_warn(&par->spi->dev,
            "9-bit SPI not available, emulating using 8-bit.\n");
        /* allocate buffer with room for dc bits */
        par->extra = devm_kzalloc(par->info->device,
            par->txbuf.len +
            (par->txbuf.len / 8) + 8,
            GFP_KERNEL);
        if (!par->extra) {
            ret = -ENOMEM;
            goto out_release;
        }
        par->fbtftops.write = fbtft_write_spi_emulate_9;
    }
}

if (!par->fbtftops.verify_gpios)

```

```

        par->fbtftops.verify_gpios = fbtft_verify_gpios;

        /* make sure we still use the driver provided functions */
        fbtft_merge_fbtftops(&par->fbtftops, &display->fbtftops);

        /* use init_sequence if provided */
        if (par->init_sequence)
            par->fbtftops.init_display = fbtft_init_display;

        /* use platform_data provided functions above all */
        fbtft_merge_fbtftops(&par->fbtftops, &pdata->display.fbtftops);

        ret = fbtft_register_framebuffer(info);
        if (ret < 0)
            goto out_release;

        return 0;

    out_release:
        fbtft_framebuffer_release(info);

        return ret;
}
EXPORT_SYMBOL(fbtft_probe_common);

/**
 * fbtft_remove_common() - Generic device remove() helper function
 * @dev: Device
 * @info: Framebuffer
 *
 * Unregisters and releases the framebuffer
 *
 * Return: 0 if successful, negative if error
 */
int fbtft_remove_common(struct device *dev, struct fb_info *info)
{
    struct fbtft_par *par;

    if (!info)
        return -EINVAL;
    par = info->par;
    if (par)
        fbtft_par_dbg(DEBUG_DRIVER_INIT_FUNCTIONS, par,
            "%s()\n", __func__);
    fbtft_unregister_framebuffer(info);
    fbtft_framebuffer_release(info);

    return 0;
}
EXPORT_SYMBOL(fbtft_remove_common);

MODULE_LICENSE("GPL");

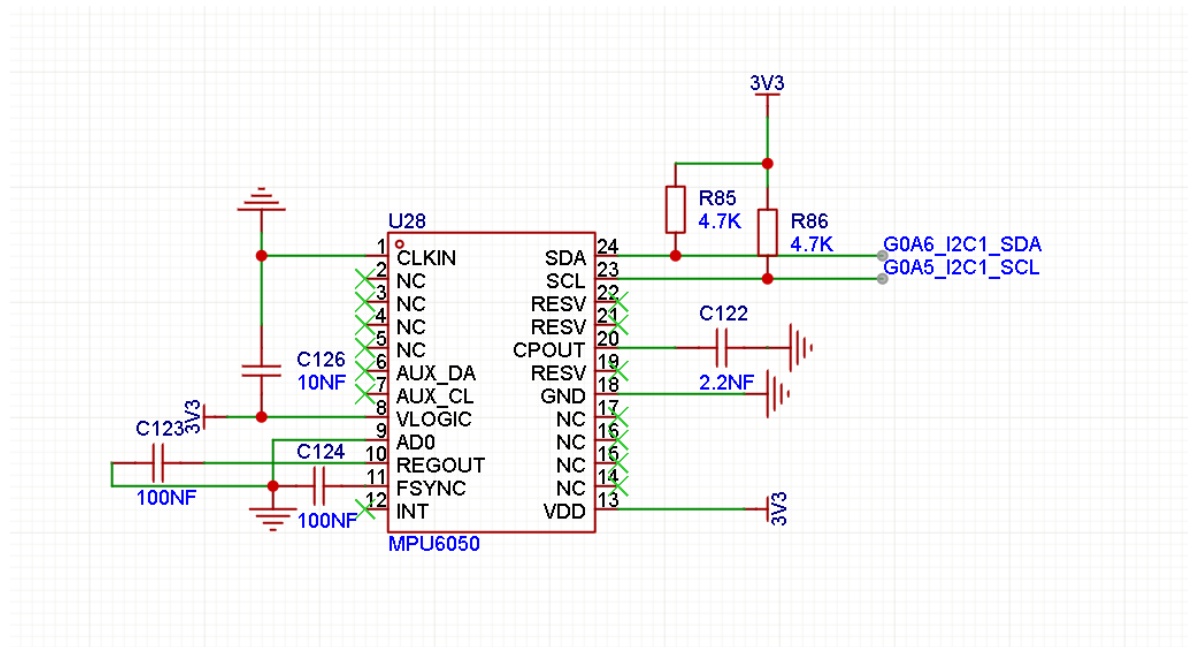
```

2.4 让陀螺仪“转”起来

接下来搞个简单的，缓一下。

2.4.1 设备树及内核修改

老样子，我们来查看硬件资源引脚，可以看出我们的i2c资源和官方声明的不一样，所以我们还要改dtsi。



打开设备树文件`rv1106-luckfox-pico-pro-max-ipc.dtsi`，确保`i2c1`节点被注释

```
Open  rv1106-luckfox-pico-pro-max-ipc.dtsi  Save  =  -  □  ×
/ luckfox-pico-main/sysdrv/source/kernel/arch/arm/boot/dts

321 };
322 */
323 };
324
325
326 // I2C
327 /*
328 &i2c0 {
329     pinctrl-0 = <&i2c0m2_xfer>;
330 };
331 */
332
333 /*
334
335 &i2c1 {
336     pinctrl-0 = <&i2c1m1_xfer>;
337 };
338
339 */
340
341 &i2c3 {
342     pinctrl-0 = <&i2c3m1_xfer &i2c3m0_xfer>;
343 };
344
345 // &i2c4 {
346 //     pinctrl-0 = <&i2c4m0_xfer>;
347 // };
348
349 // UART
350 /*
351 &uart0 {
352     pinctrl-0 = <&uart0m0_xfer &uart0m1_xfer>;
353 };
354 */
355 &uart1 {
356     pinctrl-0 = <&uart1m1_xfer>;
357 };
358 &uart3 {
```

修改黄色所标部位，将m1改成m0,这样i2c1 就会使用定义好的m0组引脚。

```
Open  [icon] rv1106.dtsi /luckFox-pico-main/sysdrv/source/kernel/arch/arm/boot/dts Save [icon] [icon] [icon] [icon]
442         #address-cells = <1>;
443         #size-cells = <0>;
444         clocks = <&cru CLK_I2C0>, <&cru PCLK_I2C0>;
445         clock-names = "i2c", "pclk";
446         pinctrl-names = "default";
447         pinctrl-0 = <&i2c0m0_xfer>;
448         status = "disabled";
449     };
450
451     i2c1: i2c@ff320000 {
452         compatible = "rockchip,rv1106-i2c", "rockchip,rk3399-i2c";
453         reg = <0xff320000 0x1000>;
454         interrupts = <GIC_SPI 19 IRQ_TYPE_LEVEL_HIGH>;
455         #address-cells = <1>;
456         #size-cells = <0>;
457         clocks = <&cru CLK_I2C1>, <&cru PCLK_I2C1>;
458         clock-names = "i2c", "pclk";
459         pinctrl-names = "default";
460         pinctrl-0 = <&i2c1m0_xfer>;
461         status = "disabled";
462     };
463
464     dsm: codec-digital@ff340000 {
465         compatible = "rockchip,rv1106-codec-digital", "rockchip,codec-digital-v1";
466         reg = <0xff340000 0x1000>;
467         clocks = <&cru MCLK_DSM>, <&cru PCLK_DSM>;
468         clock-names = "dac", "pclk";
469         resets = <&cru SRST_M_DSM>;
470         reset-names = "reset";
471         rockchip,grf = <&grf>;
472         rockchip,pwm-output-mode;
473         #sound-dai-cells = <0>;
474         pinctrl-names = "default";
475         pinctrl-0 = <&dsmaudio_pins>;
476         status = "disabled";
477     };
478
479     ...
```

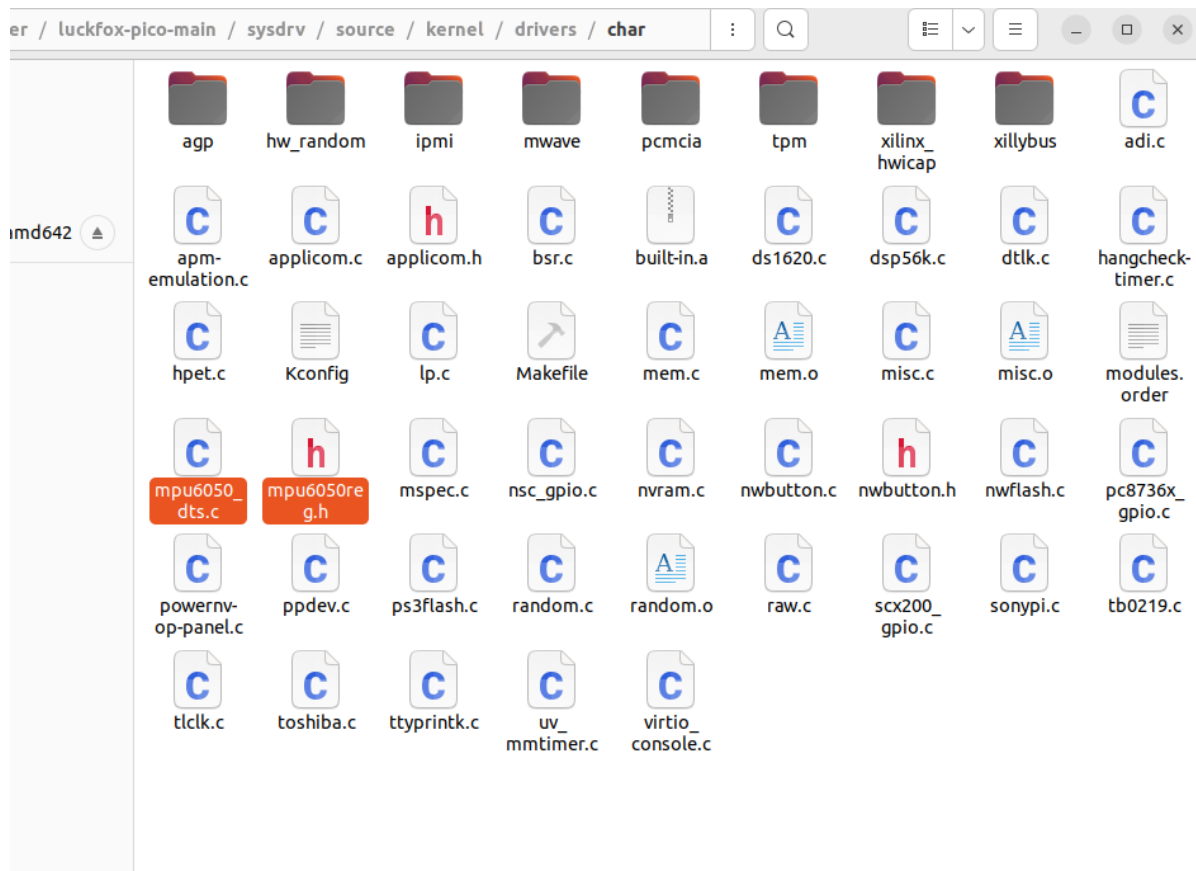
添加如下i2c设备节点。

```
239         clock-frequency = <100000>;
240     };
241
242     /* I2C1_M0 */
243     &i2c1 {
244         status = "okay";
245         clock-frequency = <400000>;
246         pinctrl-0 = <&i2c1m0_xfer>;
247         pinctrl-names = "default";
248         #address-cells = <1>;
249         #size-cells = <0>;
250
251         mpu6050:mpu6050@68{
252             compatible= "test,mpu6050";
253             reg=<0x68>;
254             status="okay";
255         };
256     };
257
258 };
259
260
```

接下来是linux内核软件配置，我们需要往内核添加代码，代码我已经写好了（可在2.3中找到）。

名称	修改日期	类型	大小
mpu6050reg.h	2024/10/5 21:27	C Header File	14 KB
mpu6050_dts.c	2025/3/19 12:42	C Source File	8 KB
mpu6050Demo.c	2025/3/19 13:35	C Source File	2 KB

进入Linux内核 drivers/char/ 下，复制代码过去，demo的不用复制，这个up有其他用处。



我们在当前文件夹下打开Makefile ,添加如图编译条件。

```
root@chen6666697-virtual-machine:/luckfox-pico-main/sysdrv/source/kernel/drivers/char# gedit Makefile
```

```
3 # Makefile for the kernel character device drivers.
4 #
5
6 obj-m += mpu6050_dts.o
7 obj-y += mem.o random.o
8 obj-$(CONFIG_TTY_PRINTK) += ttyprintk.o
9 obj-y += misc.o
10 obj-$(CONFIG_ATARI_DSP56K) += dsp56k.o
11 obj-$(CONFIG_VIRTIO_CONSOLE) += virtio_console.o
12 obj-$(CONFIG_RAW_DRIVER) += raw.o
13 obj-$(CONFIG_MSPEC) += mspec.o
14 obj-$(CONFIG_UV_MMTIMER) += uv_mmtimer.o
15 obj-$(CONFIG_I8M_BSR) += bsr.o
16
17 obj-$(CONFIG_PRINTER) += lp.o
18
19 obj-$(CONFIG_APM_EMULATION) += apm-emulation.o
20
21 obj-$(CONFIG_DTLK) += dtlk.o
22 obj-$(CONFIG_APPLICOM) += applicom.o
23 obj-$(CONFIG_SONYPI) += sonypi.o
24 obj-$(CONFIG_HPET) += hpet.o
25 obj-$(CONFIG_XILINX_HWCAP) += xilinx_hwicap/
26 obj-$(CONFIG_NVRAM) += nvram.o
27 obj-$(CONFIG_TOSHIBA) += toshiba.o
28 obj-$(CONFIG_DS1620) += ds1620.o
29 obj-$(CONFIG_HW_RANDOM) += hw_random/
30 obj-$(CONFIG_PPDEV) += ppdev.o
31 obj-$(CONFIG_NWBUTTON) += nwbutton.o
32 obj-$(CONFIG_NWFLASH) += nwflash.o
33 obj-$(CONFIG_SCX200_GPIO) += scx200_gpio.o
34 obj-$(CONFIG_PC8736X_GPIO) += pc8736x_gpio.o
35 obj-$(CONFIG_NSC_GPIO) += nsc_gpio.o
36 obj-$(CONFIG_GPIO_TB0219) += tb0219.o
37 obj-$(CONFIG_TLCLK) += tlclk.o
```

接下来，我们直接回到主sdk，编译镜像。

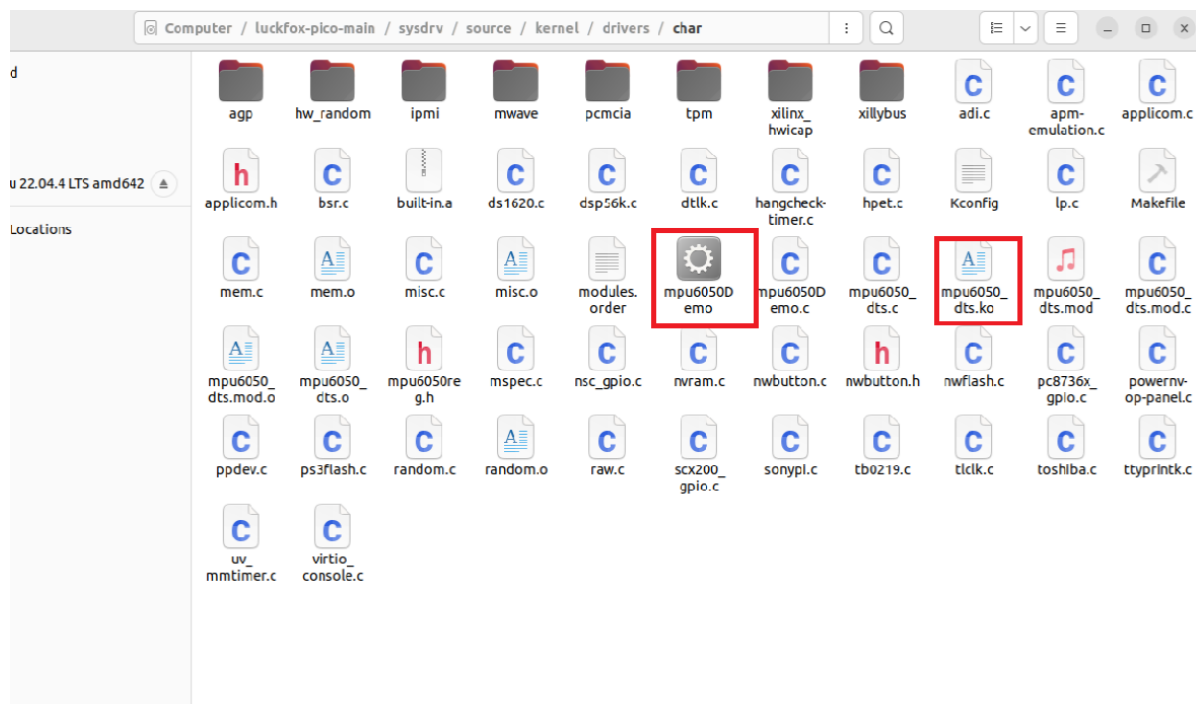
2.4.2 陀螺仪的相关操作

现在是时候使用demo了，为什么要使用他呢？驱动设备在linux中表示的是一个文件，我们要使用它，相当于去操控这个文件，因此up写了一个脚本去操控它，我们需要用交叉编译工具链去编译它。输入如下命令，如图所示。

```
arm-rockchip830-linux-uclicgnueabihf-gcc mpu6050Demo.c -o mpu6050Demo
```

```
root@chen6666697-virtual-machine:/luckfox-pico-main/sysdrv/source/kernel/drivers/char# arm-rockchip830-linux-uclicgnueabihf-gcc mpu6050Demo.c -o mpu6050Demo
```

现在我们需要将两个文件移植开发板，如图所示文件。



可以看到一个.ko文件，那什么是.ko文件呢？.ko 文件是 Linux 内核模块（Kernel Object）的后缀名，它是把驱动程序或功能代码单独编译成可加载模块的一种格式。相比直接把代码编进内核，.ko 文件可以在系统运行时通过 `insmod` 或 `modprobe` 动态加载，也能用 `rmmod` 卸载，不需要重启或重新编译整个内核。这让驱动开发、调试和维护更加灵活：只要内核版本、配置和符号匹配，模块就能像插件一样被挂载进内核，提供对应的硬件驱动或功能扩展。

下面我们来挂载设备以及读取它的值，执行如下命令，即可读到所需值。

```
insmod mpu6050_dts.ko
```

```
./mpu6050Demo
```

```
[root@luckfox root]# insmod mpu6050_dts.ko
[ 6238.345623] mpu6050 ID = 0X68
[root@luckfox root]# chmod +x mpu6050Demo
[root@luckfox root]# ./mpu6050Demo

ADC value:
gx = 14, gy = -55, gz = 6
ax = 54, ay = 7, az = -2085
temp = -1605
TRUE value:act gx = 0.85°/S, act gy = -3.35°/S, act gz = 0.37°/S
act ax = 0.03g, act ay = 0.00g, act az = -1.02g
act temp = 20.01°C

ADC value:
gx = 12, gy = -62, gz = 5
ax = 48, ay = 0, az = -2090
temp = -1598
TRUE value:act gx = 0.73°/S, act gy = -3.78°/S, act gz = 0.30°/S
act ax = 0.02g, act ay = 0.00g, act az = -1.02g
act temp = 20.03°C

ADC value:
gx = 10, gy = -61, gz = 4
ax = 64, ay = -1, az = -2087
temp = -1595
TRUE value:act gx = 0.61°/S, act gy = -3.72°/S, act gz = 0.24°/S
act ax = 0.03g, act ay = -0.00g, act az = -1.02g
```

至此，陀螺仪已经驱动完成。

2.4.3 驱动相关代码

mpu6050reg.h

```
#ifndef _MPU6050_REG_H_
#define _MPU6050_REG_H_

#define MPU6050_ADDRESS_AD0_LOW 0x68 // address pin low (GND), default for
InvenSense evaluation board
#define MPU6050_ADDRESS_AD0_HIGH 0x69 // address pin high (VCC)
#define MPU6050_DEFAULT_ADDRESS MPU6050_ADDRESS_AD0_LOW

#define MPU6050_RA_XG_OFFS_TC 0x00 //[7] PWR_MODE, [6:1] XG_OFFS_TC, [0]
OTP_BNK_VLD
#define MPU6050_RA_YG_OFFS_TC 0x01 //[7] PWR_MODE, [6:1] YG_OFFS_TC, [0]
OTP_BNK_VLD
#define MPU6050_RA_ZG_OFFS_TC 0x02 //[7] PWR_MODE, [6:1] ZG_OFFS_TC, [0]
OTP_BNK_VLD
#define MPU6050_RA_X_FINE_GAIN 0x03 //[7:0] X_FINE_GAIN
#define MPU6050_RA_Y_FINE_GAIN 0x04 //[7:0] Y_FINE_GAIN
#define MPU6050_RA_Z_FINE_GAIN 0x05 //[7:0] Z_FINE_GAIN
#define MPU6050_RA_XA_OFFS_H 0x06 //[15:0] XA_OFFS
#define MPU6050_RA_XA_OFFS_L_TC 0x07
#define MPU6050_RA_YA_OFFS_H 0x08 //[15:0] YA_OFFS
#define MPU6050_RA_YA_OFFS_L_TC 0x09
#define MPU6050_RA_ZA_OFFS_H 0x0A //[15:0] ZA_OFFS
#define MPU6050_RA_ZA_OFFS_L_TC 0x0B
#define MPU6050_RA_XG_OFFS_USRH 0x13 //[15:0] XG_OFFS_USR
#define MPU6050_RA_XG_OFFS_USRL 0x14
#define MPU6050_RA_YG_OFFS_USRH 0x15 //[15:0] YG_OFFS_USR
#define MPU6050_RA_YG_OFFS_USRL 0x16
#define MPU6050_RA_ZG_OFFS_USRH 0x17 //[15:0] ZG_OFFS_USR
#define MPU6050_RA_ZG_OFFS_USRL 0x18
#define MPU6050_RA_SMPLRT_DIV 0x19
#define MPU6050_RA_CONFIG 0x1A
#define MPU6050_RA_GYRO_CONFIG 0x1B
#define MPU6050_RA_ACCEL_CONFIG 0x1C
#define MPU6050_RA_FF_THR 0x1D
#define MPU6050_RA_FF_DUR 0x1E
#define MPU6050_RA_MOT_THR 0x1F
#define MPU6050_RA_MOT_DUR 0x20
#define MPU6050_RA_ZRMOT_THR 0x21
#define MPU6050_RA_ZRMOT_DUR 0x22
#define MPU6050_RA_FIFO_EN 0x23
#define MPU6050_RA_I2C_MST_CTRL 0x24
#define MPU6050_RA_I2C_SLV0_ADDR 0x25
#define MPU6050_RA_I2C_SLV0_REG 0x26
#define MPU6050_RA_I2C_SLV0_CTRL 0x27
#define MPU6050_RA_I2C_SLV1_ADDR 0x28
#define MPU6050_RA_I2C_SLV1_REG 0x29
#define MPU6050_RA_I2C_SLV1_CTRL 0x2A
#define MPU6050_RA_I2C_SLV2_ADDR 0x2B
#define MPU6050_RA_I2C_SLV2_REG 0x2C
```

```
#define MPU6050_RA_I2C_SLV2_CTRL    0x2D
#define MPU6050_RA_I2C_SLV3_ADDR    0x2E
#define MPU6050_RA_I2C_SLV3_REG     0x2F
#define MPU6050_RA_I2C_SLV3_CTRL    0x30
#define MPU6050_RA_I2C_SLV4_ADDR    0x31
#define MPU6050_RA_I2C_SLV4_REG     0x32
#define MPU6050_RA_I2C_SLV4_DO      0x33
#define MPU6050_RA_I2C_SLV4_CTRL    0x34
#define MPU6050_RA_I2C_SLV4_DI      0x35
#define MPU6050_RA_I2C_MST_STATUS    0x36
#define MPU6050_RA_INT_PIN_CFG      0x37
#define MPU6050_RA_INT_ENABLE        0x38
#define MPU6050_RA_DMP_INT_STATUS    0x39
#define MPU6050_RA_INT_STATUS        0x3A
#define MPU6050_RA_ACCEL_XOUT_H      0x3B
#define MPU6050_RA_ACCEL_XOUT_L      0x3C
#define MPU6050_RA_ACCEL_YOUT_H      0x3D
#define MPU6050_RA_ACCEL_YOUT_L      0x3E
#define MPU6050_RA_ACCEL_ZOUT_H      0x3F
#define MPU6050_RA_ACCEL_ZOUT_L      0x40
#define MPU6050_RA_TEMP_OUT_H        0x41
#define MPU6050_RA_TEMP_OUT_L        0x42
#define MPU6050_RA_GYRO_XOUT_H       0x43
#define MPU6050_RA_GYRO_XOUT_L       0x44
#define MPU6050_RA_GYRO_YOUT_H       0x45
#define MPU6050_RA_GYRO_YOUT_L       0x46
#define MPU6050_RA_GYRO_ZOUT_H       0x47
#define MPU6050_RA_GYRO_ZOUT_L       0x48
#define MPU6050_RA_EXT_SENS_DATA_00  0x49
#define MPU6050_RA_EXT_SENS_DATA_01  0x4A
#define MPU6050_RA_EXT_SENS_DATA_02  0x4B
#define MPU6050_RA_EXT_SENS_DATA_03  0x4C
#define MPU6050_RA_EXT_SENS_DATA_04  0x4D
#define MPU6050_RA_EXT_SENS_DATA_05  0x4E
#define MPU6050_RA_EXT_SENS_DATA_06  0x4F
#define MPU6050_RA_EXT_SENS_DATA_07  0x50
#define MPU6050_RA_EXT_SENS_DATA_08  0x51
#define MPU6050_RA_EXT_SENS_DATA_09  0x52
#define MPU6050_RA_EXT_SENS_DATA_10  0x53
#define MPU6050_RA_EXT_SENS_DATA_11  0x54
#define MPU6050_RA_EXT_SENS_DATA_12  0x55
#define MPU6050_RA_EXT_SENS_DATA_13  0x56
#define MPU6050_RA_EXT_SENS_DATA_14  0x57
#define MPU6050_RA_EXT_SENS_DATA_15  0x58
#define MPU6050_RA_EXT_SENS_DATA_16  0x59
#define MPU6050_RA_EXT_SENS_DATA_17  0x5A
#define MPU6050_RA_EXT_SENS_DATA_18  0x5B
#define MPU6050_RA_EXT_SENS_DATA_19  0x5C
#define MPU6050_RA_EXT_SENS_DATA_20  0x5D
#define MPU6050_RA_EXT_SENS_DATA_21  0x5E
#define MPU6050_RA_EXT_SENS_DATA_22  0x5F
#define MPU6050_RA_EXT_SENS_DATA_23  0x60
#define MPU6050_RA_MOT_DETECT_STATUS 0x61
#define MPU6050_RA_I2C_SLV0_DO      0x63
#define MPU6050_RA_I2C_SLV1_DO      0x64
```

```

#define MPU6050_RA_I2C_SLV2_DO      0x65
#define MPU6050_RA_I2C_SLV3_DO      0x66
#define MPU6050_RA_I2C_MST_DELAY_CTRL 0x67
#define MPU6050_RA_SIGNAL_PATH_RESET 0x68
#define MPU6050_RA_MOT_DETECT_CTRL   0x69
#define MPU6050_RA_USER_CTRL          0x6A
#define MPU6050_RA_PWR_MGMT_1         0x6B
#define MPU6050_RA_PWR_MGMT_2         0x6C
#define MPU6050_RA_BANK_SEL           0x6D
#define MPU6050_RA_MEM_START_ADDR      0x6E
#define MPU6050_RA_MEM_R_W            0x6F
#define MPU6050_RA_DMP_CFG_1          0x70
#define MPU6050_RA_DMP_CFG_2          0x71
#define MPU6050_RA_FIFO_COUNTH        0x72
#define MPU6050_RA_FIFO_COUNTL        0x73
#define MPU6050_RA_FIFO_R_W           0x74
#define MPU6050_RA_WHO_AM_I           0x75

#define MPU6050_TC_PWR_MODE_BIT       7
#define MPU6050_TC_OFFSET_BIT         6
#define MPU6050_TC_OFFSET_LENGTH      6
#define MPU6050_TC_OTP_BNK_VLD_BIT   0

#define MPU6050_VDDIO_LEVEL_VLOGIC    0
#define MPU6050_VDDIO_LEVEL_VDD       1

#define MPU6050_CFG_EXT_SYNC_SET_BIT   5
#define MPU6050_CFG_EXT_SYNC_SET_LENGTH 3
#define MPU6050_CFG_DLPF_CFG_BIT      2
#define MPU6050_CFG_DLPF_CFG_LENGTH   3

#define MPU6050_EXT_SYNC_DISABLED      0x0
#define MPU6050_EXT_SYNC_TEMP_OUT_L   0x1
#define MPU6050_EXT_SYNC_GYRO_XOUT_L  0x2
#define MPU6050_EXT_SYNC_GYRO_YOUT_L  0x3
#define MPU6050_EXT_SYNC_GYRO_ZOUT_L  0x4
#define MPU6050_EXT_SYNC_ACCEL_XOUT_L  0x5
#define MPU6050_EXT_SYNC_ACCEL_YOUT_L  0x6
#define MPU6050_EXT_SYNC_ACCEL_ZOUT_L  0x7

#define MPU6050_DLPF_BW_256            0x00
#define MPU6050_DLPF_BW_188            0x01
#define MPU6050_DLPF_BW_98             0x02
#define MPU6050_DLPF_BW_42             0x03
#define MPU6050_DLPF_BW_20             0x04
#define MPU6050_DLPF_BW_10             0x05
#define MPU6050_DLPF_BW_5              0x06

#define MPU6050_GCONFIG_FS_SEL_BIT      4
#define MPU6050_GCONFIG_FS_SEL_LENGTH   2

#define MPU6050_GYRO_FS_250            0x00
#define MPU6050_GYRO_FS_500            0x01
#define MPU6050_GYRO_FS_1000           0x02
#define MPU6050_GYRO_FS_2000           0x03

```

```

#define MPU6050_ACONFIG_XA_ST_BIT      7
#define MPU6050_ACONFIG_YA_ST_BIT      6
#define MPU6050_ACONFIG_ZA_ST_BIT      5
#define MPU6050_ACONFIG_AFS_SEL_BIT    4
#define MPU6050_ACONFIG_AFS_SEL_LENGTH 2
#define MPU6050_ACONFIG_ACCEL_HPF_BIT   2
#define MPU6050_ACONFIG_ACCEL_HPF_LENGTH 3

#define MPU6050_ACCEL_FS_2              0x00
#define MPU6050_ACCEL_FS_4              0x01
#define MPU6050_ACCEL_FS_8              0x02
#define MPU6050_ACCEL_FS_16             0x03

#define MPU6050_DHPF_RESET              0x00
#define MPU6050_DHPF_5                  0x01
#define MPU6050_DHPF_2P5                0x02
#define MPU6050_DHPF_1P25              0x03
#define MPU6050_DHPF_0P63              0x04
#define MPU6050_DHPF_HOLD               0x07

#define MPU6050_TEMP_FIFO_EN_BIT        7
#define MPU6050_XG_FIFO_EN_BIT          6
#define MPU6050_YG_FIFO_EN_BIT          5
#define MPU6050_ZG_FIFO_EN_BIT          4
#define MPU6050_ACCEL_FIFO_EN_BIT        3
#define MPU6050_SLV2_FIFO_EN_BIT        2
#define MPU6050_SLV1_FIFO_EN_BIT        1
#define MPU6050_SLV0_FIFO_EN_BIT        0

#define MPU6050_MULT_MST_EN_BIT          7
#define MPU6050_WAIT_FOR_ES_BIT          6
#define MPU6050_SLV_3_FIFO_EN_BIT        5
#define MPU6050_I2C_MST_P_NSR_BIT        4
#define MPU6050_I2C_MST_CLK_BIT          3
#define MPU6050_I2C_MST_CLK_LENGTH      4

#define MPU6050_CLOCK_DIV_348            0x0
#define MPU6050_CLOCK_DIV_333            0x1
#define MPU6050_CLOCK_DIV_320            0x2
#define MPU6050_CLOCK_DIV_308            0x3
#define MPU6050_CLOCK_DIV_296            0x4
#define MPU6050_CLOCK_DIV_286            0x5
#define MPU6050_CLOCK_DIV_276            0x6
#define MPU6050_CLOCK_DIV_267            0x7
#define MPU6050_CLOCK_DIV_258            0x8
#define MPU6050_CLOCK_DIV_500            0x9
#define MPU6050_CLOCK_DIV_471            0xA
#define MPU6050_CLOCK_DIV_444            0xB
#define MPU6050_CLOCK_DIV_421            0xC
#define MPU6050_CLOCK_DIV_400            0xD
#define MPU6050_CLOCK_DIV_381            0xE
#define MPU6050_CLOCK_DIV_364            0xF

#define MPU6050_I2C_SLV_RW_BIT           7

```

```

#define MPU6050_I2C_SLV_ADDR_BIT      6
#define MPU6050_I2C_SLV_ADDR_LENGTH  7
#define MPU6050_I2C_SLV_EN_BIT        7
#define MPU6050_I2C_SLV_BYTE_SW_BIT   6
#define MPU6050_I2C_SLV_REG_DIS_BIT   5
#define MPU6050_I2C_SLV_GRP_BIT        4
#define MPU6050_I2C_SLV_LEN_BIT        3
#define MPU6050_I2C_SLV_LEN_LENGTH    4

#define MPU6050_I2C_SLV4_RW_BIT        7
#define MPU6050_I2C_SLV4_ADDR_BIT      6
#define MPU6050_I2C_SLV4_ADDR_LENGTH  7
#define MPU6050_I2C_SLV4_EN_BIT        7
#define MPU6050_I2C_SLV4_INT_EN_BIT    6
#define MPU6050_I2C_SLV4_REG_DIS_BIT   5
#define MPU6050_I2C_SLV4_MST_DLY_BIT   4
#define MPU6050_I2C_SLV4_MST_DLY_LENGTH 5

#define MPU6050_MST_PASS_THROUGH_BIT    7
#define MPU6050_MST_I2C_SLV4_DONE_BIT    6
#define MPU6050_MST_I2C_LOST_ARB_BIT     5
#define MPU6050_MST_I2C_SLV4_NACK_BIT    4
#define MPU6050_MST_I2C_SLV3_NACK_BIT    3
#define MPU6050_MST_I2C_SLV2_NACK_BIT    2
#define MPU6050_MST_I2C_SLV1_NACK_BIT    1
#define MPU6050_MST_I2C_SLV0_NACK_BIT    0

#define MPU6050_INTCFG_INT_LEVEL_BIT      7
#define MPU6050_INTCFG_INT_OPEN_BIT       6
#define MPU6050_INTCFG_LATCH_INT_EN_BIT   5
#define MPU6050_INTCFG_INT_RD_CLEAR_BIT   4
#define MPU6050_INTCFG_FSYNC_INT_LEVEL_BIT 3
#define MPU6050_INTCFG_FSYNC_INT_EN_BIT   2
#define MPU6050_INTCFG_I2C_BYPASS_EN_BIT   1
#define MPU6050_INTCFG_CLKOUT_EN_BIT       0

#define MPU6050_INTMODE_ACTIVEHIGH  0x00
#define MPU6050_INTMODE_ACTIVELOW   0x01

#define MPU6050_INTDRV_PUSHPULL      0x00
#define MPU6050_INTDRV_OPENDRAIN      0x01

#define MPU6050_INTLATCH_50USPULSE  0x00
#define MPU6050_INTLATCH_WAITCLEAR  0x01

#define MPU6050_INTCLEAR_STATUSREAD  0x00
#define MPU6050_INTCLEAR_ANYREAD     0x01

#define MPU6050_INTERRUPT_FF_BIT      7
#define MPU6050_INTERRUPT_MOT_BIT      6
#define MPU6050_INTERRUPT_ZMOT_BIT     5
#define MPU6050_INTERRUPT_FIFO_OFLOW_BIT 4
#define MPU6050_INTERRUPT_I2C_MST_INT_BIT 3
#define MPU6050_INTERRUPT_PLL_RDY_INT_BIT 2
#define MPU6050_INTERRUPT_DMP_INT_BIT  1

```

```

#define MPU6050_INTERRUPT_DATA_RDY_BIT    0

// TODO: figure out what these actually do
// UMPL source code is not very obvious
#define MPU6050_DMPINT_5_BIT              5
#define MPU6050_DMPINT_4_BIT              4
#define MPU6050_DMPINT_3_BIT              3
#define MPU6050_DMPINT_2_BIT              2
#define MPU6050_DMPINT_1_BIT              1
#define MPU6050_DMPINT_0_BIT              0

#define MPU6050_MOTION_MOT_XNEG_BIT       7
#define MPU6050_MOTION_MOT_XPOS_BIT       6
#define MPU6050_MOTION_MOT_YNEG_BIT       5
#define MPU6050_MOTION_MOT_YPOS_BIT       4
#define MPU6050_MOTION_MOT_ZNEG_BIT       3
#define MPU6050_MOTION_MOT_ZPOS_BIT       2
#define MPU6050_MOTION_MOT_ZRMOT_BIT      0

#define MPU6050_DELAYCTRL_DELAY_ES_SHADOW_BIT  7
#define MPU6050_DELAYCTRL_I2C_SLV4_DLY_EN_BIT  4
#define MPU6050_DELAYCTRL_I2C_SLV3_DLY_EN_BIT  3
#define MPU6050_DELAYCTRL_I2C_SLV2_DLY_EN_BIT  2
#define MPU6050_DELAYCTRL_I2C_SLV1_DLY_EN_BIT  1
#define MPU6050_DELAYCTRL_I2C_SLV0_DLY_EN_BIT  0

#define MPU6050_PATHRESET_GYRO_RESET_BIT    2
#define MPU6050_PATHRESET_ACCEL_RESET_BIT    1
#define MPU6050_PATHRESET_TEMP_RESET_BIT    0

#define MPU6050_DETECT_ACCEL_ON_DELAY_BIT    5
#define MPU6050_DETECT_ACCEL_ON_DELAY_LENGTH 2
#define MPU6050_DETECT_FF_COUNT_BIT          3
#define MPU6050_DETECT_FF_COUNT_LENGTH       2
#define MPU6050_DETECT_MOT_COUNT_BIT          1
#define MPU6050_DETECT_MOT_COUNT_LENGTH       2

#define MPU6050_DETECT_DECREMENT_RESET  0x0
#define MPU6050_DETECT_DECREMENT_1      0x1
#define MPU6050_DETECT_DECREMENT_2      0x2
#define MPU6050_DETECT_DECREMENT_4      0x3

#define MPU6050_USERCTRL_DMP_EN_BIT       7
#define MPU6050_USERCTRL_FIFO_EN_BIT      6
#define MPU6050_USERCTRL_I2C_MST_EN_BIT   5
#define MPU6050_USERCTRL_I2C_IF_DIS_BIT   4
#define MPU6050_USERCTRL_DMP_RESET_BIT     3
#define MPU6050_USERCTRL_FIFO_RESET_BIT    2
#define MPU6050_USERCTRL_I2C_MST_RESET_BIT 1
#define MPU6050_USERCTRL_SIG_COND_RESET_BIT 0

#define MPU6050_PWR1_DEVICE_RESET_BIT      7
#define MPU6050_PWR1_SLEEP_BIT             6
#define MPU6050_PWR1_CYCLE_BIT             5
#define MPU6050_PWR1_TEMP_DIS_BIT          3

```

```

#define MPU6050_PWR1_CLKSEL_BIT      2
#define MPU6050_PWR1_CLKSEL_LENGTH  3

#define MPU6050_CLOCK_INTERNAL        0x00
#define MPU6050_CLOCK_PLL_XGYRO      0x01
#define MPU6050_CLOCK_PLL_YGYRO      0x02
#define MPU6050_CLOCK_PLL_ZGYRO      0x03
#define MPU6050_CLOCK_PLL_EXT32K      0x04
#define MPU6050_CLOCK_PLL_EXT19M      0x05
#define MPU6050_CLOCK_KEEP_RESET      0x07

#define MPU6050_PWR2_LP_WAKE_CTRL_BIT  7
#define MPU6050_PWR2_LP_WAKE_CTRL_LENGTH  2
#define MPU6050_PWR2_STBY_XA_BIT        5
#define MPU6050_PWR2_STBY_YA_BIT        4
#define MPU6050_PWR2_STBY_ZA_BIT        3
#define MPU6050_PWR2_STBY_XG_BIT        2
#define MPU6050_PWR2_STBY_YG_BIT        1
#define MPU6050_PWR2_STBY_ZG_BIT        0

#define MPU6050_WAKE_FREQ_1P25         0x0
#define MPU6050_WAKE_FREQ_2P5         0x1
#define MPU6050_WAKE_FREQ_5           0x2
#define MPU6050_WAKE_FREQ_10          0x3

#define MPU6050_BANKSEL_PRFTCH_EN_BIT   6
#define MPU6050_BANKSEL_CFG_USER_BANK_BIT  5
#define MPU6050_BANKSEL_MEM_SEL_BIT     4
#define MPU6050_BANKSEL_MEM_SEL_LENGTH  5

#define MPU6050_WHO_AM_I_BIT           6
#define MPU6050_WHO_AM_I_LENGTH        6

#define MPU6050_DMP_MEMORY_BANKS        8
#define MPU6050_DMP_MEMORY_BANK_SIZE    256
#define MPU6050_DMP_MEMORY_CHUNK_SIZE   16

#endif // _MPU6050_REG_H_

```

mpu6050_dts.c

```

#include <linux/types.h>
#include <linux/kernel.h>
#include <linux/delay.h>
#include <linux/init.h>
#include <linux/module.h>
#include <linux/errno.h>
#include <linux/gpio.h>
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/of_gpio.h>
#include <linux/semaphore.h>
#include <linux/timer.h>
#include <linux/i2c.h>
#include <asm/uaccess.h>

```

```

#include <asm/io.h>
#include "mpu6050reg.h"

#define MPU6050_CNT 1
#define MPU6050_NAME "mpu6050"

struct mpu6050_dev {
    dev_t devid; /* 设备号 */
    struct cdev cdev; /* cdev */
    struct class *class; /* 类 */
    struct device *device; /* 设备 */
    struct device_node *nd; /* 设备节点 */
    int major; /* 主设备号 */
    void *private_data; /* 私有数据 */
    signed int gyro_x_adc; /* 陀螺仪X轴原始值 */
    signed int gyro_y_adc; /* 陀螺仪Y轴原始值 */
    signed int gyro_z_adc; /* 陀螺仪Z轴原始值 */
    signed int accel_x_adc; /* 加速度计X轴原始值 */
    signed int accel_y_adc; /* 加速度计Y轴原始值 */
    signed int accel_z_adc; /* 加速度计Z轴原始值 */
    signed int temp_adc; /* 温度原始值 */
};

static struct mpu6050_dev mpu6050dev;

static int mpu6050_release(struct inode *inode, struct file *filp)
{
    return 0;
}

static int mpu6050_open(struct inode *inode, struct file *filp)
{
    filp->private_data = &mpu6050dev; /* 设置私有数据 */
    return 0;
}

static s32 mpu6050_write_regs(struct mpu6050_dev *dev, u8 reg, u8 *buf, u8 len)
{
    u8 b[256];
    struct i2c_msg msg;
    struct i2c_client *client = (struct i2c_client *)dev->private_data;

    b[0] = reg; /* 寄存器首地址 */
    memcpy(&b[1], buf, len); /* 将要写入的数据拷贝到数组b里面 */

    msg.addr = client->addr; /* mpu6050地址 */
    msg.flags = 0; /* 标记为写数据 */

    msg.buf = b; /* 要写入的数据缓冲区 */
    msg.len = len + 1; /* 要写入的数据长度 */

    return i2c_transfer(client->adapter, &msg, 1);
}

static void mpu6050_write_onereg(struct mpu6050_dev *dev, u8 reg, u8 data)

```

```

{
    u8 buf = 0;
    buf = data;
    mpu6050_write_regs(dev, reg, &buf, 1);
}

static int mpu6050_read_regs(struct mpu6050_dev *dev, u8 reg, void *val, int len)
{
    int ret;
    struct i2c_msg msg[2];
    struct i2c_client *client = (struct i2c_client *)dev->private_data;

    /* msg[0]为发送要读取的首地址 */
    msg[0].addr = client->addr;          /* */
    msg[0].flags = 0;                   /* 标记为发送数据 */
    msg[0].buf = &reg;                  /* 读取的首地址 */
    msg[0].len = 1;                     /* reg长度*/

    /* msg[1]读取数据 */
    msg[1].addr = client->addr;          /* */
    msg[1].flags = I2C_M_RD;            /* 标记为读取数据*/
    msg[1].buf = val;                   /* 读取数据缓冲区 */
    msg[1].len = len;                   /* 要读取的数据长度*/

    ret = i2c_transfer(client->adapter, msg, 2);
    if(ret == 2) {
        ret = 0;
    } else {
        printk("i2c rd failed=%d reg=%06x len=%d\n", ret, reg, len);
        ret = -EREMOTEIO;
    }
    return ret;
}

static unsigned char mpu6050_read_onereg(struct mpu6050_dev *dev, u8 reg)
{
    u8 data = 0;

    mpu6050_read_regs(dev, reg, &data, 1);
    return data;

#ifdef 0
    struct i2c_client *client = (struct i2c_client *)dev->private_data;
    return i2c_smbus_read_byte_data(client, reg);
#endif
}

void mpu6050_readdata(struct mpu6050_dev *dev)
{
    unsigned char data[14];
    mpu6050_read_regs(dev, MPU6050_RA_ACCEL_XOUT_H, data, 14);

    dev->accel_x_adc = (signed short)((data[0] << 8) | data[1]);
    dev->accel_y_adc = (signed short)((data[2] << 8) | data[3]);
    dev->accel_z_adc = (signed short)((data[4] << 8) | data[5]);
}

```

```

    dev->temp_adc    = (signed short)((data[6] << 8) | data[7]);
    dev->gyro_x_adc  = (signed short)((data[8] << 8) | data[9]);
    dev->gyro_y_adc  = (signed short)((data[10] << 8) | data[11]);
    dev->gyro_z_adc  = (signed short)((data[12] << 8) | data[13]);
}

static ssize_t mpu6050_read(struct file *filp, char __user *buf, size_t cnt,
loff_t *off)
{
    signed int data[7];
    long err = 0;
    struct mpu6050_dev *dev = (struct mpu6050_dev *)filp->private_data;

    mpu6050_readdata(dev);
    data[0] = dev->gyro_x_adc;
    data[1] = dev->gyro_y_adc;
    data[2] = dev->gyro_z_adc;
    data[3] = dev->accel_x_adc;
    data[4] = dev->accel_y_adc;
    data[5] = dev->accel_z_adc;
    data[6] = dev->temp_adc;
    err = copy_to_user(buf, data, sizeof(data));
    return 0;
}

/* mpu6050操作函数 */
static const struct file_operations mpu6050_ops = {
    .owner = THIS_MODULE,
    .open = mpu6050_open,
    .read = mpu6050_read,
    .release = mpu6050_release,
};

void mpu6050_reginit(void)
{
    u8 value = 0;

    mpu6050_write_onereg(&mpu6050dev, MPU6050_RA_PWR_MGMT_1, 0x80);
    mdelay(50);
    mpu6050_write_onereg(&mpu6050dev, MPU6050_RA_PWR_MGMT_1, 0x01);
    mdelay(50);

    value = mpu6050_read_onereg(&mpu6050dev, MPU6050_RA_WHO_AM_I);
    printk("mpu6050 ID = %#x\r\n", value);

    mpu6050_write_onereg(&mpu6050dev, MPU6050_RA_SMPLRT_DIV, 0x00); /* 输出速
率是内部采样率 */
    mpu6050_write_onereg(&mpu6050dev, MPU6050_RA_GYRO_CONFIG, 0x18); /* 陀螺仪
±2000dps量程 */
    mpu6050_write_onereg(&mpu6050dev, MPU6050_RA_ACCEL_CONFIG, 0x18); /* 加速度
计±16G量程 */
    mpu6050_write_onereg(&mpu6050dev, MPU6050_RA_CONFIG, 0x04); /* 陀螺仪
低通滤波BW=20Hz */
    mpu6050_write_onereg(&mpu6050dev, MPU6050_RA_FF_THR, 0x04); /* 加速度计低通滤波
BW=21.2Hz */
}

```

```

    mpu6050_write_onereg(&mpu6050dev, MPU6050_RA_PWR_MGMT_2, 0x00);    /* 打开加
速度计和陀螺仪所有轴 */
    // mpu6050_write_onereg(&mpu6050dev, MPU6050_RA_PWR_MGMT_1,
MPU6050_PWR1_CYCLE_BIT);    /* 关闭低功耗 */
    mpu6050_write_onereg(&mpu6050dev, MPU6050_RA_FIFO_EN, 0x00);    /* 关闭
FIFO */
}

static int mpu6050_probe(struct i2c_client *client, const struct i2c_device_id
*id)
{
    int ret = 0;

    /* 1、构建设备号 */
    if (mpu6050dev.major) {
        mpu6050dev.devid = MKDEV(mpu6050dev.major, 0);
        register_chrdev_region(mpu6050dev.devid, MPU6050_CNT, MPU6050_NAME);
    } else {
        alloc_chrdev_region(&mpu6050dev.devid, 0, MPU6050_CNT, MPU6050_NAME);
        mpu6050dev.major = MAJOR(mpu6050dev.devid);
    }

    /* 2、注册设备 */
    cdev_init(&mpu6050dev.cdev, &mpu6050_ops);
    cdev_add(&mpu6050dev.cdev, mpu6050dev.devid, MPU6050_CNT);

    /* 3、创建类 */
    mpu6050dev.class = class_create(THIS_MODULE, MPU6050_NAME);
    if (IS_ERR(mpu6050dev.class)) {
        return PTR_ERR(mpu6050dev.class);
    }

    /* 4、创建设备 */
    mpu6050dev.device = device_create(mpu6050dev.class, NULL, mpu6050dev.devid,
NULL, MPU6050_NAME);
    if (IS_ERR(mpu6050dev.device)) {
        return PTR_ERR(mpu6050dev.device);
    }

    mpu6050dev.private_data = client; /* 设置私有数据 */

    mpu6050_reginit();
    return ret;
}

static int mpu6050_remove(struct i2c_client *client)
{
    /* 删除设备 */
    cdev_del(&mpu6050dev.cdev);
    unregister_chrdev_region(mpu6050dev.devid, MPU6050_CNT);

    /* 注销掉类和设备 */
    device_destroy(mpu6050dev.class, mpu6050dev.devid);
    class_destroy(mpu6050dev.class);
    return 0;
}

```

```

}

/* 传统匹配方式ID列表 */
static const struct i2c_device_id mpu6050_id[] = {
{"test,mpu6050", 0},
{}
};

/* 设备树匹配列表 */
static const struct of_device_id mpu6050_of_match[] = {
{ .compatible = "test,mpu6050" },
{ /* Sentinel */ }
};

static struct i2c_driver mpu6050_driver = {
    .probe = mpu6050_probe,
    .remove = mpu6050_remove,
    .driver = {
        .owner = THIS_MODULE,
        .name = "mpu6050",
        .of_match_table = mpu6050_of_match,
    },
    .id_table = mpu6050_id,
};

static int __init mpu6050_init(void)
{
    int ret = 0;
    ret = i2c_add_driver(&mpu6050_driver);
    return ret;
}

static void __exit mpu6050_exit(void)
{
    i2c_del_driver(&mpu6050_driver);
}

module_init(mpu6050_init);
module_exit(mpu6050_exit);
MODULE_LICENSE("GPL");
MODULE_AUTHOR("chen");

```

mpu6050Demo.c

```

#include "stdio.h"
#include "unistd.h"
#include "sys/types.h"
#include "sys/stat.h"
#include "sys/ioctl.h"
#include "fcntl.h"
#include "stdlib.h"
#include "string.h"
#include <poll.h>
#include <sys/select.h>

```

```

#include <sys/time.h>
#include <signal.h>
#include <fcntl.h>

int main(int argc, char *argv[])
{
    int fd;
    signed int buffer[7];
    signed int gyro_x_adc, gyro_y_adc, gyro_z_adc;
    signed int accel_x_adc, accel_y_adc, accel_z_adc;
    signed int temp_adc;

    float gyro_x_act, gyro_y_act, gyro_z_act;
    float accel_x_act, accel_y_act, accel_z_act;
    float temp_act;

    int ret = 0;
    char* filename = "/dev/mpu6050";
    fd = open(filename, O_RDWR);
    if(fd < 0) {
        printf("can't open file %s\r\n", filename);
        return -1;
    }

    while (1) {
        ret = read(fd, buffer, sizeof(buffer));
        if(ret == 0) {
            gyro_x_adc = buffer[0];
            gyro_y_adc = buffer[1];
            gyro_z_adc = buffer[2];
            accel_x_adc = buffer[3];
            accel_y_adc = buffer[4];
            accel_z_adc = buffer[5];
            temp_adc = buffer[6];

            gyro_x_act = (float)(gyro_x_adc) / 16.4;
            gyro_y_act = (float)(gyro_y_adc) / 16.4;
            gyro_z_act = (float)(gyro_z_adc) / 16.4;
            accel_x_act = (float)(accel_x_adc) / 2048;
            accel_y_act = (float)(accel_y_adc) / 2048;
            accel_z_act = (float)(accel_z_adc) / 2048;
            temp_act = ((float)(temp_adc) - 25) / 326.8 + 25;

            printf("\r\nADC value:\r\n");
            printf("gx = %d, gy = %d, gz = %d\r\n", gyro_x_adc, gyro_y_adc,
gyro_z_adc);
            printf("ax = %d, ay = %d, az = %d\r\n", accel_x_adc, accel_y_adc,
accel_z_adc);
            printf("temp = %d\r\n", temp_adc);
            printf("TRUE value:");
            printf("act gx = %.2f°/S, act gy = %.2f°/S, act gz = %.2f°/S\r\n",
gyro_x_act, gyro_y_act, gyro_z_act);
            printf("act ax = %.2fg, act ay = %.2fg, act az = %.2fg\r\n",
accel_x_act, accel_y_act, accel_z_act);
            printf("act temp = %.2f°C\r\n", temp_act);
        }
    }
}

```

好的，接下来修改设备树（2.5中会提供相关文件），关于spi的节点删除了哦，不然引脚会起冲突的，注意display-timings需要根据你的屏幕实际配置，up主使用的屏幕是480*272的，其余参数都是根据屏幕的参数进行填写的。

```

54
55 /* LCD 面板描述 */
56 panel: panel {
57     compatible = "simple-panel";
58     status = "okay";
59
60     bus-format = <MEDIA_BUS_FMT_RGB666_1X18>;
61     width-mm = <95>;
62     height-mm = <54>;
63
64     display-timings {
65         native-mode = <&timing0>;
66         timing0: timing0 {
67             clock-frequency = <90000000>;
68             hactive = <480>;
69             vactive = <272>;
70             hsync-len = <43>;
71             hback-porch = <2>;
72             hfront-porch = <2>;
73             vsync-len = <12>;
74             vback-porch = <2>;
75             vfront-porch = <2>;
76             hsync-active = <0>;
77             vsync-active = <0>;
78             de-active = <0>;
79             pixelclk-active = <0>;
80         };
81     };
82
83     port {
84         panel_in_rgb: endpoint {
85             remote-endpoint = <&rgb_out_panel>;
86         };
87     };
88 };

```

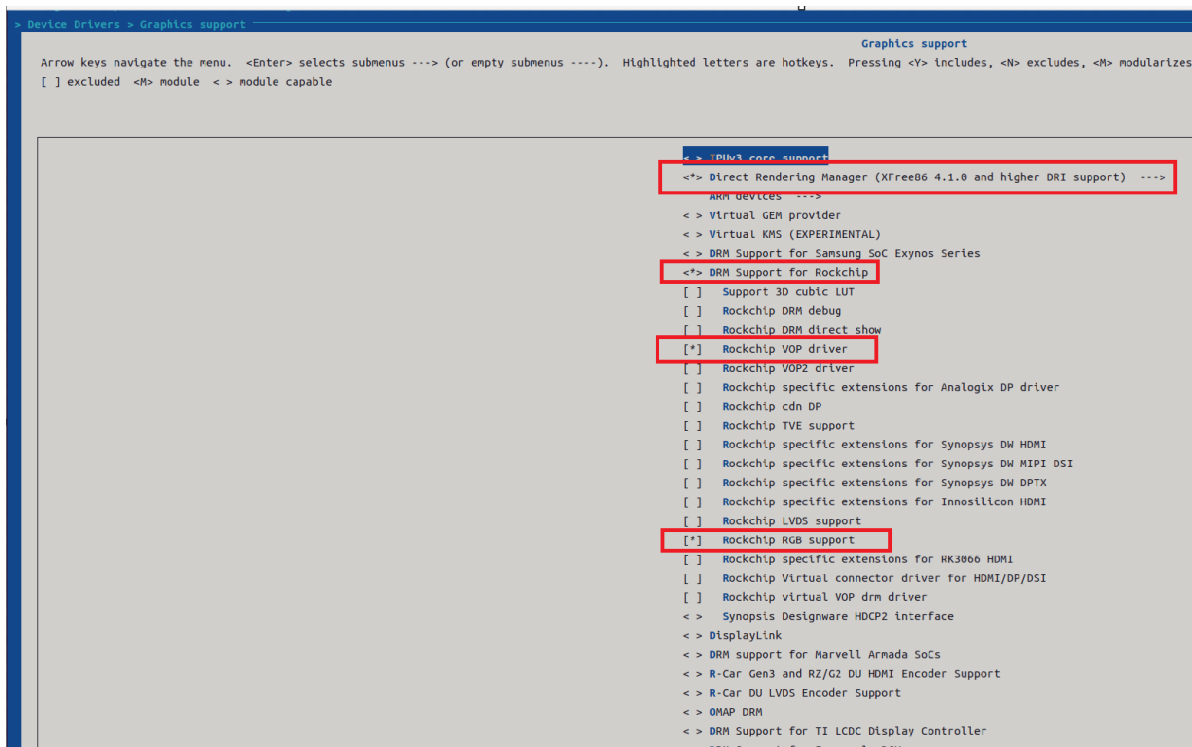
Plain Text Tab Width: 8 Ln 71, Col 58 INS

```

102 };
103
104 &rgb {
105     status = "okay";
106     pinctrl-names = "default";
107     pinctrl-0 = <&lcd_pins>;
108
109     ports {
110         rgb_out: port@1 {
111             reg = <1>;
112             #address-cells = <1>;
113             #size-cells = <0>;
114
115             rgb_out_panel: endpoint@0 {
116                 reg = <0>;
117                 remote-endpoint = <&panel_in_rgb>;
118             };
119         };
120     };
121 };
122
123
124 &display_subsystem {
125     status = "okay";
126 };
127
128
129
130 &rgb_in_vop {
131     status = "okay";
132 };
133
134 &route_rgb {
135     status = "okay";
136 };
137
138 &vop {
139     status = "okay";
140 };
141
142 /*****FLASH*****/

```

接下来修改内核配置，执行老样子命令。



这里使用了DRM驱动框架，简单介绍一下，DRM（Direct Rendering Manager）驱动是Linux内核图形子系统的核心组件之一，主要负责GPU和显示控制器（Display Controller）的统一管理。它为用户空间的图形栈（如Mesa、Xorg、Wayland）提供标准化接口，用于显存分配、缓冲区管理、渲染命令提交、显示模式设置、VSync同步等关键功能。相比早期的Framebuffer驱动，DRM不仅支持2D/3D硬件加速和多显示输出，还能确保多进程间访问显卡资源时的安全性和性能隔离。它通过内核模块与具体硬件（如Rockchip、Qualcomm、Intel、AMD的GPU或显示控制器）对应，使得不同硬件平台上的图形应用可以复用同一套上层接口，大幅减少开发难度并提升渲染效率。

编译并烧录。

2.5.2 大屏幕的相关操作

相信你的屏幕已经点亮了，接下来我们介绍操控屏幕的一些命令。

查看一下屏幕配置。

```
fbset -fb /dev/fb0
```

```
[root@luckfox root]# fbset -fb /dev/fb0

mode "480x272-0"
    # D: 0.000 MHz, H: 0.000 kHz, V: 0.000 Hz
    geometry 480 272 480 272 32
    timings 0 0 0 0 0 0 0
    accel true
    rgba 8/16,8/8,8/0,0/0
endmode
```

我准备了一些图片，和视频（2.5节资料中）来点亮试试。

显示图片

```
ffmpeg -i OIP2.jpg -vf "scale=480:272" -pix_fmt bgra -frames:v 1 -f fbdev /dev/fb0
```



播放视频

```
ffmpeg -re -i output_dong_480_272.mp4 -vf "scale=480:272,fps=60" -pix_fmt bgra -f fbdev /dev/fb0  
#或者(带音频输出)  
ffmpeg -re -i output_dong_480_272.mp4 -vf "scale=480:272,fps=60" -pix_fmt bgra -f fbdev /dev/fb0 -f wav - | aplay -D hw:0,0
```



2.5.3 设备树相关文件

rv1106g-luckfox-pico-pro-max.dts

```
// SPDX-License-Identifier: (GPL-2.0+ OR MIT)
/*
 * Copyright (c) 2022 Rockchip Electronics Co., Ltd.
 */

/dts-v1/;

#include "rv1106.dtsi"
#include "rv1106-evb.dtsi"
#include "rv1106-luckfox-pico-pro-max-ipc.dtsi"
#include <dt-bindings/input/input.h>
#include <dt-bindings/display/media-bus-format.h>

/ {
    model = "Luckfox Pico Max";
    compatible = "rockchip,rv1103g-38x38-ipc-v10", "rockchip,rv1106";
```

```

leds {
    compatible = "gpio-leds";

    heartbeat_led {
        label = "heartbeat";
        gpios = <&gpio1 RK_PA0 GPIO_ACTIVE_LOW>;
        default-state = "on";
        //max-brightness = <255>; // 手动设置最大亮度
        linux,default-trigger = "heartbeat";
        status = "okay"; // 这应该在每个 LED 节点下
    };
};

sdio_pwrseq: sdio-pwrseq {
    compatible = "mmc-pwrseq-simple";
    pinctrl-names = "default";
    pinctrl-0 = <&wifi_enable_h>;
    reset-gpios = <&gpio1 RK_PA2 GPIO_ACTIVE_LOW>;//随便给的
};

wireless_wlan: wireless-wlan {
    compatible = "wlan-platdata";
    wifi_chip_type = "rtl8723ds";
    pinctrl-names = "default";
    WIFI,host_wake_irq = <&gpio3 RK_PC6 GPIO_ACTIVE_HIGH>;
    status = "okay";
};

reserved_memory: reserved-memory {
    status = "okay";
    #address-cells = <1>;
    #size-cells = <1>;
    ranges;
    linux,cma {
        status = "okay";
        compatible = "shared-dma-pool";
        inactive;
        reusable;
        size = <0xA00000>; //10M
        linux,cma-default;
    };
};

/* LCD 面板描述 */
panel: panel {
    compatible = "simple-panel";
    status = "okay";

    bus-format = <MEDIA_BUS_FMT_RGB666_1X18>;
    width-mm = <95>;
    height-mm = <54>;

    display-timings {

```

```

        native-mode = <&timing0>;
        timing0: timing0 {
            clock-frequency = <9000000>;
            hactive = <480>;
            vactive = <272>;
            hsync-len = <43>;
            hback-porch = <2>;
            hfront-porch = <2>;
            vsync-len = <12>;
            vback-porch = <2>;
            vfront-porch = <2>;
            hsync-active = <0>;
            vsync-active = <0>;
            de-active = <0>;
            pixelclk-active = <0>;
        };
    };

    port {
        panel_in_rgb: endpoint {
            remote-endpoint = <&rgb_out_panel>;
        };
    };
};

&rgb {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&lcd_pins>;

    ports {
        rgb_out: port@1 {
            reg = <1>;
            #address-cells = <1>;
            #size-cells = <0>;

            rgb_out_panel: endpoint@0 {
                reg = <0>;
                remote-endpoint = <&panel_in_rgb>;
            };
        };
    };
};

&display_subsystem {
    status = "okay";
};

&rgb_in_vop {

```

```

        status = "okay";
};

&route_rgb {
    status = "okay";
};

&vop {
    status = "okay";
};

/*****FLASH*****/
&sfc {
    status = "okay";
    flash@0 {
        compatible = "spi-nand";
        reg = <0>;
        spi-max-frequency = <75000000>;
        spi-rx-bus-width = <4>;
        spi-tx-bus-width = <1>;
    };
};

/*****SDMMC*****/
&sdmmc {
    max-frequency = <50000000>;
    bus-width = <4>;
    #address-cells = <1>;
    #size-cells = <0>;
    cap-sd-highspeed;
    cap-sdio-irq;
    keep-power-in-suspend;
    non-removable;
    supports-sdio;
    mmc-pwrseq = <&sdio_pwrseq>;
    pinctrl-names = "default";
    pinctrl-0 = <&sdmmc0_clk &sdmmc0_cmd &sdmmc0_bus4 &sdmmc0_det>;
    status = "okay";

    wifi@1 {
        reg = <1>; // function 1 是标准 SDIO WLAN 的 function
        compatible = "realtek,rt18723ds";
        interrupt-parent = <&gpio3>;
        interrupts = <RK_PC6 IRQ_TYPE_LEVEL_HIGH>; // 与 wireless_wlan 中的保持一致
        rockchip,wl-host-wake-gpios = <&gpio3 RK_PC6 GPIO_ACTIVE_HIGH>; // 可选，
        用于唤醒
        status = "okay";
    };
};

/*****ETH*****/
&gmac {

```

```

        status = "disabled";
};

/*****USB*****/
&usbdrd_dwc3 {
    status = "okay";
    dr_mode = "peripheral";
};

&pinctrl {
    sdmmc0{
        sdmmc0_det: sdmmc0-det {
            rockchip,pins =
                /* sdmmc0_det */
                <3 RK_PA1 1 &pcfg_pull_down>;
        };
    };

    wireless-wlan {
        wifi_host_wake_irq: wifi-host-wake-irq {
            rockchip,pins = <3 RK_PC6 RK_FUNC_GPIO &pcfg_pull_none>;
        };
    };

    wifi-enable{
        wifi_enable_h:wifi-enable-h{
            rockchip,pins = <1 RK_PA2 RK_FUNC_GPIO &pcfg_pull_none>;
        };
    };

};

/*****I2C*****/
/* I2C3_M1 */
&i2c3 {
    status = "disabled";
    clock-frequency = <100000>;
};

/* I2C1_M0 */
&i2c1 {
    status = "okay";
    clock-frequency = <400000>;
    pinctrl-0=<&i2c1m0_xfer>;
    pinctrl-names = "default";
    #address-cells = <1>;
    #size-cells = <0>;

    mpu6050:mpu6050@68{
        compatible= "test,mpu6050";
        reg=<0x68>;
    };
};

```

```

        status="okay";

    };

};

/*****UART*****/
/* UART3_M1 */
&uart3 {
    status = "disabled";
};

/* UART4_M1 */
&uart4 {
    status = "disabled";
};

/*****RTC*****/
&rtc {
    status = "okay";
};

```

2.6 wifi无线网卡（RTL8723DS）

这个就上难度了哦。

2.6.1 设备树及内核修改

我们先来查看板子的硬件资源，可以看出使用的是SDIO接口，这个接口同时也是wifi的接口，使用了它，就意味着无法使用tf卡了。

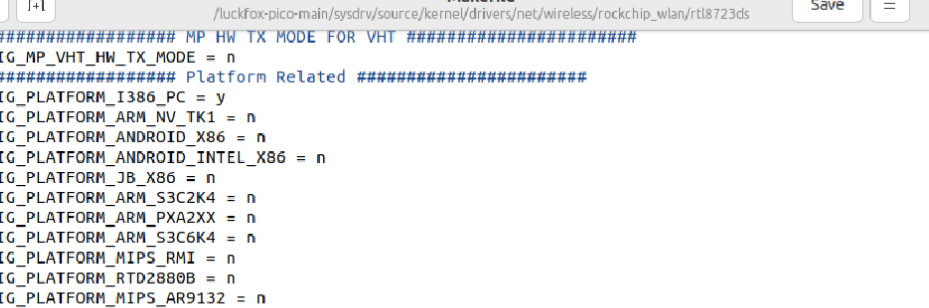

```
wifi_chip_type = "rt18723ds"
```

指定 Wi-Fi 芯片型号是 RTL8723DS (Realtek SDIO+BT二合一芯片), 这个信息给驱动或固件加载程序用, 方便加载对应固件和模块.

```
WIFI,host_wake_irq = <&gpio3 RK_PC6 GPIO_ACTIVE_HIGH>;
```

指定 Wi-Fi 模块唤醒主机的 GPIO 引脚 (Host Wake IRQ), 当 Wi-Fi 有数据时, 会拉高这个 GPIO 通知 CPU 从休眠唤醒。

接着修改linux内核配置，及代码，我们切换到 /luckfox-pico-main/sysdrv/source/kernel/drivers/net/wireless/rockchip_wlan，这个路径下，把我写好的rtl8723ds文件夹替换过去，随后我们进入rtl8723ds中，修改makefile，和kconfig（我提供的是已经修改好了的），添加如下配置。



```
root@chen6666697:~# virtual-machine: /luckfox-pico-main/sysdrv/source/kernel/drivers/net/wireless/rockchip_wlan# ls
.config  .makefile  modules.order

root@chen6666697:~# virtual-machine: /luckfox-pico-main/sysdrv/source/kernel/drivers/net/wireless/rockchip_wlan/rtl8723ds# cat Makefile
##### MP HW TX MODE FOR VHT #####
90 CONFIG_MP_VHT_HW_TX_MODE = n
91 ##### Platform Related #####
92 CONFIG_PLATFORM_I386_PC = y
93 CONFIG_PLATFORM_ARM_NV_TK1 = n
94 CONFIG_PLATFORM_ANDROID_X86 = n
95 CONFIG_PLATFORM_ANDROID_INTEL_X86 = n
96 CONFIG_PLATFORM_JB_X86 = n
97 CONFIG_PLATFORM_ARM_S3C2K4 = n
98 CONFIG_PLATFORM_ARM_PXA2XX = n
99 CONFIG_PLATFORM_ARM_S3C6K4 = n
100 CONFIG_PLATFORM_MIPS_RMI = n
101 CONFIG_PLATFORM_RTD2880B = n
102 CONFIG_PLATFORM_MIPS_AR9132 = n
103 CONFIG_PLATFORM_RTK_DMP = n
104 CONFIG_PLATFORM_MIPS_PLM = n
105 CONFIG_PLATFORM_MSTAR389 = n
106 CONFIG_PLATFORM_MT53XX = n
107 CONFIG_PLATFORM_ARM_MX51_241H = n
108 CONFIG_PLATFORM_FS_MX61 = n
109 CONFIG_PLATFORM_ACTIONS_ATJ227X = n
110 CONFIG_PLATFORM_TEGRA3_CARDHU = n
111 CONFIG_PLATFORM_TEGRA4_DALMORE = n
112 CONFIG_PLATFORM_ARM_TCC8900 = n
113 CONFIG_PLATFORM_ARM_TCC8920 = n
114 CONFIG_PLATFORM_ARM_TCC8920_JB42 = n
115 CONFIG_PLATFORM_ARM_TCC8930_JB42 = n
116 CONFIG_PLATFORM_ARM_RK2818 = n
117 CONFIG_PLATFORM_ARM_RK3066 = n
118 CONFIG_PLATFORM_ARM_RK3188 = n
119 CONFIG_PLATFORM_ARM_RV1106 = y
120 CONFIG_PLATFORM_ARM_ORBITER = n
121 CONFIG_PLATFORM_ARM_TI_PANDA = n
122 CONFIG_PLATFORM_MIPS_JZ4760 = n
123 CONFIG_PLATFORM_DMP_PHILIPS = n
124 CONFIG_PLATFORM_MSTAR_TITANIA12 = n
125 CONFIG_PLATFORM_MSTAR = n
126 CONFIG_PLATFORM_SZEBOOK = n
```

```
Open ▾ [?] Makefile /luckfox-pico-main/sysdrv/source/kernel/drivers/net/wireless/rockchip_wlan/rtl8723ds Save [≡] [—] [□] [×]
1342 # default setting for power control
1343 EXTRA_CFLAGS += -DRTW_ENABLE_WIFI_CONTROL_FUNC
1344 EXTRA_CFLAGS += -DRTW_SUPPORT_PLATFORM_SHUTDOWN
1345 # default setting for special function
1346 ARCH := arm
1347 CROSS_COMPILE := arm-rockchip830-linux-uclibcgnueabi-
1348 KSRC := /luckfox-pico-main/sysdrv/source/kernel
1349 MODULE_NAME := wlan
1350 endif
1351
1352 ifeq ($(CONFIG_PLATFORM_ARM_RV1106), y)
1353 EXTRA_CFLAGS += -DCONFIG_LITTLE_ENDIAN
1354 EXTRA_CFLAGS += -DCONFIG_CONCURRENT_MODE
1355 EXTRA_CFLAGS += -DCONFIG_IOCTL_CFG80211 -DRTW_USE_CFG80211_STA_EVENT
1356
1357 ifeq ($(CONFIG_USB_HCI), y)
1358 EXTRA_CFLAGS += -DCONFIG_USE_USB_BUFFER_ALLOC_TX -DCONFIG_FIX_NR_BULKIN_BUFFER
1359 endif
1360
1361 ARCH ?= arm
1362 CROSS_COMPILE ?= arm-rockchip830-linux-uclibcgnueabi-
1363 KSRC ?= /luckfox-pico-main/sysdrv/source/kernel
1364 MODULE_NAME ?= rtl8723ds
1365 endif
1366
1367
1368
1369 ifeq ($(CONFIG_PLATFORM_ARM_RK3066), y)
1370 EXTRA_CFLAGS += -DCONFIG_PLATFORM_ARM_RK3066
1371 EXTRA_CFLAGS += -DRTW_ENABLE_WIFI_CONTROL_FUNC
1372 EXTRA_CFLAGS += -DCONFIG_LITTLE_ENDIAN
1373 EXTRA_CFLAGS += -DCONFIG_CONCURRENT_MODE
1374 EXTRA_CFLAGS += -DCONFIG_IOCTL_CFG80211
1375 ifeq ($(CONFIG_SDIO_HCI), y)
1376 EXTRA_CFLAGS += -DRTW_SUPPORT_PLATFORM_SHUTDOWN
1377 endif
1378 EXTRA_CFLAGS += -fno-pic
1379 ARCH := arm
```

```
Open ▾ [?] Kconfig /luckfox-pico-main/sysdrv/source/kernel/drivers/net/wireless/rockchip_wlan/rtl8723ds Save [≡] [—] [□] [×]
1 config RTL8723DS
2     tristate "Realtek 8723D SDIO or SPI WiFi"
3     help
4     Help message of RTL8723DS
```

接着我们还要修改代码，代码一很多处编译报错，这里就不演示了，我们回到上一级，修改 rockchip_wlan 下的 makefile 和 kconfig。

```
Open ▾ [?] Makefile /luckfox-pico-main/sysdrv/source/kernel/drivers/net/wireless/rockchip_wlan Save [≡] [—] [□] [×]
1 # SPDX-License-Identifier: GPL-2.0
2 obj-$(CONFIG_BCM2HD) += rkwifi/
3 obj-$(CONFIG_CYW_BCM2HD) += cywdhd/
4 obj-$(CONFIG_INFINEON_DHD) += infineon/
5 obj-$(CONFIG_RTL8723DS) += rtl8723ds/
```

```
Open [v] [K] Kconfig /luckfox-pico-main/sysdrv/source/kernel/drivers/net/wireless/rockchip_wlan Save [≡] [–] [□]
6 select MAC80211
7 select MAC80211
8 help
9 Enable compatible Wifi drivers for Rockchip platform.
0
1 if WL_ROCKCHIP
2
3 config WIFI_BUILD_MODULE
4 bool "build wifi ko modules"
5 default n
6 help
7 Wifi drivers will compile as ko module
8
9 config WIFI_LOAD_DRIVER_WHEN_KERNEL_BOOTUP
0 bool "Wifi load driver when kernel bootup"
1 default n
2 help
3 Wifi driver will be load (use late_initcall) when kernel bootup
4
5 config WIFI_GENERATE_RANDOM_MAC_ADDR
6 bool "Wifi generate random mac address and save to vendor storage for cob chip"
7 default n
8 help
9 Wifi generate random mac address and save to vendor storage for cob chip
0
1 menuconfig BCM2HD
2 bool "Broadcom Wireless Device Driver Support"
3 default y
4
5 if BCM2HD
6 source "drivers/net/wireless/rockchip_wlan/rkwifi/Kconfig"
7 endif
8
9 source "drivers/net/wireless/rockchip_wlan/cywdhd/Kconfig"
0 source "drivers/net/wireless/rockchip_wlan/infineon/Kconfig"
1 source "drivers/net/wireless/rockchip_wlan/rtl8723ds/Kconfig"
2
3 endif # WL_ROCKCHIP
```

好了，终于修改好了，接下来我们就开始配置linux内核吧，将对应的功能模块编译设置成*，进入将RTL8723DS编译成模块。

```
config - Linux/arm 5.10.160 Kernel Configuration
Search (WL_ROCKCHIP) > Wireless LAN

Wireless LAN

Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----). Highlighted letters are hotkeys. Pressing <Y>
includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] built-in [ ]
excluded <M> module <> module capable

^(-)
[ ] Microchip devices
[ ] Ralink devices
[ ] Realtek devices
[ ] Redpine Signals Inc devices
[ ] STMicroelectronics devices
[ ] Texas Instrument devices
[ ] ZyDAS devices
[ ] Quantenna wireless cards support
[*] Rockchip Wireless LAN support --->
<> Simulated radio testing tool for mac80211
<> Wireless RNDIS USB support
<> Wifi wrapper for ethernet drivers

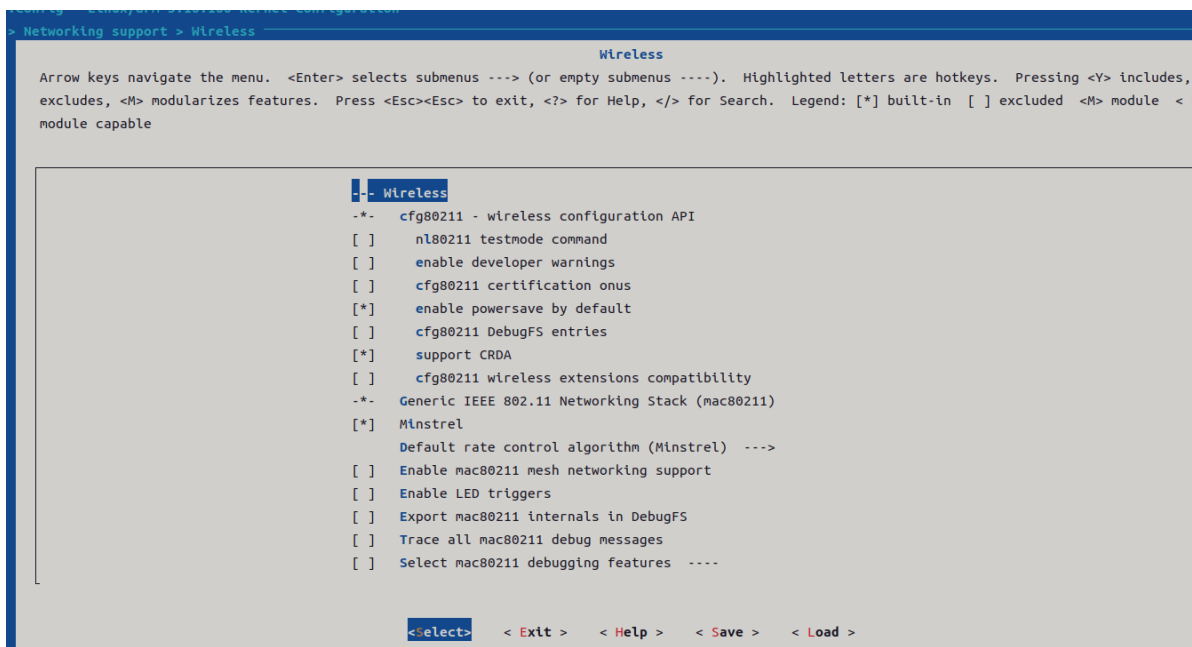
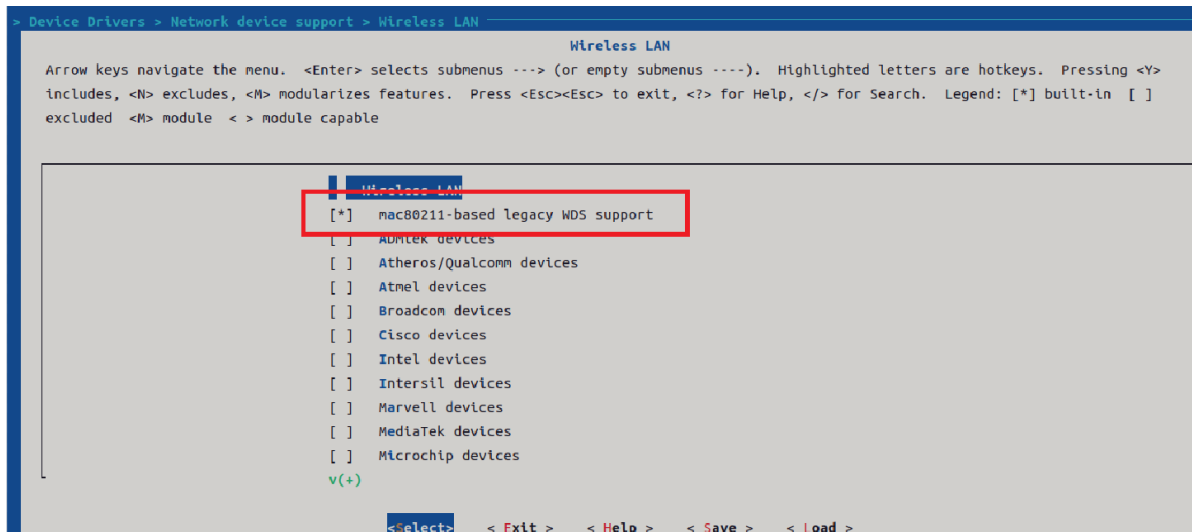
<Select> < Exit > < Help > < Save > < Load >
```

```

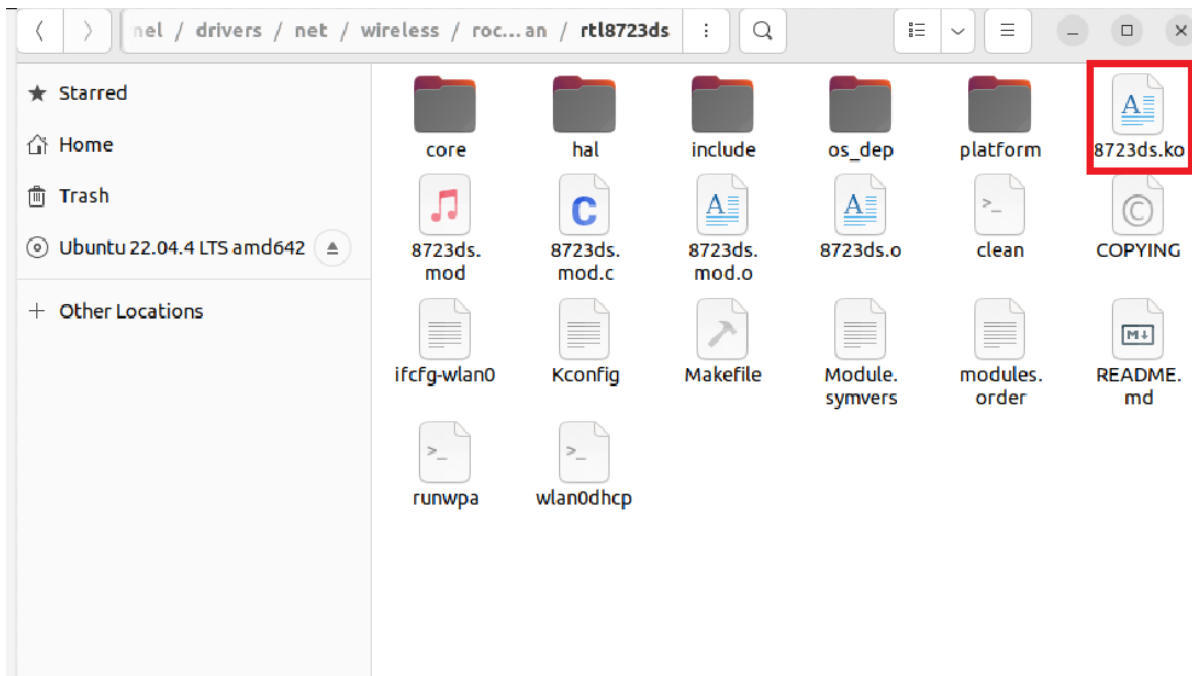
--- Rockchip Wireless LAN support
[*]  build wifi ko modules
[ ]  Wifi load driver when kernel bootup
[ ]  Wifi generate random mac address and save to vendor storage for cob chip
[ ]  Broadcom Wireless Device Driver Support ----
< > Cypress wireless sdio cards support
< > Infineon wireless sdio cards support
<M> Realtek 8723D SDIO or SPI WiFi

```

wifi的驱动还需要的其他库的支持，cfg80211库很重要，没有它，wifi网卡挂载会报错。



接下来进行编译，烧录即可，记得把编译好的文件放到开发板备用。



2.6.2 wifi的使用

好了我们开加载驱动,可以看到wlan0和wlan1 这代表我们的无线网卡已经成功加载了

```
insmod 8723ds.ko
```

```
[root@luckfox root]# insmod 8723ds.ko
[root@luckfox root]# ifconfig -a
lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:684  errors:0  dropped:0  overruns:0  frame:0
          TX packets:684  errors:0  dropped:0  overruns:0  carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:50508 (49.3 KiB)  TX bytes:50508 (49.3 KiB)

usb0      Link encap:Ethernet  HWaddr 46:0A:40:91:7A:03
          inet addr:172.32.0.93  Bcast:172.32.255.255  Mask:255.255.0.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:232  errors:0  dropped:69  overruns:0  frame:0
          TX packets:41  errors:0  dropped:0  overruns:0  carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:37754 (36.8 KiB)  TX bytes:6548 (6.3 KiB)

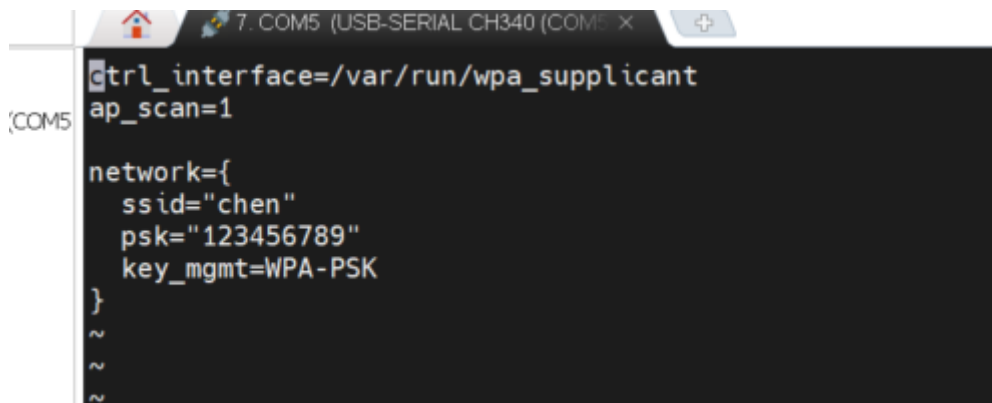
wlan0     Link encap:Ethernet  HWaddr 34:75:63:29:C4:F3
          BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0  errors:0  dropped:0  overruns:0  frame:0
          TX packets:0  errors:0  dropped:0  overruns:0  carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

wlan1     Link encap:Ethernet  HWaddr 36:75:63:29:C4:F3
          BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0  errors:0  dropped:0  overruns:0  frame:0
          TX packets:0  errors:0  dropped:0  overruns:0  carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```

接下来我们来连接网络，配置下联网的文件，ssid是你网络的名字（确保你的网络是2.4g的），psk 是密码。

vi 命令的使用，麻烦百度下，这里不过多讲解。

```
vi /etc/wpa_supplicant.conf
```



```
ctrl_interface=/var/run/wpa_supplicant
ap_scan=1

network={
    ssid="chen"
    psk="123456789"
    key_mgmt=WPA-PSK
}
~
~
~
```

执行命令，如图所示，网络已经解决成功了，网卡驱动对up来讲是最难的，耗费了up许多时间。

```
wpa_supplicant -B -i wlan0 -c /etc/wpa_supplicant.conf
# 分配ip地址
udhcpc -i wlan0
# 网络测试
ping www.baidu.com
```

```
[root@luckfox root]# wpa_supplicant -B -i wlan0 -c /etc/wpa_supplicant.conf
Successfully initialized wpa_supplicant
rfkill: Cannot open RFKILL control device
[root@luckfox root]# udhcpc -i wlan0
udhcpc: started, v1.36.1
udhcpc: broadcasting discover
udhcpc: broadcasting select for 192.168.48.236, server 192.168.48.18
udhcpc: lease of 192.168.48.236 obtained from 192.168.48.18, lease time 3599
deleting routers
adding dns 192.168.48.18
[root@luckfox root]# ping www.baidu.com
PING www.baidu.com (183.2.172.17): 56 data bytes
64 bytes from 183.2.172.17: seq=0 ttl=51 time=44.652 ms
64 bytes from 183.2.172.17: seq=1 ttl=51 time=40.909 ms
64 bytes from 183.2.172.17: seq=2 ttl=51 time=34.567 ms
64 bytes from 183.2.172.17: seq=3 ttl=51 time=34.438 ms
64 bytes from 183.2.172.17: seq=4 ttl=51 time=32.754 ms
^C
--- www.baidu.com ping statistics ---
5 packets transmitted, 5 packets received, 0% packet loss
round-trip min/avg/max = 32.754/37.464/44.652 ms
```

我们可以再执行，可以看到我们的网络已经正确连接了，显示了ip地址。

```
ifconfig -a
```

```

9 packets transmitted, 9 packets received, 0% packet loss
round-trip min/avg/max = 32.754/37.464/44.652 ms
[root@luckfox root]# ifconfig -a
lo                Link encap:Local Loopback
                  inet addr:127.0.0.1  Mask:255.0.0.0
                  UP LOOPBACK RUNNING  MTU:65536  Metric:1
                  RX packets:1516  errors:0  dropped:0  overruns:0  frame:0
                  TX packets:1516  errors:0  dropped:0  overruns:0  carrier:0
                  collisions:0 txqueuelen:1000
                  RX bytes:112076 (109.4 KiB)  TX bytes:112076 (109.4 KiB)

usb0              Link encap:Ethernet  HWaddr 46:0A:40:91:7A:03
                  inet addr:172.32.0.93  Bcast:172.32.255.255  Mask:255.255.0.0
                  UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
                  RX packets:346  errors:0  dropped:98  overruns:0  frame:0
                  TX packets:69  errors:0  dropped:0  overruns:0  carrier:0
                  collisions:0 txqueuelen:1000
                  RX bytes:59882 (58.4 KiB)  TX bytes:12986 (12.6 KiB)

wlan0             Link encap:Ethernet  HWaddr 34:75:63:29:C4:F3
                  inet addr:192.168.48.236  Bcast:192.168.48.255  Mask:255.255.255.0
                  UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
                  RX packets:37  errors:0  dropped:4  overruns:0  frame:0
                  TX packets:80  errors:0  dropped:0  overruns:0  carrier:0
                  collisions:0 txqueuelen:1000
                  RX bytes:7500 (7.3 KiB)  TX bytes:11794 (11.5 KiB)

wlan1             Link encap:Ethernet  HWaddr 36:75:63:29:C4:F3
                  BROADCAST MULTICAST  MTU:1500  Metric:1
                  RX packets:0  errors:0  dropped:0  overruns:0  frame:0
                  TX packets:0  errors:0  dropped:0  overruns:0  carrier:0
                  collisions:0 txqueuelen:1000
                  RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

[root@luckfox root]#

```

如果出现这样的情况，是因为lib80211 库没有加载，必须确保它被编译进内核中。

```

[root@luckfox root]# insmod 8723ds.ko
[ 293.955884] 8723ds: Unknown symbol cfg80211_inform_bss_frame_data (err -2)
[ 293.956028] 8723ds: Unknown symbol cfg80211_scan_done (err -2)
[ 293.956093] 8723ds: Unknown symbol cfg80211_remain_on_channel_expired (err -2)
insmod: can't insert '8723ds.ko': unknown symbol in module, or unknown parameter
[ 293.956158] 8723ds: Unknown symbol cfg80211_new_sta (err -2)
[root@luckfox root]# [ 293.956179] 8723ds: Unknown symbol cfg80211_disconnected (err -2)
[ 293.956216] 8723ds: Unknown symbol wiphy_new_nm (err -2)
[ 293.956231] 8723ds: Unknown symbol cfg80211_ready_on_channel (err -2)
[ 293.956257] 8723ds: Unknown symbol wiphy_register (err -2)
[ 293.956271] 8723ds: Unknown symbol __cfg80211_alloc_reply_skb (err -2)
[ 293.956283] 8723ds: Unknown symbol __cfg80211_alloc_event_skb (err -2)
[ 293.956297] 8723ds: Unknown symbol cfg80211_put_bss (err -2)
[ 293.956305] 8723ds: Unknown symbol cfg80211_roamed (err -2)
[ 293.956336] 8723ds: Unknown symbol ieee80211_get_channel_khz (err -2)
[ 293.956363] 8723ds: Unknown symbol __cfg80211_send_event_skb (err -2)
[ 293.956388] 8723ds: Unknown symbol cfg80211_ibss_joined (err -2)
[ 293.956421] 8723ds: Unknown symbol cfg80211_michael_mic_failure (err -2)
[ 293.956432] 8723ds: Unknown symbol wiphy_apply_custom_regulatory (err -2)
[ 293.956447] 8723ds: Unknown symbol cfg80211_rx_mgmt_khz (err -2)

```

你可能会发现为啥会有 wlan0 和 wlan1，是因为无线网卡 **同时创建了两个逻辑接口**，或者是内核驱动/固件支持 **多角色 (multi-role) 模式**，wlan0：用于 STA (Station，即连接 AP) 模式，wlan1：用于 AP (Access Point，即热点) 模式，我们使用 wlan0 上网即可。

2.6.3 makefile及kconfig配置

Kconfig

```

# SPDX-License-Identifier: GPL-2.0
menuconfig WL_ROCKCHIP
    tristate "Rockchip wireless LAN support"
    select WIRELESS_EXT
    select WEXT_PRIV

```

```

select CFG80211
select MAC80211
help
    Enable compatible wifi drivers for Rockchip platform.

if WL_ROCKCHIP

config WIFI_BUILD_MODULE
    bool "build wifi ko modules"
    default n
    help
        wifi drivers will compile as ko module

config WIFI_LOAD_DRIVER_WHEN_KERNEL_BOOTUP
    bool "wifi load driver when kernel bootup"
    default n
    help
        wifi driver will be load (use late_initcall) when kernel bootup

config WIFI_GENERATE_RANDOM_MAC_ADDR
    bool "wifi generate random mac address and save to vendor storage for cob
chip"
    default n
    help
        wifi generate random mac address and save to vendor storage for cob chip

menuconfig BCM2HD
    bool "Broadcom wireless Device Driver Support"
    default y

if BCM2HD
source "drivers/net/wireless/rockchip_wlan/rkwifi/kconfig"
endif

source "drivers/net/wireless/rockchip_wlan/cywdhd/kconfig"
source "drivers/net/wireless/rockchip_wlan/infineon/kconfig"
source "drivers/net/wireless/rockchip_wlan/rtl8723ds/kconfig"

endif # WL_ROCKCHIP

```

Makefile

```

# SPDX-License-Identifier: GPL-2.0
obj-$(CONFIG_BCM2HD) += rkwifi/
obj-$(CONFIG_CYW_BCM2HD) += cywdhd/
obj-$(CONFIG_INFINEON_DHD) += infineon/
obj-$(CONFIG_RTL8723DS) += rtl8723ds/

```

2.7 音频驱动

这块官方已经适配好了，所以我们可以直接使用，不需要额外配置。

2.7.1 扬声器

下面我们使用命令将MP4的音频数据剥离出来，并播放音频

```
ffmpeg -i input.mp4 -vn -acodec pcm_s16le output.wav  
  
aplay -D hw:0,0 output.wav
```

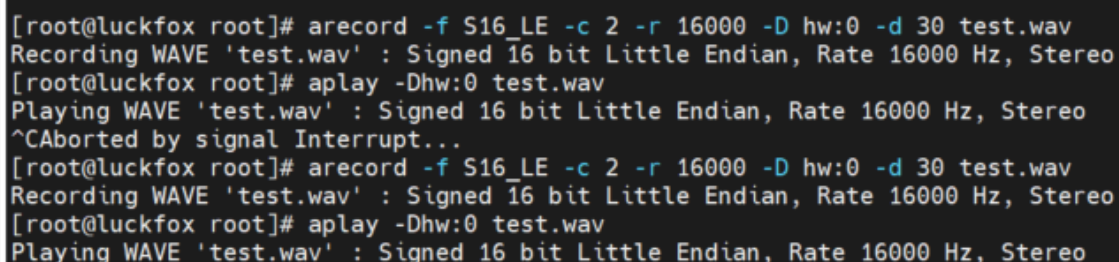
现在你应该已经听到动听的音乐声了吧？

2.7.2 麦克风

下面我们来使用麦克风,他将录制一段音频，并将数据保存到test.wav，使用aplay 播放音频test.wav。

```
arecord -f S16_LE -c 2 -r 16000 -D hw:0 -d 30 test.wav  
  
aplay -Dhw:0 test.wav
```

相信你已经听到了你录制下来的声音了。



```
[root@luckfox root]# arecord -f S16_LE -c 2 -r 16000 -D hw:0 -d 30 test.wav  
Recording WAVE 'test.wav' : Signed 16 bit Little Endian, Rate 16000 Hz, Stereo  
[root@luckfox root]# aplay -Dhw:0 test.wav  
Playing WAVE 'test.wav' : Signed 16 bit Little Endian, Rate 16000 Hz, Stereo  
^CAborted by signal Interrupt...  
[root@luckfox root]# arecord -f S16_LE -c 2 -r 16000 -D hw:0 -d 30 test.wav  
Recording WAVE 'test.wav' : Signed 16 bit Little Endian, Rate 16000 Hz, Stereo  
[root@luckfox root]# aplay -Dhw:0 test.wav  
Playing WAVE 'test.wav' : Signed 16 bit Little Endian, Rate 16000 Hz, Stereo
```

we support MobaXterm by subscribing to the professional edition here: <https://mobaxterm.mobatek.net>

`arecord`

ALSA 提供的录音工具（对应 `aplay` 的录音版）。

`-f S16_LE`

指定音频数据格式：

- `S16` = Signed 16-bit（有符号 16 位）
- `LE` = Little Endian（小端存储）

这是一种常见的 PCM 格式。

`-c 2`

声道数为 2，即立体声。

`-r 16000`

采样率 16000 Hz，也就是每秒采集 16000 个采样点，适合语音处理。

`-D hw:0`

指定使用第 0 号声卡的硬件接口（`hw:卡号`），绕过软件混音，直接访问硬件。

-d 30

录音时长 30 秒。

test.wav

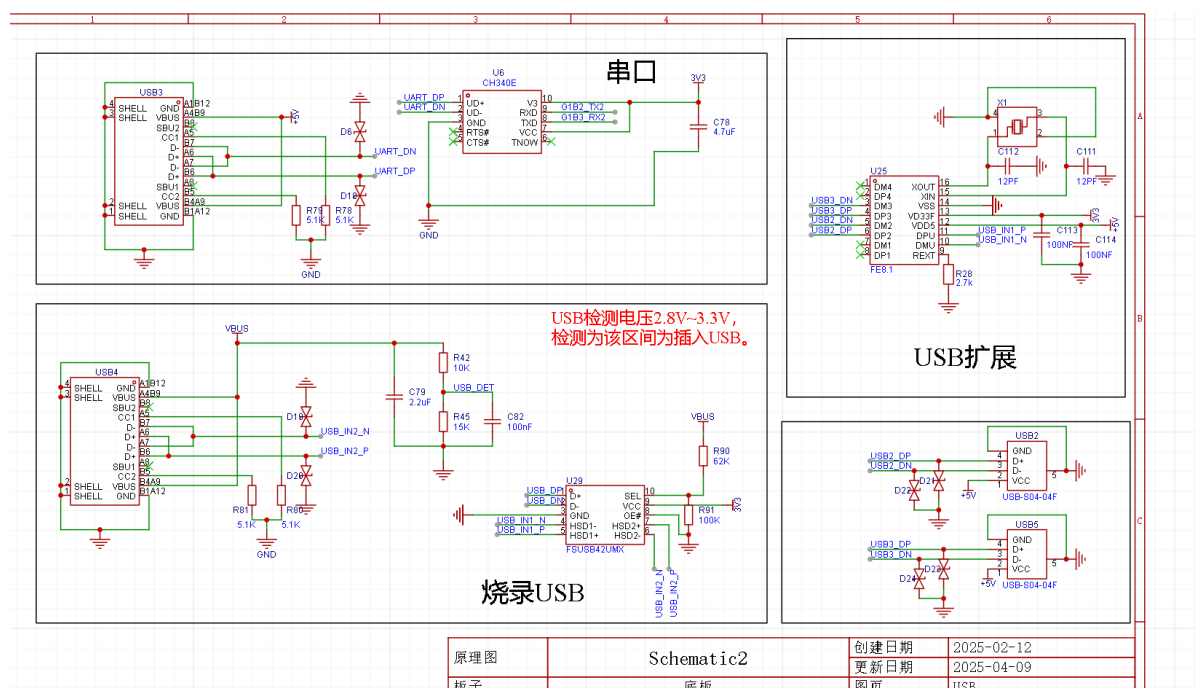
保存输出文件为 WAV 格式，名字叫 test.wav

2.8 usb驱动

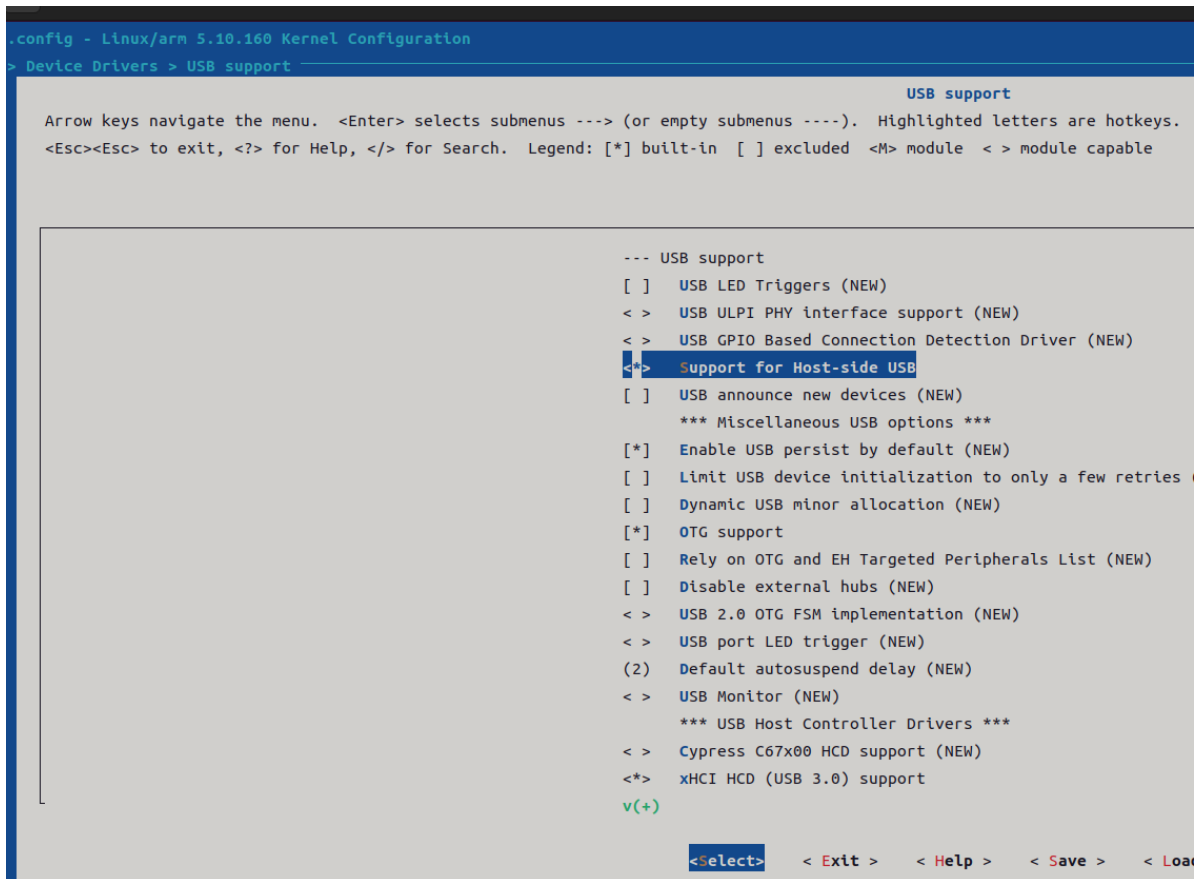
2.8.1 设备树及内核修改

这节略讲usb驱动的加载，实际上在linux内核中，usb驱动是默认开启的，我们一般不怎么用配置，我们还是先看下硬件设计。

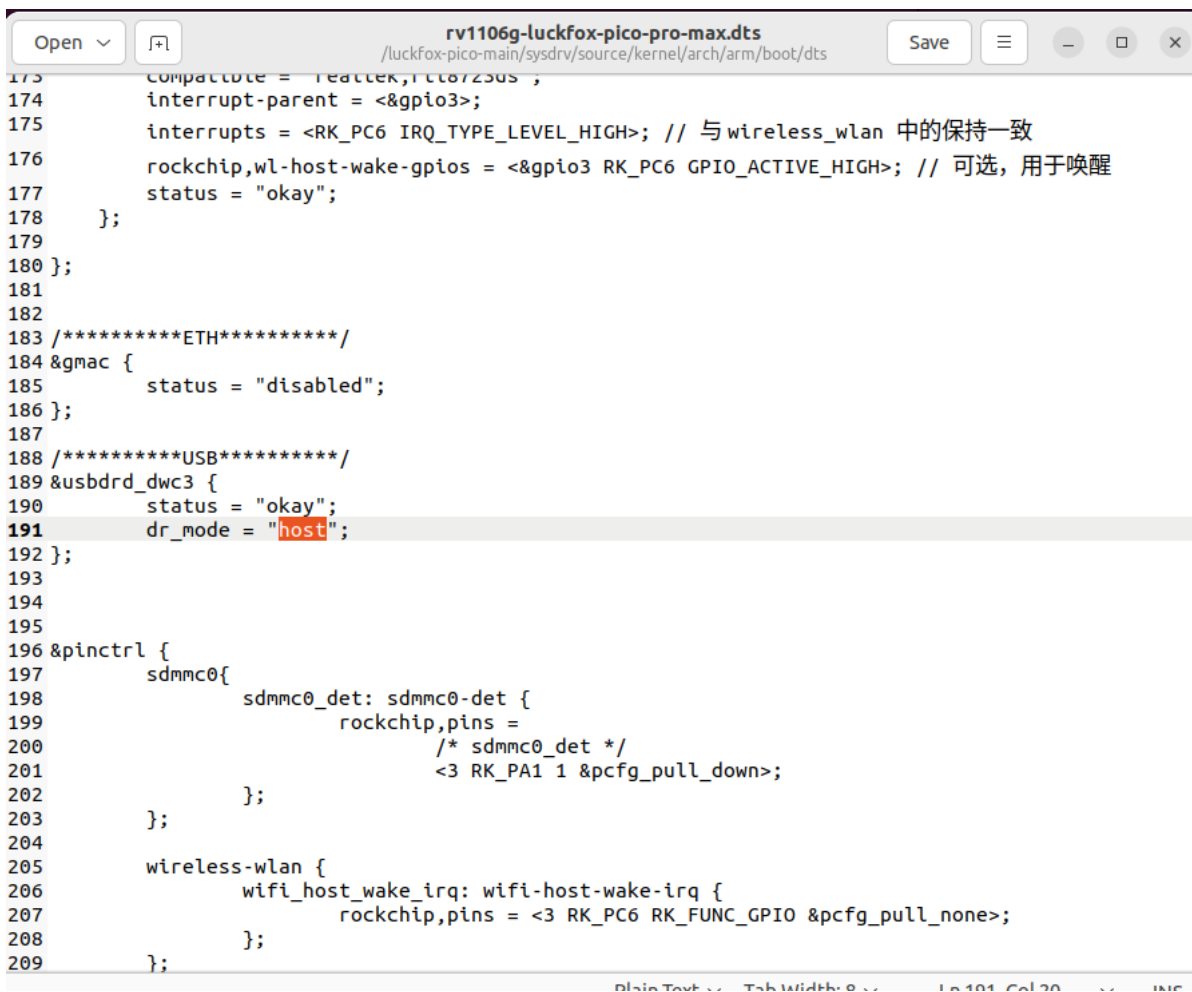
可以看出，我们的烧录type-c 口接入时，usb switch会被选择，此时我们可以进行烧录，或adb 调试。但是开启adb的话，即便选择了usb hub，usb也无法识别设备，也就是adb作为从机，无法控制usb。所以我们要把usb设置主机模式，但是每一次去设置内核然后烧录，未免太麻烦了，因此我们需要一种方法，这也是这节主要的目的。



简单配置下内核。



配置下设备树，将usb驱动模式设置为主机，编译，烧录。



```
cat /sys/devices/platform/ff3e0000.usb2-phy/otg_mode
```

```
[root@luckfox root]# cat /sys/devices/platform/ff3e0000.usb2-phy/otg_mode
host
```

可以看到usb驱动模式已经成功设置为 host模式。接下来我们插入一个usb设备，记得我们的烧录接口不能再接type了哦。

可以看到已经识别了我的U盘。

```
[root@luckfox root]#
[root@luckfox root]# cat /sys/devices/platform/ff3e0000.usb2-phy/otg_mode
host
[root@luckfox root]# [ 72.084234] usb 1-1.3: new high-speed USB device number 3 using xhci-hcd
[ 72.219303] usb-storage 1-1.3:1.0: USB Mass Storage device detected
[ 72.225871] scsi host0: usb-storage 1-1.3:1.0
[ 73.285112] scsi 0:0:0:0: Direct-Access          HIKSEMI            2.00 PQ: 0 ANSI: 4
[ 73.292330] sd 0:0:0:0: [sda] 245760000 512-byte logical blocks: (126 GB/117 GiB)
[ 73.292690] sd 0:0:0:0: [sda] Write Protect is off
[ 73.293152] sd 0:0:0:0: [sda] No Caching mode page found
[ 73.293192] sd 0:0:0:0: [sda] Assuming drive cache: write through
[ 73.299124] sda: sda1
[ 73.303705] sd 0:0:0:0: [sda] Attached SCSI removable disk
[root@luckfox root]#
```

挂载u盘：

```
mount /dev/sda1 /mnt
```

```
OIP3.jpg          output_bad_apple.mp4
[root@luckfox root]# mount /dev/sda1 /mnt
[ 1624.900073] FAT-fs (sda1): Volume was not properly unmounted. Some data may be corrupt. Please run fsck.
[root@luckfox root]# ls /mnt/
16_16点阵资料包 V1.1
234.py
Anaconda3-2023.07-0-Windows-x86_64.exe
Hi BackupSetup(V1.1.2Build220415).exe
Hi Backup使用指南.pdf
System Volume Information
depthcenterpic.py
openni2_opencv.py
yolo
乌托邦探险之旅3.2fix.zip
好看的.jpg
资料
[root@luckfox root]#
```

可以看到，我们有一张好看的图片.jpg，我们使用命令查看下：

```
ffmpeg -i /mnt/好看的.jpg -vf "scale=480:272" -pix_fmt bgra -frames:v 1 -f fbdev /dev/fb0
```

偷偷的看

如果把 USB 控制器切到 host 模式，但系统里 **没有 host 驱动加载**，这个设备就不存在于 UDC (peripheral) 框架里，也就 **无法再 bind**。

同理，如果你切回 peripheral 模式，但 UDC 驱动还没加载或者被移除，也 **无法 bind**。

所以：**bind/unbind 只能在驱动存在且设备当前模式匹配时生效**，一旦模式切错或者驱动没加载，就没法通过写 sysfs 重新绑定。

如果我们要让它重新加载，就只能重启，但是重启之后又回到老样子了，因为我们的驱动是运行在内存里的。这意味着我们要写入 flash 或者镜像中。事实上，linux 内核加载设备树会把设备树切片，供软件驱动使用。好在 linux 支持动态编译 linux 设备树切片，也就是说我们只要把驱动写入对应的设备树分区就好了，我们来看一个脚本(其中注释写的很清楚了，感兴趣的可以看看)。这个脚本是从官方配置中提炼出来的，它大致做了这些事：首先从我们的内核镜像分区中提取 前 2048 个字节（其实类似于我们的文件头，记录我们文件的信息的），解析出设备树存放的地址及大小。然后从 2048 个字节读取设备树，将我们编译好的设备树切片用 fdtoverlay 合并到我们的主设备树文件，再将设备树文件写回去。最后更新“文件头”，设置新的设备树的信息。很遗憾的是，设置失败了，原因是，官方提供的启动脚本会重新设置，覆盖我的设置，对此，我也不再深究，大概原理就是这样。

```
#!/bin/bash
# 功能：USB 设备树（dr_mode=host）永久生效
# 临时文件路径：/tmp/，Boot 分区：/dev/mtdblock3

# ===== 1. 全局配置（可修改） =====
# USB 设备树配置（修复：删除嵌套注释，仅保留单行注释）
export USB_FDT_CONTENT="/dts-v1/;
/plugin/;

&{/usbdrcd/usb@ff000000}{
    dr_mode = \"host\"; /* 设为 USB 主机模式 */
};
"

# 临时文件（全部放 /tmp/）
HDR_DTB="/tmp/fdt_hdr.dtb"
RAW_DTB="/tmp/fdt_raw.dtb"
OVERLAY_DTS="/tmp/fdt_overlay.dts"
OVERLAY_DTBO="/tmp/fdt_overlay.dtbo"
HDR_OVERLAY_DTS="/tmp/fdt_hdr_overlay.dts"
HDR_OVERLAY_DTBO="/tmp/fdt_hdr_overlay.dtbo"

# Boot 分区（已适配你的 /dev/mtdblock3）
BOOT_PART="/dev/mtdblock3"

# ===== 2. 前置检查 =====
echo -e "===== 前置检查
=====\\n"

# 检查 root 权限
if [ "${id -u}" -ne 0 ]; then
    echo "[错误] 需 root 权限，执行：sudo -i 后重试"
    exit 1
```

```

fi

# 检查核心工具（嵌入式系统常用工具集）
needed_tools=("dtc" "fdt dump" "fdtoverlay" "dd" "sha256sum" "ls" "wc")
for tool in "${needed_tools[@]}; do
    if ! command -v "$tool" &>/dev/null; then
        echo "[错误] 缺少工具: $tool (需安装 device-tree-compiler 等)"
        exit 1
    fi
done

# 检查 Boot 分区是否存在
if [ ! -b "$BOOT_PART" ]; then
    echo "[错误] Boot 分区 $BOOT_PART 不存在!"
    ls -l /dev/mtdblock* # 显示所有 mtd 设备, 方便确认
    exit 1
fi

echo "[成功] 前置检查通过"

# ===== 3. 提取原始 DTB (不用 stat, 用 ls + wc 替代)
=====
echo -e "\n===== 提取原始 DTB
===== "

# 步骤1: 提取 DTB 头部 (前 2048 字节)
echo "[1/3] 提取 DTB 头部到 $HDR_DTB"
dd if="$BOOT_PART" of="$HDR_DTB" bs=1 skip=0 count=2048 >/dev/null 2>&1

# 检查头部文件是否有效 (用 ls -l 取大小, 替代 stat)
hdr_size=$(ls -la "$HDR_DTB" | awk '{print $5}')
if [ "$hdr_size" -ne 2048 ]; then
    echo "[错误] 头部提取失败, 大小应为 2048 字节, 实际: $hdr_size"
    exit 1
fi

# 步骤2: 解析原始 DTB 大小 (从头部获取)
echo "[2/3] 解析原始 DTB 大小"
raw_size_hex=$(fdtdump "$HDR_DTB" | grep -A 5 "fdt {" | grep "data-size" | awk
'{print $3}' | tr -d ';<>')
raw_size=$(printf "%d\n" "$raw_size_hex")
if [ -z "$raw_size" ] || [ "$raw_size" -eq 0 ]; then
    echo "[错误] 无法解析 DTB 大小"
    exit 1
fi
echo "[信息] 原始 DTB 大小: $raw_size 字节"

# 步骤3: 提取完整 DTB
echo "[3/3] 提取完整 DTB 到 $RAW_DTB"
dd if="$BOOT_PART" of="$RAW_DTB" bs=1 skip=2048 count="$raw_size" >/dev/null 2>&1

# 检查完整 DTB 大小 (用 ls -l 取大小)
extracted_size=$(ls -la "$RAW_DTB" | awk '{print $5}')
if [ "$extracted_size" -ne "$raw_size" ]; then

```

```

        echo "[错误] DTB 提取不完整, 应为 $raw_size 字节, 实际: $extracted_size"
        exit 1
    fi

    echo "[成功] 原始 DTB 提取完成"

# ===== 4. 编译并叠加 USB 配置
=====
echo -e "\n===== 叠加 USB 配置
=====

# 步骤1: 写入 DTS 配置
echo "[1/5] 写入 USB 配置到 $OVERLAY_DTS"
echo "$USB_FDT_CONTENT" >"$OVERLAY_DTS"
if [ ! -f "$OVERLAY_DTS" ]; then
    echo "[错误] 无法写入 DTS 文件, 检查 /tmp/ 权限"
    exit 1
fi

# 步骤2: 编译 DTS 为 DTBO (核心步骤, 解决之前语法错误)
echo "[2/5] 编译 DTS 为 DTBO"
dtc -I dts -O dtb "$OVERLAY_DTS" -o "$OVERLAY_DTBO"
if [ $? -ne 0 ] || [ ! -f "$OVERLAY_DTBO" ]; then
    echo "[错误] DTS 编译失败! 当前配置内容: "
    cat "$OVERLAY_DTS" # 显示配置, 方便排查
    exit 1
fi

# 步骤3: 叠加 DTBO 到原始 DTB
echo "[3/5] 叠加 DTBO 到原始 DTB"
fdtoverlay -i "$RAW_DTB" -o "$RAW_DTB" "$OVERLAY_DTBO" >/dev/null 2>&1
if [ $? -ne 0 ]; then
    echo "[错误] 叠加失败! 可能节点不存在"
    echo "查看原始 DTB 中的 USB 节点: fdt dump $RAW_DTB | grep -A 5 'usbdrd'"
    exit 1
fi

# 步骤4: 校验叠加后大小 (避免覆盖内核)
echo "[4/5] 校验 DTB 大小"
overlayed_size=$(ls -la "$RAW_DTB" | awk '{print $5}')
# 解析内核偏移地址 (判断是否会覆盖)
kernel_offset_hex=$(fdtdump "$HDR_DTB" | grep -A 2 "kernel {" | grep "data-
position" | sed -n 's/.*<(0x[0-9a-fA-F]*\*)>.*\1/p')
fdt_offset_hex=$(fdtdump "$HDR_DTB" | grep -A 2 "fdt {" | grep "data-position" |
sed -n 's/.*<(0x[0-9a-fA-F]*\*)>.*\1/p')
# 十六进制转十进制
kernel_offset=$((kernel_offset_hex))
fdt_offset=$((fdt_offset_hex))
available_space=$((kernel_offset - fdt_offset))

if [ "$overlayed_size" -gt "$available_space" ]; then
    echo "[错误] DTB 过大! 可用空间: $available_space 字节, 当前: $overlayed_size"
    exit 1
fi

```

```

echo "[信息] 叠加后大小: $overlaid_size 字节, 可用空间充足"

# 步骤5: 写回 Boot 分区
echo "[5/5] 写回 DTB 到 $BOOT_PART"
dd if="$RAW_DTB" of="$BOOT_PART" bs=1 seek=2048 count="$overlaid_size"
>/dev/null 2>&1
if [ $? -ne 0 ]; then
    echo "[错误] 写回失败! 尝试重新挂载 Boot 分区为可写: "
    echo "mount -o remount,rw $BOOT_PART"
    exit 1
fi

echo "[成功] USB 配置叠加完成"

# ===== 5. 更新 DTB 头部 (含校验值) =====
echo -e "\n===== 更新 DTB 头部 ====="

# 步骤1: 计算大小和 SHA256
echo "[1/3] 计算 DTB 信息"
fdt_size=$overlaid_size
fdt_size_hex=$(printf "%x\n" "$fdt_size")
# 计算 SHA256 并转换格式
sha_raw=$(sha256sum "$RAW_DTB" | awk '{print $1}')
fdt_hash=$(echo "$sha_raw" | sed -E 's/(.)/0x\1 /g')
echo "[信息] 大小 (十六进制): 0x$fdt_size_hex, SHA256: $sha_raw"

# 步骤2: 生成头部更新 DTS
echo "[2/3] 生成头部配置"
hdr_content="
/dts-v1/;
/plugin/;

&{/images/fdt}{
    data-size=<0x$fdt_size_hex>;
    hash{
        value=<$fdt_hash>;
    };
};
"
echo "$hdr_content" >"$HDR_OVERLAY_DTS"
if [ ! -f "$HDR_OVERLAY_DTS" ]; then
    echo "[错误] 无法生成头部 DTS"
    exit 1
fi

# 步骤3: 编译并写回头部
echo "[3/3] 编译头部并写回"
dtc -I dts -O dtb "$HDR_OVERLAY_DTS" -o "$HDR_OVERLAY_DTBO"
if [ $? -ne 0 ]; then
    echo "[错误] 头部 DTS 编译失败"
    exit 1
fi

```

```

# 叠加头部
fdtoverlay -i "$HDR_DTB" -o "$HDR_DTB" "$HDR_OVERLAY_DTBO" >/dev/null 2>&1
# 写回 Boot 分区（前 2048 字节）
dd if="$HDR_DTB" of="$BOOT_PART" bs=1 seek=0 count=2048 >/dev/null 2>&1
if [ $? -ne 0 ]; then
    echo "[错误] 头部写回失败"
    exit 1
fi

echo "[成功] 头部更新完成"

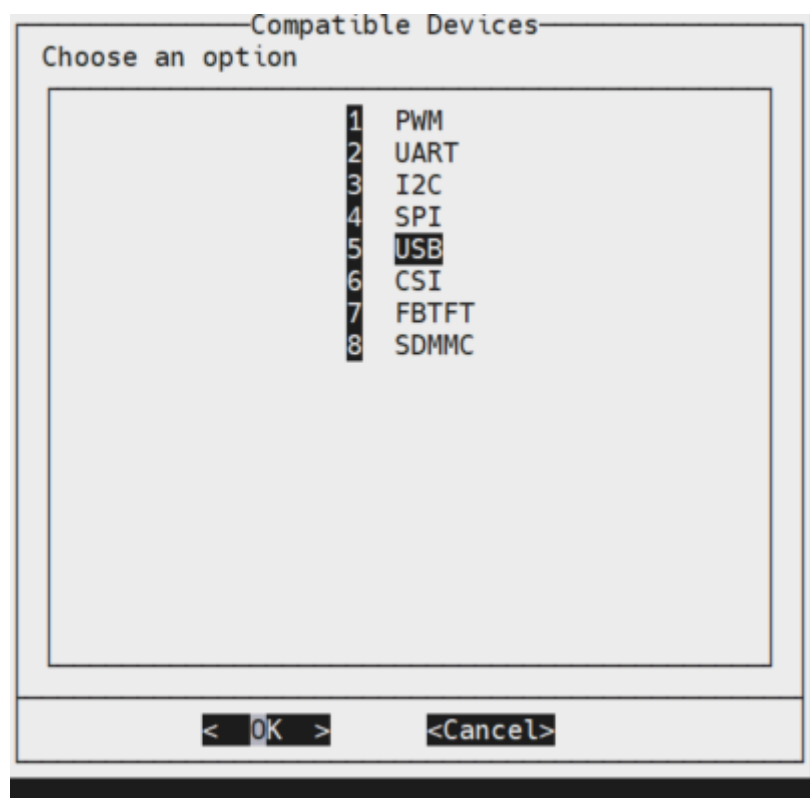
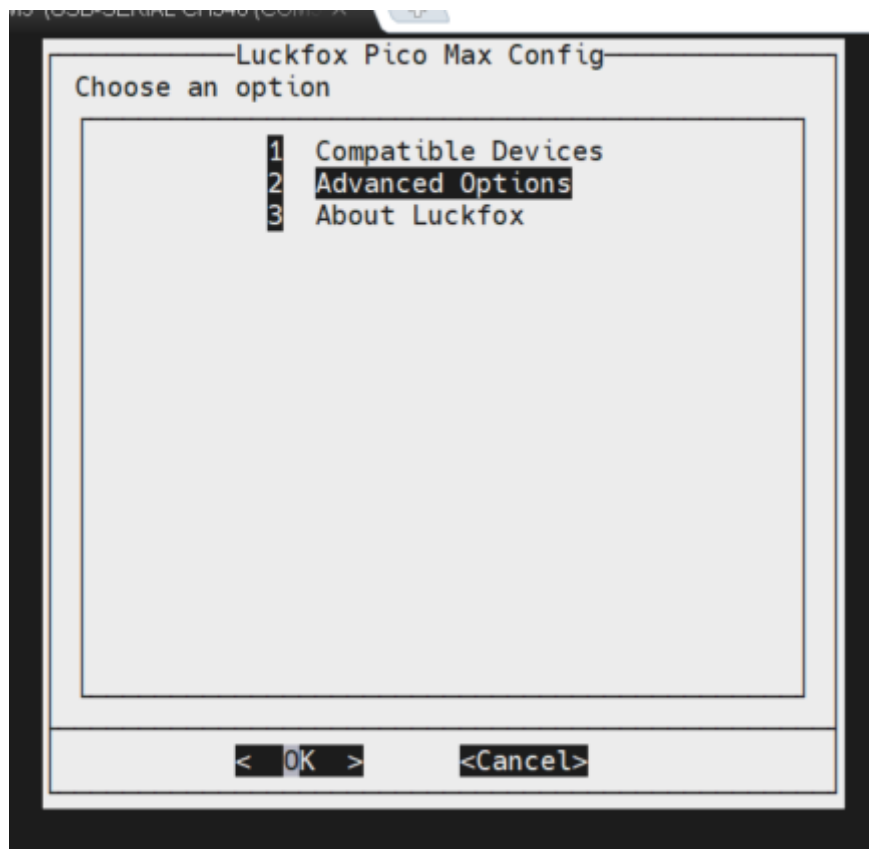
# ===== 6. 清理临时文件 =====
echo -e "\n===== 清理临时文件
=====
temp_files=("$HDR_DTB" "$RAW_DTB" "$OVERLAY_DTS" "$OVERLAY_DTBO"
"$HDR_OVERLAY_DTS" "$HDR_OVERLAY_DTBO")
for file in "${temp_files[@]}; do
    if [ -f "$file" ]; then
        rm -f "$file"
        echo "[已清理] $file"
    fi
done

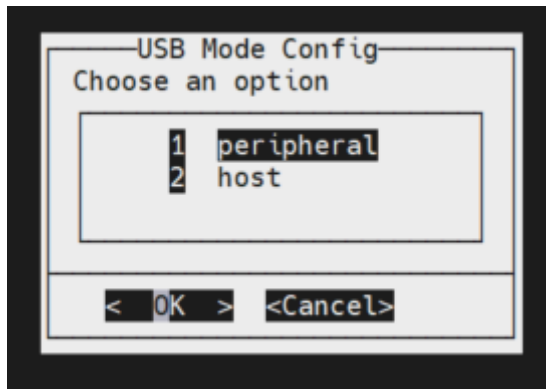
# ===== 7. 完成提示 =====
echo -e "\n===== 操作完成!
=====

```

实际上，官方已经有了更好的方案

执行luckfox-config，选择如图



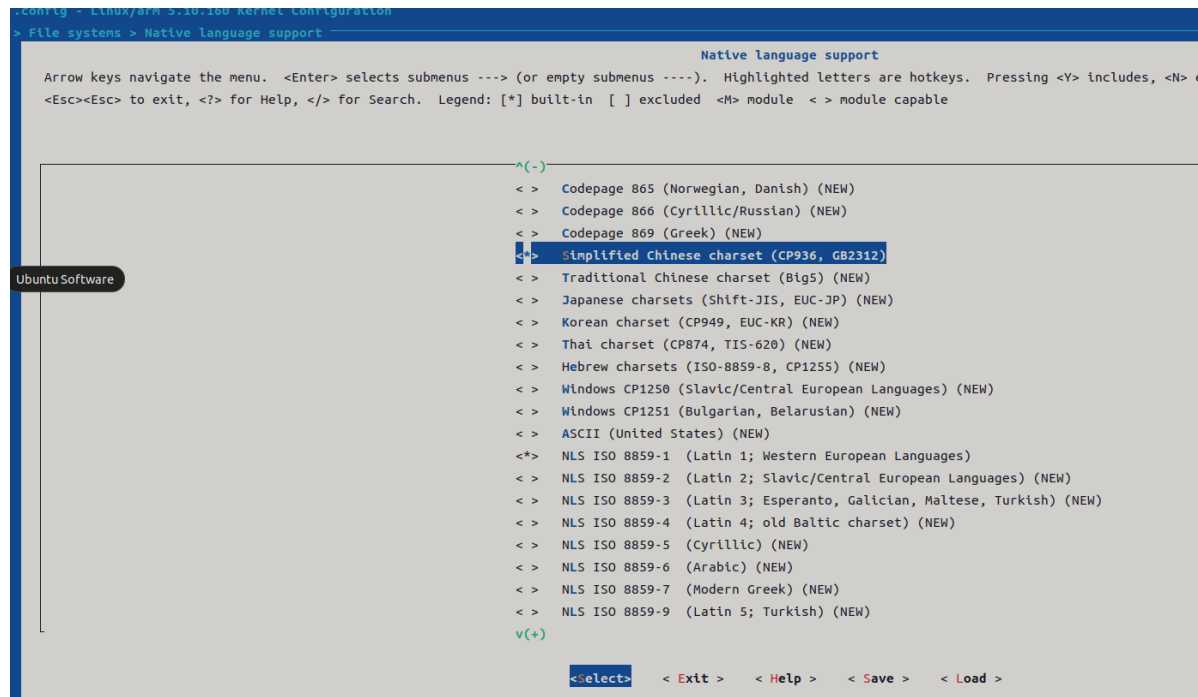


如图，便可以选择设备模式，还是主机模式了，选择完，重启即可。

2.8.3 中文字符集显示

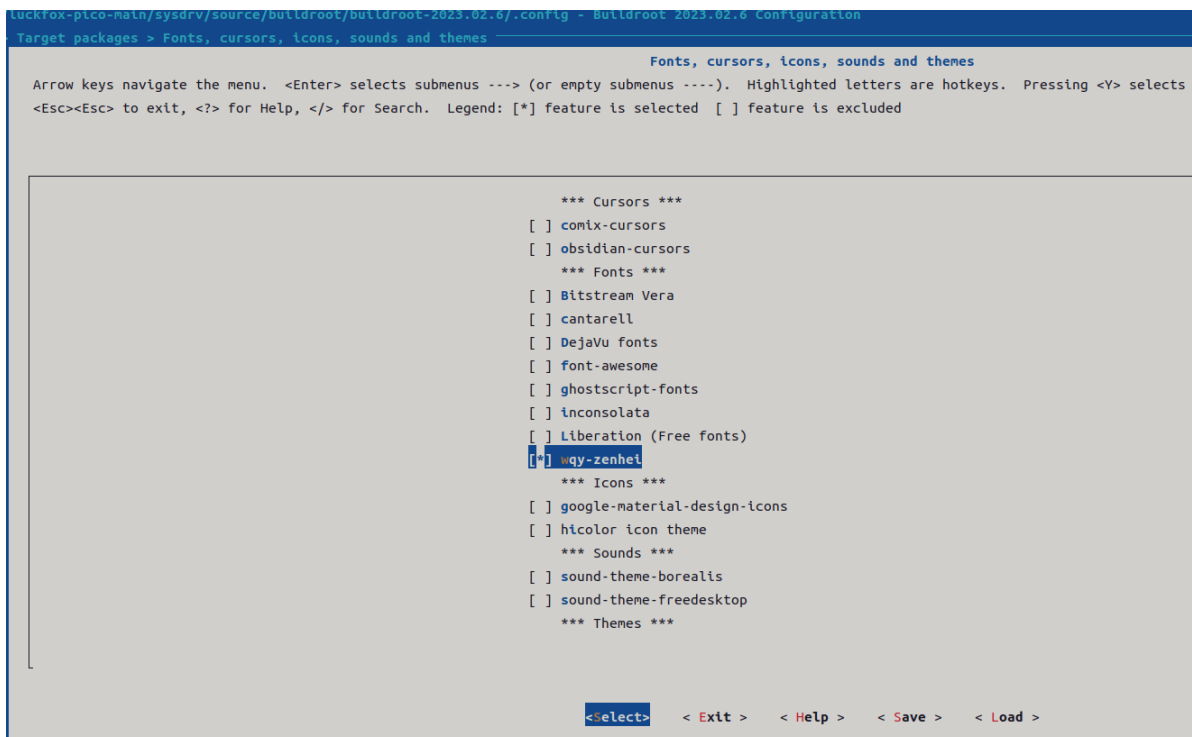
我们的linux默认是不开启中文的，支持中文字符集，对于我们命令行和图形库显示中文也很重要。

要支持中文，linux内核要开启GB编码。



buildroot 内核配置：

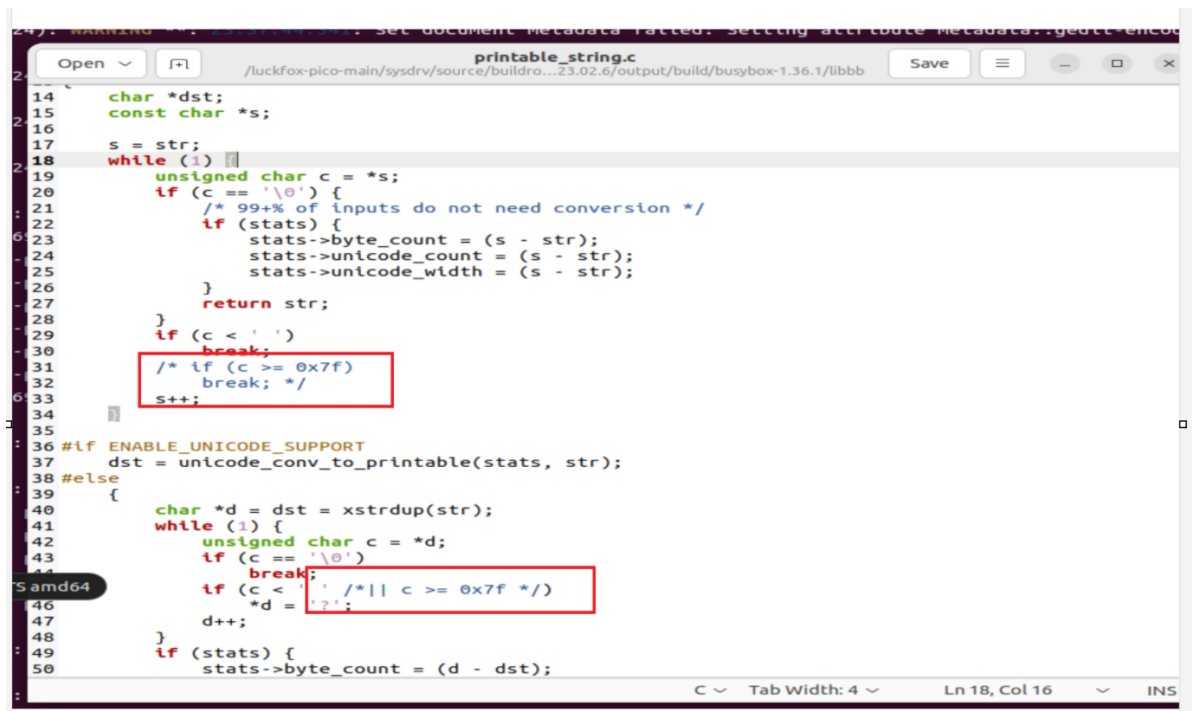
```
./build.sh buildrootconfig
```



除此之外，我们还要修改源码：

```
gedit output/build/busybox-1.36.1/libbb/printable_string.c
```

```
gedit busybox-1.36.1/libbb/printable_string.c
```





在buildroot 下执行 make busybox-clean-for-rebuild，否者更改的代码没有参与编译。

最后编译烧录即可。

第三章 Linux应用开发与适配

恭喜你已经完成了第二章！这一章我们将在做好的硬件驱动上，做出我们想要的应用。

3.1 应用自启动

这节简单，但是很重要！

3.1.1 运行型启动

有时候我们做好的应用，每次上电需要手动重新执行，好生麻烦，我们更希望，一上电就能执行我们想要的代码或者命令，实际上我们执行的命令叫shell命令，同样的我们可以写一个脚本，存放我们的一串命令，并按照我们想要的逻辑去执行，这样的脚本叫做shell脚本。在 /etc/init.d/ 中有相关的系统脚本。系统开机后，会先执行 rcS，在这个脚本里面，会循环去执行S开头脚本的start分支。系统关机时，会先执行 rcK，在这个脚本里面，也是循环去执行S开头的脚本的stop分支。所以如果我们需要添加一个脚本在开机时执行，可以按以下步骤操作：

1. 需要在 /etc/init.d/ 目录里新建一个 S??* 的脚本。
 - S:表示以S开头
 - ??:表示中间是**两个**字符串，建议加两个数字表示优先级，数字越小开机时越先执行，关机时越后执行
 - *:表示加任意字符串，用来描述脚本的功能
2. 并且在脚本里实现 start 和 stop 分支。
3. 给脚本添加权限，可以执行：

```
chmod 775 脚本名
```

```
#!/bin/sh

case $1 in
    start)
        echo "start"
        ;;
    stop)
        echo "stop"
        ;;
    *)
        exit 1
        ;;
esac
```

4. 提高系统自带的脚本优先级。

```
mv S99usb0config S90usb0config
mv S99_auto_reboot S90_auto_reboot
```

下面我们来写一个简单的shell脚本吧！

创建 `/etc/init.d/s99ffmpeg_play` 文件。

```
#!/bin/sh

case "$1" in
    start)
        echo "Starting FFmpeg video playback..."
        ffmpeg -re -i /root/output_dong_480_272.mp4 -vf "scale=480:272,fps=60" -
        pix_fmt bgra -f fbdev /dev/fb0 -f wav - | aplay -D hw:0,0 &
        ;;
    stop)
        echo "Stopping FFmpeg video playback..."
        pkill -f "ffmpeg -re -i /root/output_dong_480_272.mp4"
        ;;
    restart)
        $0 stop
        $0 start
        ;;
    *)
        echo "Usage: $0 {start|stop|restart}"
        exit 1
        ;;
esac

exit 0
```

稍微解释下。

```
#!/bin/sh
```

指定脚本解释器是 /bin/sh（系统标准 Shell）。

- **判断脚本传入参数**（init 系统会调用 `s99ffmpeg_play start`）：
 - `start`)：当启动时执行这段：
 - 打印提示 `"Starting Ffmpeg video playback..."`
 - 运行 `ffmpeg`：
 - `-re`：按输入文件原始速度读取（实时播放）。
 - `-i /root/output_dong_480_272.mp4`：输入文件路径。
 - `-vf "scale=480:272,fps=60"`：视频滤镜，缩放到 480×272，帧率 60fps。
 - `-pix_fmt bgra`：输出像素格式是 BGRA。
 - `-f fbdev /dev/fb0`：直接输出到 Linux 帧缓冲设备 `/dev/fb0`（显示屏）。
 - `-f wav -`：音频输出为 wav 格式到标准输出。
 - `| aplay -D hw:0,0`：通过 ALSA 设备 `hw:0,0` 播放音频。
 - `&`：放后台运行，让启动脚本能结束而不是卡住。

stop)：当执行 `s99ffmpeg_play stop` 时运行：

- 打印提示 `"Stopping..."`。
- 用 `kill -f` 根据命令行关键字杀掉对应的 `ffmpeg` 进程。

restart)：重启服务就是先执行 `stop`，再执行 `start`。

***)：默认情况**：如果传入参数不合法（不是 `start/stop/restart`），提示用法并退出。

我们为文件添加执行权限，重新启动，。

```
chmod +x /etc/init.d/s99ffmpeg_play
```

如果你的程序提示了以下错误

```
/etc/init.d/rcs: line 23: /etc/init.d/s99ffmpeg_play: not found
rm: can't remove '/tmp/.pin_diagram.txt': No such file or directory
```

这是由于脚本是用 Windows 编辑的（或者你文件放的有问题），可能带有 `\r\n` 换行，导致 Linux 认为它不存在，需要格式转化。

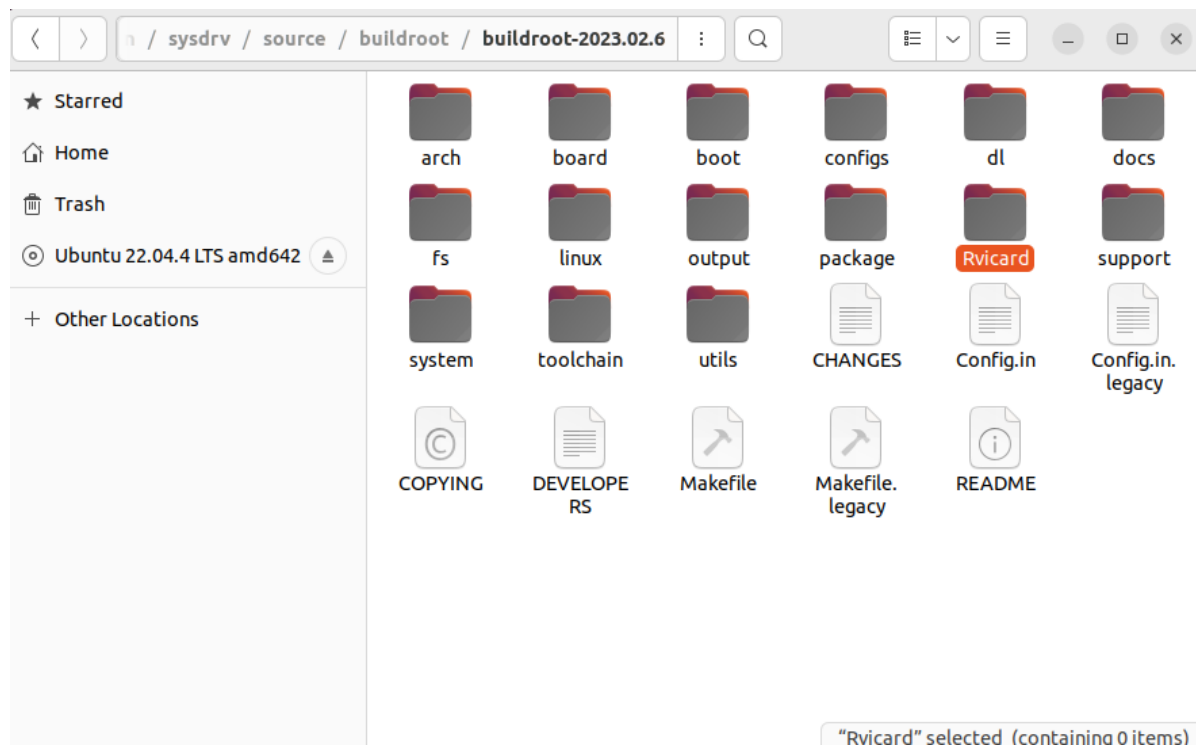
```
dos2unix /etc/init.d/s99ffmpeg_play
```

好了，相信你一启动，就开始播放音乐和视频了吧，哈哈！

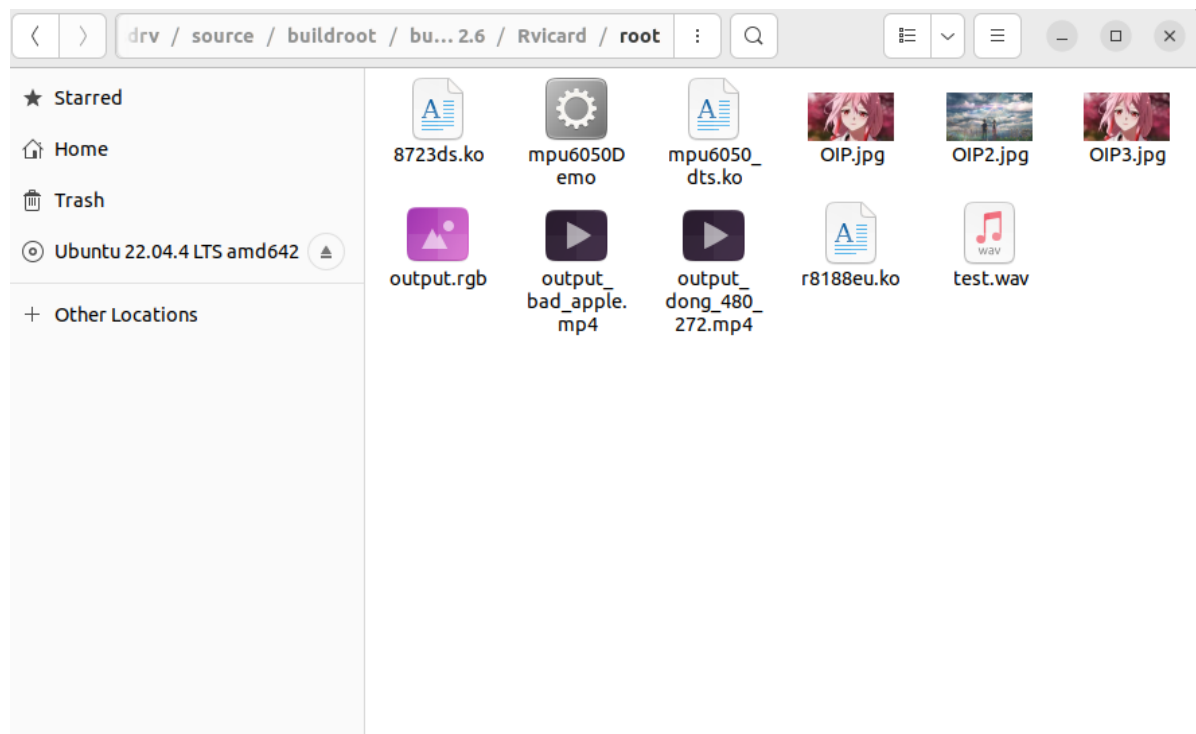
3.1.2 编译型启动

那有人就要问up了：up，up，每次都要添加并写入，好麻烦啊？有没有更快，更好的办法？有的，兄弟，有的！

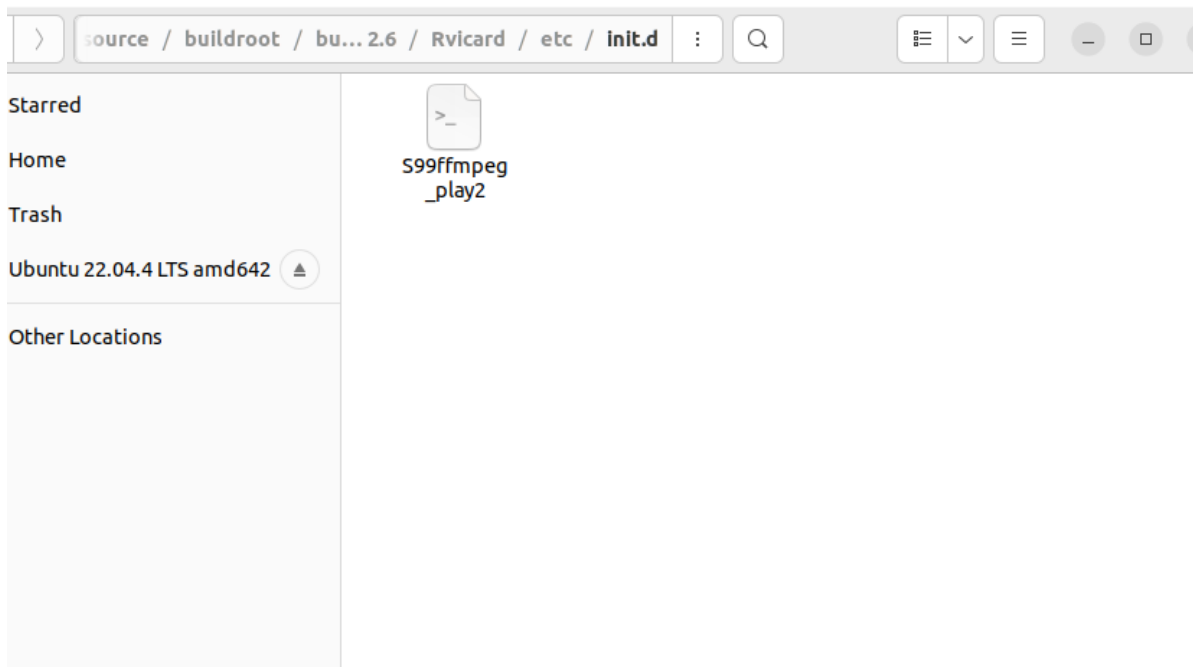
我们可以把文件编译进镜像里，下面我们进入buildroot中，添加Rvicard文件夹



进入RVicard中，创建root文件夹，添加我们常用的文件

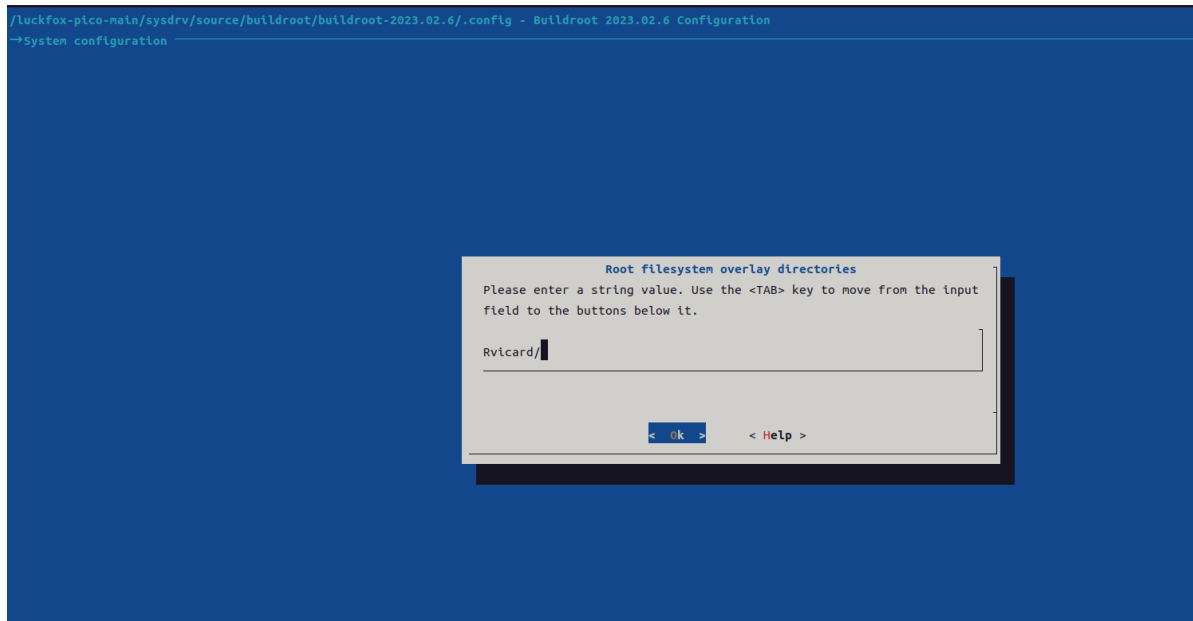


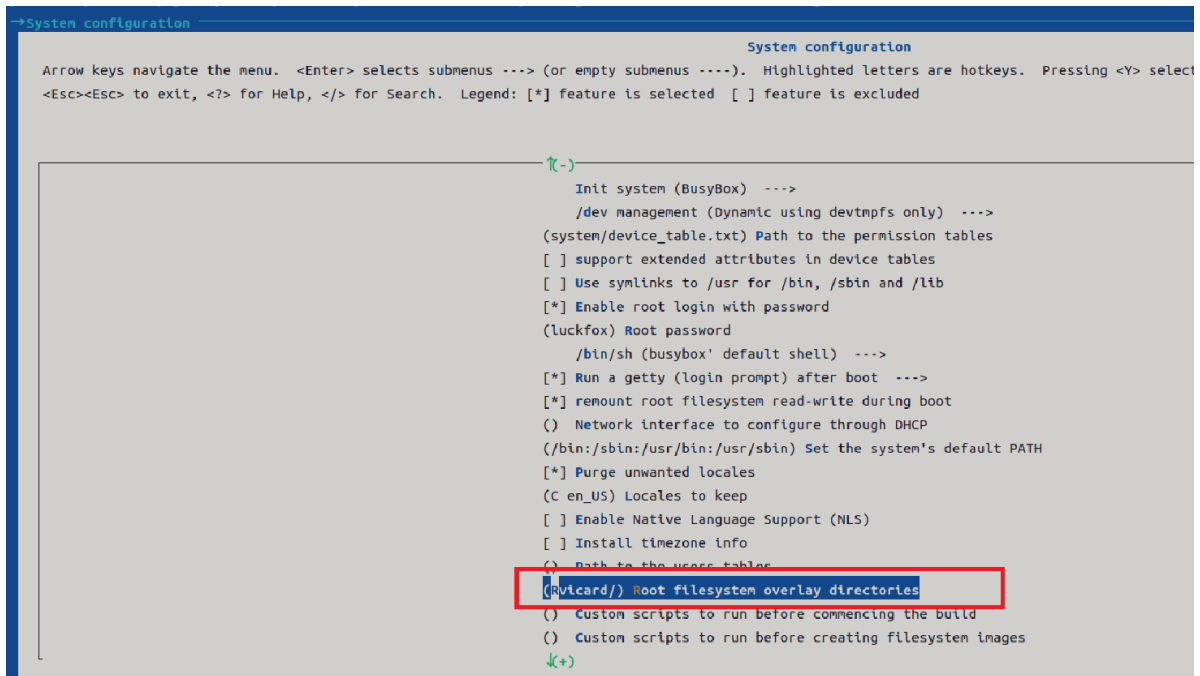
创建etc/init.d/，往里面添加我们想要的文件，不要忘记给他添加执行权限。



我们执行再SDK目录下，执行，添加Rvicard文件夹，保存退出。编译根文件系统，还要打包下。

```
./build.sh buildrootconfig  
./build.sh clean rootfs  
./build.sh rootfs  
./build.sh firmware
```





为什么这么做呢？因为buildroot 会自动把我们所需要的文件和其他文件进行合并，我们文件的布局 and 最终文件系统展示的必须一致，最终得到我们需要的镜像文件。

```

root@chen66666697-virtual-machine:/luckfox-pico-main/sysdrv/source/buildroot/buildroot-2023.02.6# ls output/target/root/
8723ds.ko  mpu6050Demo  mpu6050_dts.ko  OIP2.jpg  OIP3.jpg  OIP.jpg  output_bad_apple.mp4  output_dong_480_272.mp4  output.rgb  r8188eu.ko  test.wav
root@chen66666697-virtual-machine:/luckfox-pico-main/sysdrv/source/buildroot/buildroot-2023.02.6#
  
```

相信你已经发现我们启动的文件系统里已经加入了我们想要的文件，这样就避免了每次烧录，被覆盖或消失的问题。

S99ffmpeg_play2

```

#!/bin/sh

case "$1" in
    start)
        echo "Starting FFmpeg video playback..."
        ffmpeg -i /root/OIP3.jpg -vf "scale=480:272" -pix_fmt bgra -frames:v 1 -f
        fbdev /dev/fb0 &
        ;;
    stop)
        echo "Stopping FFmpeg video playback..."
        pkill -f "ffmpeg -re -i /root/output_dong_480_272.mp4"
        ;;
    restart)
        $0 stop
        $0 start
        ;;
    *)
        echo "Usage: $0 {start|stop|restart}"
        exit 1
        ;;
esac
  
```

3.2 移植LVGL库

LVGL(轻量级和通用图形库)是一个免费和开源的图形库，它提供了创建嵌入式GUI所需的一切，具有易于使用的图形元素，美丽的视觉效果和低内存占用。

- 1、丰富且强大的模块化图形组件：按钮、图表、列表、滑动条、图片等
- 2、高级的图形引擎：动画、抗锯齿、透明度、平滑滚动、图层混合等效果
- 3、支持多种输入设备：触摸屏、键盘、编码器、按键等
- 4、不依赖特定的硬件平台
- 5、配置可裁剪，最低资源占用：64 kB Flash，16 kB RAM
- 6、基于UTF-8的多语种支持，例如中文、日文、韩文、阿拉伯文等
- 7、可以通过类CSS的方式来设计、布局图形界面（例如：Flexbox、Grid）
- 8、支持操作系统、外置内存、以及硬件加速（已内建支持STM32 DMA2D）
- 9、即便仅有单缓冲区(frame buffer)的情况下，也可保证渲染如丝般顺滑
- 10、支持模拟器仿真，可以无硬件依托进行开发

所以我们移植LVGL，方便我们进行简单的图形化应用开发。

3.2.1 lvgl源码下载及配置

首先我们要下载相关代码，代码仓库如下：

lvgl <https://github.com/lvgl/lvgl.git> 包含了LVGL图形界面控件的源码以及少量例程

lv_drivers https://github.com/lvgl/lv_drivers.git 包含了驱动LVGL图形界面的驱动接口源代码

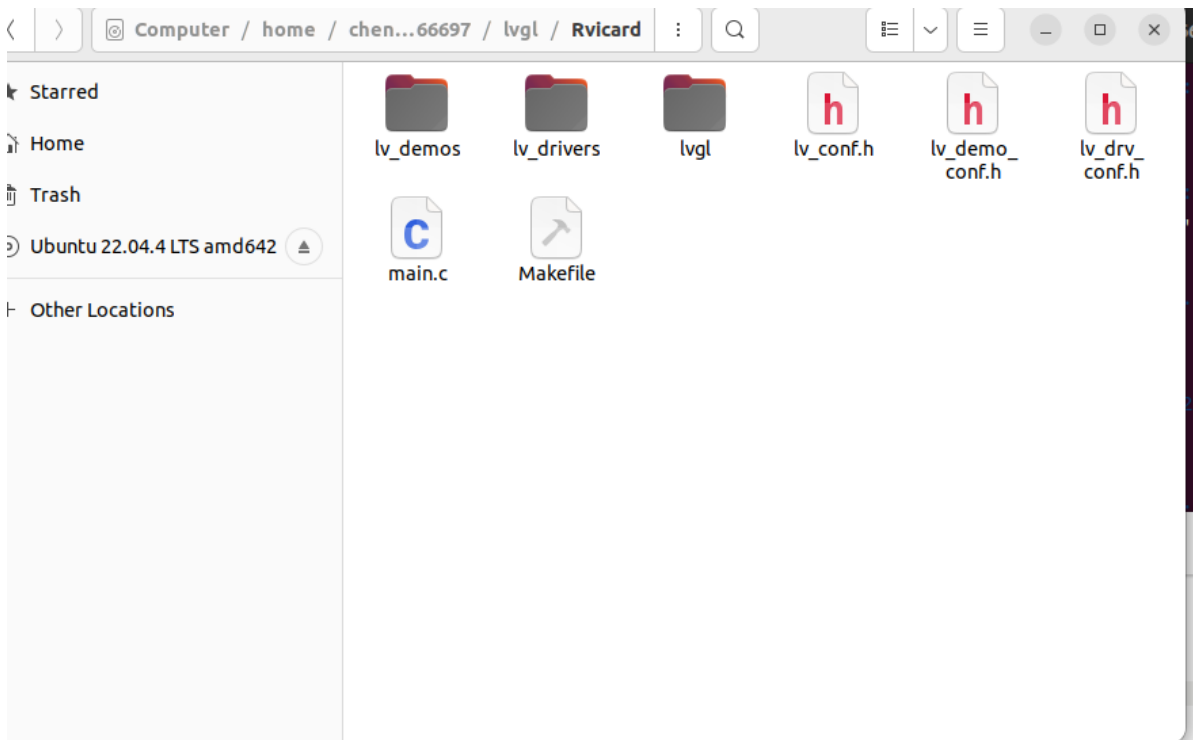
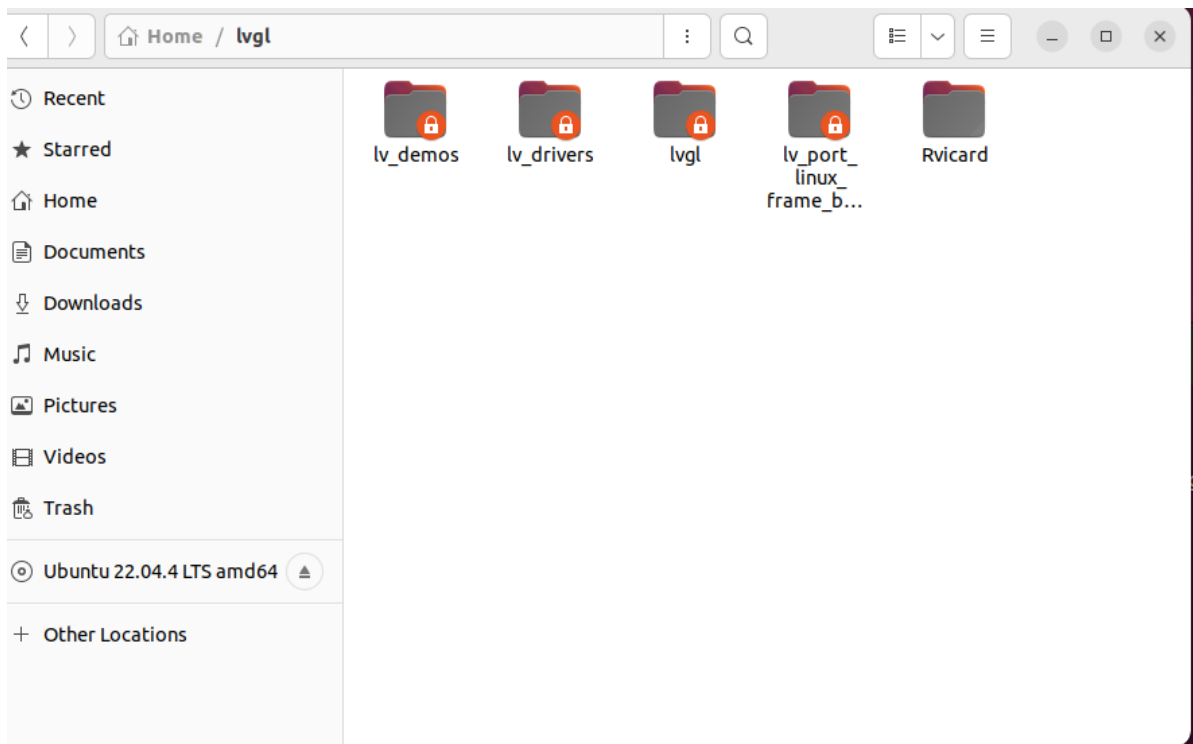
lv_demos https://github.com/lvgl/lv_demos.git LVGL的例程

lv_port_linux_frame_buffer https://github.com/lvgl/lv_port_linux_frame_buffer.git 适配有frame buffer的linux系统的接口

我们创建一个文件夹（在虚拟机环境哦），这个文件夹你随便命名

```
mkdir lvgl
cd lvgl
git clone -b v8.1.0 https://github.com/lvgl/lvgl.git
git clone -b v8.1.0 https://github.com/lvgl/lv_drivers.git
git clone -b v8.1.0 https://github.com/lvgl/lv_demos.git
git clone --branch release/v8.2 --single-branch
https://github.com/lvgl/lv_port_linux_frame_buffer.git
```

下载好，我们创建Rvicard，将lvgl文件夹中的lvgl、lv_drivers和lv_port_linux_frame_buffer/src/ 中的main.c与Makefile复制到Rvicard中。

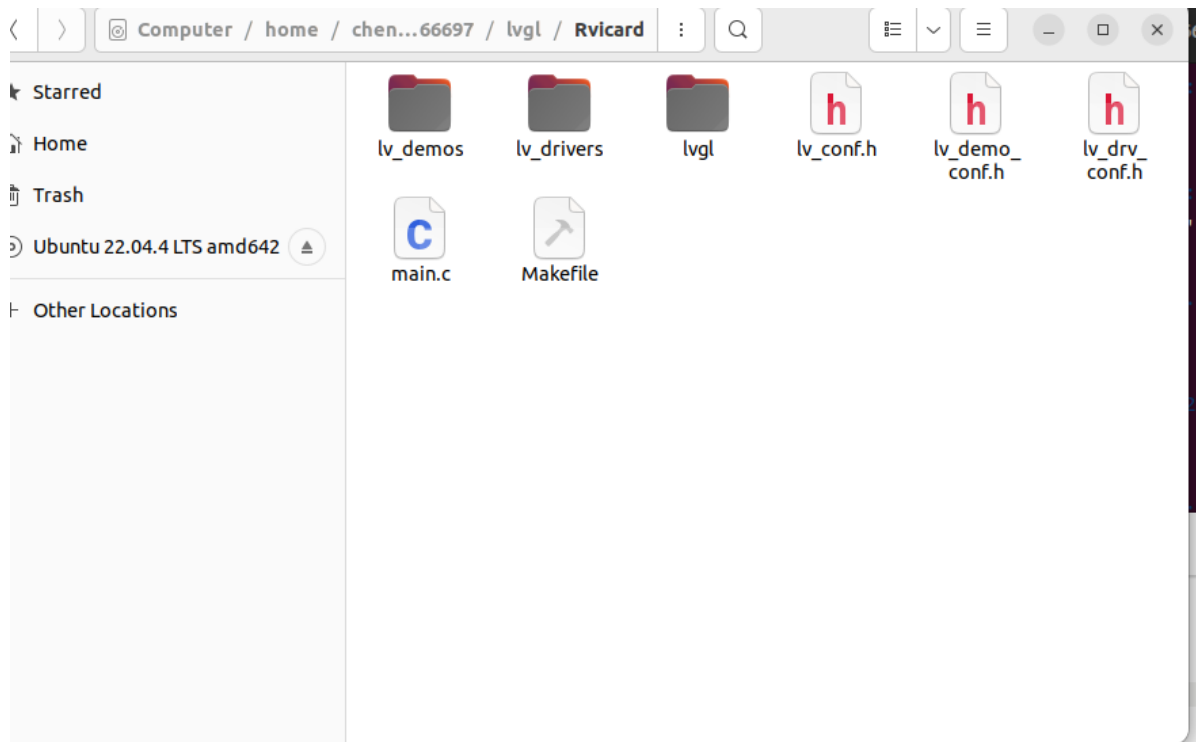


接着我们在将

lvgl中的lv_conf_template.h复制出来并且改名位lv_conf.h

lv_drivers中的lv_drv_conf_template.h复制出来并且改名为lv_drv_conf.h

lv_demos 的lv_demo_conf_template.h 复制出来并且改名为 lv_demo_conf.h



修改lv_drv_conf.h,修改这个文件的目的是为了使用linux下的frame buffer输出显示以及触控输入,需要将第11行的#ifdef改成#if 1

```
/* clang-format off */
#ifdef 1 /*Set it to "1" to enable the content*/

#ifndef LV_DRV_CONF_H
#define LV_DRV_CONF_H

#include "lv_conf.h"
```

然后328~335行, 将USE_FBDEV 的值改为1, 使能frame buffer设备

```
/*-----
 *   FreeBSD frame buffer device (/dev/fbx)
 *   .....*/
#ifndef USE_BSD_FBDEV
# define USE_BSD_FBDEV      1
#endif

#if USE_BSD_FBDEV
# define FBDEV_PATH        "/dev/fb0"
#endif

/*-----
```

修改lv_conf.h 一样需要将一开始的#ifdef 0 改成#if 1。再将 LV_MEM_POOL_EXPAND_SIZE 改为1, 开启内存扩展, 防止内存爆了。

```

/* clang-format off */
#if 1 /* Set this to "1" to enable content */

#ifndef LV_CONF_H
#define LV_CONF_H

/* If you need to include anything here, do it inside the `__ASSEMBLY__` guard */
#if 0 && defined(__ASSEMBLY__)
#include "my_include.h"
#endif

```

```

#if LV_USE_STDLIB_MALLOC == LV_STDLIB_BUILTIN
    /** Size of memory available for `lv_malloc()` in bytes (>= 2kB) */
    #define LV_MEM_SIZE (64 * 1024U)          /**< [bytes] */

    /** Size of the memory expand for `lv_malloc()` in bytes */
    #define LV_MEM_POOL_EXPAND_SIZE 1

    /** Set an address for the memory pool instead of allocating it as a normal
    array. Can be in external SRAM too. */
    #define LV_MEM_ADR 0          /**< 0: unused*/
    /* Instead of an address give a memory allocator that will be called to get a
    memory pool for LVGL. E.g. my_malloc */
    #if LV_MEM_ADR == 0
        #undef LV_MEM_POOL_INCLUDE
        #undef LV_MEM_POOL_ALLOC
    #endif

#endif /*LV_USE_STDLIB_MALLOC == LV_STDLIB_BUILTIN*/

```

这块是内存分配，以下是他的解释：

```

#if LV_USE_STDLIB_MALLOC == LV_STDLIB_BUILTIN

```

意思是 LVGL 使用自带的内存池（**builtin pool**）而不是系统 `malloc`。

如果你改为 `LV_USE_STDLIB_MALLOC == LV_STDLIB_CUSTOM`，可以使用自定义分配函数。

```

#define LV_MEM_SIZE (64 * 1024U) /**< [bytes] */

```

定义内存池大小，单位是字节，这里是 **64KB**。

这个内存池用于 `lv_malloc()` 和 LVGL 内部对象分配。

对于小屏 LCD 或嵌入式 MCU，**64KB** 已经够用；如果资源紧张可以调小，或者如果功能复杂（很多字体、图片），可以调大。

```

#define LV_MEM_POOL_EXPAND_SIZE 0

```

内存池耗尽后是否自动扩展，如果设置大于 **0**，LVGL 会尝试扩展指定大小。

这里是 **0**，表示不扩展，避免动态增加内存。

```
#define LV_MEM_ADR 0
```

可以指定一个固定地址的内存池（比如在外部 SRAM 中），而不是 LVGL 自己在 .bss 里开辟数组。

如果设为 0，则不使用固定地址，由 LVGL 自己分配

```
#if LV_MEM_ADR == 0
    #undef LV_MEM_POOL_INCLUDE
    #undef LV_MEM_POOL_ALLOC
#endif
```

如果没有指定内存池地址，则取消自定义头文件和分配函数定义。

如果你想使用自己的分配器（比如用 RTOS 的 heap 或外部 SRAM），可以定义 LV_MEM_POOL_INCLUDE 和 LV_MEM_POOL_ALLOC

如果你是这种 只用把 LV_MEM_SIZE改大点

```
39 #define LV_COLOR_MIX_ROUND_OFS (LV_COLOR_DEPTH == 32 ? 0: 128)
40
41 /*Images pixels with this color will not be drawn if they are chroma keyed*/
42 #define LV_COLOR_CHROMA_KEY lv_color_hex(0x00ff00) /*pure green*/
43
44 /*=====
45     MEMORY SETTINGS
46 *=====*/
47
48 /*1: use custom malloc/free, 0: use the built-in 'lv_mem_alloc()' and 'lv_mem_free()'*/
49 #define LV_MEM_CUSTOM 0
50 #if LV_MEM_CUSTOM == 0
51 /*Size of the memory available for 'lv_mem_alloc()' in bytes (>= 2kB)*/
52 #define LV_MEM_SIZE (360U * 1024U) /*[bytes]*/
53
54 /*Set an address for the memory pool instead of allocating it as a normal array. Can be in
   external SRAM too.*/
55 #define LV_MEM_ADR 0 /*0: unused*/
56 /*Instead of an address give a memory allocator that will be called to get a memory pool for
   LVGL. E.g. my_malloc*/
57 #if LV_MEM_ADR == 0
58 // #define LV_MEM_POOL_INCLUDE your_alloc_library /* Uncomment if using an external allocator*/
59 // #define LV_MEM_POOL_ALLOC your_alloc /* Uncomment if using an external allocator*/
60 #endif
61
62 #else /*LV_MEM_CUSTOM*/
63 #define LV_MEM_CUSTOM_INCLUDE <stdlib.h> /*Header for the dynamic memory function*/
64 #define LV_MEM_CUSTOM_ALLOC malloc
65 #define LV_MEM_CUSTOM_FREE free
66 #define LV_MEM_CUSTOM_REALLOC realloc
```

把91行的刷新时间改成10 ms

```
/** Default display refresh, input device read and animation step period. */
#define LV_DEF_REFR_PERIOD 10 /*< [ms] */
```

将lv_demo 1384行 LV_USE_DEMO_WIDGETS LV_USE_DEMO_BENCHMARK改为 1，使用官方 demo

```

#if LV_BUILD_DEMOS
    /** Show some widgets. This might be required to increase `LV_MEM_SIZE`. */
    #define LV_USE_DEMO_WIDGETS 1

    /** Demonstrate usage of encoder and keyboard. */
    #define LV_USE_DEMO_KEYPAD_AND_ENCODER 0

    /** Benchmark your system */
    #define LV_USE_DEMO_BENCHMARK 1

```

更改lv_conf tick配置

```

/home/chen6666697/lvgl/Rvicard
80 /*Default display refresh period. LVGL will redraw changed areas with this period time*/
81 #define LV_DISP_DEF_REFR_PERIOD 30      /*[ms]*/
82
83 /*Input device read period in milliseconds*/
84 #define LV_INDEV_DEF_READ_PERIOD 10     /*[ms]*/
85
86 /*Use a custom tick source that tells the elapsed time in milliseconds.
87  *It removes the need to manually update the tick with `lv_tick_inc()`*/
88 #define LV_TICK_CUSTOM 1
89 #if LV_TICK_CUSTOM
90     #define LV_TICK_CUSTOM_INCLUDE <stdint.h>          /*Header for the system time function*/
91     #define LV_TICK_CUSTOM_SYS_TIME_EXPR (custom_tick_get()) /*Expression evaluating to
current system time in ms*/
92 #endif /*LV_TICK_CUSTOM*/
93
94 /*Default Dot Per Inch. Used to initialize default sizes such as widgets sized, style paddings
95  *(Not so important, you can adjust it to modify default sizes and spaces)*/
96 #define LV_DPI_DEF 130      /*[px/inch]*/
97
98 /*=====
99  * FEATURE CONFIGURATION
100  *=====*/
101
102 /*-----
103  * Drawing
104  *-----*/
105
106 /*Enable complex draw engine.
107  *Required to draw shadow, gradient, rounded corners, circles, arc, skew lines, image
transformations or any masks*/

```

```

#define LV_TICK_CUSTOM 1
#if LV_TICK_CUSTOM
    #define LV_TICK_CUSTOM_INCLUDE <stdint.h>          /*Header for the system
time function*/
    #define LV_TICK_CUSTOM_SYS_TIME_EXPR (custom_tick_get()) /*Expression
evaluating to current system time in ms*/
#endif /*LV_TICK_CUSTOM*/

```

接下来我们在命令行中导出我们的交叉编译工具链。

```
export COMPILER=arm-rockchip830-linux-uc1ibcgnueabihf-
```

修改makefile的编译脚本。

```

#
# Makefile
#
CC = arm-rockchip830-linux-uc1ibcgnueabihf-gcc
LVGL_DIR_NAME    ?= lvgl
LVGL_DIR         ?= .

```

```

WARNINGS      := -Wall -Wshadow -Wundef -Wmissing-prototypes -Wno-discarded-
qualifiers -Wall -Wextra -Wno-unused-function -Wno-error=strict-prototypes -
Wpointer-arith \
                -fno-strict-aliasing -Wno-error=cpp -Wuninitialized -Wmaybe-
uninitialized -Wno-unused-parameter -Wno-missing-field-initializers -Wtype-limits
-Wsizeof-pointer-memaccess \
                -Wno-format-nonliteral -Wno-cast-qual -Wunreachable-code -
Wno-switch-default -Wreturn-type -Wmultichar -Wformat-security -Wno-ignored-
qualifiers -Wno-error=pedantic \
                -Wno-sign-compare -Wno-error=missing-prototypes -Wdouble-
promotion -Wclobbered -Wdeprecated -Wempty-body -Wtype-limits -Wshift-negative-
value -Wstack-usage=2048 \
                -Wno-unused-value -Wno-unused-parameter -Wno-missing-field-
initializers -Wuninitialized -Wmaybe-uninitialized -Wall -Wextra -Wno-unused-
parameter \
                -Wno-missing-field-initializers -Wtype-limits -Wsizeof-
pointer-memaccess -Wno-format-nonliteral -Wpointer-arith -Wno-cast-qual -
Wmissing-prototypes \
                -Wunreachable-code -Wno-switch-default -Wreturn-type -
Wmultichar -Wno-discarded-qualifiers -Wformat-security -Wno-ignored-qualifiers -
Wno-sign-compare -std=c99
CFLAGS        ?= -O3 -g0 -I$(LVGL_DIR)/ $(WARNINGS)
LDFLAGS       ?= -lm
BIN           = demo
BUILD_DIR     = ./build
BUILD_OBJ_DIR = $(BUILD_DIR)/obj
BUILD_BIN_DIR = $(BUILD_DIR)/bin

prefix        ?= /usr
bindir        ?= $(prefix)/bin

#Collect the files to compile
MAINSRC = $(wildcard *.*.c)

include $(LVGL_DIR)/lvgl/lvgl.mk
include $(LVGL_DIR)/lv_drivers/lv_drivers.mk
include $(LVGL_DIR)/lv_demos/lv_demo.mk

# CSRCS          +=$(LVGL_DIR)/mouse_cursor_icon.c

OBJEXT        ?= .o

AOBJS         = $(ASRCS:.S=$(OBJEXT))
COBJS         = $(CSRCS:.c=$(OBJEXT))

MAINOBJ       = $(MAINSRC:.c=$(OBJEXT))

SRCS          = $(ASRCS) $(CSRCS) $(MAINSRC)
OBJS          = $(AOBJS) $(COBJS) $(MAINOBJ)
TARGET        = $(addprefix $(BUILD_OBJ_DIR)/, $(patsubst ./%, %, $(OBJS)))

## MAINOBJ -> OBJFILES

all: default

```

```

$(BUILD_OBJ_DIR)/%.o: %.c
    @mkdir -p $(dir $@)
    @$(CC) $(CFLAGS) -c $< -o $@
    @echo "CC $<"

default: $(TARGET)
    @mkdir -p $(dir $(BUILD_BIN_DIR)/)
    $(CC) -o $(BUILD_BIN_DIR)/$(BIN) $(TARGET) $(LDFLAGS)

clean:
    rm -rf $(BUILD_DIR)

install:
    install -d $(DESTDIR)$(bindir)
    install $(BUILD_BIN_DIR)/$(BIN) $(DESTDIR)$(bindir)

uninstall:
    $(RM) -r $(addprefix $(DESTDIR)$(bindir)/,$(BIN))

```

然后 修改main函数，记得改分辨率为你的，我的是480*272

```

#include "lvgl/lvgl.h"
#include "lv_demos/lv_demo.h"
#include "lv_drivers/display/fbdev.h"
#include "lv_drivers/indev/evdev.h"
#include <unistd.h>
#include <pthread.h>
#include <time.h>
#include <sys/time.h>

#define DISP_BUF_SIZE (128 * 1024)

int main(void)
{
    /*LittlevGL init*/
    lv_init();

    /*Linux frame buffer device init*/
    fbdev_init();

    /*A small buffer for LittlevGL to draw the screen's content*/
    static lv_color_t buf[DISP_BUF_SIZE];

    /*Initialize a descriptor for the buffer*/
    static lv_disp_draw_buf_t disp_buf;
    lv_disp_draw_buf_init(&disp_buf, buf, NULL, DISP_BUF_SIZE);

    /*Initialize and register a display driver*/
    static lv_disp_drv_t disp_drv;
    lv_disp_drv_init(&disp_drv);
    disp_drv.draw_buf = &disp_buf;
    disp_drv.flush_cb = fbdev_flush;
    disp_drv.hor_res = 480;
    disp_drv.ver_res = 272;

```

```

lv_disp_drv_register(&disp_drv);

lv_demo_benchmark();

/*Handle LittlevGL tasks (tickless mode)*/
while(1) {
    lv_timer_handler();
    usleep(5000);
}

return 0;
}

/*Set in lv_conf.h as `LV_TICK_CUSTOM_SYS_TIME_EXPR`*/
uint32_t custom_tick_get(void)
{
    static uint64_t start_ms = 0;
    if(start_ms == 0) {
        struct timeval tv_start;
        gettimeofday(&tv_start, NULL);
        start_ms = (tv_start.tv_sec * 1000000 + tv_start.tv_usec) / 1000;
    }

    struct timeval tv_now;
    gettimeofday(&tv_now, NULL);
    uint64_t now_ms;
    now_ms = (tv_now.tv_sec * 1000000 + tv_now.tv_usec) / 1000;

    uint32_t time_ms = now_ms - start_ms;
    return time_ms;
}

```

在编译之前，需要把屏幕的位宽搞清楚，如果你是用大屏幕的，查看的屏幕的位宽，小屏幕RGB565 用 16。

```
cat /sys/class/graphics/fb0/bits_per_pixel
```

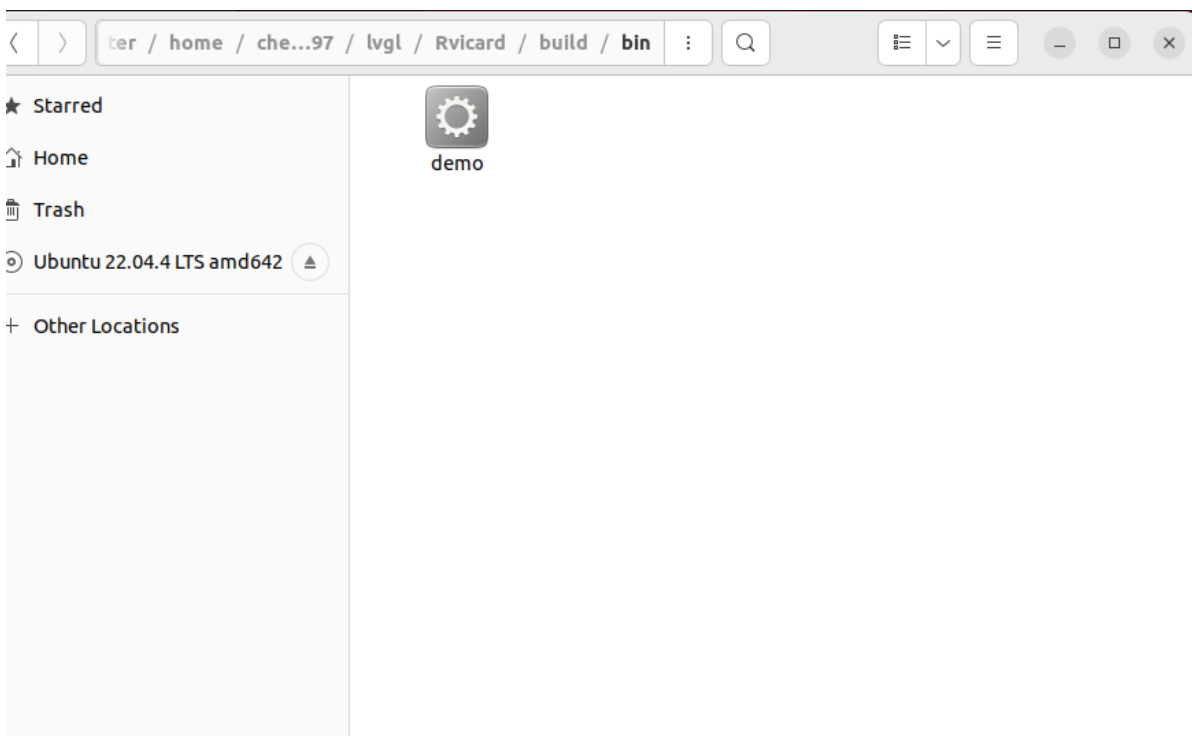
```
[root@luckfox root]# cat /sys/class/graphics/fb0/bits_per_pixel
32
```

修改如下

```
lv_conf.h
/home/chen66666697/lvgl/Rvcard

5
6 /*
7  * Copy this file as `lv_conf.h`
8  * 1. simply next to the `lvgl` folder
9  * 2. or any other places and
10  *    - define `LV_CONF_INCLUDE_SIMPLE`
11  *    - add the path as include path
12  */
13
14 /* clang-format off */
15 #if 1 /*Set it to "1" to enable content*/
16
17 #ifndef LV_CONF_H
18 #define LV_CONF_H
19
20 #include <stdint.h>
21
22 /*=====
23  COLOR SETTINGS
24  =====*/
25
26 /*Color depth: 1 (1 byte per pixel), 8 (RGB332), 16 (RGB565), 32 (ARGB8888)*/
27 #define LV_COLOR_DEPTH 32
28
29 /*Swap the 2 bytes of RGB565 color. Useful if the display has an 8-bit interface (e.g. SPI)*/
30 #define LV_COLOR_16_SWAP 0
31
```

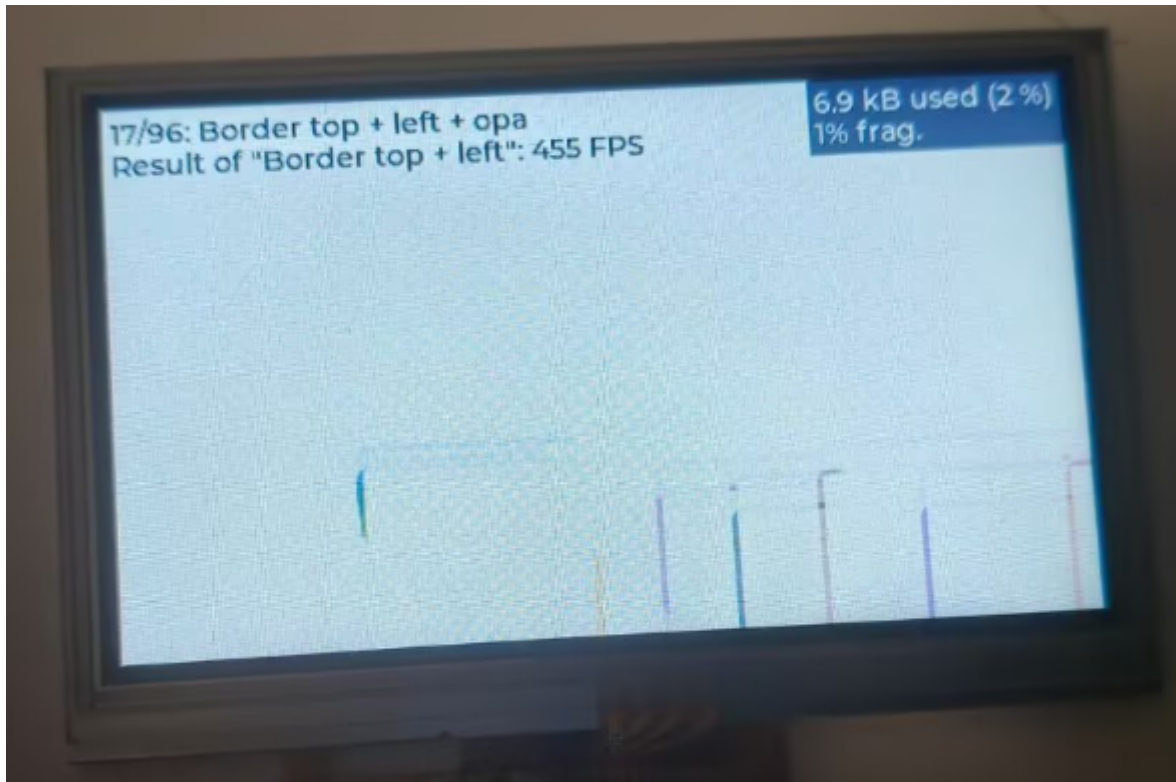
执行编译后，你就可以看到可执行程序了



3.2.2 lvgl demo运行

将文件移植到板子后执行。

运行后的效果，平均500fps。



3.3 移植QT库

QT 是 **跨平台的 C++ 应用开发框架**，最初由挪威 Trolltech 开发（现属 Qt Group），核心定位是“一次编写，多平台运行”，不仅支持 Windows/macOS/Linux 等桌面系统，还能深度适配嵌入式 Linux、Android、iOS 甚至实时操作系统（RTOS）。

QT 之所以能成为嵌入式开发的主流选择，核心在于它包含两大核心能力：

跨平台的“抽象层”——屏蔽硬件 / 系统差异：

QT 会对不同硬件（如不同芯片的显示控制器、输入设备）和系统（如嵌入式 Linux、桌面 Linux）的底层接口做“抽象封装”，比如：

- **显示抽象**：不管硬件是用帧缓冲（`/dev/fb0`）、GPU 加速（OpenGL ES）还是 X11，QT 都通过统一的“平台插件”适配；
- **输入抽象**：不管是 USB 鼠标、触摸板还是按键，QT 都通过 `evdev` 插件统一处理，开发者只需调用 `QMouseEvent` 等接口，就能响应所有输入设备；
- **文件 / 网络抽象**：不同系统的文件路径、网络接口差异，QT 也通过 `QFile`、`QTcpSocket` 等类屏蔽，代码在嵌入式和桌面系统上可直接复用。

丰富的“组件库”——覆盖嵌入式开发核心需求：

QT 自带大量开箱即用的组件，无需开发者重复造轮子，尤其适配嵌入式场景：

- **界面组件**：按钮（`QPushButton`）、文本框（`QLineEdit`）、列表（`QListWidget`）等基础控件，还有仪表盘、进度条等工业级控件（`Qt widgets` 模块）；

- **图形渲染**：支持 2D 矢量图形（`QPainter`）、图片解码（支持 JPG/PNG）、字体渲染（适配中文字体）；
- **硬件通信**：串口（`QSerialPort`，嵌入式常用）、网络（TCP/TIP、UDP）、蓝牙等，直接直接操作硬件寄存器；
- **多线程 / 定时器**：`QThread` 处理耗时任务（如数据采集），`QTimer` 实现定时控制，避免界面卡顿。

因此对于嵌入式开发，直接用“裸 C 语言 + 硬件驱动”写界面 / 应用会面临“开发慢、兼容性差、维护难”的问题，而移植 QT 后，这些痛点能被精准解决：

开发效率飙升 —— 从“硬件适配”转向“功能实现”：

嵌入式开发的核心痛点之一是“硬件差异大”：比如换一块开发板，显示驱动、输入设备可能全变，裸 C 代码几乎要重写。而 QT 移植后：

- **无需重复适配硬件**：移植时只需配置好“平台插件”（如 `linuxfb` 适配帧缓冲、`evdev` 适配鼠标），后续所有 QT 应用都能直接复用，不用再管 `fb0` 或 `eventX` 设备节点；
- **界面开发可视化**：配合 QT 的可视化设计工具 `Qt Designer`，拖放控件就能生成界面，无需手动计算控件坐标、处理刷新逻辑；
- **代码复用率高**：在 Windows 上用 QT 写好的界面代码，移植到 Rvcard 上只需重新编译，不用修改业务逻辑，到开发板上能直接跑。

功能更强大 —— 轻松实现复杂需求：

裸 C 语言在嵌入式中实现“中文显示”“图形交互”“网络通信”等需求，需要手动集成第三方库（如 `freetype` 处理字体、`libjpeg` 解码图片、`lwip` 做网络），还得解决库之间的兼容性问题。而 QT 自带这些能力：

- **图形交互更流畅**：支持界面动画（如按钮点击效果）、窗口切换，且 `Qt Widgets` 做了优化，在嵌入式低性能硬件上也能流畅运行；
- **多功能一键集成**：比如要做一个“串口数据采集 + 界面显示 + 网络上传”的程序，用 QT 只需调用 `QSerialPort`（串口）、`QTableWidget`（数据显示）、`QTcpSocket`（网络），300 行代码就能实现，而裸 C 可能需要上千行。

稳定性更高 —— 减少“底层 Bug”：

嵌入式系统对稳定性要求极高（如工业控制、物联网设备），而 QT 经过几十年迭代，底层代码非常成熟：

- **内存管理更安全**：QT 的 `QObject` 类支持“父子对象机制”—— 父对象销毁时自动销毁子对象，避免裸 C 中常见的内存泄漏；
- **错误处理更完善**：QT 的类都提供了错误信号（如 `QSerialPort` 的 `errorOccurred` 信号），能精准捕获硬件异常（如串口断开），而裸 C 需手动检查每个函数返回值；
- **兼容性经过验证**：QT 对嵌入式 Linux 的主流芯片都有适配案例，减少“硬件不兼容”导致的崩溃。

后期维护更简单 —— 代码可读性 / 扩展性强：

嵌入式设备通常需要长期维护（如迭代功能、修复 Bug），而 QT 代码的“模块化”设计让维护成本大幅降低：

- **代码结构清晰**: QT 应用按“界面层”“业务逻辑层”“硬件交互层”拆分（如界面用 Qt Designer 生成，业务逻辑写在 QMainWindow 子类中，硬件交互封装成独立类），后续改界面不用动业务代码；
- **功能扩展方便**: 比如要给 mousebuttons 程序加“键盘支持”，只需添加 QKeyEvent 的处理函数，不用修改鼠标相关代码；
- **调试工具丰富**: 配合 Qt Creator 的远程调试功能，能在电脑上实时调试 Luckfox 上的 QT 程序，查看变量值、定位崩溃原因，比裸 C 用 gdb 调试效率高很多。

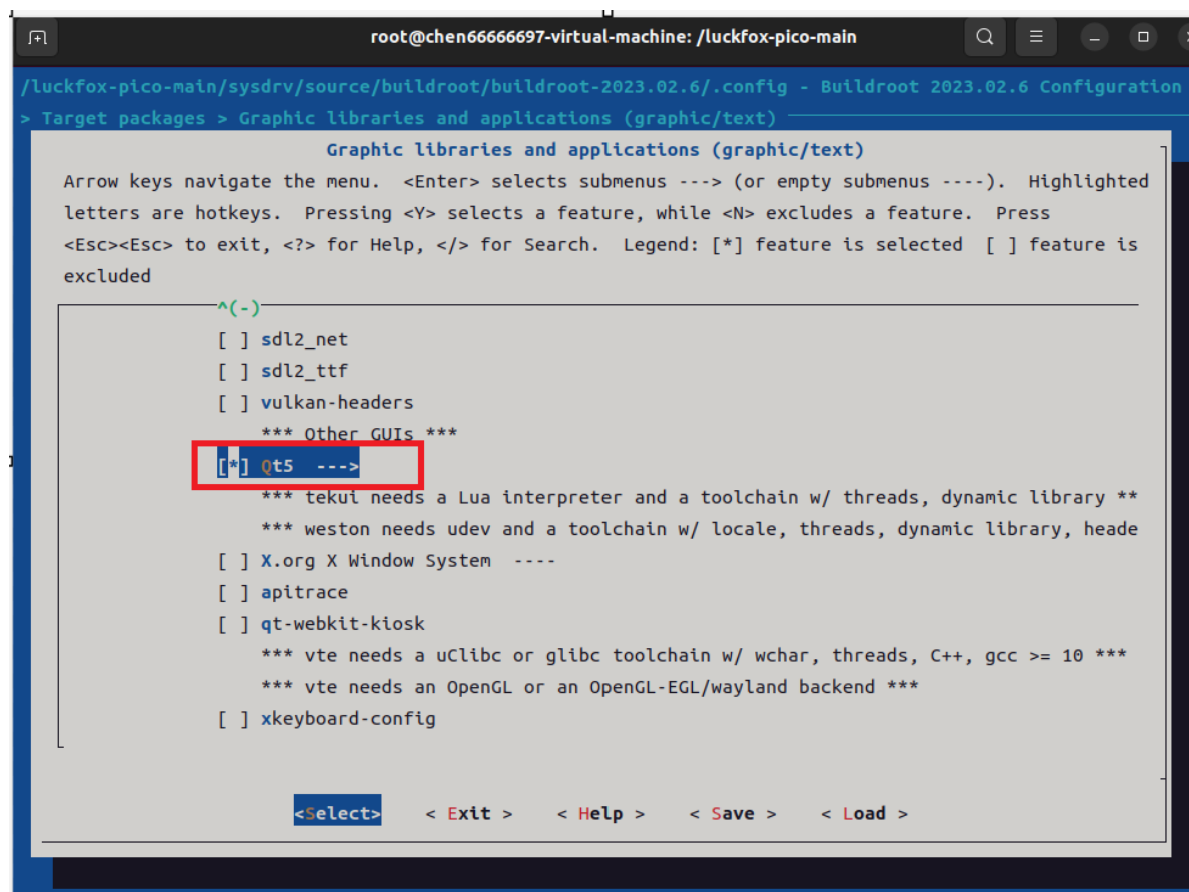
总之 QT 非常强大，非常值得学习，学习成本比其他图形库的要低，并且功能及其丰富，对于开发图形化应用非常轻松。所以这一节也是我们重点。

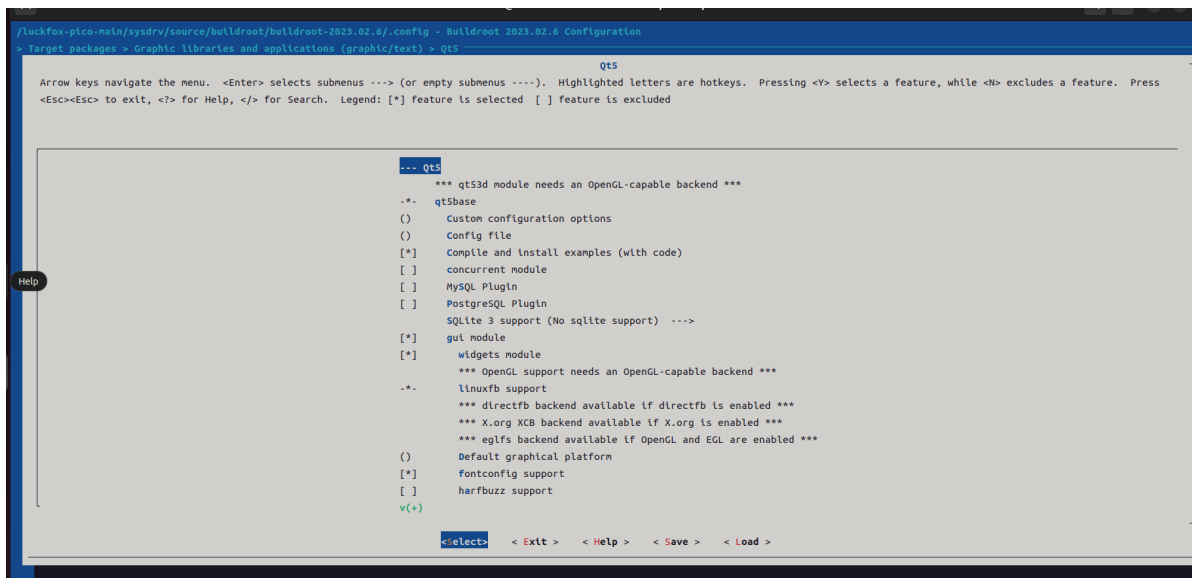
3.3.1 buildroot添加QT库

废话不多说，在SDK下执行

```
./build.sh buildrootconfig
```

勾选如下：





`Compile and install examples (with code)` : 是会编译进我们根文件镜像中的例程，会占用一部分空间，后面可以取消掉。

`fontconfig support` : 与我们的中文字体有关。

保存，退出，编译根文件，编译完后，我们的镜像会增加 20M 的空间，同时可以看到我们的QT 有许多例程。

```
[root@luckfox examples]# ls
README      gui          qtconcurrent  widgets
corelib     network     qtestlib      xml
examples.pro qpa         sql
[root@luckfox examples]# pwd
/usr/lib/qt/examples
[root@luckfox examples]#
```

我们先不急，先进入下一节，因为，我们需要输入设备与QT交互。

3.3.2 QT输入设备配置

我们一般使用QT程序，在小设备上多使用触摸，但是我们的板子无法触摸（实际上底板可以加，不过我已经画不下了），因此，需要其他的输入输出设备，键盘，鼠标才是最常用的输入设备，下面我们来为QT添加键盘，鼠标支持。

```
luckfox-pico-main/sysdrv/source/buildroot/buildroot-2023.02.6/.config - Buildroot 2023.02.6 Configuration
Target packages > Hardware handling

Hardware handling
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----). Highlighted letters are hotkeys. Pressing <Y> select
<Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] feature is selected [ ] feature is excluded

^(-)
[ ] dvb-apps
[ ] dvbsnoop
[ ] edid-decode
*** eudev needs eudev /dev management ***
[*] eudev
[*] evtest
[ ] fan-ctrl
[ ] fconfig
[ ] flashrom
[ ] fntools
[ ] freeipmi
[ ] Freescale i.MX libraries ----
[ ] fxload
*** gcnano-binaries needs a Linux kernel to be built ***
*** gcnano-binaries needs a glibc toolchain w/ threads, dynamic library ***
[ ] gpm
[ ] gpsd ----
[ ] gptfdisk
[ ] gvfs
[ ] hddtemp
v(+)
```

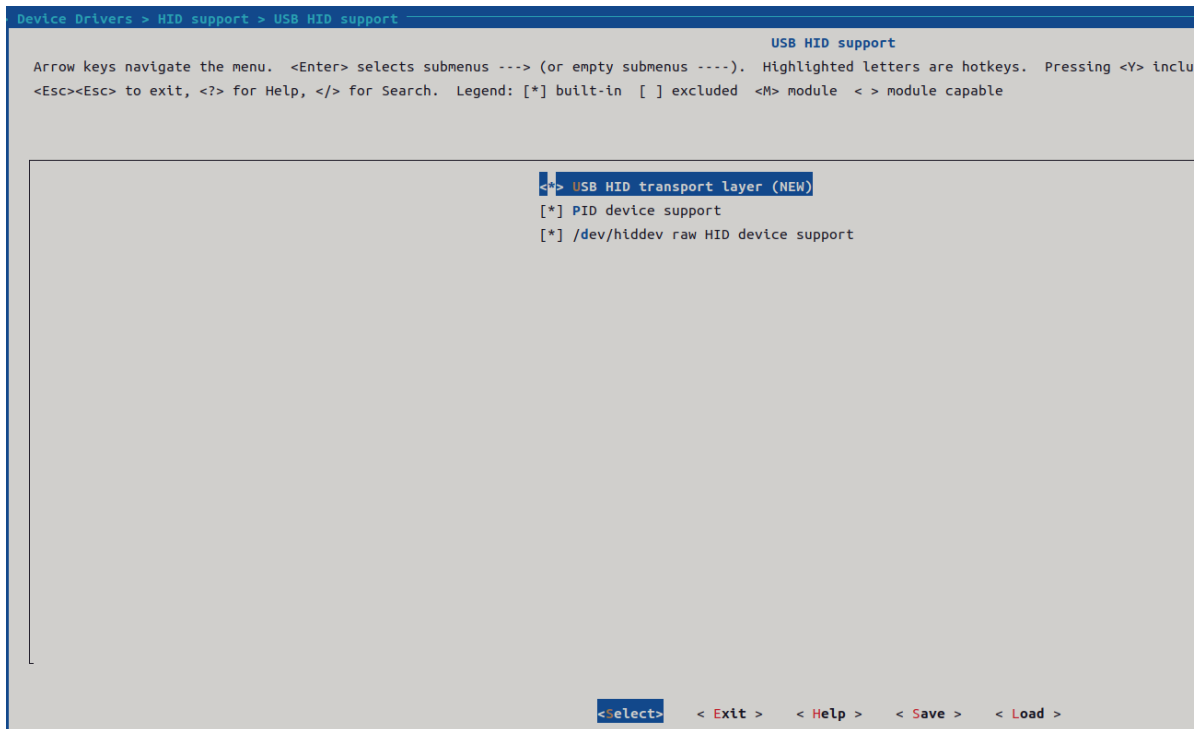
这两个是为了添加能被QT检测到的设备事件。

接下来我们要配置linux内核支持 通用 HID 设备驱动：

```
hfig - Linux/arm 5.10.160 Kernel Configuration
Device Drivers > HID support

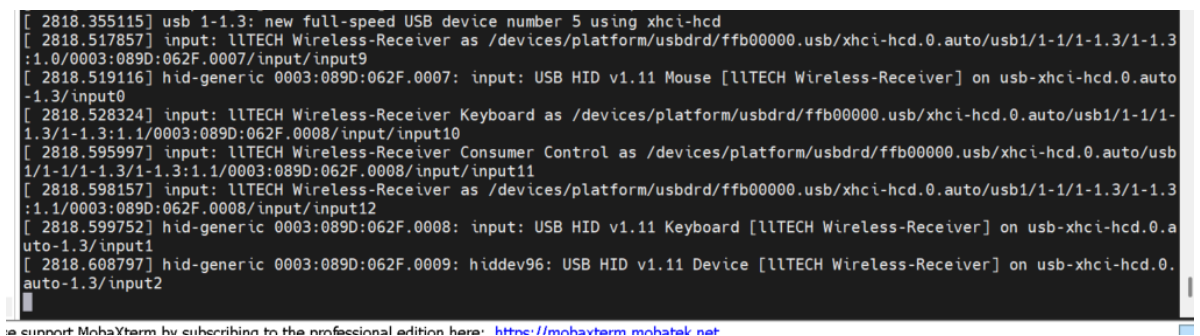
HID support
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----). Highlighted letters are hotkeys. Pressing <Y>
<Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] built-in [ ] excluded <M> module <> module capable

-- HID bus support
[ ] Battery level reporting for HID devices (NEW)
[ ] /dev/hidraw raw HID device support (NEW)
<*> User-space I/O driver support for HID subsystem
<*> Generic HID driver (NEW)
Special HID drivers --->
USB HID support --->
I2C HID support --->
```



接下来编译内核，并烧录。

插入鼠标



可以看到，已经正确识别到我的2.4g鼠标了，如果没识别到，它就只会检测到usb设备插入，不会生成相应的节点，它的打印类似于：

```
[ 57.584691] usb 1-1.3: new full-speed USB device number 3 using xhci-hcd
```

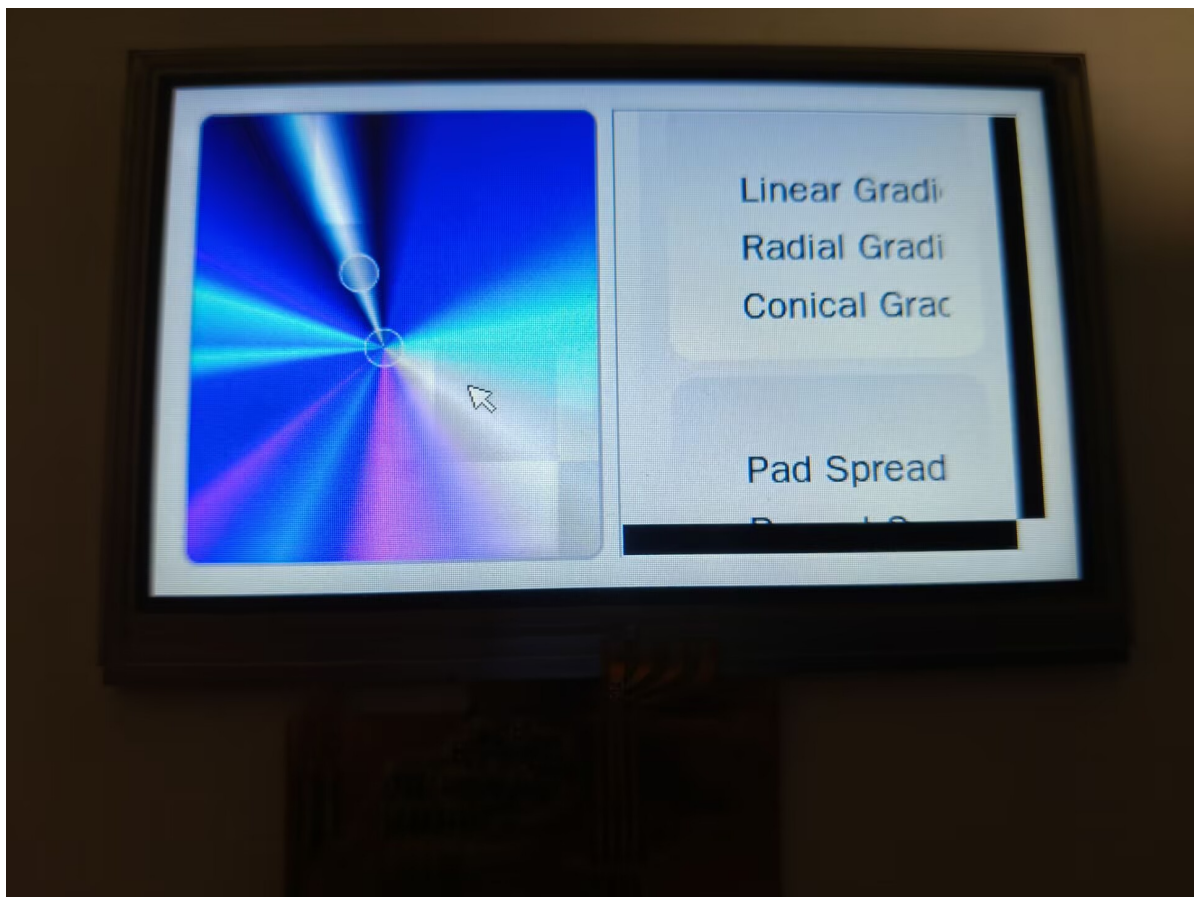
接下来，我们来运行下例程，由于我们使用的是linuxfb，所以需要声明下环境变量：

```
# 选择 framebuffer 作为输出
export QT_QPA_PLATFORM=linuxfb
# 设置键盘/鼠标输入为 evdev
export QT_QPA_GENERIC_PLUGINS=evdevmouse, evdevkeyboard
# 指定字体路径（你安装的 TTF 中文字体）
export QT_QPA_FONTDIR=/usr/share/fonts
```

执行，

```
/usr/lib/qt/examples/widgets/painting/gradients/gradients
```

应该已经成功运行了吧！！恭喜你，试着用鼠标拖动滑块或锚点吧。



如果出现这样的错误，是因为你没告诉它用linux-fb设备。

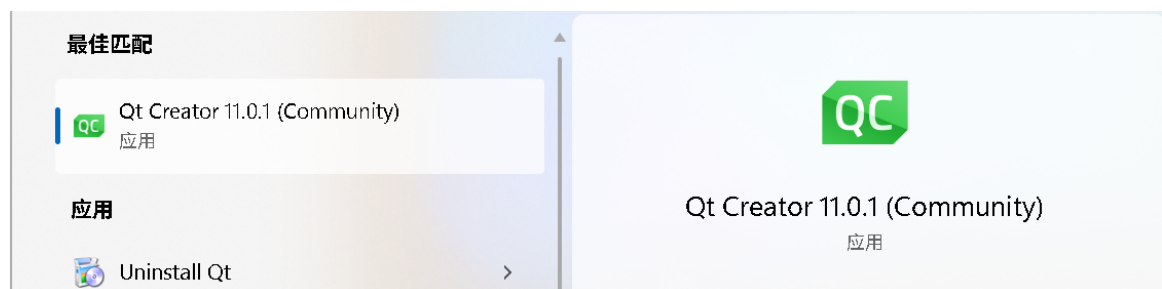
```
[root@luckfox ~]# /usr/lib/qt/examples/widgets/painting/gradients/gradients
qt.qpa.plugin: Could not find the Qt platform plugin "eglfs" in ""
This application failed to start because no Qt platform plugin could be initialized. Reinstalling the application may fix this problem.

Available platform plugins are: linuxfb, minimal, offscreen, vnc.

Aborted (core dumped)
[root@luckfox ~]#
```

3.3.3 交叉编译QT

接下来，我们讲解下，作为用户我们怎么自己设计自己的QT程序。相信大家一定听说过QT creators吧？

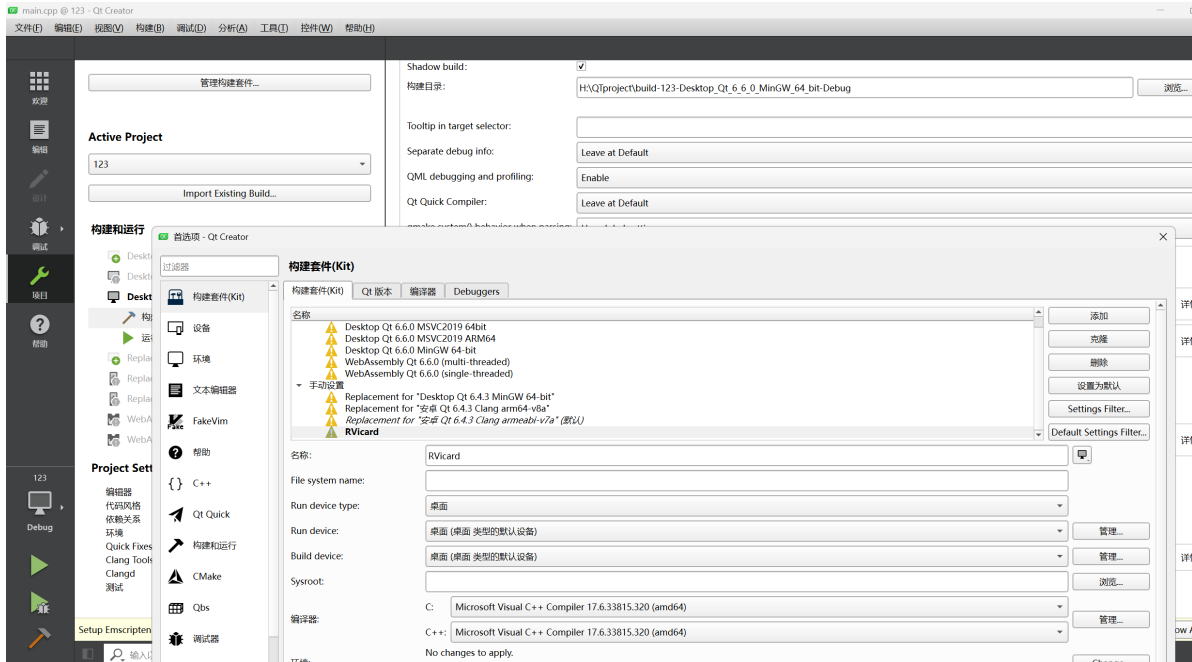


QT creators 是一款可以可视化设计QT的官方工具，设计的相当的美观（up非常喜欢QT）。这只是一个启动器，下载运行后，你需要选择QT的版本及需要的包，才能真正开始设计。不过既然是，运行在arm上的，我们一般，会把QT 编辑器下载在ubuntu中，但是，up的虚拟机已经有100多和G了，盘已经红了，所以，我就使用windows的QT 为你（读者）演示，怎么设置QT的交叉编译工具链。

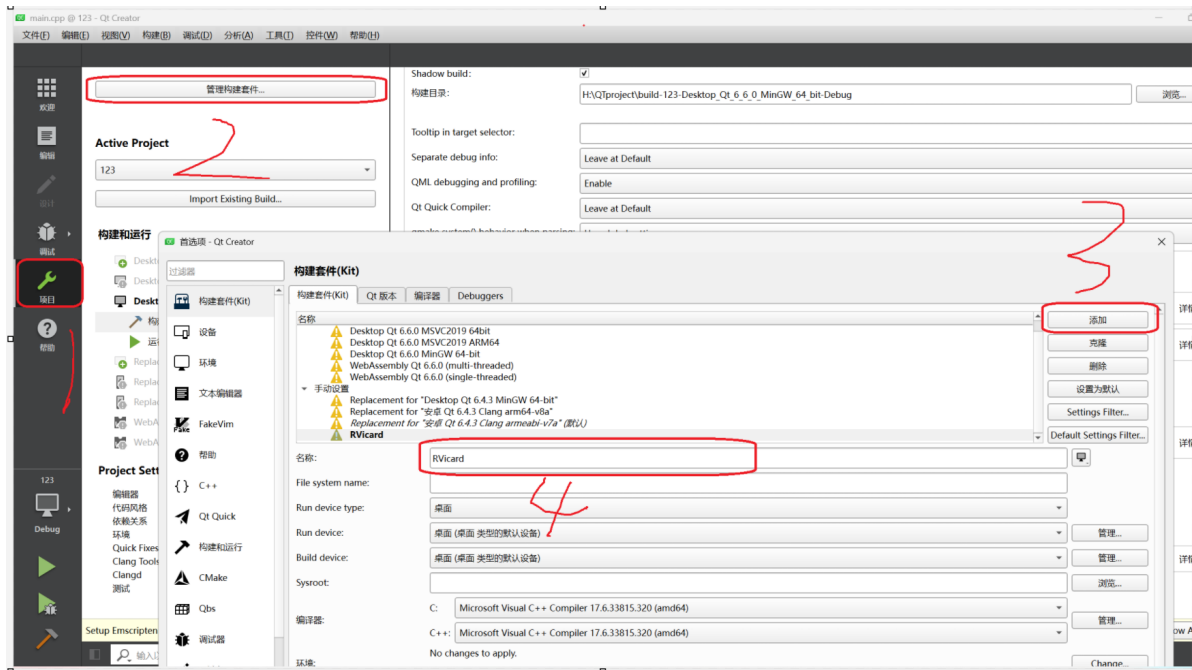
首先我们要知道Qt使用的QT 库是那个版本。

```
root@chen66666697-virtual-machine:/luckfox-pico-main/sysdrv/source/buildroot/bui
ldroot-2023.02.6/output/build/qt5base-2ffb7ad8a1079a0444b9c72affe3d19b089b60de/b
in# ./qmake -v
QMake version 3.1
Using Qt version 5.15.8 in /luckfox-pico-main/sysdrv/source/buildroot/buildroot-
2023.02.6/output/host/arm-buildroot-linux-uclibcgnueabi-hf/sysroot/usr/lib
```

可以看出是 5.15.8，我们的QT需要下载相同的库。



我们随便创建一个工程，依次点击，为我们的QT 套件命名为 RVicard。

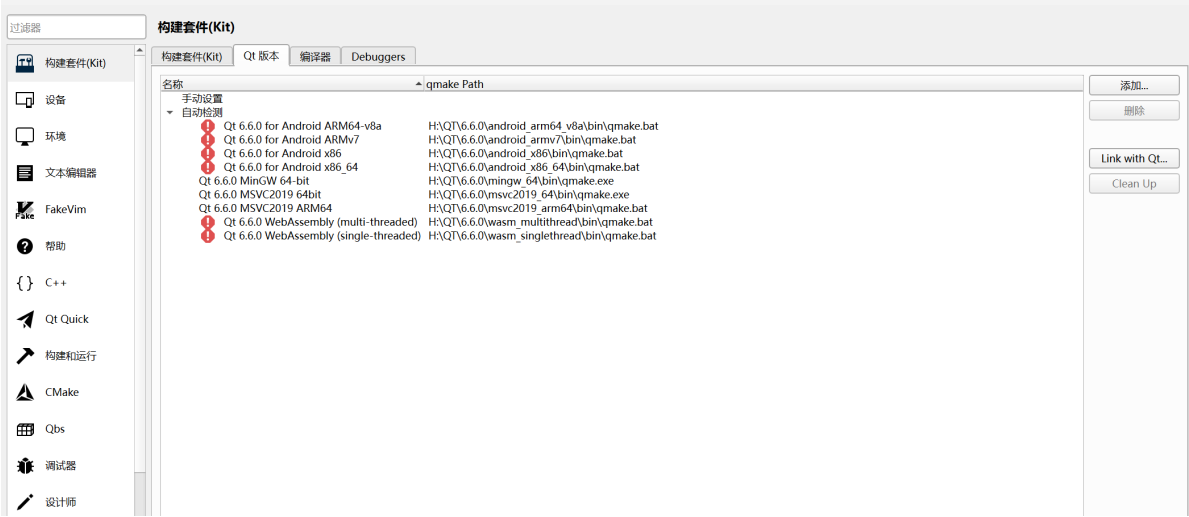


这个套件能成功设置，需要 gcc 编译器 g++ 编译器 qmake 编译器。

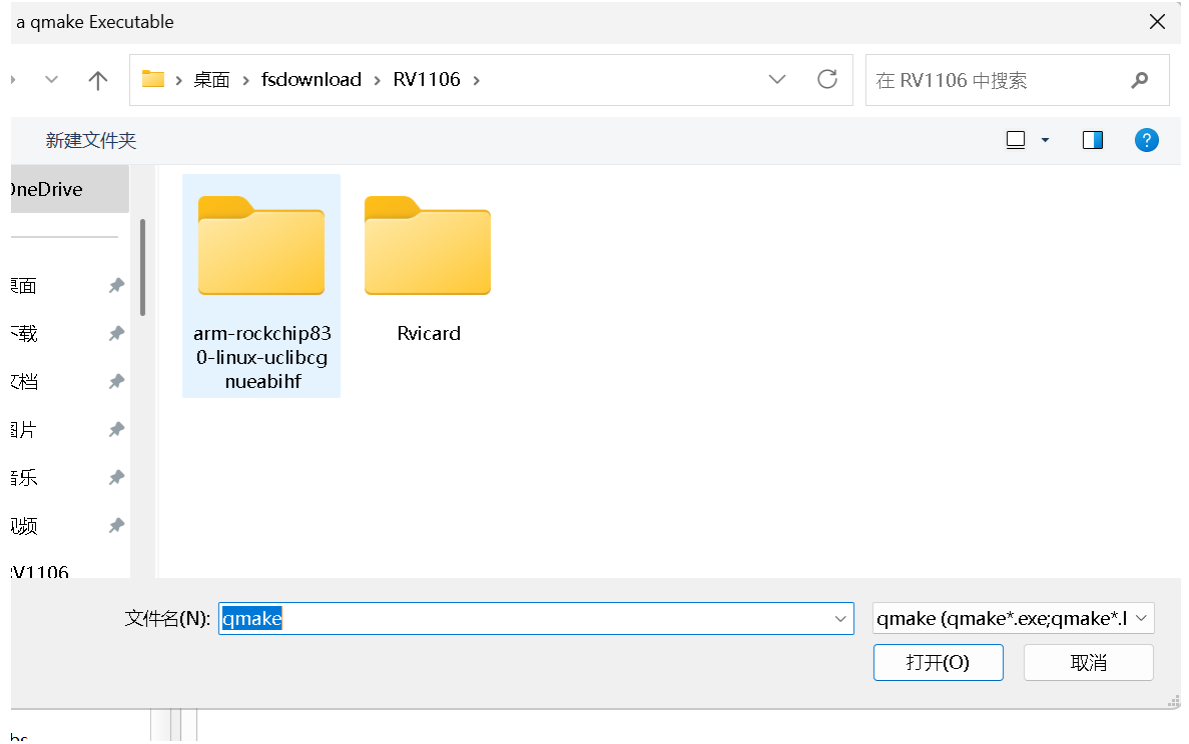
聪明的你（读者），肯定知道 gcc g++ 对吧，但是 qmake 又是什么呢，这个是我们编译QT源码获得的。

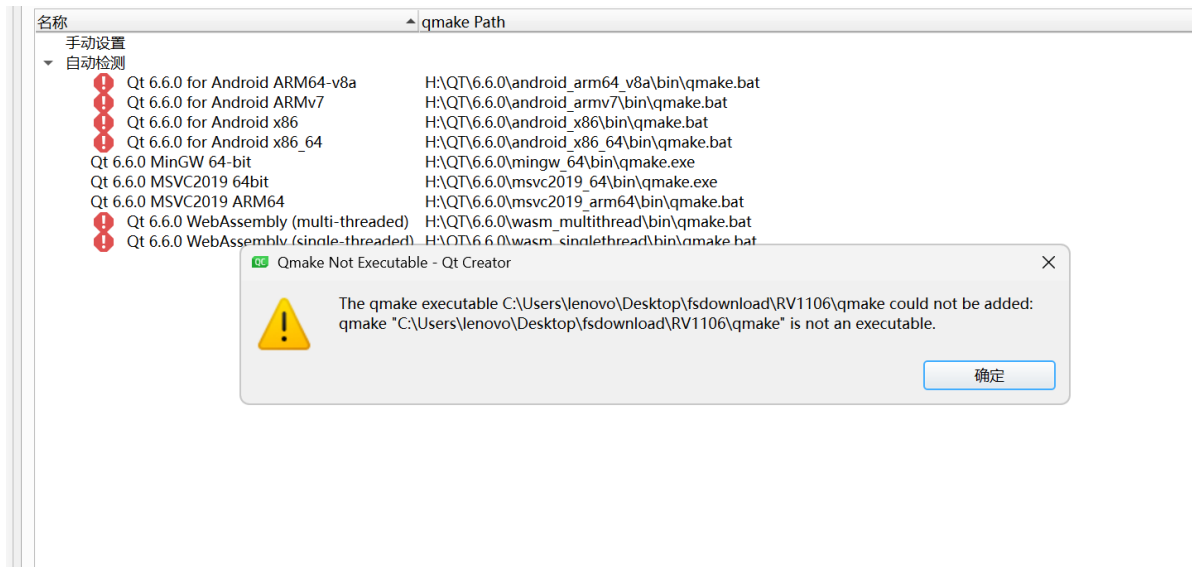
```
root@chen66666697-virtual-machine:/luckfox-pico-main/sysdrv/source/buildroot/buildroot-2023.02.6/output/build/qt5base-2ffb7ad8a1079a0444b9c72affe3d19b089b60de/b
in# ls
fixqt4headers.pl  qlalr  qt.conf  rcc      tracegen
moc               qmake  qvkgen  syncqt.pl  uic
```

下面我们需要把交叉编译工具链，和qmake 拿出来。

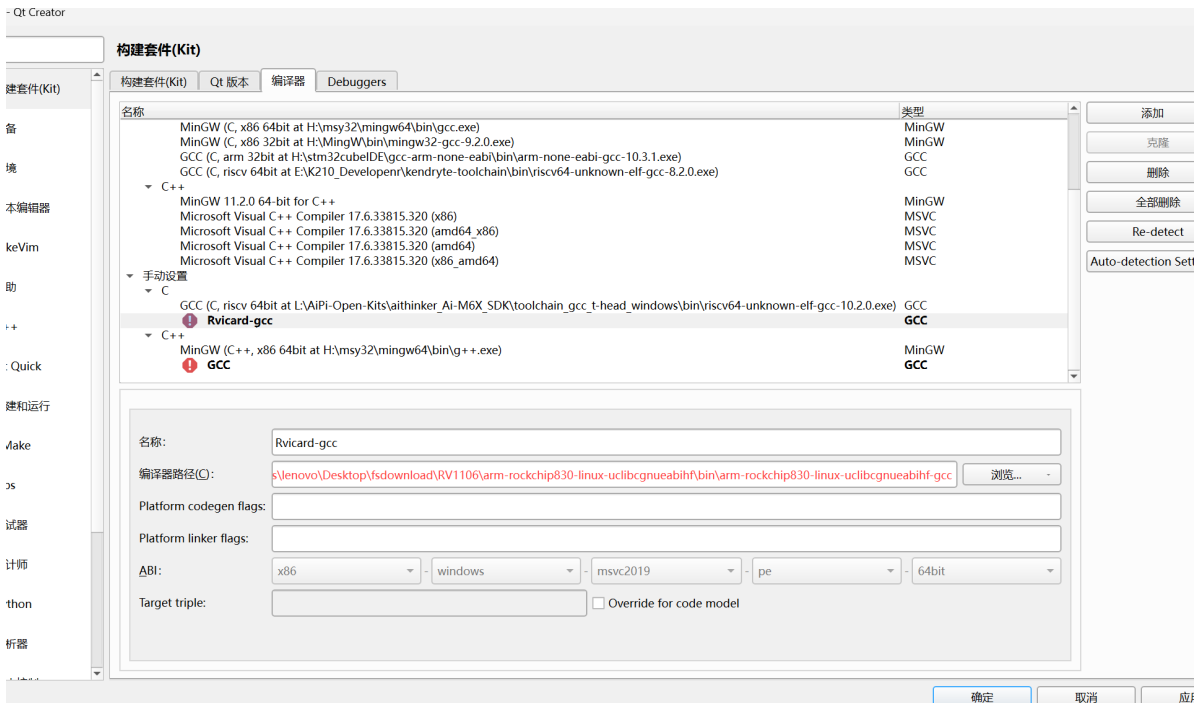


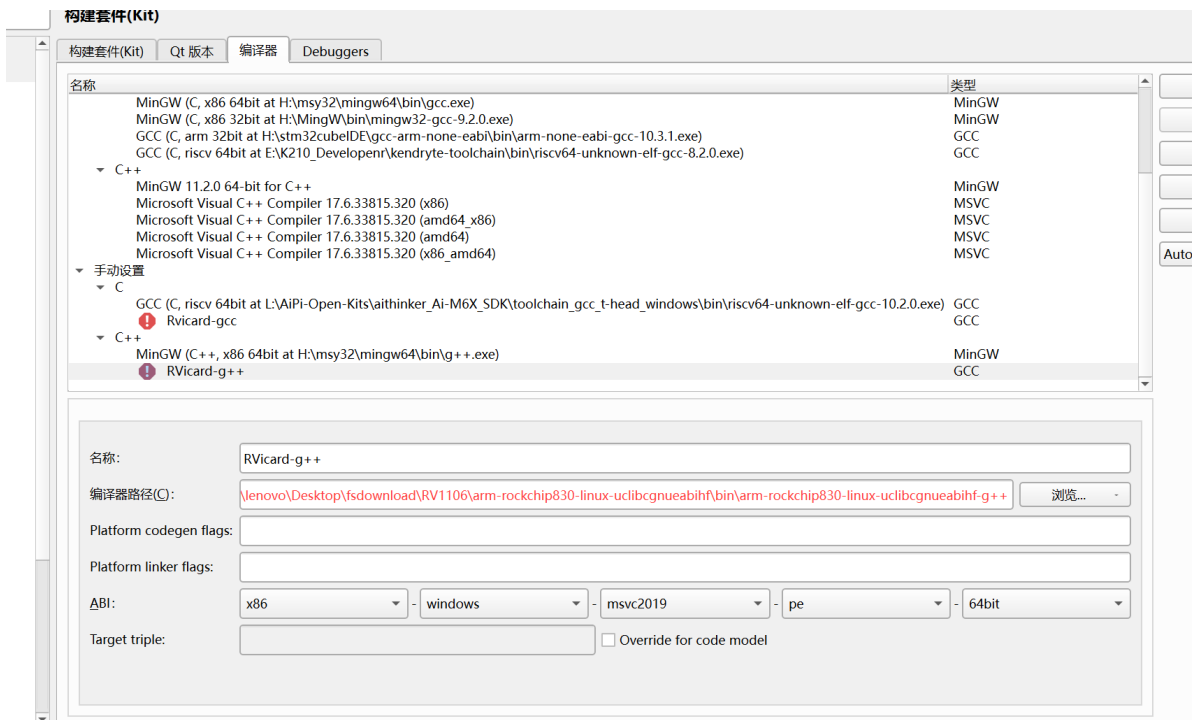
添加Qt版本，其实就是qmake。很遗憾的是windows 强制锁exe，无法添加，读者添加完后，需要命个名字，就叫RVicard-qmake



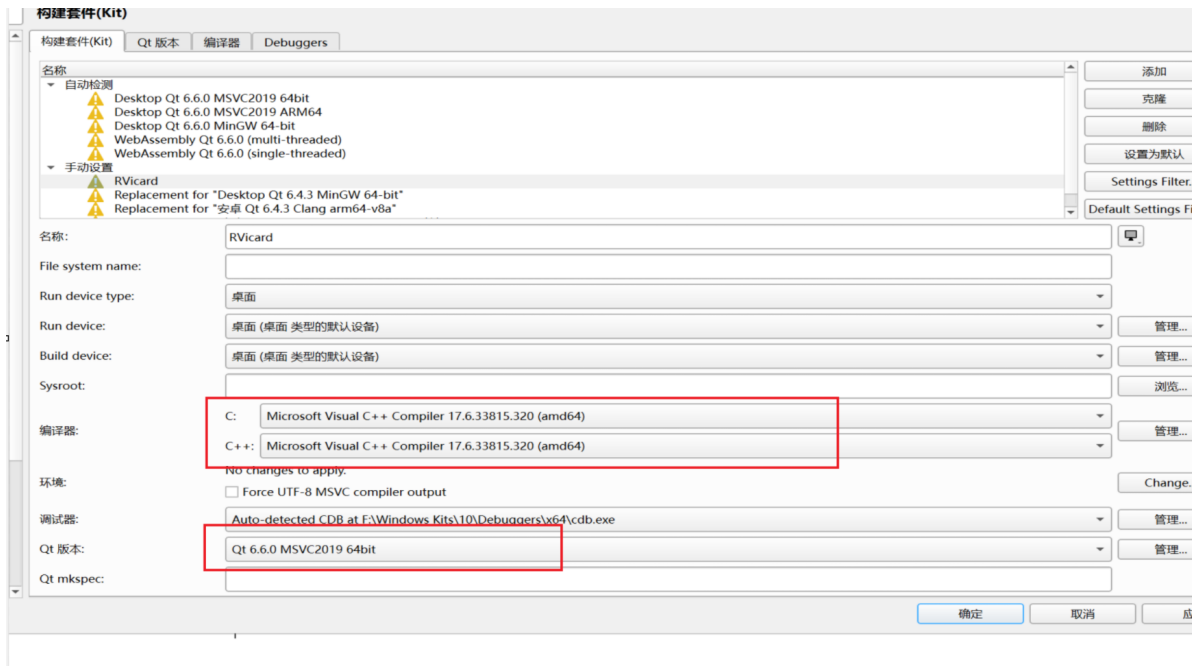


添加gcc 与g++ 编译器:

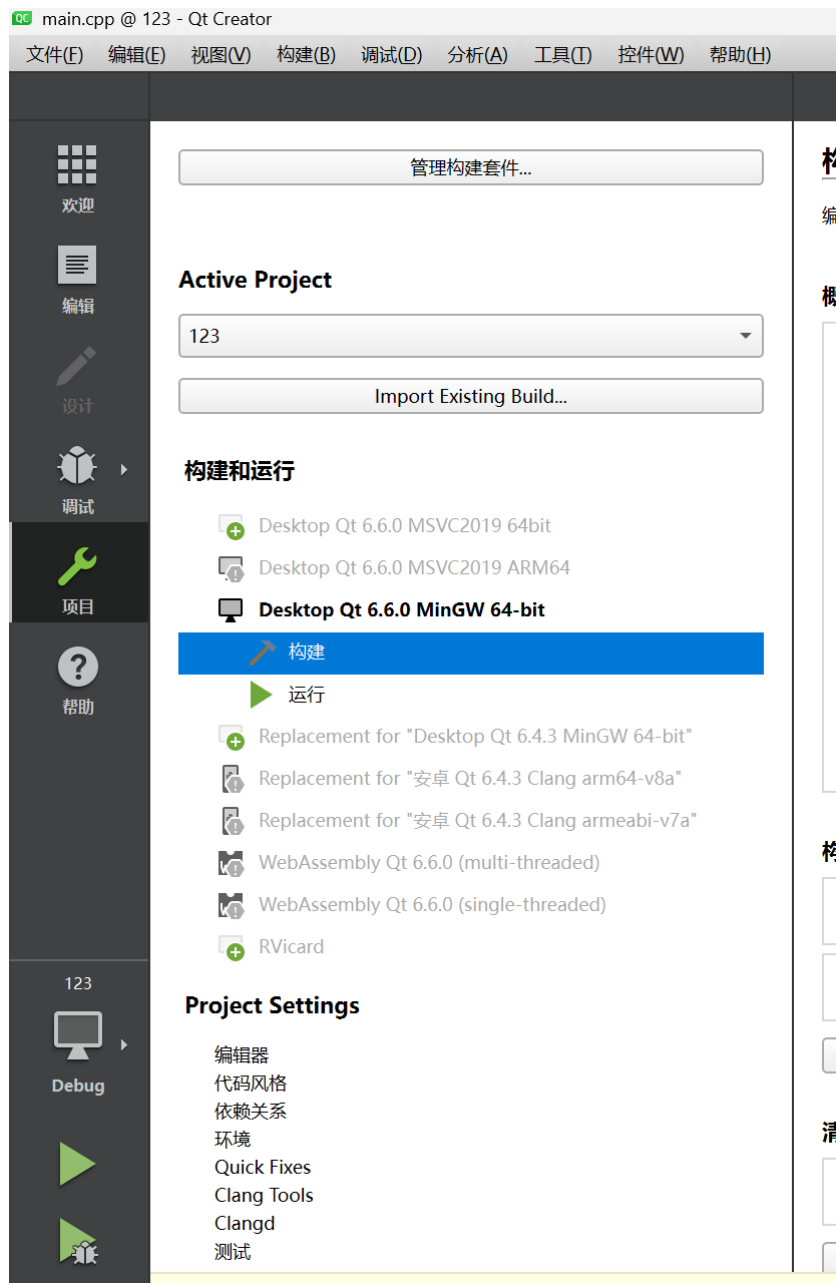




在构建中分别选择我们设置好的：

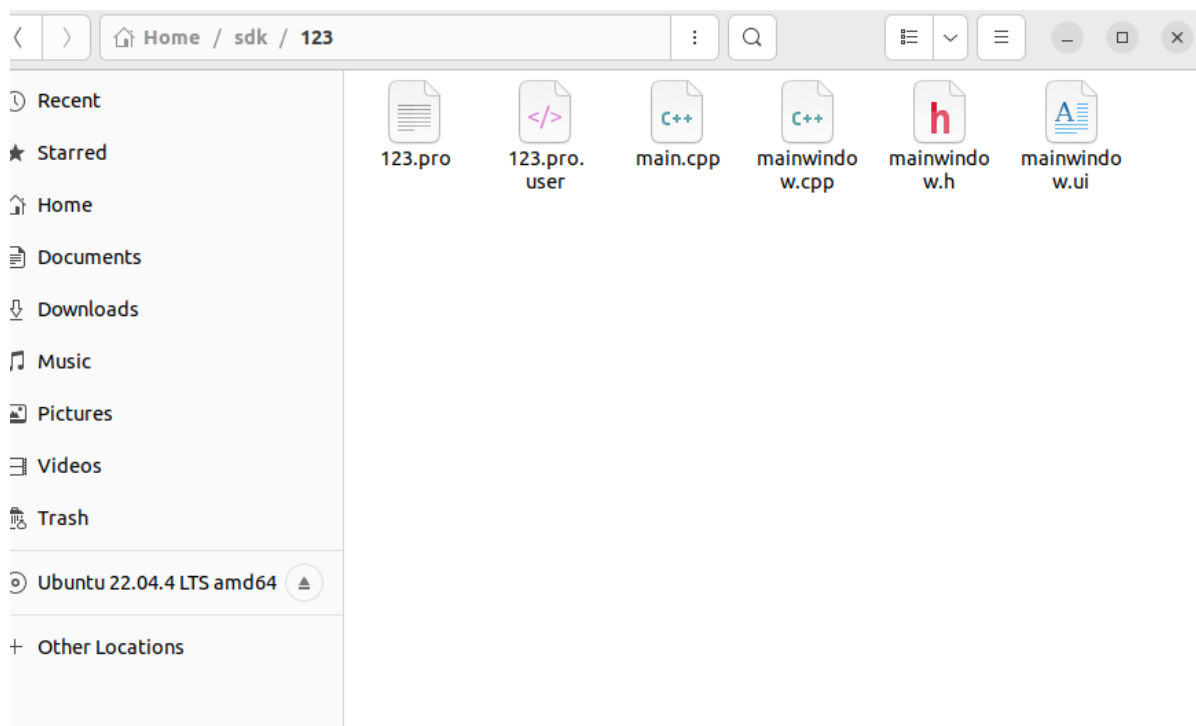


很遗憾的是，up的都报红了，看样子，交叉编译QT必须跑在linux环境中，这里up就做一个演示，之后，你就可以看到你生成了新的构件，使用这个构建可以编译，但是无法运行，毕竟我们的电脑是x86或x64架构，不是arm架构，至于运行效果，查看windows下的就行。



那么up没有Qt creator 我就不能编译代码了吗？开玩笑，怎么可能，能拦得住我。前面我们已经知道，只需要那三个编译器就可以编译了。

我们在windows中随便创建一个工程，其中，pro.usr 是windows下的，我们不需要，删除。

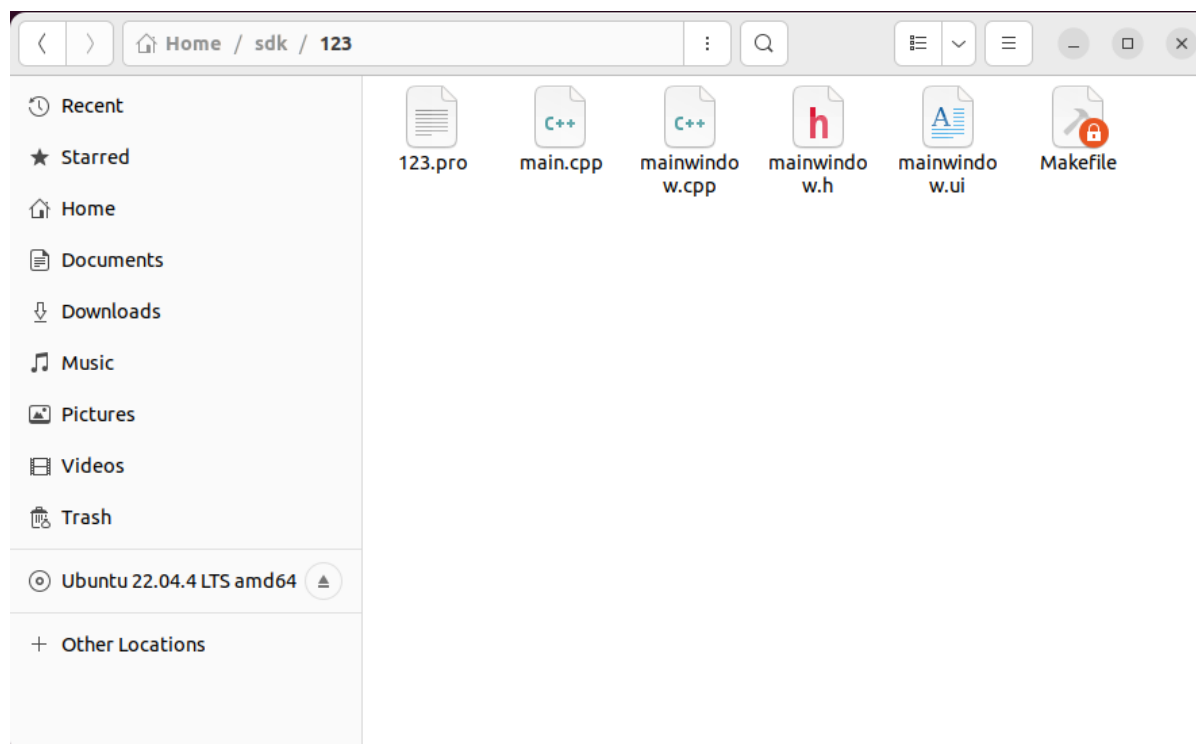


接下来我们准备编译这个工程，其中 pro 就是我们该编译的目标文件，它记载其他文件该怎么编译。

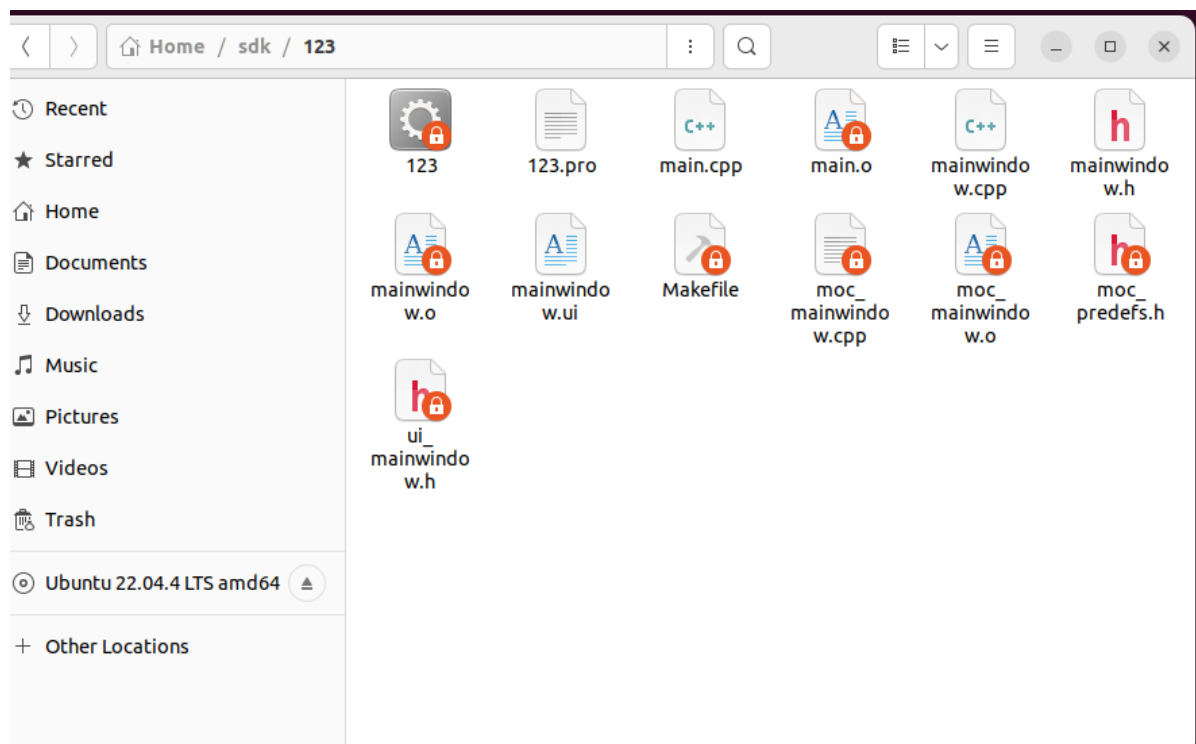
声明交叉编译工具链，并使用qmake进行编译：

```
export COMPILE=arm-rockchip830-linux-uc1bcgnueabihf-  
  
luckfox-pico-main/sysdrv/source/buildroot/buildroot-  
2023.02.6/output/build/qt5base-2ffb7ad8a1079a0444b9c72affe3d19b089b60de/bin/qmke  
123.pro
```

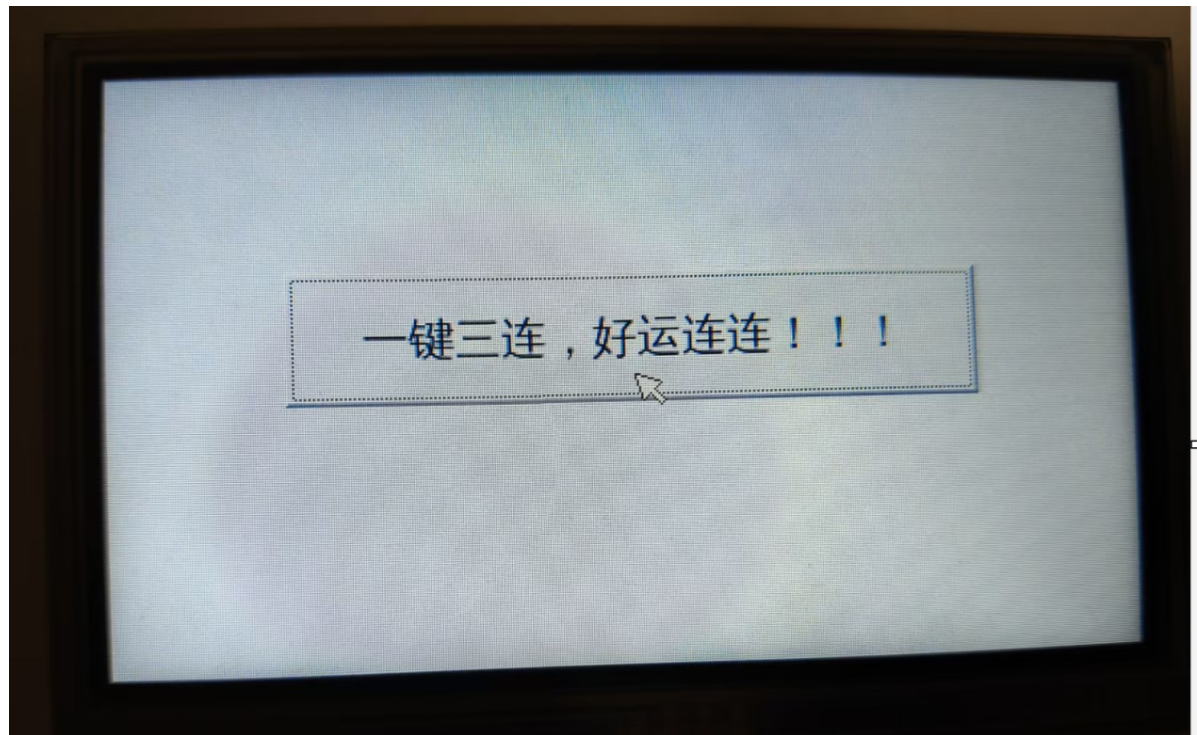
它输出了makefile文件，那么我们就可以执行make了：



如图我们已经得到了我们得目标文件。



接着我们运行下吧!!!



一键三连，好运连连!!! (鼠标疯狂点击中.jpg)

3.4 RKNN 模型量化与推理

3.4.1 RKNN-Toolkit2 下载与demo使用

我们需要下载RKNN-Toolkit2，官网有相关教程 [RKNN 推理测试 | LUCKFOX WIKI](#) 大家可以上官网学习下载。这里只简单介绍，up下这个折腾了近5个小时，网络不稳定且慢，注意up是 python 3.10

```
# 用腾讯云源安装，超时设为180秒
pip install --default-timeout=180 -i
https://mirrors.cloud.tencent.com/pypi/simple/ rknn_toolkit2-2.3.2-cp310-cp310-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl

# 或用华为云源 pip install --default-timeout=180 -i
https://mirrors.huaweicloud.com/repository/pypi/simple/ rknn_toolkit2-2.3.2-
cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl

# 如果遇到下载某个失败，单独下载它
pip install --default-timeout=180 -i
https://mirrors.cloud.tencent.com/pypi/simple/ nvidia-cublas-cu12==12.1.3.1
```

官方兼容可能没做好，onnx的库有问题，高版本缺少了 mapping。

```
I ===== WARN(0) =====
E rknn-toolkit2 version: 2.3.2
Traceback (most recent call last):
  File "rknn/api/rknn_log.py", line 344, in
rknn.api.rknn_log.error_decorator.error_catch_wrapper
  File "rknn/api/rknn_base.py", line 1579, in
rknn.api.rknn_base.RKNNBase.load_onnx
  File "rknn/api/rknn_base.py", line 613, in
rknn.api.rknn_base.RKNNBase._create_ir_and_inputs_meta
  File "rknn/api/ir_graph.py", line 84, in rknn.api.ir_graph.IRGraph.__init__
  File "rknn/api/ir_graph.py", line 665, in rknn.api.ir_graph.IRGraph.rebuild
  File "rknn/api/base_utils.py", line 34, in rknn.api.base_utils.to_np_type
AttributeError: module 'onnx' has no attribute 'mapping'

During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File
"/home/chen66666697/sdk/rknn/rknn_model_zoo/examples/yolov5/python/convert.py",
line 52, in <module>
    ret = rknn.load_onnx(model=model_path)
  File "/home/chen66666697/.local/lib/python3.10/site-packages/rknn/api/rknn.py",
line 168, in load_onnx
    return self.rknn_base.load_onnx(model, inputs, input_size_list,
input_initial_val, outputs)
  File "rknn/api/rknn_log.py", line 349, in
rknn.api.rknn_log.error_decorator.error_catch_wrapper
  File "rknn/api/rknn_log.py", line 95, in rknn.api.rknn_log.RKNNLog.e
ValueError: Traceback (most recent call last):
  File "rknn/api/rknn_log.py", line 344, in
rknn.api.rknn_log.error_decorator.error_catch_wrapper
  File "rknn/api/rknn_base.py", line 1579, in
rknn.api.rknn_base.RKNNBase.load_onnx
```

```
File "rknn/api/rknn_base.py", line 613, in
rknn.api.rknn_base.RKNNBase._create_ir_and_inputs_meta
File "rknn/api/ir_graph.py", line 84, in rknn.api.ir_graph.IRGraph.__init__
File "rknn/api/ir_graph.py", line 665, in rknn.api.ir_graph.IRGraph.rebuild
File "rknn/api/base_utils.py", line 34, in rknn.api.base_utils.to_np_type
AttributeError: module 'onnx' has no attribute 'mapping'
```

```
# 卸载 官方的 onnx 版本
pip uninstall -y onnx

#下载 低版本 onnx
pip install -i https://mirrors.aliyun.com/pypi/simple/ onnx==1.13.1
```

做这个是为了安装 RKNN。

```
# 如果成功导入，则安装成功
from rknn.api import RKNN
```

接下来我们下载 rknn_model_zoo。

```
git clone https://github.com/airockchip/rknn_model_zoo.git
```

获取 YOLOv5 ONNX模型文件

```
cd <rknn_model_zoo Path>/rknn_model_zoo/examples/yolov5/model
chmod a+x download_model.sh
./download_model.sh
```

转化模型

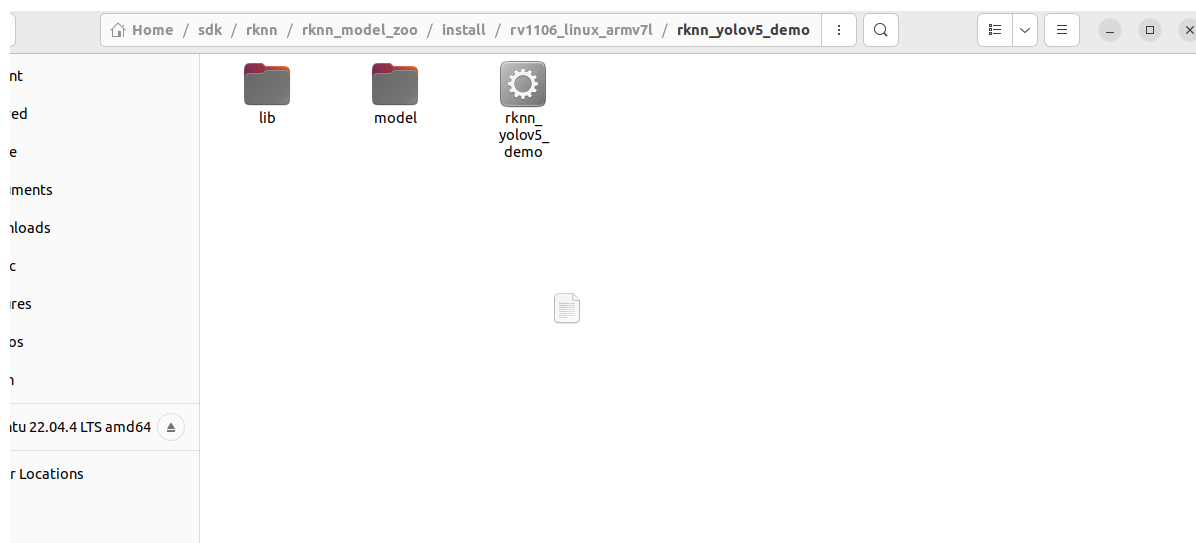
```
cd <rknn_model_zoo Path>/rknn_model_zoo/examples/yolov5/python
python3 convert.py ../model/yolov5s_relu.onnx rv1106
```

关于模型路径，也可以，在convert.py 中进行设置比

```
Open  [F1] convert.py ~/sdk/rknn/rknn_model_zoo/examples/yolov5/python Save [Menu] [Close] [X]
1 import sys
2
3 from rknn.api import RKNN
4
5 DATASET_PATH = '.././../datasets/COCO0/coco_subset_20.txt'
6 DEFAULT_RKNN_PATH = '../model/yolov5_relu.rknn'
7 DEFAULT_QUANT = True
8
9 def parse_arg():
10     if len(sys.argv) < 3:
11         print("Usage: python3 {} onnx_model_path [platform] [dtype(optional)]".format(sys.argv[0]))
12         print("        platform choose from [rk3562, rk3566, rk3568, rk3576, rk3588, rv1103, rv1106, rv1126b, rv1109, rv1126, rk1808]")
13         print("        dtype choose from [i8, fp] for [rk3562, rk3566, rk3568, rk3576, rk3588, rv1103, rv1106, rv1126b]")
14         print("        dtype choose from [u8, fp] for [rv1109, rv1126, rk1808]")
15         exit(1)
16
17 model_path = sys.argv[1]
18 platform = sys.argv[2]
19
20 do_quant = DEFAULT_QUANT
21 if len(sys.argv) > 3:
22     model_type = sys.argv[3]
23     if model_type not in ['i8', 'u8', 'fp']:
24         print("ERROR: Invalid model type: {}".format(model_type))
25         exit(1)
26     elif model_type in ['i8', 'u8']:
27         do_quant = True
```

回到根目录，执行。

```
export GCC_COMPILER=arm-rockchip830-linux-uclicgngueabihf
./build-linux.sh -t rv1106 -a armv7l -d yolov5
```

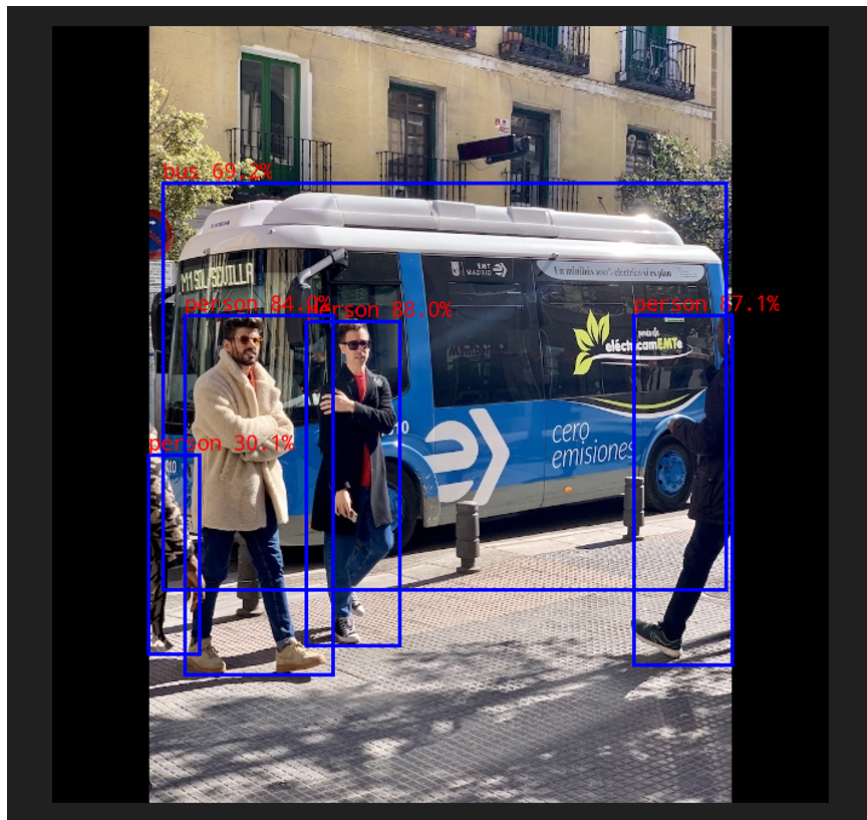


可以看到目录已经生成了，文件夹，这个就是运行所需要的库与环境。

执行 ./rknn_yolov5_demo model/yolov5_relu.rknn mode

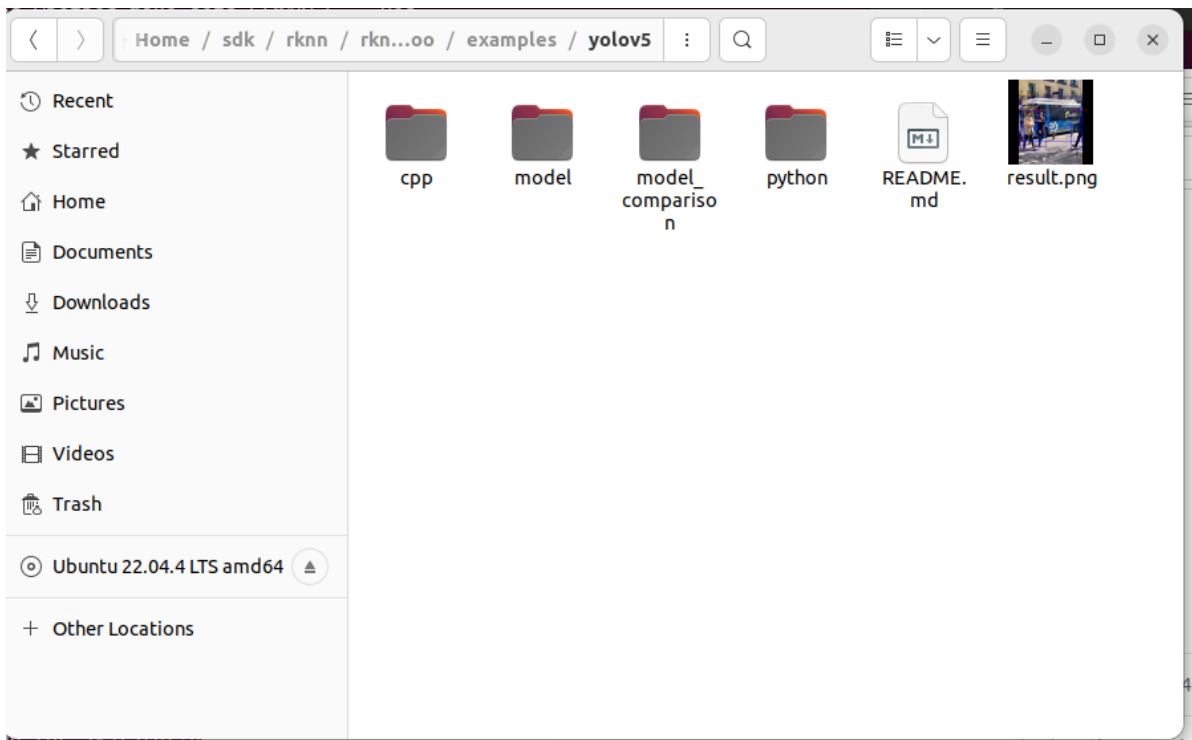
```
[root@luckfox rknn_yolov5_demo]# ./rknn_yolov5_demo model/yolov5_relu.rknn mode
l/bus.jpg
load label ./model/coco_80_labels_list.txt
model input num: 1, output num: 3
input tensors:
  index=0, name=images, n_dims=4, dims=[1, 640, 640, 3], n_elems=1228800, size=1228800, fmt=NHWC, type=INT8, qnt_type=AFFINE
  , zp=-128, scale=0.003922
output tensors:
  index=0, name=output0, n_dims=4, dims=[1, 255, 80, 80], n_elems=1632000, size=1632000, fmt=NCHW, type=INT8, qnt_type=AFFIN
  E, zp=-128, scale=0.003922
  index=1, name=286, n_dims=4, dims=[1, 255, 40, 40], n_elems=408000, size=408000, fmt=NCHW, type=INT8, qnt_type=AFFINE, zp=
  -128, scale=0.003922
  index=2, name=288, n_dims=4, dims=[1, 255, 20, 20], n_elems=102000, size=102000, fmt=NCHW, type=INT8, qnt_type=AFFINE, zp=
  -128, scale=0.003922
input_attrs[0].size_with_stride=1228800
model is NHWC input fmt
model input height=640, width=640, channel=3
origin size=640x640 crop size=640x640
input image: 640 x 640, subsampling: 4:2:0, colorspace: YCbCr, orientation: 1
scale=1.000000 dst_box=(0 0 639 639) allow_slight_change=1 _left_offset=0 _top_offset=0 padding_w=0 padding_h=0
rga_api version 1.10.1 [0]
rknn_run
@ (209 243 286 510) 0.880
@ (479 238 560 526) 0.871
@ (109 238 231 534) 0.840
@ (91 129 555 464) 0.692
@ (79 353 121 517) 0.301
write_image path: out.png width=640 height=640 channel=3 data=0xa5a3b000
[root@luckfox rknn_yolov5_demo]#
```

我们使用adb pull命令把图片拉取下来，可以看到预测成功了。



3.4.2 物品检测

使用图片预测明显是不够的，我们需要一般需要使用视频或连续的图片去预测，目前我们的开发板跑的是并不是完整的opencv，这个opencv-mobile 仅支持摄像头或单个图片的输入，这意味着我们视频源是没办法输入的，于是我把视频源拆解成一张张图片，由代码循环读取。我们还是上一个demo，进入 yolov5 的源代码目录。



进入CPP 修改main.cc

代码如下：

```
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/ioctl.h>
#include <sys/mman.h>
#include <linux/fb.h>
#include <dirent.h>
#include <vector>
#include <string>
#include <algorithm>

#include "yolov5.h"
#include "image_utils.h"
#include "file_utils.h"
#include "image_drawing.h"

#if defined(RV1106_1103)
#include "dma_alloc.hpp"
#endif

#define MODEL_WIDTH 640 // 模型输入宽度
#define MODEL_HEIGHT 640 // 模型输入高度

// framebuffer 初始化
void* fb_init(int* fb, int* width, int* height, int* pixel_size) {
    *fb = open("/dev/fb0", O_RDWR);
    if(*fb < 0) return NULL;
```

```

    struct fb_var_screeninfo vinfo;
    struct fb_fix_screeninfo finfo;
    ioctl(*fb, FBIOGET_VSCREENINFO, &vinfo);
    ioctl(*fb, FBIOGET_FSCREENINFO, &finfo);

    *width = vinfo.xres;
    *height = vinfo.yres;
    *pixel_size = vinfo.bits_per_pixel / 8;

    size_t screensize = (*width) * (*height) * (*pixel_size);
    void* framebuffer = mmap(NULL, screensize, PROT_READ | PROT_WRITE,
MAP_SHARED, *fb, 0);
    if(framebuffer == MAP_FAILED) {
        close(*fb);
        return NULL;
    }

    return framebuffer;
}

// 获取文件夹下所有 jpg/png 文件
std::vector<std::string> get_frame_files(const std::string& folder) {
    std::vector<std::string> files;
    DIR* dir = opendir(folder.c_str());
    if(!dir) return files;
    struct dirent* entry;
    while ((entry = readdir(dir)) != NULL) {
        std::string name = entry->d_name;
        if(name.size() > 4 &&
            (name.substr(name.size()-4) == ".jpg" || name.substr(name.size()-4) ==
".png"))
            files.push_back(folder + "/" + name);
    }
    closedir(dir);
    std::sort(files.begin(), files.end());
    return files;
}

// RGB24 转 XRGB8888
void rgb24_to_xrgb8888(uint8_t* rgb, uint32_t* fb, int w, int h, int fb_width,
int fb_height) {
    for(int y = 0; y < h && y < fb_height; y++) {
        for(int x = 0; x < w && x < fb_width; x++) {
            int idx = (y * w + x) * 3;
            uint8_t r = rgb[idx + 0];
            uint8_t g = rgb[idx + 1];
            uint8_t b = rgb[idx + 2];
            fb[y * fb_width + x] = (r << 16) | (g << 8) | b; // XRGB8888
        }
    }
}

int main(int argc, char **argv) {
    if(argc != 3) {
        printf("Usage: %s <model_path.rknn> <frames_folder>\n", argv[0]);
    }
}

```

```

        return -1;
    }

    const char* model_path = argv[1];
    const char* frames_folder = argv[2];

    rknn_app_context_t rknn_app_ctx;
    memset(&rknn_app_ctx, 0, sizeof(rknn_app_context_t));

    init_post_process();
    if(init_yolov5_model(model_path, &rknn_app_ctx) != 0) {
        printf("init_yolov5_model fail!\n");
        return -1;
    }

    std::vector<std::string> frames = get_frame_files(frames_folder);
    if(frames.empty()) {
        printf("No frames found: %s\n", frames_folder);
        deinit_post_process();
        release_yolov5_model(&rknn_app_ctx);
        return -1;
    }

    int fb, fb_width, fb_height, fb_pixel_size;
    void* framebuffer = fb_init(&fb, &fb_width, &fb_height, &fb_pixel_size);
    if(!framebuffer) {
        printf("framebuffer init fail!\n");
        deinit_post_process();
        release_yolov5_model(&rknn_app_ctx);
        return -1;
    }

    size_t max_img_size = MODEL_WIDTH * MODEL_HEIGHT * 3;
    uint8_t* src_buf = (uint8_t*)malloc(max_img_size);
    if(!src_buf) {
        printf("malloc src_buf fail\n");
        return -1;
    }

    for(const auto& path : frames) {
        image_buffer_t src_image;
        memset(&src_image, 0, sizeof(image_buffer_t));
        src_image.virt_addr = src_buf;
        src_image.width = MODEL_WIDTH;
        src_image.height = MODEL_HEIGHT;
        src_image.size = MODEL_WIDTH * MODEL_HEIGHT * 3;

        if(read_image(path.c_str(), &src_image) != 0) {
            printf("read_image fail: %s\n", path.c_str());
            continue;
        }
    }

#ifdef RV1106_1103
    int ret = dma_buf_alloc(RV1106_CMA_HEAP_PATH, src_image.size,
                           &rknn_app_ctx.img_dma_buf.dma_buf_fd,

```

```

(void*)&rknn_app_ctx.img_dma_buf.dma_buf_virt_addr);
    memcpy(rknn_app_ctx.img_dma_buf.dma_buf_virt_addr, src_image.virt_addr,
src_image.size);
    dma_sync_cpu_to_device(rknn_app_ctx.img_dma_buf.dma_buf_fd);
    src_image.virt_addr =
(uint8_t*)rknn_app_ctx.img_dma_buf.dma_buf_virt_addr;
    src_image.fd = rknn_app_ctx.img_dma_buf.dma_buf_fd;
    rknn_app_ctx.img_dma_buf.size = src_image.size;
#endif

    object_detect_result_list od_results;
    if(inference_yolov5_model(&rknn_app_ctx, &src_image, &od_results) != 0) {
        printf("inference fail: %s\n", path.c_str());
    }
    #if !defined(RV1106_1103)
        free(src_image.virt_addr);
    #endif

    continue;
}

char text[128];
for(int i=0; i<od_results.count; i++) {
    object_detect_result *det = &od_results.results[i];
    draw_rectangle(&src_image, det->box.left, det->box.top,
        det->box.right - det->box.left,
        det->box.bottom - det->box.top,
        COLOR_BLUE, 2);
    sprintf(text, "%s %.1f%%", coco_cls_to_name(det->cls_id), det-
>prop*100);
    draw_text(&src_image, text, det->box.left, det->box.top-20,
COLOR_RED, 10);
}

// 显示到 framebuffer
rgb24_to_xrgb8888(src_image.virt_addr, (uint32_t*)framebuffer,
    src_image.width, src_image.height,
    fb_width, fb_height);

    #if defined(RV1106_1103)
        dma_buf_free(rknn_app_ctx.img_dma_buf.size,
            &rknn_app_ctx.img_dma_buf.dma_buf_fd,
            rknn_app_ctx.img_dma_buf.dma_buf_virt_addr);
    #endif

}

free(src_buf);
munmap(framebuffer, fb_width * fb_height * fb_pixel_size);
close(fb);

deinit_post_process();
release_yolov5_model(&rknn_app_ctx);
return 0;
}

```

代码会读取指定文件夹下的图片，然后连续显示预测结果

我们回到rknn_model_zoo根路径下，执行

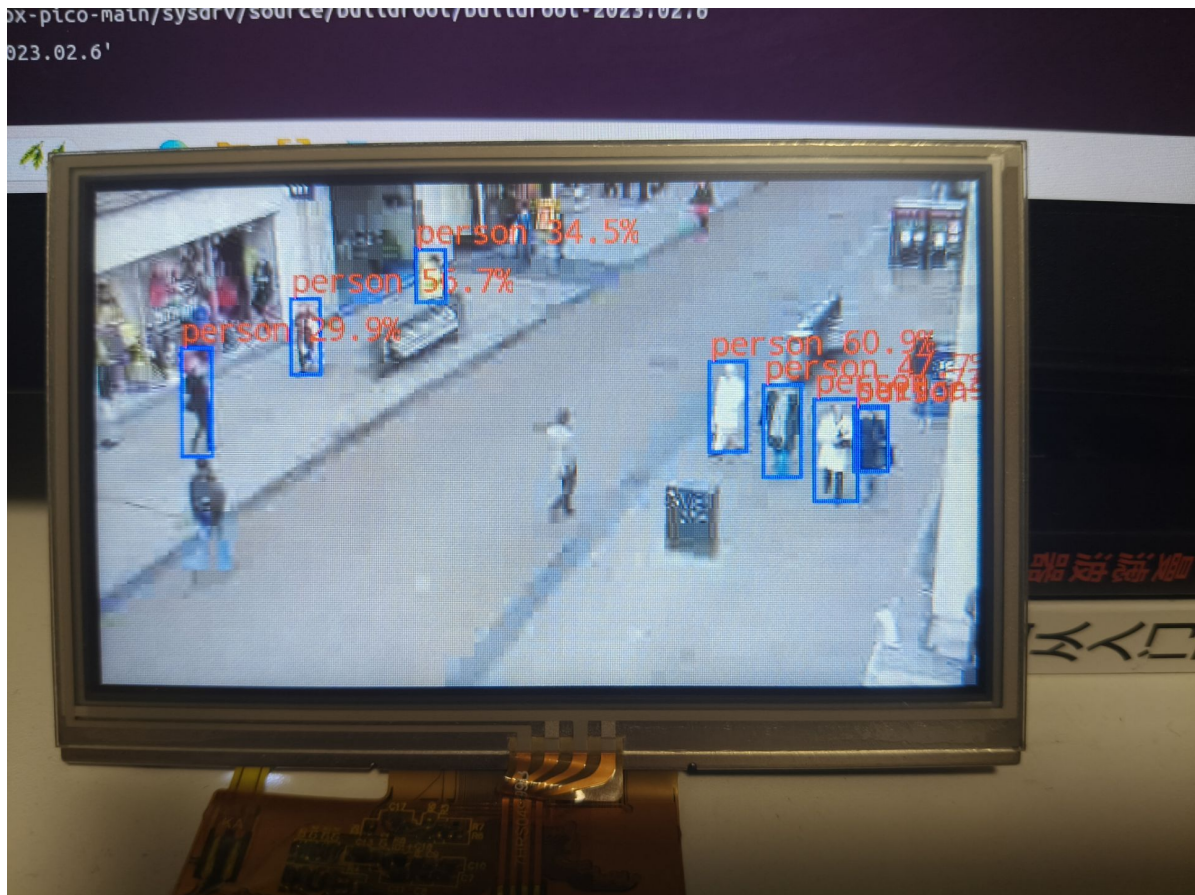
```
export GCC_COMPILER=arm-rockchip830-linux-uclicgnueabihf  
./build-linux.sh -t rv1106 -a armv7l -d yolov5
```

我们将生成的demo 移植到开发板，上传我准备的一段素材yolov5_480.mp4（up 也找不到好的视频素材）。

我们用开发板上的命令来制作需要的图片集。

```
ffmpeg -i kunkun.mp4 -vf "scale=640:640" frames/frame_%04d.jpg  
  
./rknn_yolov5_demo/rknn_yolov5_demo ./rknn_yolov5_demo/model/yolov5_relu.rknn  
./frames/
```

然后进入demo 执行命令，记住一定要进入demo中 不然预测显示的都是null。



3.4.3 人脸识别

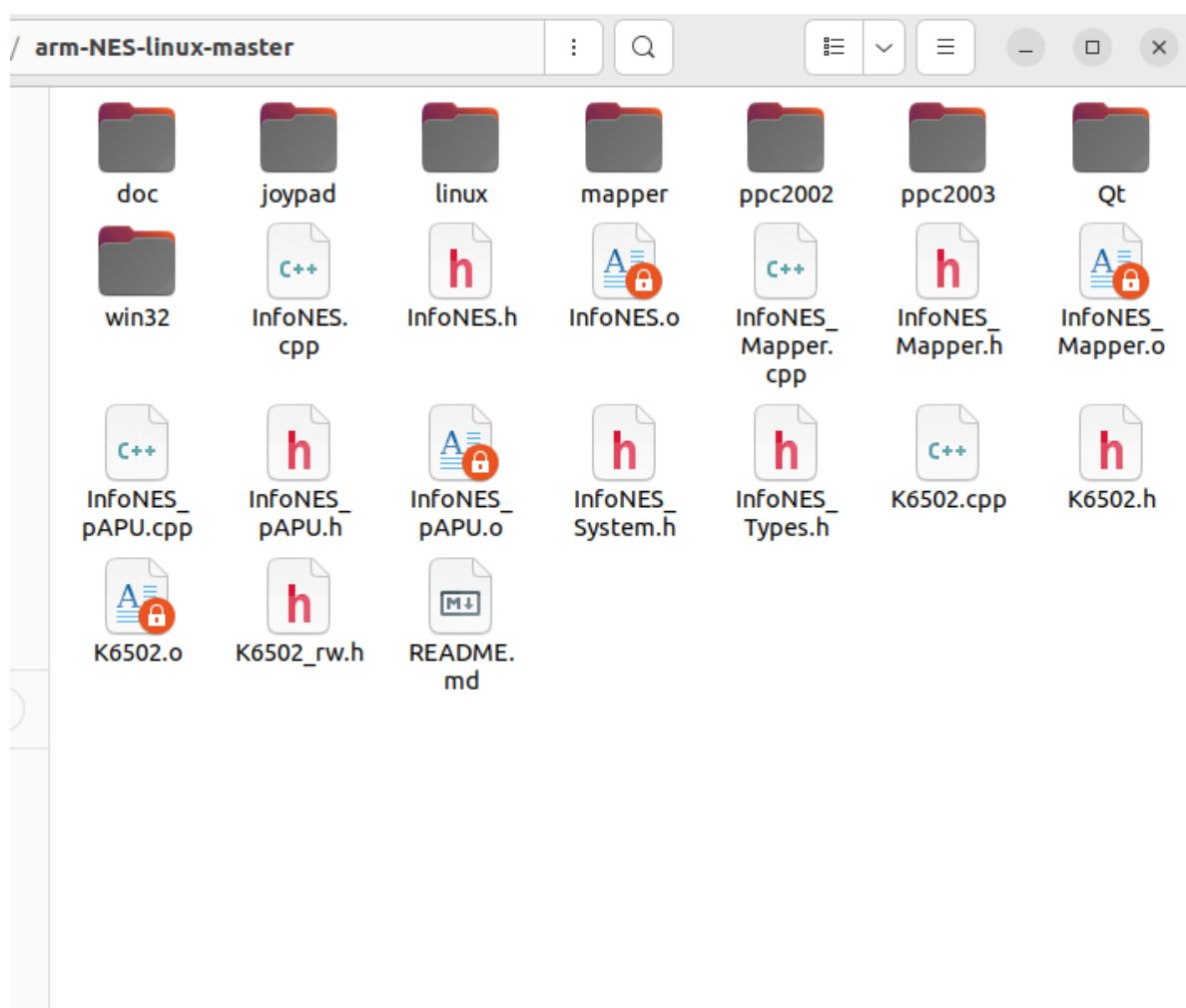
3.5 移植NES游戏模拟器

我试过移植其他NES，非常繁琐，搞了很久，还是报错。偶然发现有个NES 模拟器，被广泛移植。其源码地址在：[nejdev/arm-NES-linux: arm-NES-linux emulate for InfoNES open source](https://github.com/nejdev/arm-NES-linux)

我们把它下载下来，开始移植吧。

3.5.1 移植Info NES

下载下来目录结构大概是这样。



我们进入linux 修改Makefile 其中修改如下：

CC 就是指我们的交叉编译工具。

SYSROOT 指定我们的buildroot 的sysroot，其中有些库供工程编译使用，比如我们的扬声器 alsa 设备。

Makefile

```
# 交叉编译器
CC = arm-rockchip830-linux-uclicgncueabihf-gcc

# 打包文件名
TARBALL = InfoNES

# Buildroot sysroot 路径
```

```
SYSROOT = /luckfox-pico-main/sysdrv/source/buildroot/buildroot-2023.02.6/output/host/usr/arm-buildroot-linux-uclibcgnueabi/hf/sysroot
```

头文件和库路径

```
INCLUDES = -I$(SYSROOT)/usr/include
LDLAGS = -L$(SYSROOT)/usr/lib
```

源文件

```
CFILES = ./../K6502.cpp \
          ./../InfoNES.cpp \
          ./../InfoNES_Mapper.cpp \
          ./../InfoNES_pAPU.cpp \
          ./InfoNES_System_Linux.cpp \
          joystick_input.cpp
```

生成的目标文件

```
OFILES = $(CFILES:.cpp=.o)
```

编译参数

```
CCLAGS = -O2 -fsigned-char
```

链接库

```
LDLAGS = -lstdc++ -lasound -lm -lz -lpthread
```

默认目标

```
all: InfoNES
```

编译 InfoNES 可执行文件

```
InfoNES: $(OFILES)
    $(CC) $(INCLUDES) -o $@ $(OFILES) $(LDLAGS) $(LDLAGS)
```

生成 .o 文件

```
.cpp.o:
    $(CC) $(INCLUDES) -c $(CCLAGS) $< -o $@
```

清理生成文件

```
clean:
    rm -f $(OFILES) ../.*~ ../.*/*~ core
```

清理所有生成文件，包括可执行文件

```
cleanall: clean
    rm -f InfoNES
```

打包源码

```
tar: clean
    ( cd ..; \
      tar cvf $(TARBALL).tar ./.*; \
      gzip -f $(TARBALL).tar )
```

安装可执行文件到系统路径

```
install: InfoNES
    install ./InfoNES /usr/local/bin
```

修改好工程编译文件就可以开始编译了，但是我们还需要修改屏幕，和输入设备。

首先处理 屏幕,up 的屏幕是 rgb888的, 其中 r 是8位, 偏移16 位, g 是 8位, 偏移8位, b 是8位, 偏移0, a是0位。

```
^C
[root@luckfox root]# fbset

mode "480x272-0"
    # D: 0.000 MHz, H: 0.000 kHz, V: 0.000 Hz
    geometry 480 272 480 272 32
    timings 0 0 0 0 0 0 0
    accel true
    rgba 8/16,8/8,8/0,0/0
endmode
```

把16位颜色提取出来, 扩展为32 位

```
62
63 static inline uint32_t rgb565_to_fb32(uint16_t color16)
64 {
65     uint8_t r = (color16 >> 11) & 0x1F;
66     uint8_t g = (color16 >> 5) & 0x3F;
67     uint8_t b = color16 & 0x1F;
68
69     // 扩展到 8-bit
70     r = (r << 3) | (r >> 2);
71     g = (g << 2) | (g >> 4);
72     b = (b << 3) | (b >> 2);
73
74     // 小端内存顺序 ARGB8888 (0x00RRGGBB)
75     return (r << 16) | (g << 8) | b;
76 }
77
78 static int lcd_fb_display_px(uint16_t color16, int x, int y)
79 {
80     uint32_t *pen = (uint32_t *) (fb_mem + y * line_width + x * px_width);
81     *pen = rgb565_to_fb32(color16);
82     return 0;
83 }
84
```

InfoNES_System_Linux.cpp

```
/*=====*/
/*
/*  InfoNES_System_Linux.cpp : Linux specific File
/*
/*
/*  2001/05/18  InfoNES Project ( Sound is based on DarcNES )
/*
/*
/*=====*/

/*-----*/
/*  Include files
/*-----*/

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>

#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
```

```

#include <sys/ioctl.h>
#include <unistd.h>
#include <alsa/asoundlib.h>

#include "../InfoNES.h"
#include "../InfoNES_System.h"
#include "../InfoNES_pAPU.h"

//bool define
#define TRUE 1
#define FALSE 0

/* lcd 操作相关 头文件 */
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <linux/fb.h>
#include <sys/ioctl.h>
#include <unistd.h>
#include <string.h>
#include <sys/mman.h>
#include <termios.h>

#include <fcntl.h>

static snd_pcm_t *playback_handle;

static int fb_fd = -1;
static unsigned char *fb_mem;
static int px_width;
static int line_width;
static int screen_width;
static int lcd_width;
static int lcd_height;
static struct fb_var_screeninfo var;

static int *zoom_x_tab;
static int *zoom_y_tab;

extern int InitJoypadInput(void);
extern int GetJoypadInput(void);

static inline uint32_t rgb565_to_fb32(uint16_t color16)
{
    uint8_t r = (color16 >> 11) & 0x1F;
    uint8_t g = (color16 >> 5) & 0x3F;
    uint8_t b = color16 & 0x1F;

    // 扩展到 8-bit
    r = (r << 3) | (r >> 2);
    g = (g << 2) | (g >> 4);
    b = (b << 3) | (b >> 2);

    // 小端内存顺序 ARGB8888 (0x00RRGGBB)
    return (r << 16) | (g << 8) | b;
}

```

```

}

static int lcd_fb_display_px(uint16_t color16, int x, int y)
{
    uint32_t *pen = (uint32_t *) (fb_mem + y * line_width + x * px_width);
    *pen = rgb565_to_fb32(color16);
    return 0;
}

static int lcd_fb_init()
{
    //如果使用 mmap 打开方式 必须是 读定方式
    fb_fd = open("/dev/fb0", O_RDWR);
    if(-1 == fb_fd)
    {
        printf("cat't open /dev/fb0 \n");
        return -1;
    }
    //获取屏幕参数
    if(-1 == ioctl(fb_fd, FBIOGET_VSCREENINFO, &var))
    {
        close(fb_fd);
        printf("cat't ioctl /dev/fb0 \n");
        return -1;
    }

    //计算参数
    px_width      = var.bits_per_pixel / 8;
    line_width     = var.xres * px_width;
    screen_width  = var.yres * line_width;
    lcd_width      = var.xres;
    lcd_height     = var.yres;

    printf("fb width:%d height:%d \n", lcd_width, lcd_height);

    fb_mem = (unsigned char *)mmap(NULL, screen_width, PROT_READ | PROT_WRITE,
    MAP_SHARED, fb_fd, 0);
    if(fb_mem == (void *)-1)
    {
        close(fb_fd);
        printf("cat't mmap /dev/fb0 \n");
        return -1;
    }
    //清屏
    memset(fb_mem, 0 , screen_width);
    return 0;
}

/**
 * 生成zoom 缩放表
 */
int make_zoom_tab()
{
    int i;
    zoom_x_tab = (int *)malloc(sizeof(int) * lcd_width);

```

```

    if(NULL == zoom_x_tab)
    {
        printf("make zoom_x_tab error\n");
        return -1;
    }
    for(i=0; i<lcd_width; i++)
    {
        zoom_x_tab[i] = i*NES_DISP_WIDTH/lcd_width;
    }
    zoom_y_tab = (int *)malloc(sizeof(int) * lcd_height);
    if(NULL == zoom_y_tab)
    {
        printf("make zoom_y_tab error\n");
        return -1;
    }
    for(i=0; i<lcd_height; i++)
    {
        zoom_y_tab[i] = i*NES_DISP_HEIGHT/lcd_height;
    }
    return 1;
}

/*-----*/
/* ROM image file information */
/*-----*/

char    szRomName[256];
char    szSaveName[256];
int     nSRAM_SaveFlag;

/*-----*/
/* Constants ( Linux specific ) */
/*-----*/

#define VBOX_SIZE    7
#define SOUND_DEVICE    "/dev/dsp"
#define VERSION      "InfoNES v0.91j"

/*-----*/
/* Global variables ( Linux specific ) */
/*-----*/

/* Emulation thread */
pthread_t  emulation_tid;
int  bThread;

/* Pad state */
DWORD     dwKeyPad1;
DWORD     dwKeyPad2;
DWORD     dwKeySystem;

/* For Sound Emulation */
BYTE     final_wave[2048];
int  waveptr;

```

```

int wavflag;
int sound_fd;

/*-----*/
/*  Function prototypes ( Linux specific )      */
/*-----*/

void *emulation_thread( void *args );

void start_application( char *filename );

int LoadSRAM();

int SaveSRAM();

/* Palette data */
WORD NesPalette[64] =
{
    0x39ce, 0x1071, 0x0015, 0x2013, 0x440e, 0x5402, 0x5000, 0x3c20,
    0x20a0, 0x0100, 0x0140, 0x00e2, 0x0ceb, 0x0000, 0x0000, 0x0000,
    0x5ef7, 0x01dd, 0x10fd, 0x401e, 0x5c17, 0x700b, 0x6ca0, 0x6521,
    0x45c0, 0x0240, 0x02a0, 0x0247, 0x0211, 0x0000, 0x0000, 0x0000,
    0x7fff, 0x1eff, 0x2e5f, 0x223f, 0x79ff, 0x7dd6, 0x7dcc, 0x7e67,
    0x7ae7, 0x4342, 0x2769, 0x2ff3, 0x03bb, 0x0000, 0x0000, 0x0000,
    0x7fff, 0x579f, 0x635f, 0x6b3f, 0x7f1f, 0x7f1b, 0x7ef6, 0x7f75,
    0x7f94, 0x73f4, 0x57d7, 0x5bf9, 0x4ffe, 0x0000, 0x0000, 0x0000
};

/*=====*/
/*
main() : Application main
*/
/*=====*/

/* Application main */
int main( int argc, char **argv )
{
    char cmd;

    /*-----*/
    /*  Pad Control      */
    /*-----*/

    /* Initialize a pad state */
    dwKeyPad1  = 0;
    dwKeyPad2  = 0;
    dwKeySystem = 0;

    /*-----*/
    /*  Load Cassette & Create Thread      */
    /*-----*/

```

```

/* Initialize thread state */
bThread = FALSE;

int i;

InitJoypadInput();

lcd_fb_init();

//初始化 zoom 缩放表
make_zoom_tab();

/* If a rom name specified, start it */
if ( argc == 2 )
{
    start_application( argv[1] );
}

//主循环中处理输入事件 声音播放
while(1)
{
    dwKeyPad1 = GetJoypadInput();
    //主线程休息一下 让子线程用一下 CPU
    usleep(300);
}
return(0);
}

/*=====*/
/*
/*      emulation_thread() : Thread Hooking Routine      */
/*
/*=====*/

void *emulation_thread( void *args )
{
    InfoNES_Main();
}

/*=====*/
/*
/*      start_application() : Start NES Hardware      */
/*
/*=====*/
void start_application( char *filename )
{
    /* Set a ROM image name */
    strcpy( szRomName, filename );

    /* Load cassette */
    if ( InfoNES_Load( szRomName ) == 0 )
    {
        /* Load SRAM */

```

```

        LoadSRAM();

        /* Create Emulation Thread */
        bThread = TRUE;
        pthread_create( &emulation_tid, NULL, emulation_thread, NULL );
    }
}

/*=====*/
/*
/*          LoadSRAM() : Load a SRAM
/*
/*
/*=====*/
int LoadSRAM()
{
/*
*   Load a SRAM
*
*   Return values
*       0 : Normally
*      -1 : SRAM data couldn't be read
*/

    FILE          *fp;
    unsigned char  pSrcBuf[SRAM_SIZE];
    unsigned char  chData;
    unsigned char  chTag;
    int            nRunLen;
    int            nDecoded;
    int            nDecLen;
    int            nIdx;

    /* It doesn't need to save it */
    nSRAM_SaveFlag = 0;

    /* It is finished if the ROM doesn't have SRAM */
    if ( !ROM_SRAM )
        return(0);

    /* There is necessity to save it */
    nSRAM_SaveFlag = 1;

    /* The preparation of the SRAM file name */
    strcpy( szSaveName, szRomName );
    strcpy( strrchr( szSaveName, '.' ) + 1, "srm" );

    /*-----*/
    /*   Read a SRAM data
    /*-----*/

    /* Open SRAM file */
    fp = fopen( szSaveName, "rb" );
    if ( fp == NULL )
        return(-1);

```

```

/* Read SRAM data */
fread( pSrcBuf, SRAM_SIZE, 1, fp );

/* Close SRAM file */
fclose( fp );

/*-----*/
/* Extract a SRAM data */
/*-----*/

nDecoded    = 0;
nDecLen     = 0;

chTag = pSrcBuf[nDecoded++];

while ( nDecLen < 8192 )
{
    chData = pSrcBuf[nDecoded++];

    if ( chData == chTag )
    {
        chData = pSrcBuf[nDecoded++];
        nRunLen = pSrcBuf[nDecoded++];
        for ( nIdx = 0; nIdx < nRunLen + 1; ++nIdx )
        {
            SRAM[nDecLen++] = chData;
        }
    }else {
        SRAM[nDecLen++] = chData;
    }
}

/* Successful */
return(0);
}

/*=====*/
/*
/*          SaveSRAM() : Save a SRAM
/*
/*=====*/
int SaveSRAM()
{
/*
* Save a SRAM
*
* Return values
* 0 : Normally
* -1 : SRAM data couldn't be written
*/

FILE      *fp;
int      nUsedTable[256];

```

```

unsigned char    chData;
unsigned char    chPrevData;
unsigned char    chTag;
int             nIdx;
int             nEncoded;
int             nEncLen;
int             nRunLen;
unsigned char    pDstBuf[SRAM_SIZE];

if ( !nSRAM_SaveFlag )
    return(0); /* It doesn't need to save it */

/*-----*/
/*  Compress a SRAM data                                */
/*-----*/

memset( nUsedTable, 0, sizeof nUsedTable );

for ( nIdx = 0; nIdx < SRAM_SIZE; ++nIdx )
{
    ++nUsedTable[SRAM[nIdx]];
}
for ( nIdx = 1, chTag = 0; nIdx < 256; ++nIdx )
{
    if ( nUsedTable[nIdx] < nUsedTable[chTag] )
        chTag = nIdx;
}

nEncoded    = 0;
nEncLen     = 0;
nRunLen     = 1;

pDstBuf[nEncLen++] = chTag;

chPrevData = SRAM[nEncoded++];

while ( nEncoded < SRAM_SIZE && nEncLen < SRAM_SIZE - 133 )
{
    chData = SRAM[nEncoded++];

    if ( chPrevData == chData && nRunLen < 256 )
        ++nRunLen;
    else{
        if ( nRunLen >= 4 || chPrevData == chTag )
        {
            pDstBuf[nEncLen++] = chTag;
            pDstBuf[nEncLen++] = chPrevData;
            pDstBuf[nEncLen++] = nRunLen - 1;
        }else {
            for ( nIdx = 0; nIdx < nRunLen; ++nIdx )
                pDstBuf[nEncLen++] = chPrevData;
        }

        chPrevData = chData;
        nRunLen    = 1;
    }
}

```

```

    }
}
if ( nRunLen >= 4 || chPrevData == chTag )
{
    pDstBuf[nEncLen++] = chTag;
    pDstBuf[nEncLen++] = chPrevData;
    pDstBuf[nEncLen++] = nRunLen - 1;
}else {
    for ( nIdx = 0; nIdx < nRunLen; ++nIdx )
        pDstBuf[nEncLen++] = chPrevData;
}

/*-----*/
/* Write a SRAM data */
/*-----*/

/* Open SRAM file */
fp = fopen( szSaveName, "wb" );
if ( fp == NULL )
    return(-1);

/* Write SRAM data */
fwrite( pDstBuf, nEncLen, 1, fp );

/* Close SRAM file */
fclose( fp );

/* Successful */
return(0);
}

/*=====*/
/*
/* InfoNES_Menu() : Menu screen
/*
/*=====*/
int InfoNES_Menu()
{
/*
* Menu screen
*
* Return values
* 0 : Normally
* -1 : Exit InfoNES
*/

/* If terminated */
if ( bThread == FALSE )
{
    return(-1);
}

/* Nothing to do here */
return(0);

```

```

}

/*=====*/
/*
/*          InfoNES_ReadRom() : Read ROM image file          */
/*
/*=====*/
int InfoNES_ReadRom( const char *pszFileName )
{
/*
* Read ROM image file
*
* Parameters
*   const char *pszFileName      (Read)
*
* Return values
*   0 : Normally
*  -1 : Error
*/

FILE *fp;

/* Open ROM file */
fp = fopen( pszFileName, "rb" );
if ( fp == NULL )
    return(-1);

/* Read ROM Header */
fread( &NesHeader, sizeof NesHeader, 1, fp );
if ( memcmp( NesHeader.byID, "NES\x1a", 4 ) != 0 )
{
    /* not .nes file */
    fclose( fp );
    return(-1);
}

/* Clear SRAM */
memset( SRAM, 0, SRAM_SIZE );

/* If trainer presents Read Trainer at 0x7000-0x71ff */
if ( NesHeader.byInfo1 & 4 )
{
    fread( &SRAM[0x1000], 512, 1, fp );
}

/* Allocate Memory for ROM Image */
ROM = (BYTE *) malloc( NesHeader.byRomSize * 0x4000 );

/* Read ROM Image */
fread( ROM, 0x4000, NesHeader.byRomSize, fp );

if ( NesHeader.byVRomSize > 0 )
{
    /* Allocate Memory for VROM Image */

```

```

        VROM = (BYTE *) malloc( NesHeader.byVRomSize * 0x2000 );

        /* Read VROM Image */
        fread( VROM, 0x2000, NesHeader.byVRomSize, fp );
    }

    /* File close */
    fclose( fp );

    /* Successful */
    return(0);
}

/*=====*/
/*
/*          InfoNES_ReleaseRom() : Release a memory for ROM          */
/*
/*=====*/
void InfoNES_ReleaseRom()
{
    /*
    *   Release a memory for ROM
    *
    */

    if ( ROM )
    {
        free( ROM );
        ROM = NULL;
    }

    if ( VROM )
    {
        free( VROM );
        VROM = NULL;
    }
}

/*=====*/
/*
/*          InfoNES_MemoryCopy() : memcpy          */
/*
/*=====*/
void *InfoNES_MemoryCopy( void *dest, const void *src, int count )
{
    /*
    *   memcpy
    *
    *   Parameters
    *       void *dest                (write)
    *       Points to the starting address of the copied block's destination
    *
    *       const void *src           (Read)
    */

```

```

*      Points to the starting address of the block of memory to copy
*
*      int count                                (Read)
*      Specifies the size, in bytes, of the block of memory to copy
*
* Return values
*      Pointer of destination
*/

memcpy( dest, src, count );
return(dest);
}

/*=====*/
/*
/*      InfoNES_MemorySet() : memset
/*
/*
/*=====*/
void *InfoNES_MemorySet( void *dest, int c, int count )
{
/*
*      memset
*
* Parameters
*      void *dest                                (Write)
*      Points to the starting address of the block of memory to fill
*
*      int c                                    (Read)
*      Specifies the byte value with which to fill the memory block
*
*      int count                                (Read)
*      Specifies the size, in bytes, of the block of memory to fill
*
* Return values
*      Pointer of destination
*/

memset( dest, c, count );
return(dest);
}

/*=====*/
/*
/*      InfoNES_LoadFrame() :
/*      Transfer the contents of work frame on the screen
/*
/*
/*=====*/
void InfoNES_LoadFrame()
{
    int x,y;
    int line_width;
    WORD wColor;

```

```

//修正 即便没有 LCD 也可以出声
if(0 < fb_fd)
{
    for (y = 0; y < lcd_height; y++ )
    {
        line_width = zoom_y_tab[y] * NES_DISP_WIDTH;
        for (x = 0; x < lcd_width; x++ )
        {
            wColor = workFrame[line_width + zoom_x_tab[x]];
            lcd_fb_display_px(wColor, x, y);
        }
    }
}

/*=====*/
/*
/*          InfNES_PadState() : Get a joypad state          */
/*
/*=====*/
void InfNES_PadState( DWORD *pdwPad1, DWORD *pdwPad2, DWORD *pdwSystem )
{
/*
*   Get a joypad state
*
*   Parameters
*       DWORD *pdwPad1           (write)
*       Joypad 1 State
*
*       DWORD *pdwPad2           (write)
*       Joypad 2 State
*
*       DWORD *pdwSystem         (write)
*       Input for InfNES
*
*/

/* Transfer joypad state */
*pdwPad1    = dwKeyPad1;
*pdwPad2    = dwKeyPad2;
*pdwSystem  = dwKeySystem;

//取消重置手柄 在 输入函数中自行处理
//dwKeyPad1 = 0;
}

/*=====*/
/*
/*          InfNES_SoundInit() : Sound Emulation Initialize */
/*
/*=====*/
void InfNES_SoundInit( void )
{

```

```

}

/*=====*/
/*                                          */
/*      InfONES_SoundOpen() : Sound Open      */
/*                                          */
/*=====*/
int InfONES_SoundOpen( int samples_per_sync, int sample_rate )
{
    //sample_rate 采样率 44100
    //samples_per_sync 735
    unsigned int rate      = sample_rate;
    snd_pcm_hw_params_t *hw_params;

    if(0 > snd_pcm_open(&playback_handle, "default", SND_PCM_STREAM_PLAYBACK, 0))
    {
        printf("snd_pcm_open err\n");
        return -1;
    }

    if(0 > snd_pcm_hw_params_malloc(&hw_params))
    {
        printf("snd_pcm_hw_params_malloc err\n");
        return -1;
    }

    if(0 > snd_pcm_hw_params_any(playback_handle, hw_params))
    {
        printf("snd_pcm_hw_params_any err\n");
        return -1;
    }

    if(0 > snd_pcm_hw_params_set_access(playback_handle, hw_params,
SND_PCM_ACCESS_RW_INTERLEAVED))
    {
        printf("snd_pcm_hw_params_any err\n");
        return -1;
    }

    //8bit PCM 数据
    if(0 > snd_pcm_hw_params_set_format(playback_handle, hw_params,
SND_PCM_FORMAT_U8))
    {
        printf("snd_pcm_hw_params_set_format err\n");
        return -1;
    }

    if(0 > snd_pcm_hw_params_set_rate_near(playback_handle, hw_params, &rate, 0))
    {
        printf("snd_pcm_hw_params_set_rate_near err\n");
        return -1;
    }

    //单声道 非立体声

```

```

    if(0 > snd_pcm_hw_params_set_channels(playback_handle, hw_params, 1))
    {
        printf("snd_pcm_hw_params_set_channels err\n");
        return -1;
    }

    if(0 > snd_pcm_hw_params(playback_handle, hw_params))
    {
        printf("snd_pcm_hw_params err\n");
        return -1;
    }

    snd_pcm_hw_params_free(hw_params);

    if(0 > snd_pcm_prepare(playback_handle))
    {
        printf("snd_pcm_prepare err\n");
        return -1;
    }
    return 1;
}

/*=====*/
/*                                          */
/*      InfoNES_SoundClose() : Sound Close      */
/*                                          */
/*=====*/
void InfoNES_SoundClose( void )
{
    snd_pcm_close(playback_handle);
}

/*=====*/
/*                                          */
/*      InfoNES_SoundOutput() : Sound Output 5 Waves      */
/*                                          */
/*=====*/
void InfoNES_SoundOutput( int samples, BYTE *wave1, BYTE *wave2, BYTE *wave3,
BYTE *wave4, BYTE *wave5 )
{
    int i;
    int ret;
    unsigned char wav;
    unsigned char *pcmBuf = (unsigned char *)malloc(samples);

    for (i=0; i < samples; i++)
    {
        wav = (wave1[i] + wave2[i] + wave3[i] + wave4[i] + wave5[i]) / 5;
        //单声道 8位数据
        pcmBuf[i] = wav;
    }
    ret = snd_pcm_writei(playback_handle, pcmBuf, samples);
    if(-EPIPE == ret)

```

```

    {
        snd_pcm_prepare(playback_handle);
    }
    free(pcmBuf);
    return ;
}

/*=====*/
/*
/*      InfoNES_wait() : Wait Emulation if required      */
/*
/*=====*/
void InfoNES_wait()
{
}

/*=====*/
/*
/*      InfoNES_MessageBox() : Print System Message      */
/*
/*=====*/
void InfoNES_MessageBox(const char *pszMsg, ... )
{
    printf( "MessageBox: %s \n", pszMsg );
}

/*
 * End of InfoNES_System_Linux.cpp
 */

```

接下来处理输入，由于up 没有手柄，用的是键盘，故使用键盘输入，按照我的代码修改即可。

代码可以看看，有键盘对手柄输入的映射，修改完成后，我们编译 linux下的makefile。

jopad_input.cpp

```

// jopad_input.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <pthread.h>
#include <dirent.h>
#include <linux/input.h>
#include <sys/ioctl.h>
#include <errno.h>
#include <stdint.h>

#define UP_MASK      (1<<0)
#define DOWN_MASK    (1<<1)

```

```

#define LEFT_MASK    (1<<2)
#define RIGHT_MASK   (1<<3)
#define A_MASK       (1<<4)
#define B_MASK       (1<<5)
#define START_MASK   (1<<6)
#define SELECT_MASK  (1<<7)

/* ----- 框架保持与原工程一致 ----- */
typedef struct JoypadInput{
    int (*DevInit)(void);
    int (*DevExit)(void);
    int (*GetJoypad)(void);
    struct JoypadInput *ptNext;
    pthread_t tTreadID;
} T_JoypadInput, *PT_JoypadInput;

static unsigned char g_InputEvent;
static pthread_mutex_t g_tMutex = PTHREAD_MUTEX_INITIALIZER;
static pthread_cond_t g_tConVar = PTHREAD_COND_INITIALIZER;
static PT_JoypadInput g_ptJoypadInputHead = NULL;

/* ----- 键盘扫描与读取 ----- */
static int keyboard_fd = -1;
static char keyboard_dev_path[256] = {0};

static int test_bit(int bit, const unsigned long *array)
{
    return (array[bit/ (8*sizeof(unsigned long))] >> (bit % (8*sizeof(unsigned long)))) & 1;
}

static void dump_bits(const char *tag, const unsigned long *bits, int maxbit)
{
    printf("[DEBUG] %s:", tag);
    for (int i = 0; i <= maxbit; ++i)
        if (test_bit(i, bits)) printf(" %d", i);
    printf("\n");
}

static int looks_like_keyboard(int fd, char *name_buf, size_t name_len, int
*has_rel, int *has_abs, int *has_key_a)
{
    unsigned long ev_bits[(EV_MAX+1) / (8*sizeof(unsigned long)) + 1];
    unsigned long key_bits[(KEY_MAX+1)/(8*sizeof(unsigned long)) + 1];
    unsigned long rel_bits[(REL_MAX+1)/(8*sizeof(unsigned long)) + 1];
    unsigned long abs_bits[(ABS_MAX+1)/(8*sizeof(unsigned long)) + 1];
    memset(ev_bits,0,sizeof(ev_bits));
    memset(key_bits,0,sizeof(key_bits));
    memset(rel_bits,0,sizeof(rel_bits));
    memset(abs_bits,0,sizeof(abs_bits));

    if (ioctl(fd, EVIOCGNAME(name_len), name_buf) < 0) {
        snprintf(name_buf, name_len, "unknown");
    }
}

```

```

    if (ioctl(fd, EVIOCGBIT(0, sizeof(ev_bits)), ev_bits) < 0) return 0;

    int ev_key = test_bit(EV_KEY, ev_bits);
    int ev_rel = test_bit(EV_REL, ev_bits);
    int ev_abs = test_bit(EV_ABS, ev_bits);

    if (has_rel) *has_rel = ev_rel;
    if (has_abs) *has_abs = ev_abs;

    if (!ev_key) return 0;

    if (ioctl(fd, EVIOCGBIT(EV_KEY, sizeof(key_bits)), key_bits) < 0) return 0;
    if (ioctl(fd, EVIOCGBIT(EV_REL, sizeof(rel_bits)), rel_bits) < 0)
memset(rel_bits, 0, sizeof(rel_bits));
    if (ioctl(fd, EVIOCGBIT(EV_ABS, sizeof(abs_bits)), abs_bits) < 0)
memset(abs_bits, 0, sizeof(abs_bits));

    int hasA = test_bit(KEY_A, key_bits);
    if (has_key_a) *has_key_a = hasA;

    /* 调试输出（如太吵可注释） */
    printf("[DEBUG] name=\"%s\" ev:key=%d rel=%d abs=%d has KEY_A=%d\n",
        name_buf, ev_key, ev_rel, ev_abs, hasA);

    /* 进一步加分项：名字里含 keyboard */
    int name_keyboard = (strcasestr(name_buf, "keyboard") != NULL);

    /* 规则：必须能产生标准键（含 KEY_A），且最好不是相对/绝对坐标设备 */
    return (hasA || name_keyboard);
}

static int open_event_force(const char *path)
{
    int fd = open(path, O_RDONLY | O_NONBLOCK);
    if (fd < 0) {
        perror("[DEBUG] open forced event");
        return -1;
    }
    strncpy(keyboard_dev_path, path, sizeof(keyboard_dev_path)-1);
    keyboard_dev_path[sizeof(keyboard_dev_path)-1] = 0;
    printf("[DEBUG] Forced event opened: %s\n", keyboard_dev_path);
    return fd;
}

static int findKeyboardDevice(char *devPath, size_t len)
{
    /* 优先：允许用户强制指定 */
    const char *force = getenv("INPUT_EVENT");
    if (force && strlen(force) > 0) {
        int fd = open(force, O_RDONLY | O_NONBLOCK);
        if (fd >= 0) {
            close(fd);
            strncpy(devPath, force, len-1);
            devPath[len-1] = 0;
            printf("[DEBUG] Using forced INPUT_EVENT=%s\n", devPath);
        }
    }
}

```

```

        return 0;
    } else {
        fprintf(stderr, "[WARN] INPUT_EVENT=%s open failed: %s\n", force,
strerror(errno));
    }
}

DIR *dir = opendir("/dev/input");
if (!dir) {
    perror("opendir /dev/input");
    return -1;
}

/* 先收集候选，再评分选择最优 */
char best_path[256] = {0};
int best_score = -1;

struct dirent *entry;
while ((entry = readdir(dir))) {
    if (strncmp(entry->d_name, "event", 5) != 0) continue;
    char path[256];
    snprintf(path, sizeof(path), "/dev/input/%s", entry->d_name);

    int fd = open(path, O_RDONLY | O_NONBLOCK);
    if (fd < 0) continue;

    char name[256] = {0};
    int has_rel=0, has_abs=0, has_key_a=0;
    int kbd_like = looks_like_keyboard(fd, name, sizeof(name), &has_rel,
&has_abs, &has_key_a);

    /* 给个评分：
    +1000 如果名字里含keyboard
    +100  如果含KEY_A
    -200  如果有EV_REL
    -200  如果有EV_ABS
    */
    int score = -100000;
    if (kbd_like) {
        score = 0;
        if (strcasestr(name, "keyboard")) score += 1000;
        if (has_key_a) score += 100;
        if (has_rel) score -= 200;
        if (has_abs) score -= 200;
    }

    printf("[DEBUG] candidate %s name=\"%s\" score=%d\n", path, name, score);

    if (score > best_score) {
        best_score = score;
        strncpy(best_path, path, sizeof(best_path)-1);
    }

    close(fd);
}

```

```

        closedir(dir);

        if (best_score < 0) {
            fprintf(stderr, "[ERROR] No suitable keyboard-like event device
found.\n");
            return -1;
        }

        strncpy(devPath, best_path, len-1);
        devPath[len-1] = 0;
        printf("[DEBUG] Selected keyboard device: %s (score=%d)\n", devPath,
best_score);
        return 0;
    }

static int usbKeyboardDevInit(void)
{
    if (findKeyboardDevice(keyboard_dev_path, sizeof(keyboard_dev_path)) < 0) {
        fprintf(stderr, "No keyboard device found!\n");
        return -1;
    }

    keyboard_fd = open(keyboard_dev_path, O_RDONLY | O_NONBLOCK);
    if (keyboard_fd < 0) {
        perror("open keyboard device");
        return -1;
    }

    /* 可选：独占抓取，避免字符被终端吃掉（必要时打开） */
    // if (ioctl(keyboard_fd, EVIOCGRAB, 1) < 0) {
    //     perror("[WARN] EVIOCGRAB");
    // }

    /* 打印最终确认信息 */
    char name[256] = {0};
    ioctl(keyboard_fd, EVIOCGNAME(sizeof(name)), name);
    printf("[DEBUG] keyboard device opened: %s, name=\"%s\"\n",
keyboard_dev_path, name[0]?name:"unknown");

    /* 再打印能力位，帮助诊断 */
    unsigned long ev_bits[(EV_MAX+1) / (8*sizeof(unsigned long)) + 1] = {0};
    unsigned long key_bits[(KEY_MAX+1)/(8*sizeof(unsigned long)) + 1] = {0};
    if (ioctl(keyboard_fd, EVIOCGBIT(0, sizeof(ev_bits)), ev_bits) == 0) {
        dump_bits("EV bits", ev_bits, EV_MAX);
    }

    if (ioctl(keyboard_fd, EVIOCGBIT(EV_KEY, sizeof(key_bits)), key_bits) == 0) {
        printf("[DEBUG] KEY_A=%d KEY_W=%d KEY_S=%d KEY_A(left)=%d KEY_D=%d J=%d
K=%d U=%d I=%d\n",
            test_bit(KEY_A, key_bits),
            test_bit(KEY_W, key_bits),
            test_bit(KEY_S, key_bits),
            test_bit(KEY_A, key_bits),
            test_bit(KEY_D, key_bits),
            test_bit(KEY_J, key_bits),
            test_bit(KEY_K, key_bits),

```

```

        test_bit(KEY_U, key_bits),
        test_bit(KEY_I, key_bits));
    }

    return 0;
}

static int usbKeyboardDevExit(void)
{
    if(keyboard_fd >= 0) {
        // ioctl(keyboard_fd, EVIOCGRAB, 0); // 若之前抓取，这里释放
        close(keyboard_fd);
        keyboard_fd = -1;
    }
    printf("[DEBUG] keyboard device closed\n");
    return 0;
}

/* 可同时支持两套映射：WASD / 方向键；J/K 或 Z/X；U/I 或 Enter/Space */
static int usbKeyboardGet(void)
{
    static unsigned char joypad = 0;
    struct input_event ev;
    ssize_t n;

    while ((n = read(keyboard_fd, &ev, sizeof(ev))) == sizeof(ev)) {
        if (ev.type == EV_SYN) continue;
        if (ev.type != EV_KEY) continue;

        int pressed = (ev.value != 0); // 1=down, 0=up (某些设备2=autorepeat)
        if (ev.value == 2) pressed = 1; // 将重复也视作按下

        switch (ev.code) {
            /* 方向：WASD & 方向键 */
            case KEY_W:
            case KEY_UP:
                joypad = pressed ? (joypad | UP_MASK) : (joypad & ~UP_MASK);
            break;

            case KEY_S:
            case KEY_DOWN:
                joypad = pressed ? (joypad | DOWN_MASK) : (joypad & ~DOWN_MASK);
            break;

            case KEY_A: /* 左字母A 与 左方向区有冲突？注意 KEY_A(字母A) 与 KEY_LEFT(方向键左) 是不同code */
            case KEY_LEFT:
                joypad = pressed ? (joypad | LEFT_MASK) : (joypad & ~LEFT_MASK);
            break;

            case KEY_D:
            case KEY_RIGHT:
                joypad = pressed ? (joypad | RIGHT_MASK) : (joypad & ~RIGHT_MASK);
            break;

            /* A/B: J/K 或 Z/X 或 N/M */
            case KEY_J:
            case KEY_Z:

```

```

        case KEY_N:
            joypad = pressed ? (joypad | A_MASK) : (joypad & ~A_MASK); break;
        case KEY_K:
        case KEY_X:
        case KEY_M:
            joypad = pressed ? (joypad | B_MASK) : (joypad & ~B_MASK); break;

        /* START/SELECT: U/I 或 Enter/Space */
        case KEY_U:
        case KEY_ENTER:
            joypad = pressed ? (joypad | START_MASK) : (joypad &
~START_MASK); break;
        case KEY_I:
        case KEY_SPACE:
            joypad = pressed ? (joypad | SELECT_MASK) : (joypad &
~SELECT_MASK); break;

        default:
            /* 其他键仅打印 */
            break;
    }

    printf("[DEBUG] ev.code=%d (%s) %s -> joypad=0x%02X\n",
           ev.code,

(ev.code==KEY_W?"W":ev.code==KEY_A?"A":ev.code==KEY_S?"S":ev.code==KEY_D?"D":

ev.code==KEY_UP?"UP":ev.code==KEY_LEFT?"LEFT":ev.code==KEY_DOWN?"DOWN":ev.code==
KEY_RIGHT?"RIGHT":

ev.code==KEY_J?"J":ev.code==KEY_K?"K":ev.code==KEY_Z?"Z":ev.code==KEY_X?"X":

ev.code==KEY_U?"U":ev.code==KEY_I?"I":ev.code==KEY_ENTER?"ENTER":ev.code==KEY_SP
ACE?"SPACE":"OTHER"),
           pressed ? "pressed" : "released",
           joypad);
    }

    if (n < 0 && errno != EAGAIN && errno != EWOULDBLOCK) {
        perror("[ERROR] read");
    }

    return joypad;
}

/* ----- 线程与注册 ----- */
static void *InputEventThreadFunction(void *pvoid)
{
    int (*GetJoypad)(void) = (int (*)(void))pvoid;
    while (1) {
        unsigned char val = GetJoypad();
        pthread_mutex_lock(&g_tMutex);
        g_InputEvent = val;
        pthread_cond_signal(&g_tConVar);
        pthread_mutex_unlock(&g_tMutex);
    }
}

```

```

        usleep(10000); // 10ms
    }
    return NULL;
}

static int RegisterJoypadInput(PT_JoypadInput ptJoypadInput)
{
    PT_JoypadInput tmp;
    if(ptJoypadInput->DevInit()) return -1;

    if(pthread_create(&ptJoypadInput->tTreadID, NULL, InputEventThreadFunction,
(void*)ptJoypadInput->GetJoypad) != 0) {
        perror("pthread_create");
        ptJoypadInput->DevExit();
        return -1;
    }

    if(!g_ptJoypadInputHead) g_ptJoypadInputHead = ptJoypadInput;
    else {
        tmp = g_ptJoypadInputHead;
        while(tmp->ptNext) tmp = tmp->ptNext;
        tmp->ptNext = ptJoypadInput;
    }
    ptJoypadInput->ptNext = NULL;
    return 0;
}

static T_JoypadInput usbkeyboardInput = {
    usbKeyboardDevInit,
    usbKeyboardDevExit,
    usbKeyboardGet,
};

int InitJoypadInput(void)
{
    return RegisterJoypadInput(&usbkeyboardInput);
}

int GetJoypadInput(void)
{
    unsigned char val;
    pthread_mutex_lock(&g_tMutex);
    pthread_cond_wait(&g_tConVar, &g_tMutex);
    val = g_InputEvent;
    pthread_mutex_unlock(&g_tMutex);
    return val;
}

```

将编译出来的 InfoNES 移植到Rvicard 上，同时可以移植.nes 游戏文件，up的资料会提供部分游戏（游戏名不可带中文）。

执行可执行程序 加空格 就可以运行目标游戏(接上扬声器可以听见音效哦)

```
./InfoNES hun.nes
```

```

01P3.jpg test.wav
Super Mario Bros. 3 (USA).nes test_fb
[root@luckfox root]# ./InfoNES hun.nes
[DEBUG] name="SEM USB Keyboard System Control" ev:key=1 rel=0 abs=0 has KEY_A=0
[DEBUG] candidate /dev/input/event3 name="SEM USB Keyboard System Control" score=1000
[DEBUG] name="SEM USB Keyboard Consumer Control" ev:key=1 rel=1 abs=1 has KEY_A=0
[DEBUG] candidate /dev/input/event2 name="SEM USB Keyboard Consumer Control" score=600
[DEBUG] name="SEM USB Keyboard" ev:key=1 rel=0 abs=0 has KEY_A=1
[DEBUG] candidate /dev/input/event1 name="SEM USB Keyboard" score=1100
[DEBUG] name="adc-keys" ev:key=1 rel=0 abs=0 has KEY_A=0
[DEBUG] candidate /dev/input/event0 name="adc-keys" score=-100000
[DEBUG] Selected keyboard device: /dev/input/event1 (score=1100)
[DEBUG] Keyboard device opened: /dev/input/event1, name="SEM USB Keyboard"
fb width:480 height:272

```

Please support MobaXterm by subscribing to the professional edition here: <https://mobaxterm.mobatek.net>

可以按键，试试如何操作界面吧，你的键盘输入，将会输出调试信息，赶快玩起来吧！！



3.6 该移植什么好呢？